
DATA SCIENCE

Série 1 — Volume IV

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume IV : Sujets avancés et modernes

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données, modèles et environnement	2
Lancer le code	2
Vérifier son installation	3
Plan du cahier	4
Chapitre 1 : Deep learning — exercices et applications	5
Préparation	5
Applications	5
Application 1.1 — La rétropropagation sur un nouveau réseau (section 1.1.5) . . .	6
Application 1.2 — Optimiseurs et pas d'apprentissage (section 1.1.7)	6
Application 1.3 — Arrêt précoce et pénalité sur les poids (section 1.1.8)	7
Application 1.4 — Une convolution écrite à la main (section 1.2.2)	7
Application 1.5 — Mesurer l'effet de l'augmentation de données (section 1.2.6) . .	8
Application 1.6 — LSTM ou GRU ? Fenêtre courte ou longue ? (section 1.3.4) . . .	9
Application 1.7 — Lire des factures dégradées : OCR et inclinaison (section 1.5) . .	10
Application 1.8 — Réseau ou boosting : l'effet de la taille des données (section 1.6)	10
Exercices	12
Exercice 1.1 ★ — Le gradient du softmax (section 1.1.4)	12
Exercice 1.2 ★ — Compter les paramètres (sections 1.1.3 et 1.2.5)	12
Exercice 1.3 ★★ — Rétropropagation avec une ReLU (section 1.1.5)	12
Exercice 1.4 ★ — Choisir une activation (section 1.1.2)	12
Exercice 1.5 ★★ — Le dropout « inversé » (section 1.1.8)	12
Exercice 1.6 ★ — Taille de sortie d'une convolution (section 1.2.2)	12
Exercice 1.7 ★★ — Le champ réceptif (section 1.2.4)	12
Exercice 1.8 ★★ — Un pas de LSTM à la main (section 1.3.3)	13
Exercice 1.9 ★ — Le naïf saisonnier à la main (section 1.3.4)	13
Exercice 1.10 ★ — Le bug de la boucle (section 1.4.3)	13
Exercice 1.11 ★ — Le taux d'erreur par caractère (section 1.5)	13
Exercice 1.12 ★ — Choisir la méthode (sections 1.2 à 1.6)	13
Corrigés	13
Corrigé 1.1	13
Corrigé 1.2	13
Corrigé 1.3	14
Corrigé 1.4	14
Corrigé 1.5	14
Corrigé 1.6	14

Corrigé 1.7	14
Corrigé 1.8	14
Corrigé 1.9	15
Corrigé 1.10	15
Corrigé 1.11	15
Corrigé 1.12	15
Chapitre 2 : NLP et modèles de langage — exercices et applications	16
Préparation	16
Applications	17
Application 2.1 — Du texte au TF-IDF, sans bibliothèque	17
Application 2.2 — La référence et les phrases hors gabarit	18
Application 2.3 — L'attention de zéro	19
Application 2.4 — Entraîner un mini-transformer, et l'interroger	20
Application 2.5 — Le décodage de zéro	21
Application 2.6 — Le BPE à la main	22
Application 2.7 — Recherche sémantique avec MiniLM	23
Application 2.8 — Un RAG minimal	24
Exercices	26
Exercice 2.1 ★ — Mots vides et négation (section 2.1.1)	26
Exercice 2.2 ★ — Racinisation à la hache (section 2.1.1)	26
Exercice 2.3 ★★ — L'idf de scikit-learn (section 2.1.3)	26
Exercice 2.4 ★★ — Les mots inconnus (section 2.1.5)	26
Exercice 2.5 ★★ — Une attention à deux jetons (section 2.2.2)	26
Exercice 2.6 ★★ — La variance des scores et l'équivariance (sections 2.2.3 et 2.2.4)	26
Exercice 2.7 ★★ — Compter les paramètres (section 2.2.5)	27
Exercice 2.8 ★★ — Le prix de la longueur (section 2.2.6)	27
Exercice 2.9 ★ — La température (section 2.3.4)	27
Exercice 2.10 ★★ — Top-k contre top-p (section 2.3.4)	27
Exercice 2.11 ★★ — Le BPE à la main (section 2.3.2)	27
Exercice 2.12 ★★ — Évaluer une recherche (section 2.4.2)	27
Exercice 2.13 ★★ — Le vocabulaire qui explose (section 2.4.4)	27
Exercice 2.14 ★★ — Évaluer un RAG (section 2.5.3)	28
Corrigés	28
Corrigé 2.1	28
Corrigé 2.2	28
Corrigé 2.3	29
Corrigé 2.4	29
Corrigé 2.5	29
Corrigé 2.6	30
Corrigé 2.7	30
Corrigé 2.8	30
Corrigé 2.9	30
Corrigé 2.10	31
Corrigé 2.11	31
Corrigé 2.12	31
Corrigé 2.13	32
Corrigé 2.14	32
Chapitre 3 : Big data et calcul distribué — exercices et applications	33
Applications	33
Préparation commune	33

Application 3.1 — Un MapReduce complet sur les avis (section 3.1.4)	34
Application 3.2 — La loi d'Amdahl sur votre machine (section 3.1.5)	35
Application 3.3 — Lire des plans d'exécution et choisir sa jointure (sections 3.2.4 et 3.2.7)	36
Application 3.4 — Asymétrie des clés et salage (section 3.2.9)	37
Application 3.5 — Un journal de messages et un consommateur idempotent (sec- tion 3.3.3)	38
Application 3.6 — Fenêtres et filigrane (sections 3.3.4 et 3.3.5)	39
Exercices	40
Exercice 3.1 ★ — Quelle mémoire pour quelle table ? (section 3.1.1)	40
Exercice 3.2 ★ — Partitionner par hachage (section 3.1.3)	40
Exercice 3.3 ★★ — Un combineur à la main (section 3.1.4)	40
Exercice 3.4 ★ — Appliquer la loi d'Amdahl (section 3.1.5)	40
Exercice 3.5 ★★ — Retrouver la fraction parallélisable (section 3.1.5)	41
Exercice 3.6 ★★ — Combien de copies ? (section 3.1.7)	41
Exercice 3.7 ★ — Compter les étapes d'un plan (section 3.2.4)	41
Exercice 3.8 ★★ — Étroit ou large ? (section 3.2.4)	41
Exercice 3.9 ★★ — Combien de données voyagent ? (section 3.2.7)	41
Exercice 3.10 ★★ — Dimensionner les partitions (section 3.2.6)	41
Exercice 3.11 ★★ — L'asymétrie en chiffres (section 3.2.9)	41
Exercice 3.12 ★★★ — Une médiane ne se combine pas (sections 3.1.4 et 3.2)	42
Exercice 3.13 ★★ — Petits fichiers (section 3.2.10)	42
Exercice 3.14 ★ — Partitions et décalages d'un journal (section 3.3.3)	42
Exercice 3.15 ★★ — Garanties de livraison (section 3.3.3)	42
Exercice 3.16 ★★ — Fenêtres fixes et glissantes (section 3.3.4)	42
Exercice 3.17 ★★ — Le filigrane à la main (section 3.3.5)	42
Exercice 3.18 ★★★ — Fenêtres de session (section 3.3.4)	42
Corrigés	43
Corrigé 3.1	43
Corrigé 3.2	43
Corrigé 3.3	43
Corrigé 3.4	43
Corrigé 3.5	44
Corrigé 3.6	44
Corrigé 3.7	44
Corrigé 3.8	44
Corrigé 3.9	44
Corrigé 3.10	45
Corrigé 3.11	45
Corrigé 3.12	45
Corrigé 3.13	45
Corrigé 3.14	46
Corrigé 3.15	46
Corrigé 3.16	46
Corrigé 3.17	46
Corrigé 3.18	47
Nettoyage	47
Chapitre 4 : MLOps — exercices et applications	48
Préparation	48
Applications	49
Application 4.1 — Une batterie de tests pour un lot de données	49

Application 4.2 — Une API de scoring, de la requête à l'erreur	50
Application 4.3 — Un déploiement canari avec retour arrière automatique	51
Application 4.4 — Le journal des prédictions et les étiquettes tardives	52
Application 4.5 — Orchestrer un pipeline d'entraînement	54
Application 4.6 — Suivre une recherche d'hyperparamètres avec MLflow	55
Application 4.7 — Une porte de qualité, une « intégration continue » et un service protégé	57
Application 4.8 — Un mois de production : quelles variables ont dérivé, et est-ce grave ?	58
Application 4.9 — Une chaîne complète, de l'entraînement à la supervision	60
Exercices	61
Exercice 4.1 ★ — Que manque-t-il pour reproduire ? (section 4.1)	61
Exercice 4.2 ★★ — Une échelle de satisfaction qui change (section 4.1)	61
Exercice 4.3 ★★ — Exporter la régression logistique en ONNX (section 4.2)	61
Exercice 4.4 ★★ — Latence, débit, disponibilité (sections 4.2 et 4.3)	62
Exercice 4.5 ★★★ — Choisir la règle d'alerte au moindre coût (section 4.3)	62
Exercice 4.6 ★★ — Rendre une publication idempotente (section 4.4)	62
Exercice 4.7 ★★ — Un seul octet change tout... ou presque (section 4.5)	62
Exercice 4.8 ★★ — Tester les cas limites d'une API (section 4.6)	62
Exercice 4.9 ★★ — Limiter le débit par fenêtre de temps (section 4.6)	62
Exercice 4.10 ★★ — Un PSI à la main (section 4.7)	62
Exercice 4.11 ★★★ — Boucle de rétroaction et groupe témoin (section 4.7)	63
Exercice 4.12 ★★★ — Écrire une politique de ré-entraînement (section 4.7)	63
Corrigés	63
Corrigé 4.1	63
Corrigé 4.2	63
Corrigé 4.3	64
Corrigé 4.4	64
Corrigé 4.5	64
Corrigé 4.6	65
Corrigé 4.7	66
Corrigé 4.8	66
Corrigé 4.9	67
Corrigé 4.10	67
Corrigé 4.11	67
Corrigé 4.12	68
Chapitre 5 : Ingénierie des données — exercices et applications	69
Préparation	69
Applications	70
Application 5.1 — Un ETL pas à pas (sections 5.1.2 à 5.1.5)	70
Application 5.2 — La même transformation en SQL avec DuckDB (section 5.1.6)	71
Application 5.3 — Un chargement incrémental idempotent (section 5.1.7)	72
Application 5.4 — Règles de qualité, score et seuils (sections 5.2.1 à 5.2.5)	73
Application 5.5 — Un contrat de données exécutable (section 5.2.7)	75
Application 5.6 — Rapprocher les produits (sections 5.3.2 à 5.3.4)	76
Application 5.7 — Rapprocher les clients (sections 5.3.5 à 5.3.7)	77
Application 5.8 — Lignage et pseudonymisation (sections 5.4.2 à 5.4.5)	78
Application 5.9 — Collecter sur le site local (sections 5.5.3 à 5.5.6)	80
Exercices	81
Exercice 5.1 ★ — L'équation de conservation (section 5.1.4)	81
Exercice 5.2 ★★ — Idempotent ou pas ? (section 5.1.7)	81

Exercice 5.3 ★★★ — Le filigrane qui perd des lignes (section 5.1.7)	82
Exercice 5.4 ★ — Quelle dimension ? (section 5.2.2)	82
Exercice 5.5 ★★ — Un score sans se tromper de formule (section 5.2.4)	82
Exercice 5.6 ★★ — Une règle statistique et son seuil (section 5.2.6)	82
Exercice 5.7 ★ — Levenshtein à la main (section 5.3.3)	82
Exercice 5.8 ★★ — Précision, rappel, coûts (section 5.3.6)	83
Exercice 5.9 ★★★ — Choisir une clé de blocage (section 5.3.5)	83
Exercice 5.10 ★ — Le k-anonymat à la main (section 5.4.5)	83
Exercice 5.11 ★★ — Un sel public protège-t-il ? (section 5.4.5)	83
Exercice 5.12 ★★ — Un extracteur qui survit au changement de page (sections 5.5.4 à 5.5.6)	83
Corrigés	84
Corrigé 5.1	84
Corrigé 5.2	84
Corrigé 5.3	84
Corrigé 5.4	85
Corrigé 5.5	85
Corrigé 5.6	85
Corrigé 5.7	86
Corrigé 5.8	86
Corrigé 5.9	86
Corrigé 5.10	87
Corrigé 5.11	88
Corrigé 5.12	88
Chapitre 6 : Plateformes cloud — exercices et applications	89
Préparation	89
Applications	90
Application 6.1 — Capex ou opex ? (section 6.1.1)	90
Application 6.2 — Dimensionner, et ce que l'élasticité change (section 6.1.2)	90
Application 6.3 — Disponibilité d'une architecture (section 6.1.4)	91
Application 6.4 — Simulateur d'autoscaling (section 6.2.1)	92
Application 6.5 — Quel palier de stockage ? (section 6.2.2)	92
Application 6.6 — Auditer une politique d'accès (section 6.3.4)	93
Application 6.7 — Comparateur de modes d'achat (section 6.3.1)	94
Application 6.8 — Une grille de décision, et sa sensibilité (section 6.3.6)	94
Exercices	95
Exercice 6.1 ★ — Le seuil à la main (section 6.1.1)	95
Exercice 6.2 ★ — Pic et moyenne (section 6.1.2)	95
Exercice 6.3 ★★ — Latence minimale (section 6.1.4)	95
Exercice 6.4 ★★ — Les maillons d'une chaîne (section 6.1.4)	96
Exercice 6.5 ★★ — Qui est responsable ? (section 6.1.5)	96
Exercice 6.6 ★ — Quel palier ? (section 6.2.2)	96
Exercice 6.7 ★★ — L'autoscaling à la main (section 6.2.1)	96
Exercice 6.8 ★★ — Quelle base de données ? (section 6.2.3)	96
Exercice 6.9 ★★ — Combiner les modes d'achat (section 6.3.1)	96
Exercice 6.10 ★★ — Fonction ou machine ? (section 6.3.1)	97
Exercice 6.11 ★★ — La facture de sortie (section 6.3.3)	97
Exercice 6.12 ★★★ — Le dossier de décision (section 6.3.6)	97
Corrigés	97
Corrigé 6.1	97
Corrigé 6.2	97

Corrigé 6.3	98
Corrigé 6.4	98
Corrigé 6.5	98
Corrigé 6.6	99
Corrigé 6.7	99
Corrigé 6.8	99
Corrigé 6.9	99
Corrigé 6.10	100
Corrigé 6.11	100
Corrigé 6.12	100
Chapitre 7 : ☉ Applications de démonstration — exercices et applications	101
Préparation	101
Applications	102
Application 7.1 — Construire l'application de résiliation pas à pas (sections 7.1 et 7.2)	102
Application 7.2 — L'incertitude et l'explication (sections 7.1.4 et 7.1.5)	104
Application 7.3 — Écrire la suite de tests (section 7.2.5)	106
Application 7.4 — Mesurer l'effet du cache sur un pipeline (section 7.2.4)	107
Application 7.5 — Étendre le mini système réactif (section 7.3.2)	109
Application 7.6 — Préparer la mise en service (sections 7.4.1 à 7.4.4 et 7.4.7)	110
Application 7.7 — Application ou API ? (section 7.4.9)	112
Exercices	113
Exercice 7.1 ★ — Démonstration, prototype ou produit ? (section 7.1.1)	113
Exercice 7.2 ★★ — Une combinaison absente (section 7.1.3)	114
Exercice 7.3 ★★ — Lire l'intervalle (section 7.1.4)	114
Exercice 7.4 ★ — Combien d'exécutions ? (sections 7.2.1 et 7.2.2)	114
Exercice 7.5 ★★ — cache_data ou cache_resource ? (section 7.2.4)	114
Exercice 7.6 ★★ — Écrire un test (section 7.2.5)	114
Exercice 7.7 ★★★ — Le compteur qui reste à 1 (section 7.2.6)	114
Exercice 7.8 ★ — Streamlit, Shiny ou autre ? (section 7.3.5)	114
Exercice 7.9 ★★ — Qui est recalculé ? (sections 7.3.1 et 7.3.2)	115
Exercice 7.10 ★★★ — Lire sans dépendre (section 7.3.4)	115
Exercice 7.11 ★ — Que mettre dans le journal ? (section 7.4.4)	115
Exercice 7.12 ★★ — Choisir la couleur du texte (section 7.4.6)	115
Exercice 7.13 ★★ — Repérer les licences à examiner (section 7.4.7)	115
Exercice 7.14 ★★★ — Application ou API : le dossier (section 7.4.9)	115
Corrigés	115
Corrigé 7.1	115
Corrigé 7.2	116
Corrigé 7.3	116
Corrigé 7.4	116
Corrigé 7.5	117
Corrigé 7.6	117
Corrigé 7.7	118
Corrigé 7.8	118
Corrigé 7.9	119
Corrigé 7.10	119
Corrigé 7.11	119
Corrigé 7.12	120
Corrigé 7.13	120
Corrigé 7.14	121

Projet du volume et auto-évaluation — exercices et applications	122
Projet du volume	122
P.1 Le cahier des charges	122
P.2 Étape 1 : structurer et configurer	123
P.3 Étape 2 : un pipeline testé	123
P.4 Étape 3 : suivre les expériences	125
P.5 Étape 4 : servir par une API qui valide ses entrées	126
P.6 Étape 5 : journaliser et superviser	127
P.7 Étape 6 : la dérive abîme-t-elle les performances ?	128
P.8 Étape 7 : automatiser	130
P.9 Étape 8 : le rapport de mise en production	131
P.10 Les limites de l'étude	131
Auto-évaluation	132
Deep learning (chapitre 1)	132
NLP et modèles de langage (chapitre 2)	132
Big data et calcul distribué (chapitre 3)	132
MLOps (chapitre 4)	133
Ingénierie des données (chapitre 5)	133
Plateformes cloud (chapitre 6)	133
Applications de démonstration (chapitre 7)	133
Corrigés des questions	134
Votre grille d'auto-évaluation	137

Mode d'emploi

« On comprend en lisant, on retient en construisant. »

Ce cahier est le **compagnon du livre** du volume IV (*Sujets avancés et modernes*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (déployer un modèle avec un pipeline et une supervision) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.
2. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre : on peut chercher sans voir la réponse.
3. **Tapez le code vous-même** plutôt que de le copier : modifiez-le, cassez-le, lisez les messages d'erreur. Dans ce volume, on apprend surtout en changeant un réglage (le pas d'apprentissage, le nombre de partitions, une unité en amont) et en observant ce que devient le résultat.
4. **Faites les applications dans l'ordre** : chacune raconte une petite étude, avec une question, des étapes, du code et une lecture des résultats.
5. **Mesurez avant de conclure.** Un réseau « qui apprend », un service « qui répond » ou un traitement « plus rapide » sont des affirmations qui se vérifient : chaque application demande un chiffre, pas une impression.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une formule ou d'une idée vue dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, une démonstration ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données, modèles et environnement

Les données sont celles du livre, dans le dossier `donnees/` ; les modèles pré-entraînés, dans `modeles/`. Les jeux de la boutique sont **simulés** avec des graines fixes (script `build/donnees4.py`) : vos résultats seront identiques à ceux du livre.

Fichier	Contenu
<code>mnist_sous_ensemble.npz</code>	jeu réel : 12 000 images de chiffres manuscrits (10 000 pour apprendre, 2 000 pour tester)
<code>ventes_quotidiennes.csv</code>	ventes journalières de la boutique : date, ventes, promo, jour_semaine, mois
<code>avis_clients.csv</code>	avis en français : id_avis, categorie, canal, note, sujet, texte
<code>clients_ml.csv</code>	les 12 000 clients du volume III, avec la cible <code>churn_90j</code>
<code>sources/*.csv</code>	exports « bruts » volontairement sales : commandes, CRM, catalogues, et les fichiers <code>verite_*</code> qui donnent la bonne réponse
<code>gros_volume</code> (fonction de <code>donnees4.py</code>)	des millions de transactions écrites à la demande au format Parquet, dans un dossier ignoré par Git

Les colonnes de `clients_ml.csv` ont le même statut qu'au volume III (identifiant, 19 entrées, cible, et trois colonnes à **ne pas utiliser en entrée** : `depense_6m`, `segment_vrai`, `commandes_apres_cible`). Le dictionnaire des autres jeux figure dans la section « Les données, les modèles et l'environnement du volume » du livre. Chaque chapitre du cahier est **autonome** : il commence par recharger ses données et refaire ses imports.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/` et `modeles/`), car les chemins sont relatifs (`donnees/clients_ml.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, ou dans un terminal interactif.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Le temps de calcul. Les applications sont dimensionnées pour tourner en quelques dizaines de secondes sur un ordinateur ordinaire, **sans carte graphique**. Un réseau y est donc petit, un corpus restreint, un « grand volume » de quelques millions de lignes. Si l'une d'elles est trop lente chez vous, réduisez le nombre d'époques ou l'échantillon, sans changer le raisonnement. Le lancement de Spark prend une quinzaine de secondes : ce n'est pas un blocage.

Hors ligne. Après l'installation unique décrite dans le livre, plus aucun téléchargement n'est nécessaire. Si un programme tente d'aller sur le réseau, c'est qu'une variable d'environnement manque (voir la dernière vérification ci-dessous).

Vérifier son installation

Avant de commencer, ces trois courts blocs vérifient que les bibliothèques sont installées, que les modèles pré-entraînés se chargent sans réseau, que Java et Tesseract sont trouvés, et qu'un petit réseau à graine fixe donne le résultat attendu. Les numéros de version peuvent différer des nôtres (voir le livre, section « L'environnement »), à condition que tout s'exécute.

1. Les bibliothèques.

```
import sys
from importlib import import_module
from importlib.metadata import version
print("python", sys.version.split()[0])
for p in ["torch", "transformers", "pyspark", "duckdb", "fastapi", "mlflow", "streamlit"]:
    import_module(p)
    print(f"{p:13s}", version(p))
```

```
python 3.13.3
torch      2.14.1+cpu
transformers 5.18.0
pyspark    4.2.0
duckdb     1.5.6
fastapi    0.142.2
mlflow     3.16.1
streamlit  1.65.0
```

2. Les modèles, Java et Tesseract.

```
import os, subprocess
os.environ.setdefault("HF_HOME", "modeles/hf")
os.environ.setdefault("HF_HUB_OFFLINE", "1")
from sentence_transformers import SentenceTransformer
st = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2", device="cpu")
v = st.encode(["La livraison a été rapide.", "Colis reçu très vite."], normalize_embeddings=True)
print("plongement :", v.shape, "; similarité", round(float(v[0] @ v[1]), 3))
print("java :", subprocess.run(["java", "-version"], capture_output=True,
    ↪ text=True).stderr.splitlines()[0])
import pytesseract
print("tesseract :", pytesseract.get_tesseract_version(), "; français :", "fra" in
    ↪ pytesseract.get_languages())
```

```
plongement : (2, 384) ; similarité 0.728
java : openjdk version "21.0.9" 2025-10-21
tesseract : 5.5.0 ; français : True
```

3. Un petit réseau, reproductible.

```
import torch
torch.manual_seed(0)
x = torch.linspace(-1, 1, 64).unsqueeze(1)
y = 3 * x + 0.5 # la droite à retrouver
net = torch.nn.Linear(1, 1)
opt = torch.optim.SGD(net.parameters(), lr=0.5)
for _ in range(200):
    opt.zero_grad()
    perte = torch.nn.functional.mse_loss(net(x), y)
    perte.backward()
    opt.step()
print("pente", round(net.weight.item(), 3), "ordonnée", round(net.bias.item(), 3), "perte",
      ↪ f"{perte.item():.1e}")
```

```
pente 3.0 ordonnée 0.5 perte 2.4e-14
```

Vous devez lire une version pour chaque bibliothèque, une similarité positive entre les deux phrases, une version de Java et de Tesseract avec français : True, puis une pente proche de 3 et une ordonnée proche de 0,5. Si le deuxième bloc échoue en cherchant un modèle sur internet, c'est que `modeles/` est absent : lancez `bash setup-env.sh` (voir le livre).

Plan du cahier

Chapitre	Contenu
1. Deep learning	rétropropagation à la main, réseau à entraîner, convolutions sur les chiffres manuscrits, prévision par réseau récurrent ; ☉ frameworks, apprentissage par transfert, deep learning tabulaire
2. NLP et modèles de langage	du texte aux nombres, plongements, attention, modèle de langage en pratique et son évaluation ; ☉ text mining, sentiments, recherche par plongements
3. Big data et calcul distribué	partitions, mini-MapReduce, requêtes Spark sur un grand volume ; ☉ flux d'événements
4. MLOps	pipelines reproductibles, service d'un modèle, supervision et dérive ; ☉ orchestration, suivi d'expériences, intégration continue, API
5. Ingénierie des données	ETL, contrôles de qualité, réconciliation d'exports sales ; ☉ gouvernance, collecte par API
6 et 7 (facultatifs)	plateformes cloud (coûts, risques) ; applications de démonstration
Projet et auto-évaluation	déployer un modèle avec un pipeline et une supervision, puis des questions pour vérifier ses acquis

Chapitre 1 : Deep learning — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre (réseaux de neurones, convolutifs, récurrents, frameworks, transfert et OCR, tabulaire). Les **applications** reprennent, avec leur code, des expériences dont le livre ne montre que les résultats ; les **exercices** se font à la main ou avec quelques lignes ; les **corrigés** des exercices sont à la fin.

Préparation

Toutes les applications utilisent le sous-ensemble de MNIST et les fichiers du dossier `donnees/`, ainsi que quelques fonctions utilitaires fournies avec le livre (`build/outils_ch01.py`) : `graine`, `charger_mnist`, `nb_parametres`, `exactitude` et `entraîner` (la boucle d'entraînement de la section 1.4.3 du livre, avec arrêt précoce optionnel).

```
import sys, warnings
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, torch, torch.nn as nn
sys.path.insert(0, "build")
from outils_ch01 import graine, charger_mnist, nb_parametres, exactitude, entraîner

graine(0)
xi, yi, xte, yte = charger_mnist()           # images (N, 1, 28, 28), valeurs dans [0, 1]
xt, yt, xv, yv = xi[:4000], yi[:4000], xi[8000:9000], yi[8000:9000]
xt_p, xv_p, xte_p = xt.flatten(1), xv.flatten(1), xte.flatten(1)  # versions aplaties (N, 784) pour
↳ les réseaux denses
print(xt.shape, xv.shape, xte.shape)

torch.Size([4000, 1, 28, 28]) torch.Size([1000, 1, 28, 28]) torch.Size([2000, 1, 28, 28])
```

Applications

Application 1.1 — La rétropropagation sur un nouveau réseau (section 1.1.5)

Énoncé. Écrivez en numpy une fonction qui calcule, pour un réseau « deux entrées, n neurones cachés (tanh), une sortie (sigmoïde) » avec l'entropie croisée binaire, la perte **et tous les gradients**. Vérifiez-la par **différences finies** sur chaque paramètre, puis faites un pas de descente de gradient et constatez que la perte diminue. Entrée $x = (0,5, -1)$, étiquette $y = 0$, trois neurones cachés, poids tirés au hasard avec la graine 0.

```
def avant_arriere(theta, x, y):
    W1, b1, W2, b2 = theta[:6].reshape(3, 2), theta[6:9], theta[9:12], theta[12]
    a1 = np.tanh(W1 @ x + b1) # couche cachée
    p = 1 / (1 + np.exp(-(W2 @ a1 + b2))) # sortie : probabilité de la classe 1
    perte = -(y * np.log(p) + (1 - y) * np.log(1 - p))
    d2 = p - y # gradient en sortie : prédiction moins vérité
    d1 = (W2 * d2) * (1 - a1 ** 2) # retour vers la couche cachée (dérivée de tanh)
    return perte, np.concatenate([np.outer(d1, x).ravel(), d1, d2 * a1, [d2]]) # gradient de tous
    ↪ les paramètres

theta = np.random.default_rng(0).normal(0, 0.5, 13) # 6 poids cachés, 3 biais, 3 poids de sortie, 1
    ↪ biais
x, y = np.array([0.5, -1.0]), 0.0
perte, grad = avant_arriere(theta, x, y)

ecart = 0.0 # différences finies, un paramètre à la fois
for k in range(13):
    h = np.zeros(13); h[k] = 1e-6
    numerique = (avant_arriere(theta + h, x, y)[0] - avant_arriere(theta - h, x, y)[0]) / 2e-6
    ecart = max(ecart, abs(numerique - grad[k]))
print("perte :", round(perte, 4), "| écart maximal avec les différences finies :", f"{ecart:.1e}")

perte : 0.1618 | écart maximal avec les différences finies : 4.1e-11
```

Corrigé. Les gradients de la fonction concordent avec les différences finies à 4×10^{-11} près : la rétropropagation est correcte. Le point à retenir est la **structure** : le signal d'erreur $d2 = p - y$ remonte à la couche cachée en étant multiplié par les poids de sortie et par la dérivée de l'activation ($1 - a^2$ pour la tanh). Un pas de descente de gradient (pas de 0,5) fait passer la perte de 0,1618 à 0,1363.

Application 1.2 — Optimiseurs et pas d'apprentissage (section 1.1.7)

Énoncé. Entraînez le même réseau 784-64-10 pendant 6 époques avec le SGD simple, le SGD avec moment et Adam, pour quatre valeurs du pas d'apprentissage (10^{-3} , 10^{-2} , 10^{-1} et $3 \cdot 10^{-1}$ pour le SGD ; 10^{-4} , 10^{-3} , 10^{-2} et 10^{-1} pour Adam). Quel optimiseur est le plus sensible au pas ?

```
def essai(fabrique_opt):
    graine(0)
    m = nn.Sequential(nn.Linear(784, 64), nn.ReLU(), nn.Linear(64, 10))
    return entrainer(m, xt_p, yt, xv_p, yv, epoques=6, optimiseur=fabrique_opt)["acc_val"][-1]

pas_sgd, pas_adam = [1e-3, 1e-2, 1e-1, 3e-1], [1e-4, 1e-3, 1e-2, 1e-1]
tab = pd.DataFrame({
    "pas SGD": [f"{l:g}" for l in pas_sgd], "SGD": [essai(lambda p, l=l: torch.optim.SGD(p, lr=l)) for
    ↪ l in pas_sgd],
    "SGD + moment": [essai(lambda p, l=l: torch.optim.SGD(p, lr=l, momentum=0.9)) for l in pas_sgd],
    "pas Adam": [f"{l:g}" for l in pas_adam], "Adam": [essai(lambda p, l=l: torch.optim.Adam(p, lr=l))
    ↪ for l in pas_adam]})
print(tab.round(3).to_string(index=False))
```

pas SGD	SGD	SGD + moment	pas Adam	Adam
0.001	0.171	0.670	0.0001	0.768
0.01	0.690	0.888	0.001	0.907

0.1	0.892	0.908	0.01	0.917
0.3	0.916	0.920	0.1	0.838

Corrigé. L'exactitude de validation varie selon le pas de 17,1 % à 91,6 % pour le SGD simple, de 67,0 % à 92,0 % pour le SGD avec moment, et de 76,8 % à 91,7 % pour Adam. Le **SGD simple** est le plus sensible : avec un pas trop petit (10^{-3}), il n'a presque rien appris en 6 époques, et il n'atteint son meilleur niveau qu'avec le plus grand pas essayé. Le **moment** accélère nettement l'apprentissage aux petits pas. **Adam** donne de bons résultats sur une plage plus large (de 10^{-3} à 10^{-2} ici), mais il n'est **pas insensible** : un pas de 10^{-4} est trop lent pour 6 époques, et un pas de 10^{-1} dégrade la validation. Retenez : Adam rend le réglage du pas **moins délicat**, pas inutile.

Application 1.3 — Arrêt précoce et pénalité sur les poids (section 1.1.8)

Énoncé. Sur seulement 1 000 images d'entraînement, entraînez un réseau 784-256-128-10 pendant 40 époques, avec (a) rien, (b) une pénalité de poids (weight_decay), (c) un arrêt précoce (patience 5). Comparez l'exactitude d'entraînement, de validation et de test.

```
def modele_grand():
    graine(0); return nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 128), nn.ReLU(),
    ↪ nn.Linear(128, 10))

resultats = {}
for nom, args in {"rien": {}, "pénalité de poids": {"wd": 1e-3}, "arrêt précoce": {"patience":
    ↪ 5}}.items():
    m = modele_grand(); h = entrainer(m, xt_p[:1000], yt[:1000], xv_p, yv, epoques=40, lr=1e-3, **args)
    resultats[nom] = (len(h["perte"]), h["acc_train"][-1], exactitude(m, xv_p, yv), exactitude(m,
    ↪ xte_p, yte))
print(pd.DataFrame(resultats, index=["époques", "train", "val", "test"]).T.round(3).to_string())
```

	époques	train	val	test
rien	40.0	1.000	0.885	0.890
pénalité de poids	40.0	1.000	0.874	0.883
arrêt précoce	14.0	0.977	0.889	0.876

Corrigé. Sans rien, le réseau mémorise les 1 000 images (100,0 % en entraînement) et obtient 88,9 % au test. La pénalité de poids donne 88,3 % ; l'arrêt précoce, qui s'arrête après 14 époques, 87,6 %. Les écarts entre ces trois lignes sont de l'ordre du **bruit d'une graine** (voir la section 1.1.8 : plusieurs graines sont nécessaires pour conclure) ; l'enseignement robuste est que l'écart entre entraînement et test **montre** le surapprentissage, et que l'arrêt précoce économise du calcul.

Application 1.4 — Une convolution écrite à la main (section 1.2.2)

Énoncé. Écrivez avec deux boucles une convolution 3×3 sans rembourrage ; vérifiez-la contre `torch.nn.functional.conv2d`, puis appliquez à un vrai chiffre de MNIST le filtre de détection des contours verticaux de la section 1.2.2 et celui des contours horizontaux (le transposé).

```
import torch.nn.functional as Fn
def convolution(image, K):
    k = K.shape[0]; n = image.shape[0] - k + 1
    sortie = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            sortie[i, j] = (image[i:i + k, j:j + k] * K).sum()
    return sortie

V = np.array([[-1, 0, 1]] * 3, dtype=float); H = V.T # contours verticaux, horizontaux
chiffre = xte[0, 0].numpy()
ref = Fn.conv2d(xte[:1], torch.tensor(V, dtype=torch.float32).reshape(1, 1, 3, 3))[0, 0].numpy()
```

```
print("identique à PyTorch :", np.allclose(convolution(chiffre, V), ref, atol=1e-5), "| taille de la  
↪ sortie :", convolution(chiffre, V).shape)
```

```
identique à PyTorch : True | taille de la sortie : (26, 26)
```

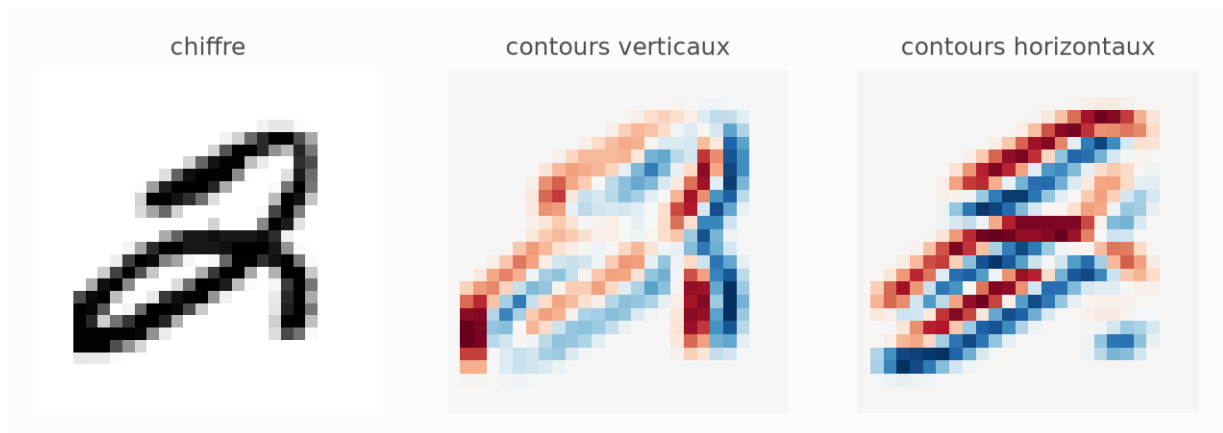


Figure 1: Un chiffre de test, puis les cartes obtenues avec le filtre vertical et le filtre horizontal : le premier met en valeur les traits verticaux, le second les traits horizontaux.

Corrigé. La convolution écrite à la main est identique à celle de PyTorch (vérification : True). La sortie fait 26×26 : $28 - 3 + 1$. Les couleurs de la figure montrent que chaque filtre répond aux transitions d'une orientation donnée (positives en rouge, négatives en bleu) : c'est ce type de détecteur que les 8 filtres du livre apprennent seuls.

Application 1.5 — Mesurer l'effet de l'augmentation de données (section 1.2.6)

Énoncé. Entraînez un petit réseau convolutif sur 2 000 images, avec et sans augmentation (chaque image est dupliquée avec un décalage aléatoire de -3 à $+3$ pixels dans chaque direction). Évaluez sur le test, puis sur le test décalé de 3 pixels horizontalement.

```
def petit_cnn():
    graine(0); return nn.Sequential(nn.Conv2d(1, 8, 3), nn.ReLU(), nn.MaxPool2d(2), nn.Conv2d(8, 16,
    ↪ 3), nn.ReLU(), nn.MaxPool2d(2), nn.Flatten(), nn.Linear(400, 10))

def decale(x, g):
    gen = torch.Generator().manual_seed(g)
    return torch.stack([torch.roll(im, shifts=tuple(torch.randint(-3, 4, (2,)),
    ↪ generator=gen).tolist()), dims=(1, 2)) for im in x])

x2, y2 = xt[:2000], yt[:2000]
x2a, y2a = torch.cat([x2, decale(x2, 1)]), torch.cat([y2, y2])
lignes = {}
for nom, (x_, y_) in {"sans augmentation": (x2, y2), "avec augmentation": (x2a, y2a)}.items():
    m = petit_cnn(); entrainer(m, x_, y_, xv, yv, epoques=8, lr=3e-3)
    lignes[nom] = (exactitude(m, xte, yte), exactitude(m, torch.roll(xte, 3, dims=3), yte))
print(pd.DataFrame(lignes, index=["test", "test décalé de 3 px"]).T.round(3).to_string())
```

	test	test décalé de 3 px
sans augmentation	0.902	0.532
avec augmentation	0.944	0.874

Corrigé. Sans augmentation : 90,2 % au test et 53,2 % sur le test décalé. Avec augmentation : 94,3 % et 87,4 %. Le gain est **spectaculaire sur les images décalées** (de 53,2 % à 87,4 %) : le réseau a vu pendant son entraînement les situations qu'il rencontre au test. Il se voit aussi, plus modestement,

sur le test non décalé, parce que l'augmentation double aussi la quantité d'exemples d'un jeu de seulement 2 000 images (une graine seulement ici : ne retenez que l'ordre de grandeur).

Application 1.6 — LSTM ou GRU ? Fenêtre courte ou longue ? (section 1.3.4)

Énoncé. Reprenez la prévision des ventes quotidiennes du livre (entraînement 2023-2024, test 2025). Comparez un LSTM et un GRU de 16 unités, avec une fenêtre de 7 puis de 28 jours, sur deux graines. Rapportez le nombre de paramètres et l'erreur absolue moyenne (MAE) en euros.

```
vd = pd.read_csv("donnees/ventes_quotidiennes.csv", parse_dates=["date"]); vd["ly"] =
↳ np.log(vd.ventes)
tr_i, te_i = vd[vd.date < "2025-01-01"].index, vd[vd.date >= "2025-01-01"].index
mu, sg = vd.ly[tr_i].mean(), vd.ly[tr_i].std(); z = ((vd.ly - mu) / sg).values
promo = vd.promo.values.astype(float); doy = vd.date.dt.dayofyear.values; dow =
↳ np.eye(7)[vd.jour_semaine.values]

def jeux(W, idx):
    X = np.stack([np.column_stack([z[i - W:i], promo[i - W:i]]) for i in idx])
    C = np.column_stack([promo[idx], dow[idx], np.sin(2 * np.pi * doy[idx] / 365), np.cos(2 * np.pi *
↳ doy[idx] / 365)])
    return [torch.tensor(a, dtype=torch.float32) for a in (X, C, z[idx])]

class Prevision(nn.Module):
    def __init__(self, cellule):
        super().__init__(); self.rnn = cellule(2, 16, batch_first=True); self.tete =
↳ nn.Sequential(nn.Linear(26, 16), nn.ReLU(), nn.Linear(16, 1))
    def forward(self, f, c): return self.tete(torch.cat([self.rnn(f)[0][:, -1], c], 1)).squeeze(1)

def evaluer(cellule, W, g):
    Xa, Ca, ya = jeux(W, [i for i in tr_i if i >= 28]); Xb, Cb, _ = jeux(W, list(te_i))
    graine(g); m = Prevision(cellule); opt = torch.optim.Adam(m.parameters(), 2e-3)
    for e in range(25):
        perm = torch.randperm(len(Xa))
        for i in range(0, len(Xa), 64):
            idx = perm[i:i + 64]; opt.zero_grad(); nn.functional.mse_loss(m(Xa[idx], Ca[idx]),
↳ ya[idx]).backward(); opt.step()
        with torch.no_grad(): p = np.exp(m.eval()(Xb, Cb).numpy() * sg + mu)
        return nb_parametres(m), float(np.mean(np.abs(vd.ventes.values[te_i] - p)))

res = {(n, W): [evaluer(c, W, g) for g in (0, 1)] for n, c in [("LSTM", nn.LSTM), ("GRU", nn.GRU)] for
↳ W in (7, 28)}
print(pd.DataFrame({f"{n}", "fenêtre {W}": [v[0][0], round(np.mean([a[1] for a in v]), 2)] for (n, W), v
↳ in res.items()}), index=["paramètres", "MAE (€)"].T.to_string())
```

	paramètres	MAE (€)
LSTM, fenêtre 7	1729.0	21.36
LSTM, fenêtre 28	1729.0	21.40
GRU, fenêtre 7	1409.0	21.03
GRU, fenêtre 28	1409.0	20.84

Corrigé. Le GRU a 1 409 paramètres contre 1 729 pour le LSTM (trois blocs de portes au lieu de quatre). Les MAE moyennes (2 graines) sont de 21,36 € (LSTM, fenêtre de 7 jours), 21,40 € (LSTM, 28 jours), 21,03 € (GRU, 7 jours) et 20,84 € (GRU, 28 jours). Avec deux graines seulement, de petits écarts ne sont pas interprétables ; ce qui compte est l'**ordre de grandeur** (ces MAE, obtenues avec 25 époques, sont un peu plus élevées que celles du livre, obtenues avec 40) et la règle : à performance voisine, on prend le modèle le plus simple.

Application 1.7 — Lire des factures dégradées : OCR et inclinaison (section 1.5)

Énoncé. Générez huit factures synthétiques avec Pillow, inclinez chaque image d'un angle de 0°, 5°, 10° ou 15° et mesurez le taux d'erreur par caractère (CER) de Tesseract. Redresser l'image de l'angle connu (en pratique, on l'estime) améliore-t-il la lecture ?

```
from PIL import Image, ImageDraw, ImageFont
import pytesseract
from rapidfuzz.distance import Levenshtein
police = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 22)
rng = np.random.default_rng(1)

def facture(i):
    lignes = [f"FACTURE N° 2025-{i:04d}", f"Client : Ville {chr(65 + i % 8)}", f"Article A{i % 9 + 1}"]
    ↪ x{i % 3 + 1} {rng.uniform(5, 120):.2f} €", f"TOTAL TTC : {rng.uniform(20, 400):.2f} €"]
    im = Image.new("L", (620, 170), 255); dr = ImageDraw.Draw(im)
    for k, l in enumerate(lignes): dr.text((15, 10 + 36 * k), l, font=police, fill=0)
    return im, "\n".join(lignes)

def cer(lu, vrai): return Levenshtein.distance(lu, vrai) / len(vrai)
data = [facture(i) for i in range(1, 9)]
tab = {}
for ang in (0, 5, 10, 15):
    brut = [cer(pytesseract.image_to_string(im.rotate(ang, fillcolor=255), lang="fra").strip(), t)
    ↪ for im, t in data]
    redr = [cer(pytesseract.image_to_string(im.rotate(ang, fillcolor=255).rotate(-ang,
    ↪ fillcolor=255), lang="fra").strip(), t) for im, t in data]
    tab[ang] = (np.mean(brut), np.mean(redr))
print(pd.DataFrame(tab, index=["CER incliné", "CER redressé"]).T.round(3).to_string())
```

	CER incliné	CER redressé
0	0.053	0.053
5	0.090	0.043
10	0.864	0.105
15	1.000	0.210

Corrigé. À 0° le CER est de 0,053. Il augmente avec l'inclinaison : 0,090 à 5°, puis **0,864 à 10°** (la lecture est devenue presque entièrement fausse) et 1,000 à 15° (Tesseract ne reconnaît plus rien : un CER de 1 signifie que le texte lu est entièrement faux ou absent). Redresser avant la lecture ramène le CER à 0,043 (5°), 0,105 (10°) et 0,210 (15°). Le redressement n'est pas parfait : l'image inclinée a perdu ses coins (la rotation n'agrandissait pas le cadre) et l'interpolation adoucit les caractères, d'où un CER encore supérieur à celui de l'image d'origine. En pratique, l'angle est **estimé** (par la direction des lignes de texte) ; cette étape de prétraitement est souvent plus rentable que de changer de moteur OCR.

Application 1.8 — Réseau ou boosting : l'effet de la taille des données (section 1.6)

Énoncé. Sur les clients du volume III, comparez l'AUC d'un réseau à plongements et d'un Hist-GradientBoosting pour des tailles d'entraînement de 500, 1 500, 4 500 et 9 000 clients (test fixe de 3 000 clients, trois graines pour le réseau). Le réseau rattrape-t-il le boosting quand les données augmentent ?

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score
from sklearn.preprocessing import StandardScaler
cl = pd.read_csv("donnees/clients_ml.csv"); yc = cl.churn_90j.values
cats = ["ville", "canal_acquisition", "appareil", "categorie_preferee"]
nums = [k for k in cl.columns if k not in cats + ["id_client", "churn_90j", "depense_6m",
    ↪ "segment_vrai", "commandes_apres_cible"]]
codes = np.column_stack([pd.Categorical(cl[k].fillna("manquant")).codes for k in
    ↪ cats]).astype(np.int64)
```

```

nbmod = [int(codes[:, i].max()) + 1 for i in range(len(cats))]
N = cl[nums]; manq = N.isna().astype(float).loc[:, N.isna().any()].values
X_hgb = pd.concat([N, pd.DataFrame(codes, columns=cats)], axis=1)
test = np.arange(9000, 12000); score = {}

class Reseau(nn.Module):
    def __init__(self, d):
        super().__init__(); self.e = nn.ModuleList([nn.Embedding(n, min(8, n)) for n in nbmod])
        self.f = nn.Sequential(nn.Linear(sum(min(8, n) for n in nbmod) + d, 64), nn.ReLU(),
                                nn.Linear(64, 1))
    def forward(self, xn, xc): return self.f(torch.cat([e(xc[:, i]) for i, e in enumerate(self.e)] +
                                                         [xn, 1])).squeeze(1)

def auc_reseau(n, g):
    tr = np.arange(n); A = N.fillna(N.iloc[tr].median()); sc = StandardScaler().fit(A.iloc[tr])
    Z = torch.tensor(np.column_stack([sc.transform(A), manq]), dtype=torch.float32); C =
    torch.tensor(codes)
    graine(g); m = Reseau(Z.shape[1]); opt = torch.optim.Adam(m.parameters(), 2e-3); yt_ =
    torch.tensor(yc, dtype=torch.float32)
    for e in range(25):
        perm = torch.randperm(n)
        for i in range(0, n, 128):
            idx = perm[i:i + 128]; opt.zero_grad();
            nn.functional.binary_cross_entropy_with_logits(m(Z[idx], C[idx]), yt_[idx]).backward();
            opt.step()
        with torch.no_grad(): return roc_auc_score(yc[test], m.eval()(Z[test], C[test])).numpy()

for n in (500, 1500, 4500, 9000):
    hg = HistGradientBoostingClassifier(random_state=0,
    categorical_features=[X_hgb.columns.get_loc(c) for c in cats]).fit(X_hgb.iloc[:n], yc[:n])
    score[n] = (roc_auc_score(yc[test], hg.predict_proba(X_hgb.iloc[test])[:, 1]),
    np.mean([auc_reseau(n, g) for g in range(3)]))
print(pd.DataFrame(score, index=["boosting", "réseau"]).T.round(4).to_string())

```

	boosting	réseau
500	0.8439	0.8277
1500	0.8713	0.8505
4500	0.8834	0.8680
9000	0.8951	0.8743

Corrigé.

Clients d'entraînement	AUC du boosting	AUC du réseau	Écart
500	0,844	0,828	0,016
1 500	0,871	0,851	0,021
4 500	0,883	0,868	0,015
9 000	0,895	0,874	0,021

Les deux méthodes progressent avec les données, mais **l'écart ne se réduit pas** : le réseau ne rattrape pas le boosting dans cette plage de tailles. Une courbe qui se resserrerait avec le volume serait la signature habituelle du deep learning, qui profite davantage de l'abondance de données que les arbres ; ici, à 9 000 clients, nous n'y sommes pas. Un test fixe de 3 000 clients et trois graines donnent une incertitude de quelques millièmes d'AUC : un écart constant d'environ 0,02 est donc au-delà du bruit, mais les petites variations de cet écart d'une taille à l'autre ne se lisent pas.

Exercices

Exercice 1.1 ★ — Le gradient du softmax (section 1.1.4)

Un réseau produit les scores (« logits ») $\mathbf{z} = (2, 1, 0)$ pour trois classes ; la vraie classe est la première. (a) Calculez les probabilités softmax $\mathbf{p}$. (b) Calculez la perte d'entropie croisée $L = -\ln p_1$. (c) Montrez que $\partial L / \partial z_k = p_k - y_k$ (où y_k vaut 1 pour la vraie classe, 0 sinon), et donnez numériquement le gradient.

Exercice 1.2 ★ — Compter les paramètres (sections 1.1.3 et 1.2.5)

- (a) Un réseau dense 784-256-128-10 : combien de paramètres ? (b) Un réseau convolutif : convolution à 16 filtres 5×5 sur une image à 1 canal, puis convolution à 32 filtres 3×3 , puis une couche dense de 10 sorties appliquée à 32 cartes de 4×4 aplaties : combien de paramètres ?

Exercice 1.3 ★★ — Rétropropagation avec une ReLU (section 1.1.5)

Un réseau à une entrée, deux neurones cachés (ReLU) et une sortie linéaire, avec la perte quadratique $L = \frac{1}{2}(\hat{y} - y)^2$. Entrée $x = 2$, cible $y = 1$. Couche cachée : poids $w = (0,5, -1)$, biais $b = (0,1, 0,2)$. Sortie : poids $v = (1,5, 0,5)$, biais $c = 0$. Calculez la passe avant, puis tous les gradients, puis le gradient par rapport à w_2 . Que vaut-il, et pourquoi ?

Exercice 1.4 ★ — Choisir une activation (section 1.1.2)

Pour chaque situation, choisissez la sortie et la perte adaptées, et justifiez en une phrase : (a) prédire si un client résilie (oui/non) ; (b) prédire le chiffre sur une image (10 classes exclusives) ; (c) prédire les ventes du lendemain en euros ; (d) étiqueter un article avec plusieurs thèmes possibles à la fois.

Exercice 1.5 ★★ — Le dropout « inversé » (section 1.1.8)

On applique un dropout de probabilité $p = 0,5$ à $\mathbf{a} = (1, 2, 3, 4)$: chaque valeur est mise à zéro avec la probabilité p , et les valeurs conservées sont **multipliées par** $1/(1 - p)$. (a) Pourquoi cette mise à l'échelle ? (b) Vérifiez par simulation (10 000 tirages) que l'espérance de la sortie est $\mathbf{a}$. (c) Que fait le dropout à la prédiction ?

Exercice 1.6 ★ — Taille de sortie d'une convolution (section 1.2.2)

Une image de 64×64 passe par : convolution 5×5 (pas 1, sans rembourrage), pooling 2×2 , convolution 3×3 avec rembourrage 1, pooling 2×2 , convolution 3×3 de pas 2 sans rembourrage. Donnez la taille après chaque étape.

Exercice 1.7 ★★ — Le champ réceptif (section 1.2.4)

Même réseau qu'à l'exercice 1.6. Quel est le **champ réceptif** (en pixels de l'image) d'un neurone situé après la troisième convolution ? On rappelle la récurrence : à chaque couche de filtre k et de pas s , le champ vaut $r \leftarrow r + (k - 1)j$, puis le « saut » devient $j \leftarrow js$ (au départ, $r = 1$ et $j = 1$).

Exercice 1.8 ★★ — Un pas de LSTM à la main (section 1.3.3)

Un LSTM à une unité, entrée $x = -1$, $h_{t-1} = 0$, $c_{t-1} = 1$. Poids (entrée, état, biais) : entrée (1; 0; 0), oubli (0; 0; 2), candidat (1; 0; 0), sortie (0; 0; 0). Calculez les portes, c_t et h_t . Que fait ce LSTM de sa mémoire ?

Exercice 1.9 ★ — Le naïf saisonnier à la main (section 1.3.4)

Les ventes de deux semaines sont : semaine 1 = (10, 12, 11, 13, 15, 20, 14) et semaine 2 = (11, 13, 10, 14, 16, 22, 13). Prévoyez la semaine 2 par le naïf saisonnier (même jour, semaine précédente). Calculez l'erreur absolue moyenne, et comparez-la à celle du naïf « hier ».

Exercice 1.10 ★ — Le bug de la boucle (section 1.4.3)

Un étudiant écrit une boucle d'entraînement où il a oublié `optimiseur.zero_grad()`. (a) Que se passe-t-il pour les gradients d'un lot à l'autre ? (b) Montrez-le sur un exemple minimal : un seul paramètre $w = 1$, la perte $L = 3w$ calculée et rétropropagée trois fois de suite.

Exercice 1.11 ★ — Le taux d'erreur par caractère (section 1.5)

La vraie ligne est FACTURE N° 2025. Tesseract a lu FACTURF N° 2025 (la lettre O à la place du chiffre 0). Calculez la distance d'édition de Levenshtein et le CER. Que vaudrait le CER si la lecture avait **supprimé** l'espace entre N° et 2025 ?

Exercice 1.12 ★ — Choisir la méthode (sections 1.2 à 1.6)

Pour chaque cas, dites si vous commenceriez par un réseau de neurones, et lequel : (a) 5 000 photos de produits à classer en 12 rayons ; (b) un tableau de 20 000 clients, 40 colonnes, prédire la résiliation ; (c) la prévision hebdomadaire des ventes de 3 000 produits ; (d) 200 factures scannées par mois à saisir automatiquement.

Corrigés

Corrigé 1.1

(a) $p_k = e^{z_k} / \sum_j e^{z_j}$; $\mathbf{p} = (0,6652, 0,2447, 0,0900)$. (b) $L = -\ln p_1 = 0,4076$. (c) Avec $L = -z_1 + \ln \sum_j e^{z_j}$, la dérivée par rapport à z_k est $-y_k + e^{z_k} / \sum_j e^{z_j} = p_k - y_k$. Numériquement : $(p_1 - 1, p_2, p_3) = (-0,3348, 0,2447, 0,0900)$; les trois composantes somment à zéro, et PyTorch retrouve les mêmes valeurs (vérification : True).

Corrigé 1.2

(a) $784 \times 256 + 256 = 200\,960$, puis $256 \times 128 + 128 = 32\,896$, puis $128 \times 10 + 10 = 1\,290$: au total **235 146**. (b) Première convolution : $16 \times (1 \times 5 \times 5 + 1) = 416$; seconde : $32 \times (16 \times 3 \times 3 + 1) = 4\,640$; couche dense : $(32 \times 4 \times 4) \times 10 + 10 = 5\,130$; au total **10 186**. Les deux comptes sont confirmés par `nb_parametres`. Remarque : la convolution a très peu de paramètres comparée à sa taille ; c'est la couche dense finale qui pèse le plus.

Corrigé 1.3

Passe avant : $z = (1, 10, -1, 80)$, activations ReLU $a = (1, 10, 0, 0)$, sortie $\hat{y} = 1,5 \times 1,10 + 0,5 \times 0 = 1,65$, perte $L = 0,211$. Passe arrière : $\partial L / \partial \hat{y} = \hat{y} - y = 0,65$; gradients de sortie $\partial L / \partial v = (0,715, 0,000)$; pour les neurones cachés, le signal est multiplié par la dérivée de la ReLU (1 si $z > 0$, sinon 0) : $\partial L / \partial b = (0,975, 0,000)$ et $\partial L / \partial w = (1,950, 0,000)$. Le gradient par rapport à w_2 est **nul** : le neurone 2 est « éteint » ($z_2 < 0$), donc aucun signal ne le traverse ; il ne bougera pas pour cet exemple. C'est le risque de la ReLU (neurones « morts ») dont l'initialisation de He et un pas raisonnable protègent.

Corrigé 1.4

- (a) **Une sortie, sigmoïde, entropie croisée binaire** : une probabilité pour la classe « résilie ».
- (b) **Dix sorties, softmax, entropie croisée** : des probabilités qui somment à 1 pour des classes exclusives.
- (c) **Une sortie linéaire** (aucune activation), perte quadratique ou absolue : une valeur réelle, pas bornée.
- (d) **Une sigmoïde par thème**, entropie croisée binaire pour chaque sortie : les thèmes ne s'excluent pas, donc pas de softmax.

Corrigé 1.5

- (a) Si l'on se contentait de couper la moitié des valeurs, la somme reçue par la couche suivante serait **divisée par deux** à l'entraînement, mais pas à la prédiction. La mise à l'échelle par $1/(1-p)$ compense : l'**espérance** de chaque sortie est $a \cdot (1-p) \cdot \frac{1}{1-p} = a$.
- (b) La moyenne simulée de la sortie vaut $(1,00, 2,04, 2,97, 4,00)$: c'est **a**, à l'erreur de simulation près.
- (c) À la prédiction, le dropout est **désactivé** (`modele.eval()`) : toutes les unités sont actives, sans mise à l'échelle supplémentaire, grâce à la compensation faite à l'entraînement.

Corrigé 1.6

Avec $\lfloor (n+2p-k)/s \rfloor + 1$: $64 \rightarrow 60$ (conv. 5×5) $\rightarrow 30$ (pooling) $\rightarrow 30$ (conv. 3×3 avec rembourrage, la taille est conservée) $\rightarrow 15$ (pooling) $\rightarrow 7$ (conv. 3×3 de pas 2).

Corrigé 1.7

On applique la récurrence couche par couche : champ de 5 pixels après la première convolution, 6 après le premier pooling, 10 après la deuxième convolution, 12 après le second pooling, puis 20 après la troisième convolution : un carré de 20 pixels de côté, sur une image de 64×64 . Un neurone de la dernière couche voit donc une large portion de l'image, alors que chaque filtre n'est que de 3×3 ou 5×5 : c'est l'effet de l'empilement.

Corrigé 1.8

Portes : entrée $i = \sigma(-1) = 0,269$, oubli $f = \sigma(2) = 0,881$, candidat $g = \tanh(-1) = -0,762$, sortie $o = \sigma(0) = 0,500$. Mémoire : $c_t = f \cdot 1 + i \cdot g = 0,676$; état : $h_t = o \cdot \tanh(c_t) = 0,294$. La porte d'oubli est proche de 1 : la mémoire (1) est presque conservée ; l'information nouvelle (négative) en retranche un peu. C'est le comportement « mémoire stable » qui permet de relier des instants éloignés.

Corrigé 1.9

Le naïf saisonnier prédit la semaine 1 pour la semaine 2 : les erreurs absolues sont (1, 1, 1, 1, 1, 2, 1), d'où une MAE de **1,14**. Le naïf « hier » prédit chaque jour par la veille ; sa MAE vaut **4,14**, bien plus grande, parce qu'il ignore le rythme hebdomadaire (le samedi est fort, le dimanche plus faible). C'est pourquoi le naïf **saisonnier** est la bonne référence ici.

Corrigé 1.10

- (a) Par défaut, PyTorch **additionne** les nouveaux gradients à ceux déjà présents : sans `zero_grad()`, chaque pas utilise la somme des gradients de tous les lots précédents, et l'entraînement diverge ou se comporte de façon incohérente. (b) Avec $L = 3w$, le gradient vaut 3 à chaque calcul ; mais le champ `w.grad` prend successivement les valeurs 3, 6 puis 9 : il **s'accumule**. (Cette accumulation est parfois utile, pour simuler de gros lots, mais elle doit être voulue.)

Corrigé 1.11

La vraie ligne compte 15 caractères. La lecture comporte deux substitutions ($E \rightarrow F$, $0 \rightarrow O$) : la distance de Levenshtein est 2 et le CER vaut $2/15 =$ **0,133**. Si l'espace avait été supprimé, ce serait une seule suppression : distance 1, CER 0,067. Le CER compte les erreurs de n'importe quel type, mais ne dit pas leur gravité : 0 lu O dans un montant est bien plus grave qu'un espace manquant ; en pratique on ajoute des contrôles métier (le total est-il la somme des lignes ?).

Corrigé 1.12

- (a) **Oui, un réseau convolutif**, mais surtout **par transfert** : avec 5 000 photos, on réutilise un réseau pré-entraîné et l'on n'entraîne que la fin (section 1.5). (b) **Non** : régression logistique puis boosting ; un réseau à plongements ne sera qu'un essai de plus, rarement meilleur (section 1.6). (c) **Peut-être** : avec des milliers de séries, un modèle global (un seul réseau pour toutes les séries) devient intéressant, mais **après** les références classiques (naïf saisonnier, boosting à retards). (d) **Un OCR existant** (Tesseract ou service équivalent), avec un prétraitement et des contrôles métier ; entraîner un réseau de reconnaissance sur 200 documents par mois n'a pas de sens.

Chapitre 2 : NLP et modèles de langage — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Les **applications** refont pas à pas, sur les avis de la boutique, les calculs que le livre n'a fait que résumer (TF-IDF, attention, décodage, BPE, recherche sémantique, RAG) ; les **exercices** se travaillent d'abord à la main, et leurs **corrigés** viennent à la fin. Chaque exercice indique la section du livre qu'il met en pratique. Tous les modèles s'exécutent hors ligne sur un ordinateur ordinaire.

Préparation

Une seule cellule charge les bibliothèques et les 8 000 avis (simulés, générés par des gabarits de phrases : voir le livre, introduction du chapitre), répartit les avis nets (positifs : note ≥ 4 , négatifs : note ≤ 2) en entraînement (75 %) et test (25 %), et ajuste la **référence** de la section 2.1.4 : TF-IDF (mots et paires de mots) + régression logistique. Les applications suivantes en réutilisent les noms : avis (tous les avis), tr, te (entraînement, test), vec et modele (la référence), et 0.SONDES (48 phrases écrites à la main, hors gabarits, avec leurs étiquettes 0.Y_SONDES). Le module outils_ch02, dans le dossier build/, contient les outils du chapitre (tokenisation, mini-transformer, décodage) ; ses fonctions sont lisibles.

```
import sys, warnings, math, os
import numpy as np, pandas as pd, torch
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch02 as o
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

torch.set_num_threads(2)
avis = o.etiqueter_tranches(o.charger_avis())
d = o.polarite(avis)
tr, te = train_test_split(d, test_size=0.25, random_state=0, stratify=d["y"])
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2)
modele = LogisticRegression(max_iter=3000, C=3).fit(vec.fit_transform(tr["texte"]), tr["y"])
acc_te = modele.score(vec.transform(te["texte"]), te["y"])
acc_so = ((modele.predict(vec.transform(0.SONDES))) == 0.Y_SONDES).mean())
print(len(avis), len(tr), len(te), "| exactitude test :", round(acc_te, 3), "| sur les 48 phrases hors
↳ gabarit :", round(acc_so, 3))
```

```
8000 5074 1692 | exactitude test : 0.942 | sur les 48 phrases hors gabarit : 0.625
```

Le test du corpus donne environ 94 %, et les phrases écrites à la main environ 62 % : c'est l'écart que le livre explique en section 2.1.5.

Applications

Application 2.1 — Du texte au TF-IDF, sans bibliothèque

Sections du livre : 2.1.1 à 2.1.3. Objectif : refaire le TF-IDF de zéro (jetons, comptages, idf, poids, cosinus), puis vérifier que la variante de scikit-learn donne la même idée avec des valeurs différentes, et que notre version de zéro classe aussi bien.

Étape 1 — Jetons et vocabulaire. On découpe chaque avis en jetons avec `0.tokeniser` (minuscules, apostrophe conservée), puis on construit le vocabulaire des mots présents au moins deux fois.

```
from collections import Counter
jetons_tr = [0.tokeniser(t) for t in tr["texte"]]
df_mots = Counter(w for j in jetons_tr for w in set(j))           # nombre de documents
    ↪ contenant chaque mot
vocab = sorted(w for w, n in df_mots.items() if n >= 2)
indice = {w: i for i, w in enumerate(vocab)}
print(len(vocab), "mots ; exemple de découpage :", jetons_tr[0][:8])
```

```
321 mots ; exemple de découpage : ['dans', "l'ensemble", 'tarif', 'habituel', 'pour', 'ce', 'type',
    ↪ "d'article"]
```

Étape 2 — La matrice TF-IDF. Pour chaque document, $tf = n_{t,d}/|d|$; pour chaque mot, $idf = \ln(N/df)$; le poids est leur produit.

```
N = len(jetons_tr)
idf = np.array([math.log(N / df_mots[w]) for w in vocab])
def tfidf(jetons):
    x = np.zeros(len(vocab))
    for w, n in Counter(jetons).items():
        if w in indice:
            x[indice[w]] = n / len(jetons) * idf[indice[w]]
    return x
X_tr = np.array([tfidf(j) for j in jetons_tr])
X_te = np.array([tfidf(0.tokeniser(t)) for t in te["texte"]])
top = np.argsort(-idf)[:3]; bas = np.argsort(idf)[:3]
print("mots les plus rares :", [vocab[i] for i in top], "| les plus fréquents :", [vocab[i] for i in
    ↪ bas])
```

```
mots les plus rares : ['as', 'bine', 'u'] | les plus fréquents : ['pas', 'le', 'la']
```

Étape 3 — Classifier avec notre TF-IDF.

```
m0 = LogisticRegression(max_iter=3000, C=30).fit(X_tr, tr["y"])
print("exactitude (TF-IDF de zéro, mots seuls) :", round(m0.score(X_te, te["y"]), 3))
```

```
exactitude (TF-IDF de zéro, mots seuls) : 0.942
```

Étape 4 — Le cosinus entre deux avis. Les deux avis les plus proches d'un avis donné, parmi les 500 premiers du jeu d'entraînement.

```
def cos(a, b):
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-12))
q = 0
sims = [cos(X_tr[q], X_tr[i]) for i in range(1, 500)]
```

```
voisin = 1 + int(np.argmax(sims))
print("avis :", tr["texte"].iloc[q][:70], "\nvoisin :", tr["texte"].iloc[voisin][:70], "| cosinus",
      ↪ round(max(sims), 3))
```

```
avis : Dans l'ensemble, tarif habituel pour ce type d'article. Aucune casse,
voisin : Service correct. Tarif habituel pour ce type d'article. | cosinus 0.606
```

Les mots les plus rares ne sont pas les plus utiles pour classer : ce sont des **fautes de frappe** (le plus petit df). Un min_df de 2 les élimine en grande partie. Notre TF-IDF de zéro, avec les mots seuls, atteint à peu près l'exactitude de la référence (mots et paires de mots) : sur ce corpus, **les paires de mots n'apportent presque rien**.

Pour aller plus loin. Ajoutez les paires de mots (bigrammes) à votre vocabulaire et vérifiez que l'exactitude monte d'au plus quelques dixièmes de point. Puis remplacez $\ln(N/df)$ par la variante de scikit-learn, $\ln \frac{1+N}{1+df} + 1$, et comparez.

Application 2.2 — La référence et les phrases hors gabarit

Sections du livre : 2.1.4 et 2.1.5. **Objectif** : voir où la référence se trompe sur les phrases écrites à la main, mesurer la part de mots inconnus, et écrire vos propres phrases de test.

Étape 1 — Les erreurs sur les 48 phrases.

```
p = modele.predict_proba(vec.transform(0.SONDES))[:, 1]
faux = [i for i in range(len(0.SONDES)) if (p[i] > 0.5) != 0.Y_SONDES[i]]
print(len(faux), "erreurs sur", len(0.SONDES))
for i in faux[:6]:
    print(f"    p(positif)={p[i]:.2f}    attendu={0.Y_SONDES[i]}    {0.SONDES[i]}")
```

```
18 erreurs sur 48
p(positif)=0.45    attendu=1    Rapport qualité-prix imbattable.
p(positif)=0.50    attendu=1    Tout est arrivé intact, merci beaucoup.
p(positif)=0.71    attendu=0    Interminable : trois semaines pour recevoir un simple colis.
p(positif)=0.93    attendu=0    Rien n'a fonctionné, c'est une arnaque.
p(positif)=0.66    attendu=0    Ce n'est pas du tout ce que j'espérais.
p(positif)=0.89    attendu=0    Je regrette amèrement d'avoir commandé.
```

Étape 2 — La part de mots inconnus. Pour chaque phrase, la proportion de ses mots absents du vocabulaire appris ; on compare les phrases bien et mal classées (l'hypothèse naturelle : les erreurs viennent des mots inconnus).

```
connus = set(vec.vocabulary_)
def part_inconnue(s):
    j = 0.tokeniser(s)
    return np.mean([w not in connus for w in j]) if j else 0.0
inc = np.array([part_inconnue(s) for s in 0.SONDES])
ok = np.ones(len(0.SONDES), bool); ok[faux] = False
print("part moyenne de mots inconnus : phrases justes", round(inc[ok].mean(), 3), "| phrases
      ↪ fausses", round(inc[-ok].mean(), 3))
```

```
part moyenne de mots inconnus : phrases justes 0.411 | phrases fausses 0.385
```

Étape 3 — Vos propres phrases. Écrivez six phrases (3 positives, 3 négatives) qui n'emploient aucun gabarit du corpus, et testez la référence dessus.

```
mes_phrases = ["Colis reçu avec une semaine d'avance, bravo.", "Aucun souci, tout est conforme à la
      ↪ description.", "Un achat que je ne regrette pas.",
                "Décevant : la couleur n'a rien à voir avec la photo.", "Le vendeur n'a jamais répondu à
      ↪ mes relances.", "Article arrivé fendu, c'est inadmissible."]
mes_y = np.array([1, 1, 1, 0, 0, 0])
```

```
pm = modele.predict_proba(vec.transform(mes_phrases))[:, 1]
for s, pp, yy in zip(mes_phrases, pm, mes_y):
    print(f"p(positif)={pp:.2f} attendu={yy} {s}")
```

```
p(positif)=0.79 attendu=1 Colis reçu avec une semaine d'avance, bravo.
p(positif)=0.94 attendu=1 Aucun souci, tout est conforme à la description.
p(positif)=0.06 attendu=1 Un achat que je ne regrette pas.
p(positif)=0.83 attendu=0 Décevant : la couleur n'a rien à voir avec la photo.
p(positif)=0.67 attendu=0 Le vendeur n'a jamais répondu à mes relances.
p(positif)=0.47 attendu=0 Article arrivé fendu, c'est inadmissible.
```

La part de mots inconnus est **élevée partout** (environ 40 % des mots, que la phrase soit bien ou mal classée) : elle ne suffit donc pas à expliquer les erreurs. Ce qui compte est le **signe** porté par les quelques mots connus, souvent ambigus (« rien », « colis », « pas »). Sur les six phrases que nous avons écrites, la référence se trompe dans la moitié des cas : elle n'a mémorisé que le vocabulaire et les tournures du corpus.

Pour aller plus loin. Étendez votre jeu à 30 phrases et calculez l'incertitude de l'exactitude : l'écart-type d'une proportion est $\sqrt{p(1-p)/n}$.

Application 2.3 — L'attention de zéro

Sections du livre : 2.2.2 à 2.2.6. **Objectif** : écrire l'attention multi-têtes avec seulement numpy, la comparer à PyTorch, et visualiser l'effet du masque causal.

Étape 1 — Une tête, avec des projections quelconques. Quatre jetons de dimension 4, des projections W_Q, W_K, W_V tirées au hasard.

```
rng = np.random.default_rng(0)
n, d = 4, 4
X = rng.standard_normal((n, d))
Wq, Wk, Wv = (rng.standard_normal((d, d)) for _ in range(3))
def softmax_lignes(S):
    e = np.exp(S - S.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)
Q, K, V = X @ Wq, X @ Wk, X @ Wv
A = softmax_lignes(Q @ K.T / np.sqrt(d))
sortie = A @ V
print(A.round(2)); print("somme des lignes :", A.sum(axis=1))
```

```
[[0.03 0.01 0.31 0.65]
 [0. 0. 0.13 0.87]
 [0.07 0.08 0.24 0.61]
 [0.14 0.07 0.42 0.37]]
somme des lignes : [1. 1. 1. 1.]
```

Étape 2 — Comparaison avec PyTorch.

```
import torch.nn.functional as F
ref = F.scaled_dot_product_attention(*(torch.tensor(M, dtype=torch.float64)[None] for M in (Q, K,
↪ V)))[0].numpy()
print("écart maximal avec PyTorch :", float(np.abs(ref - sortie).max()))
```

```
écart maximal avec PyTorch : 8.881784197001252e-16
```

Étape 3 — Le masque causal. On remplace par $-\infty$ les scores de la partie triangulaire supérieure avant le softmax.

```
S = Q @ K.T / np.sqrt(d)
S_masque = np.where(np.triu(np.ones((n, n), bool), 1), -np.inf, S)
```

```
A_c = softmax_lignes(S_masque)
print(A_c.round(2))
```

```
[[1.  0.  0.  0. ]
 [1.  0.  0.  0. ]
 [0.18 0.2  0.62 0. ]
 [0.14 0.07 0.42 0.37]]
```

Étape 4 — Plusieurs têtes. On découpe les dimensions en 2 têtes de dimension 2, on calcule chaque attention, on concatène.

```
def tete(X, Wq, Wk, Wv):
    q, k, v = X @ Wq, X @ Wk, X @ Wv
    return softmax_lignes(q @ k.T / np.sqrt(q.shape[1])) @ v
h = 2
sorties = [tete(X, Wq[:, i * 2:(i + 1) * 2], Wk[:, i * 2:(i + 1) * 2], Wv[:, i * 2:(i + 1) * 2]) for i
    ↪ in range(h)]
multi = np.concatenate(sorties, axis=1)
print(multi.shape, "| différence avec la tête unique :", float(np.abs(multi - sortie).max()))
```

```
(4, 4) | différence avec la tête unique : 2.9175532574183047
```

La somme des lignes vaut 1, l'écart avec PyTorch est de l'ordre de 10^{-16} , et le masque causal rend la matrice triangulaire inférieure : la première ligne ne regarde que le premier jeton. Les deux têtes de dimension 2 ne redonnent **pas** la tête unique de dimension 4 : une attention multi-têtes **n'est pas** la même fonction qu'une tête large, elle calcule plusieurs attentions indépendantes qu'une dernière projection remélange.

Pour aller plus loin. Ajoutez la projection de sortie W_O et vérifiez que le nombre de paramètres de l'ensemble (W_Q, W_K, W_V, W_O) vaut $4d^2$.

Application 2.4 — Entraîner un mini-transformer, et l'interroger

Sections du livre : 2.2.5 à 2.2.8. **Objectif :** entraîner le mini-transformer sur un sous-échantillon, le comparer à la référence, et mesurer sa **sensibilité à l'ordre** des mots, que TF-IDF (sans paires de mots) ne peut pas avoir.

Étape 1 — Entraînement sur 2 000 avis.

```
petit = tr.sample(2000, random_state=0)
voc = 0.Vocabulaire(petit["texte"].tolist(), min_freq=2)
mini = 0.entrainer_classifieur(petit["texte"].tolist(), petit["y"].to_numpy(), voc, epoques=4)
p_te = 0.predire_classe(mini, voc, te["texte"].tolist())
print("paramètres :", sum(p.numel() for p in mini.parameters()), "| exactitude test :", round(((p_te
    ↪ > 0.5) == te["y"].to_numpy()).mean(), 3))
```

```
paramètres : 70274 | exactitude test : 0.937
```

Étape 2 — Sur les phrases hors gabarit.

```
ps = 0.predire_classe(mini, voc, 0.SONDES)
print("exactitude sur les 48 phrases :", round(((ps > 0.5) == 0.Y_SONDES).mean(), 3))
```

```
exactitude sur les 48 phrases : 0.583
```

Étape 3 — Sensibilité à l'ordre. On mélange l'ordre des mots de 300 avis du test et l'on mesure la variation moyenne de la probabilité prédite, pour le mini-transformer et pour un TF-IDF de mots seuls.

```

rng = np.random.default_rng(0)
echant = te["texte"].iloc[:300].tolist()
melange = [" ".join(rng.permutation(t.split())) for t in echant]
vec1 = TfidfVectorizer(min_df=2).fit(tr["texte"]); lr1 = LogisticRegression(max_iter=3000,
    ↪ C=3).fit(vec1.transform(tr["texte"]), tr["y"])
ecart_mini = np.abs(0.predire_classe(mini, voc, echant) - 0.predire_classe(mini, voc, melange)).mean()
ecart_tfidf = np.abs(lr1.predict_proba(vec1.transform(echant))[:, 1] -
    ↪ lr1.predict_proba(vec1.transform(melange))[:, 1]).mean()
print("variation moyenne de la probabilité : mini-transformer", round(ecart_mini, 3), "| TF-IDF de
    ↪ mots", round(ecart_tfidf, 6))

```

```
variation moyenne de la probabilité : mini-transformer 0.001 | TF-IDF de mots 0.0
```

Le TF-IDF de mots seuls est **parfaitement insensible** à l'ordre (variation nulle : même sac de mots). Le mini-transformer l'est **presque** aussi (variation de l'ordre du millièmè) : malgré l'encodage positionnel, il a appris, sur ce corpus où l'ordre apporte peu, à se comporter surtout comme un sac de mots. Sur les phrases hors gabarit, il ne fait pas mieux que la référence : l'attention ne remplace pas le pré-entraînement (section 2.2.7). Un modèle capable d'être sensible à l'ordre ne l'est que si **les données l'exigent**.

Pour aller plus loin. Entraînez avec couches=1 puis couches=3 et comparez exactitude et nombre de paramètres ; mesurez la variation due à la graine (graine=0, 1, 2).

Application 2.5 — Le décodage de zéro

Sections du livre : 2.3.3 et 2.3.4. **Objectif :** implémenter température, top-k et top-p sur un petit vocabulaire, et vérifier empiriquement par simulation que les fréquences tirées suivent les probabilités.

Étape 1 — La température. Cinq jetons, des scores arbitraires.

```

mots = ["rapide", "lent", "correct", "cher", "parfait"]
z = np.array([2.0, 0.5, 1.2, -0.3, 1.8])
def probas(z, T=1.0):
    e = np.exp((z - z.max()) / T)
    return e / e.sum()
tab = pd.DataFrame({"T={T}": probas(z, T) for T in (0.3, 1.0, 3.0)}, index=mots).round(3)
tab.loc["entropie (bits)"] = [round(0.entropie(probas(z, T)), 2) for T in (0.3, 1.0, 3.0)]
print(tab.to_string())

```

	T=0.3	T=1.0	T=3.0
rapide	0.629	0.386	0.265
lent	0.004	0.086	0.161
correct	0.044	0.173	0.203
cher	0.000	0.039	0.123
parfait	0.323	0.316	0.248
entropie (bits)	1.180	1.980	2.270

Étape 2 — Top-k et top-p.

```

def top_k(p, k):
    q = np.where(p >= np.sort(p)[-k], p, 0.0)
    return q / q.sum()
def top_p(p, seuil):
    ordre = np.argsort(-p); cum = np.cumsum(p[ordre])
    garde = ordre[: np.searchsorted(cum, seuil) + 1]
    q = np.zeros_like(p); q[garde] = p[garde]
    return q / q.sum()
p1 = probas(z)
print("top-2 :", top_k(p1, 2).round(3), "| top-p 0,8 :", top_p(p1, 0.8).round(3))

```

```
top-2 : [0.55 0.    0.    0.45] | top-p 0,8 : [0.441 0.    0.198 0.    0.361]
```

Étape 3 — Vérification par simulation. On tire 100 000 jetons à $T = 1$ et l'on compare les fréquences aux probabilités.

```
rng = np.random.default_rng(0)
tirages = rng.choice(len(mots), size=100000, p=p1)
freq = np.bincount(tirages, minlength=len(mots)) / len(tirages)
print(pd.DataFrame({"théorique": p1, "observé": freq}, index=mots).round(3).to_string())
print("écart maximal :", round(float(np.abs(p1 - freq).max()), 4))
```

	théorique	observé
rapide	0.386	0.384
lent	0.086	0.088
correct	0.173	0.173
cher	0.039	0.039
parfait	0.316	0.315
écart maximal :	0.0021	

À $T = 0,3$ la masse se concentre sur les deux meilleurs jetons (« rapide » 0,63 et « parfait » 0,32 ; entropie faible) ; à $T = 3$, la distribution est proche de l'uniforme (entropie proche du maximum $\log_2 5 \approx 2,32$ bits). Le top-p 0,8 retient les jetons les plus probables jusqu'à atteindre 80 % de masse : ici trois jetons, et leur proportion relative est conservée.

Pour aller plus loin. Vérifiez que, pour $T \rightarrow 0$, la sortie tend vers le décodage glouton (un seul jeton à probabilité 1), et que top-p = 1 ne change rien.

Application 2.6 — Le BPE à la main

Section du livre : 2.3.2. Objectif : entraîner un BPE sur le vocabulaire des avis, puis voir comment il découpe un mot qu'il n'a jamais vu et une faute de frappe.

Étape 1 — L'algorithme. Un mot est une suite de symboles terminée par la marque `·` ; à chaque étape, on fusionne la paire voisine la plus fréquente (pondérée par la fréquence des mots).

```
def apprendre_bpe(freq_mots, n_fusions):
    corpus = {tuple(m) + ("·",): f for m, f in freq_mots.items()}
    regles = []
    for _ in range(n_fusions):
        paires = Counter()
        for mot, f in corpus.items():
            for a, b in zip(mot, mot[1:]):
                paires[(a, b)] += f
        if not paires:
            break
        a, b = max(paires, key=paires.get)
        regles.append((a, b))
        corpus = {fusionner(mot, a, b): f for mot, f in corpus.items()}
    return regles

def fusionner(mot, a, b):
    sortie, i = [], 0
    while i < len(mot):
        if i < len(mot) - 1 and mot[i] == a and mot[i + 1] == b:
            sortie.append(a + b); i += 2
        else:
            sortie.append(mot[i]); i += 1
    return tuple(sortie)
```

Étape 2 — Entraînement sur les avis. 200 fusions sur les mots des avis d'entraînement.

```
freq_mots = Counter(w for j in jetons_tr for w in j)
regles = apprendre_bpe(freq_mots, 200)
print(regles[:8], "...", regles[-3:])
```

```
[('e', ' '), ('t', ' '), ('s', ' '), ('e', 'n'), ('o', 'n'), ('a', 'i'), ('a', 'n'), ('l', 'e')] ...
↳ [('i', 's'), ('auc', 'une'), ('t', 'on')]
```

Étape 3 — Découper un mot quelconque. On applique les règles dans l'ordre où elles ont été apprises.

```
def decouper(mot, regles):
    s = tuple(mot) + (".",)
    for a, b in regles:
        s = fusionner(s, a, b)
    return s
for mot in ["livraison", "interminable", "livriason", "emballage"]:
    print(f"{mot:13s} ->", " ".join(decouper(mot, regles)))
```

```
livraison      -> livraison.
interminable   -> i n t e r m i n a b l e.
livriason      -> l i v r i a s o n.
emballage      -> emballage.
```

Les mots fréquents (« livraison », « emballage ») sont réduits à un ou deux symboles ; un mot **jamais vu** comme « interminable » est découpé en **morceaux plus petits** que le BPE connaît, sans jamais être « inconnu » ; la **faute de frappe** « livriason » est découpée en cinq morceaux courts (« l i v r i a s o n »). C'est la grande différence avec un vocabulaire de mots entiers.

Pour aller plus loin. Faites varier le nombre de fusions (50, 200, 800) et tracez la longueur moyenne de la segmentation d'un mot en fonction de ce nombre.

Application 2.7 — Recherche sémantique avec MiniLM

Sections du livre : 2.4.2 et 2.2.7. **Objectif :** construire un petit moteur de recherche sur 2 000 avis avec des plongements de phrases, et le comparer à TF-IDF.

Étape 1 — Encoder les avis. Le modèle paraphrase-multilingual-MiniLM-L12-v2 (118 M de paramètres) transforme chaque avis en un vecteur de 384 nombres ; les vecteurs sont normalisés, donc le produit scalaire est le cosinus.

```
from sentence_transformers import SentenceTransformer
st = SentenceTransformer("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2", device="cpu")
corpus = avis.sample(2000, random_state=1).reset_index(drop=True)
E = st.encode(corpus["texte"].tolist(), batch_size=128, normalize_embeddings=True)
print(E.shape)
```

```
(2000, 384)
```

Étape 2 — Deux moteurs.

```
tf = TfidfVectorizer(min_df=2).fit(corpus["texte"]); X_c = tf.transform(corpus["texte"])
def chercher(requete, methode, k=5):
    if methode == "tfidf":
        s = (X_c @ tf.transform([requete]).T).toarray()[0, :]
    else:
        s = E @ st.encode([requete], normalize_embeddings=True)[0]
    return np.argsort(-s)[:k]
for q in ["Mon colis est arrivé très en retard", "Je trouve ça trop cher"]:
    print("Requête :", q)
    for m in ("tfidf", "minilm"):
```

```
print(f"    {m:7s}", [corpus['sujet'][j] + "/" + str(corpus['note'][j]) for j in chercher(q,
↪ m)])
```

Requête : Mon colis est arrivé très en retard

tfidf ['qualite/2', 'livraison/1', 'service/2', 'livraison/4', 'service/2']

minilm ['livraison/1', 'prix/2', 'service/2', 'livraison/2', 'livraison/3']

Requête : Je trouve ça trop cher

tfidf ['prix/1', 'prix/2', 'prix/1', 'prix/2', 'prix/1']

minilm ['prix/1', 'prix/1', 'prix/2', 'emballage/2', 'prix/1']

Étape 3 — Précision au rang 5 sur les 15 requêtes du livre. Un résultat est **pertinent** s'il a pour sujet principal celui de la requête et une note ≤ 2 .

```
res = []
for sj, qs in 0.REQUETES.items():
    pert = set(np.where((corpus["sujet"] == sj) & (corpus["note"] <= 2))[0])
    for q in qs:
        res.append([0.precision_au_rang(chercher(q, m, 5), pert, 5) for m in ("tfidf", "minilm")])
res = np.array(res)
print("précision moyenne au rang 5 : TF-IDF", res[:, 0].mean().round(3), "| MiniLM", res[:,
↪ 1].mean().round(3), "| requêtes :", len(res))
```

```
précision moyenne au rang 5 : TF-IDF 0.6 | MiniLM 0.493 | requêtes : 15
```

Sur ce sous-corpus, TF-IDF est **devant** (environ 0,60 contre 0,49), alors que sur les 8 000 avis du livre l'écart est plus faible (0,77 contre 0,69) : les chiffres varient beaucoup avec le corpus, preuve du bruit de la mesure. MiniLM retrouve des avis de même **thème** même sans mot commun, mais ignore souvent le **ton** (il renvoie parfois des avis positifs sur le même thème), et la mesure dépend de l'étiquette « sujet principal », imparfaite pour des avis qui mêlent plusieurs sujets.

Pour aller plus loin. Combinez les deux scores (par exemple la somme des rangs inversés) : une recherche **hybride** fait-elle mieux que chacune ?

Application 2.8 — Un RAG minimal

Section du livre : 2.5.3. Objectif : construire la chaîne *indexer* → *retrouver* → *construire le prompt*, et l'évaluer à chaque maillon. Aucun modèle de génération n'est utilisé ici : on s'intéresse à ce qu'on envoie au modèle.

Étape 1 — Base de connaissances et questions. Huit passages (inventés) et neuf questions, dont une sans réponse dans la base.

```
BASE = ["Les retours sont acceptés pendant 14 jours après la réception, avec le produit dans son
↪ emballage d'origine.",
        "Les frais de port sont offerts à partir de 60 € d'achat ; en dessous, ils s'élèvent à 5,90 €.",
        "Le service client répond du lundi au vendredi, de 9 h à 17 h, par courriel ou par téléphone.",
        "Le remboursement est effectué sous 7 jours ouvrés après réception du retour, sur le moyen de
↪ paiement d'origine.",
        "La livraison standard prend 3 à 5 jours ouvrés ; la livraison express prend 24 heures pour un
↪ supplément de 9 €.",
        "Un produit endommagé à la réception est remplacé gratuitement si la réclamation est faite
↪ sous 48 heures avec une photo.",
        "Les cartes cadeaux sont valables un an et ne sont pas remboursables.",
        "Le produit A est garanti deux ans contre les défauts de fabrication."]
QUESTIONS = ["Combien de temps ai-je pour renvoyer un article ?", "À partir de quel montant la
↪ livraison est-elle gratuite ?",
               "Quand puis-je joindre quelqu'un au téléphone ?", "Au bout de combien de temps serai-je
↪ remboursé ?",
               "Peut-on recevoir sa commande le lendemain ?", "Mon colis est arrivé cassé, que faire ?",
               "Combien de temps dure la garantie du produit A ?", "Puis-je me faire rembourser une
↪ carte cadeau ?", "Quel est le prix du produit A ?"]
```

```

ATTENDU = [0, 1, 2, 3, 4, 5, 7, 6, None]
E_base = st.encode(BASE, normalize_embeddings=True); E_q = st.encode(QUESTIONS,
↪ normalize_embeddings=True)
scores = E_q @ E_base.T
rang = np.argsort(-scores, axis=1)
repondables = [i for i, a in enumerate(ATTENDU) if a is not None]
for k in (1, 3):
    print(f"succès au rang {k} :", sum(ATTENDU[i] in rang[i, :k] for i in respondables), "sur",
↪ len(repondables))

```

```

succès au rang 1 : 6 sur 8
succès au rang 3 : 8 sur 8

```

Étape 2 — Construire le prompt. On insère les trois meilleurs passages, numérotés pour que le modèle puisse **citer**, et la consigne de refuser s'il ne sait pas.

```

def construire_prompt(question, k=3):
    passages = "\n".join(f"[{n + 1}] {BASE[j]}" for n, j in enumerate(rang[QUESTIONS.index(question),
↪ :k]))
    return ("Réponds en français à la question à partir des passages ci-dessous et cite le numéro du
↪ passage utilisé. "
           f"Si la réponse n'y figure pas, réponds « Je ne sais pas ».\n\nPassages
↪ :\n{passages}\n\nQuestion : {question}")
print(construire_prompt(QUESTIONS[5]))

```

```

Réponds en français à la question à partir des passages ci-dessous et cite le numéro du passage
↪ utilisé. Si la réponse n'y figure pas, réponds « Je ne sais pas ».

```

Passages :

```

[1] Un produit endommagé à la réception est remplacé gratuitement si la réclamation est faite sous 48
↪ heures avec une photo.
[2] Les cartes cadeaux sont valables un an et ne sont pas remboursables.
[3] Le produit A est garanti deux ans contre les défauts de fabrication.

```

Question : Mon colis est arrivé cassé, que faire ?

Étape 3 — Peut-on refuser avec un seuil ? Le meilleur score de la question hors base, comparé à celui des questions avec réponse.

```

meilleur = scores.max(axis=1)
print("scores des questions avec réponse :", meilleur[repondables].round(2))
print("score de la question hors base    :", meilleur[8].round(2))

```

```

scores des questions avec réponse : [0.42 0.6  0.45 0.66 0.41 0.05 0.61 0.69]
score de la question hors base    : 0.38

```

Le passage attendu est en première position pour 6 questions sur 8 et dans les trois premiers pour les 8 : fournir **trois** passages plutôt qu'un rattrape les erreurs de recherche. La question hors base a un score **dans la plage des questions avec réponse** (0,38, entre 0,05 et 0,69) : aucun seuil ne sépare proprement les deux cas, et un modèle de génération à qui l'on fournirait ce passage pourrait **inventer** une réponse. D'où les citations et les tests de refus du livre (section 2.5.3).

Pour aller plus loin. Branchez un modèle de génération (SmolLM2, voir le livre) sur `construire_prompt`, et évaluez, sur les neuf questions, la part de réponses correctes **et** la part de refus justifiés.

Exercices

Exercice 2.1 ★ — Mots vides et négation (section 2.1.1)

1. Sur nos avis d'entraînement, quelle est la proportion d'avis négatifs qui contiennent le jeton « pas », et quelle est celle des avis positifs ?
2. Retirez « pas » du texte (remplacez-le par une chaîne vide) avant de vectoriser, et comparez l'exactitude de la référence (TF-IDF de mots seuls, régression logistique) avec et sans ce retrait.
3. Pourquoi retirer « pas » comme mot vide est-il dangereux pour l'analyse de sentiments ?

Exercice 2.2 ★ — Racinisation à la hache (section 2.1.1)

Écrivez une fonction qui retire les terminaisons « -s », « -es », « -ement » et « -ions » d'un mot (si le mot restant fait au moins quatre lettres). Appliquez-la à « livraisons », « livraison », « rapidement », « rapide », « chères », « cher ». Quels mots sont regroupés à raison ? Lesquels à tort ou pas du tout ?

Exercice 2.3 ★★ — L'idf de scikit-learn (section 2.1.3)

Soit trois documents « alpha beta », « alpha gamma » et « alpha delta ». Calculez à la main $\text{idf}(\text{alpha})$ et $\text{idf}(\text{beta})$ avec $\ln(N/\text{df})$, puis avec la formule lissée de scikit-learn, $\ln \frac{1+N}{1+\text{df}} + 1$. Vérifiez avec `TfidfVectorizer`. Quelle est la propriété de la formule lissée pour un mot présent dans tous les documents ?

Exercice 2.4 ★★ — Les mots inconnus (section 2.1.5)

Parmi les 48 phrases hors gabarit, quelle proportion de **mots** (jetons) est absente du vocabulaire de la référence ? Cette proportion est-elle plus élevée pour les phrases mal classées que pour les phrases bien classées ? Que concluez-vous sur l'explication « les erreurs viennent des mots inconnus » ?

Exercice 2.5 ★★ — Une attention à deux jetons (section 2.2.2)

Deux jetons $x_1 = (1, 0)$, $x_2 = (0, 1)$; $W_Q = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$, $W_K = W_V = I$. Calculez à la main la matrice des poids d'attention A et la sortie, avec $d_k = 2$, et vérifiez numériquement.

Exercice 2.6 ★★★ — La variance des scores et l'équivariance (sections 2.2.3 et 2.2.4)

1. Démontrez que si les composantes de q et k sont indépendantes, centrées, de variance 1, alors $\text{Var}(q \cdot k) = d_k$.
2. Démontrez que l'attention sans positions est équivariante par permutation : si P est une matrice de permutation, $\text{Att}(PX) = P \text{Att}(X)$.
3. Vérifiez la première propriété par simulation pour $d_k = 16$.

Exercice 2.7 ★★ — Compter les paramètres (section 2.2.5)

Un transformer a 12 blocs de dimension $d = 768$ (réseau par jeton de largeur $4d$), un vocabulaire de 30 000 jetons. Estimez le nombre de paramètres des blocs (formule $12d^2 + 13d$ par bloc) et des plongements d'entrée. Quelle part du total représentent les plongements ?

Exercice 2.8 ★★ — Le prix de la longueur (section 2.2.6)

Un modèle a 32 têtes et 40 couches. Si l'on stockait la matrice d'attention **complète** en flottants de 16 bits (2 octets) pour un texte de n jetons, quelle mémoire faut-il pour $n = 2\,048$ et pour $n = 32\,768$? Par quel facteur la mémoire est-elle multipliée quand n est multiplié par 16 ? Pourquoi les implémentations modernes évitent-elles de stocker cette matrice ?

Exercice 2.9 ★ — La température (section 2.3.4)

Des scores $z = (2, 1, 0)$. Calculez les probabilités du softmax pour $T = 1$, $T = 0,5$ et $T = 2$. Que valent les limites $T \rightarrow 0$ et $T \rightarrow \infty$? Que devient l'entropie ?

Exercice 2.10 ★★ — Top-k contre top-p (section 2.3.4)

Deux distributions sur dix jetons : (a) $(0,91, 0,02, 0,01, \dots)$ où la masse restante est répartie sur les neuf autres ; (b) quasi uniforme $(0,1$ chacun). Pour chacune, combien de jetons retiennent le top-k avec $k = 5$ et le top-p avec $p = 0,85$? Quel est le défaut du top-k que cela illustre ?

Exercice 2.11 ★★ — Le BPE à la main (section 2.3.2)

Corpus (mot : fréquence) : « bas » 5, « basse » 2, « passe » 6, « passage » 3. En partant des caractères avec marque de fin de mot, effectuez à la main les **trois premières** fusions du BPE (en cas d'égalité, retenez la première paire rencontrée dans l'ordre du corpus) et donnez le découpage de « passe ». Vérifiez avec le code de l'application 2.6.

Exercice 2.12 ★★ — Évaluer une recherche (section 2.4.2)

Une requête a 4 documents pertinents (parmi 1 000). Le moteur renvoie, dans l'ordre, des documents dont la pertinence est : 1, 0, 1, 0, 0, 1, 0, 0, 0, 1. Calculez la précision au rang 3, au rang 5, au rang 10, le rappel au rang 5, et le **rang réciproque** (inverse du rang du premier pertinent). Quelles limites a une moyenne de la précision au rang 5 sur quinze requêtes ?

Exercice 2.13 ★★ — Le vocabulaire qui explose (section 2.4.4)

Mesurez, sur les avis, la **croissance du vocabulaire** : le nombre de mots distincts en fonction du nombre d'avis lus (100, 500, 2 000, 8 000), avec les formes de surface, puis après passage en minuscules et après une racinisation à la hache. Que constatez-vous sur ce corpus ? Pourquoi une langue qui accole des articles et des pronoms aux mots verrait-elle sa courbe monter plus vite ?

Exercice 2.14 ★★ — Évaluer un RAG (section 2.5.3)

Un RAG répond à 100 questions : la recherche place le bon passage au rang 1 pour 70 questions et au rang 2 ou 3 pour 15 ; il ne figure pas dans les trois premiers pour 15. Quand le bon passage est dans le prompt, le modèle répond juste dans 90 % des cas ; quand il ne l'est pas, dans 10 % des cas (par chance ou connaissance générale). Quelle est la part de réponses justes si l'on fournit **un** passage (rang 1) ? Si l'on en fournit **trois** ? Donnez deux raisons de ne pas fournir tous les passages disponibles.

Corrigés

Corrigé 2.1

1. « pas » apparaît dans environ 54 % des avis négatifs contre 34 % des positifs (voir le calcul ci-dessous).
2. Le retrait de « pas » ne change **pas** l'exactitude sur ce corpus (0,944 dans les deux cas) : d'autres indices subsistent dans chaque avis. Cela ne rend pas le retrait anodin ailleurs.
3. « Pas » inverse le sens des mots voisins : sans lui, « pas satisfait » et « satisfait » deviennent identiques.

```
neg = tr[tr["y"] == 0]["texte"].apply(lambda s: "pas" in 0.tokeniser(s)).mean()
pos = tr[tr["y"] == 1]["texte"].apply(lambda s: "pas" in 0.tokeniser(s)).mean()
def exactitude(retrait):
    f = (lambda s: " ".join(w for w in 0.tokeniser(s) if w != "pas")) if retrait else (lambda s: s)
    v = TfidfVectorizer(min_df=2).fit(tr["texte"].map(f))
    m = LogisticRegression(max_iter=3000, C=3).fit(v.transform(tr["texte"].map(f)), tr["y"])
    return m.score(v.transform(tr["texte"].map(f)), tr["y"])
print(f"« pas » dans {neg:.1%} des avis négatifs et {pos:.1%} des positifs | exactitude avec :
↳ {exactitude(False):.3f}, sans : {exactitude(True):.3f}")
```

« pas » dans 54.3% des avis négatifs et 33.8% des positifs | exactitude avec : 0.944, sans : 0.944

Corrigé 2.2

« livraisons » et « livraison » sont regroupés (« livraison »), « rapidement » devient « rapid » et « rapide » reste « rapide » : à **tort** non regroupés ; « chères » devient « chère » et « cher » reste « cher » : non regroupés non plus. La racinisation à la hache est **simple et imparfaite** : elle regroupe bien le pluriel et rate des variantes (« -e », accents).

```
def racine(mot):
    for suf in ("ement", "ions", "es", "s"):
        if mot.endswith(suf) and len(mot) - len(suf) >= 4:
            return mot[: -len(suf)]
    return mot
for m in ["livraisons", "livraison", "rapidement", "rapide", "chères", "cher"]:
    print(f"{m:11s} -> {racine(m)}")
```

```
livraisons -> livraison
livraison  -> livraison
rapidement -> rapid
rapide     -> rapide
chères     -> chère
cher       -> cher
```

Corrigé 2.3

Avec $N = 3$: « alpha » est dans les 3 documents, « beta » dans un seul. Version de cours : $\text{idf}(\text{alpha}) = \ln 1 = 0$, $\text{idf}(\text{beta}) = \ln 3 \approx 1,099$. Version lissée : $\text{idf}(\text{alpha}) = \ln \frac{4}{4} + 1 = 1$, $\text{idf}(\text{beta}) = \ln \frac{4}{2} + 1 \approx 1,693$. La formule lissée garantit qu'un mot présent partout garde un **poids strictement positif** (1), au lieu d'être effacé (0).

```
tv = TfidfVectorizer(norm=None).fit(["alpha beta", "alpha gamma", "alpha delta"])
print(dict(zip(tv.get_feature_names_out(), tv.idf_.round(3))), "| à la main :", round(math.log(4 / 2)
↪ + 1, 3), round(math.log(3), 3))
```

```
{'alpha': np.float64(1.0), 'beta': np.float64(1.693), 'delta': np.float64(1.693), 'gamma':
↪ np.float64(1.693)} | à la main : 1.693 1.099
```

Corrigé 2.4

La part de mots inconnus est **voisine** pour les phrases bien et mal classées (environ 40 % dans les deux cas, un peu moins pour les phrases fausses). L'hypothèse « les erreurs viennent des mots inconnus » ne tient donc pas telle quelle : *toutes* les phrases hors gabarit contiennent beaucoup de mots inconnus ; la phrase est bien classée quand les quelques mots connus portent le bon signe, et mal classée quand ils sont ambigus. Une hypothèse se **teste**, elle ne se suppose pas.

```
connus = set(vec.vocabulary_)
p = modele.predict_proba(vec.transform(0.SONDES))[:, 1]
juste = (p > 0.5) == 0.Y_SONDES
inc = np.array([np.mean([w not in connus for w in 0.tokeniser(s)]) for s in 0.SONDES])
print("part de mots inconnus : phrases justes", inc[juste].mean().round(3), "| phrases fausses",
↪ inc[~juste].mean().round(3))
```

```
part de mots inconnus : phrases justes 0.411 | phrases fausses 0.385
```

Corrigé 2.5

Les scores bruts QK^T : $q_1 = (2, 0)$, $q_2 = (0, 1)$, $k_1 = (1, 0)$, $k_2 = (0, 1)$ donnent $\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$; divisés par

$\sqrt{2}$: $\begin{pmatrix} 1,414 & 0 \\ 0 & 0,707 \end{pmatrix}$. Le softmax ligne à ligne donne $A_{1.} = (0,804, 0,196)$ et $A_{2.} = (0,330, 0,670)$

; la sortie (avec $V = X$) est donc $\begin{pmatrix} 0,804 & 0,196 \\ 0,330 & 0,670 \end{pmatrix}$ (la matrice A elle-même, car $V = I$).

```
X = np.array([[1., 0.], [0., 1.]]) ; Wq = np.diag([2., 1.])
S = (X @ Wq) @ X.T / np.sqrt(2)
A = np.exp(S) / np.exp(S).sum(axis=1, keepdims=True)
print(A.round(3)); print((A @ X).round(3))
```

```
[[0.804 0.196]
 [0.33 0.67 ]]
[[0.804 0.196]
 [0.33 0.67 ]]
```

Corrigé 2.6

1. $q \cdot k = \sum_m q_m k_m$. Les termes $q_m k_m$ sont indépendants, de moyenne $E[q_m]E[k_m] = 0$ et de variance $E[q_m^2 k_m^2] = E[q_m^2]E[k_m^2] = 1$ (indépendance de q_m et k_m). La variance d'une somme de termes indépendants est la somme des variances : $\text{Var}(q \cdot k) = d_k$.
2. Pour $X' = PX : Q' = PXW_Q = PQ$, de même $K' = PK$, $V' = PV$. Alors $Q'K'^T = PQK^T P^T$. Le softmax ligne à ligne commute avec la permutation des lignes **et** des colonnes (il normalise chaque ligne, sur un ensemble de colonnes que P^T ne fait que réordonner) : $\text{softmax}(PSP^T) = P \text{softmax}(S)P^T$. Donc $A'V' = PAP^T PV = PAV$ car $P^T P = I$: la sortie est permutée comme l'entrée.
3. Voir le code.

```
rng = np.random.default_rng(0)
q = rng.standard_normal((200000, 16)); k = rng.standard_normal((200000, 16))
print("variance de q.k :", round(float((q * k).sum(axis=1).var()), 2), "(théorie : 16)")
```

```
variance de q.k : 15.96 (théorie : 16)
```

Corrigé 2.7

Par bloc : $12 \times 768^2 + 13 \times 768 = 7\,077\,888 + 9\,984 = 7\,087\,872$; pour 12 blocs : environ 85,1 M. Les plongements d'entrée : $30\,000 \times 768 = 23,04$ M. Total ≈ 108 M, dont les plongements font environ 21 %.

```
d, L, V = 768, 12, 30000
blocs = L * (12 * d**2 + 13 * d); emb = V * d
print(f"blocs : {blocs / 1e6:.1f} M | plongements : {emb / 1e6:.2f} M | part des plongements : {emb / (blocs + emb):.1%}")
```

```
blocs : 85.1 M | plongements : 23.04 M | part des plongements : 21.3%
```

Corrigé 2.8

La matrice complète coûte $n^2 \times 2$ octets, par tête et par couche, donc $n^2 \times 2 \times 32 \times 40$ octets. Pour $n = 2048$: environ 10,7 Go ; pour $n = 32\,768$: environ **2,7 To**. La mémoire est multipliée par $16^2 = 256$. Les implémentations modernes calculent l'attention **par blocs**, sans jamais écrire la matrice complète en mémoire rapide (et recalculent ce qu'il faut pour la rétropropagation).

```
for n in (2048, 32768):
    print(f"n = {n:6d} : {n * n * 2 * 32 * 40 / 1e9:10.1f} Go")
```

```
n = 2048 : 10.7 Go
n = 32768 : 2748.8 Go
```

Corrigé 2.9

Le softmax de $(2, 1, 0)$: $e^2 = 7,389$, $e^1 = 2,718$, $e^0 = 1$, somme 11,107 : $T = 1$ donne $(0,665, 0,245, 0,090)$. À $T = 0,5$ ($z/T = (4, 2, 0)$) : $(0,867, 0,117, 0,016)$; à $T = 2$ ($(1, 0, 5, 0)$) : $(0,506, 0,307, 0,186)$. Limites : $T \rightarrow 0$ donne la distribution $(1, 0, 0)$ (décodage glouton, entropie 0) ; $T \rightarrow \infty$ donne $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ (entropie maximale $\log_2 3 \approx 1,585$ bits).

```
z = np.array([2., 1., 0.])
for T in (0.01, 0.5, 1, 2, 100):
    pT = 0.softmax(z / T)
    print(f"T={T:<5} p={pT.round(3)} entropie={0.entropie(pT):.3f} bits")
```

```

T=0.01  p=[1. 0. 0.]  entropie=0.000 bits
T=0.5   p=[0.867 0.117 0.016]  entropie=0.636 bits
T=1     p=[0.665 0.245 0.09 ]  entropie=1.201 bits
T=2     p=[0.506 0.307 0.186]  entropie=1.472 bits
T=100   p=[0.337 0.333 0.33 ]  entropie=1.585 bits

```

Corrigé 2.10

- (a) Une distribution concentrée : le top-p 0,85 ne retient **qu'un jeton** ($0,91 \geq 0,85$), alors que le top-k avec $k = 5$ en retient 5, dont quatre quasi nuls. (b) Une distribution plate : le top-p 0,85 retient **9 jetons** ($9 \times 0,1 = 0,9 \geq 0,85$), le top-k avec $k = 5$ n'en retient que 5 et **élimine arbitrairement** des jetons aussi probables que ceux qu'il garde. Le top-k ignore la **forme** de la distribution ; le top-p s'y adapte.

```

pa = np.array([0.91] + [0.09 / 9] * 9); pb = np.full(10, 0.1)
for nom, p_ in (("a) concentrée", pa), ("b) plate", pb)):
    print(nom, "| top-k=5 :", int((0.decoder_probas(np.log(p_), top_k=5) > 0).sum()), "jetons |
    ↪ top-p=0,85 :", int((0.decoder_probas(np.log(p_), top_p=0.85) > 0).sum()), "jetons")

```

```

(a) concentrée | top-k=5 : 5 jetons | top-p=0,85 : 1 jetons
(b) plate | top-k=5 : 5 jetons | top-p=0,85 : 9 jetons

```

Corrigé 2.11

Mots (avec fin) : b a s · (5), b a s s e · (2), p a s s e · (6), p a s s a g e · (3). Paires : (a,s) = 5+2+6+3 = 16, (s,s) = 2+6+3 = 11, (e,·) = 2+6+3 = 11, (p,a) = 9, (s,e) = 8, (b,a) = 7. **Fusion 1** : (a, s), la plus fréquente (16), donne « as ». Après cette fusion : (as, s) = 11, (e, ·) = 11, (p, as) = 9, (s, e) = 8. Il y a égalité entre (as, s) et (e, ·) ; (as, s) est rencontrée en premier (dans « basse »), d'où la **fusion 2** : « ass ». Après elle : (e, ·) = 11 reste la plus fréquente (devant (p, ass) = 9 et (ass, e) = 8) : **fusion 3** : « e· ». Découpage de « passe » : « p », « ass », « e· ».

```

f = apprendre_bpe({"bas": 5, "basse": 2, "passe": 6, "passage": 3}, 3)
print(f, "| passe ->", decouper("passe", f))

```

```

[('a', 's'), ('as', 's'), ('e', '·')] | passe -> ('p', 'ass', 'e·')

```

Corrigé 2.12

Pertinence : 1, 0, 1, 0, 0, 1, 0, 0, 0, 1. Précision au rang 3 = $2/3 \approx 0,667$; au rang 5 = $2/5 = 0,4$; au rang 10 = $4/10 = 0,4$. Rappel au rang 5 = $2/4 = 0,5$. Rang réciproque = $1/1 = 1$. Une moyenne de précision au rang 5 sur 15 requêtes est **très bruitée** (un seul résultat déplace 0,2 par requête, soit 0,013 de la moyenne), dépend de la qualité des jugements de pertinence, ignore l'ordre dans les cinq premiers et ne dit rien du rappel.

```

pert = np.array([1, 0, 1, 0, 0, 1, 0, 0, 0, 1])
print({k: round(pert[:k].mean(), 3) for k in (3, 5, 10)}, "| rappel@5 :", pert[:5].sum() / 4, "| rang
    ↪ réciproque :", 1 / (1 + int(np.argmax(pert))))

```

```

{3: np.float64(0.667), 5: np.float64(0.4), 10: np.float64(0.4)} | rappel@5 : 0.5 | rang réciproque :
    ↪ 1.0

```

Corrigé 2.13

Le vocabulaire grandit **moins vite** que le nombre d'avis (200 formes à 100 avis, 659 à 8 000 : loi de Heaps). Sur ce corpus de gabarits, déjà normalisé, les minuscules et la racinisation ne le réduisent presque pas (659 → 655) ; sur un vrai corpus, l'écart serait bien plus grand. Dans une langue qui accole articles, conjonctions et pronoms aux mots, chaque mot de base apparaît sous de nombreuses formes de surface rares : la **courbe monte plus vite et ne se stabilise pas**, d'où un vocabulaire plus grand, davantage de mots inconnus, et la nécessité de normaliser ou de passer à des sous-mots.

```
rng = np.random.default_rng(0)
ordre = rng.permutation(len(avis))
def croissance(f):
    sortie = {}
    for n in (100, 500, 2000, 8000):
        sortie[n] = len({f(w) for i in ordre[:n] for w in 0.tokeniser(avis["texte"].iloc[i])})
    return sortie
print(pd.DataFrame({"formes de surface": croissance(lambda w: w), "minuscules": croissance(lambda w:
↳ w.lower()), "racinisation": croissance(racine)}).to_string())
```

	formes de surface	minuscules	racinisation
100	200	200	200
500	297	297	295
2000	406	406	404
8000	659	659	655

Corrigé 2.14

Avec **un** passage : $0,70 \times 0,9 + 0,30 \times 0,1 = 0,63 + 0,03 = 0,66$. Avec **trois** : le bon passage est dans le prompt dans $70 + 15 = 85$ cas : $0,85 \times 0,9 + 0,15 \times 0,1 = 0,765 + 0,015 = 0,78$. Fournir trop de passages est mauvais pour deux raisons : le **coût et la latence** augmentent avec la longueur du prompt (la fenêtre de contexte est limitée), et des passages non pertinents **distraient** le modèle et augmentent le risque de réponse fausse (hypothèse que l'on teste en mesurant la part de réponses justes en fonction de k).

```
p1 = 0.70 * 0.9 + 0.30 * 0.1; p3 = 0.85 * 0.9 + 0.15 * 0.1
print("un passage :", round(p1, 3), "| trois passages :", round(p3, 3))
```

```
un passage : 0.66 | trois passages : 0.78
```

Chapitre 3 : Big data et calcul distribué — exercices et applications

Ce chapitre du cahier accompagne le chapitre 3 du livre. Les **applications** refont, pas à pas et avec le code, les études du livre (MapReduce, loi d'Amdahl, plans Spark, asymétrie, journal de messages, fenêtres et filigrane) ; les **exercices** se travaillent d'abord à la main, puis se vérifient par le calcul. Données : un historique de **un million de transactions** simulées de la boutique (fonction `gros_volume` de `build/donnees4.py`, écrite dans un dossier temporaire) et les 8 000 avis clients (`donnees/avis_clients.csv`). Prérequis : le chapitre 3 du livre ; Python avec pandas, PySpark et DuckDB (et un Java récent, nécessaire à Spark).

Applications

Préparation commune

À exécuter une fois. Elle importe les outils, génère le million de transactions dans un dossier temporaire (effacé à la fin du chapitre) et démarre Spark en mode local. Comptez une quinzaine de secondes pour le démarrage de Spark.

```
import os, re, sys, shutil, tempfile, time, zlib, collections, warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from pyspark.sql import SparkSession, functions as F, Window

sys.path.insert(0, "build")
import donnees4

TMP = tempfile.mkdtemp(prefix="v4c3cahier_")
DOSSIER = donnees4.gros_volume(1_000_000, 4, dossier=os.path.join(TMP, "gros_volume"))
spark = (SparkSession.builder.master("local[2]").appName("cahier3")
        .config("spark.ui.enabled", "false").config("spark.sql.shuffle.partitions", "4")
        .config("spark.sql.adaptive.enabled", "false")
        .config("spark.sql.warehouse.dir", TMP + "/entrepot").getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
ventes = spark.read.parquet(DOSSIER)
print("fichiers :", len(os.listdir(DOSSIER)), "; lignes :", ventes.count(), "; partitions :",
      ↪ ventes.rdd.getNumPartitions())

def plan(df):
    ↪ chemins
    t = df._jdf.queryExecution().executedPlan().toString()
    t = re.sub(r"#\d+L?", "", t)
```

plan physique, sans identifiants internes ni

```

t = re.sub(r", \[plan_id=\d+\]", "", t)
t = re.sub(
↪ r"(Location|Batched|ReadSchema|PushedFilters|PartitionFilters|DataFilters|Format|InputPaths)(^\n)*"
↪ , "", t)
return "\n".join(l.rstrip() for l in t.splitlines() if l.strip())

```

```
fichiers : 4 ; lignes : 1000000 ; partitions : 2
```

Application 3.1 — Un MapReduce complet sur les avis (section 3.1.4)

Contexte. La gérante veut la **note moyenne par catégorie de produit** sur les 8 000 avis. **Objectif.** Écrire les trois phases (map, mélange, reduce) à la main, avec un combineur, et mesurer ce que le combineur épargne au réseau.

Étape 1 — Découper en partitions et appliquer la fonction map. Quatre partitions, comme quatre machines ; chaque machine émet des paires (catégorie, (note, 1)).

```

avis = pd.read_csv("donnees/avis_clients.csv")
lignes = list(zip(avis["categorie"], avis["note"]))
partitions = [lignes[i::4] for i in range(4)]

def map_partition(lignes):
    return [(cat, (note, 1)) for cat, note in lignes]

paires = [map_partition(p) for p in partitions]
print("partitions :", [len(p) for p in paires], "; paires émises au total :", sum(len(p) for p in
↪ paires))

```

```
partitions : [2000, 2000, 2000, 2000] ; paires émises au total : 8000
```

Étape 2 — Le combineur. Avant le mélange, chaque machine additionne localement ses paires de même catégorie.

```

def combiner(paires):
    local = {}
    for cat, (s, n) in paires:
        a, b = local.get(cat, (0, 0))
        local[cat] = (a + s, b + n)
    return list(local.items())

combinees = [combiner(p) for p in paires]
print("paires à mélanger sans combineur :", sum(len(p) for p in paires))
print("paires à mélanger avec combineur :", sum(len(p) for p in combinees))

```

```

paires à mélanger sans combineur : 8000
paires à mélanger avec combineur : 16

```

Étape 3 — Le mélange. On envoie chaque paire à la machine hachage(catégorie) mod 3 (trois machines *reduce*). On utilise `zlib.crc32`, un hachage **stable** (celui de Python, hash, change d'une exécution à l'autre pour les chaînes).

```

reducteurs = [collections.defaultdict(list) for _ in range(3)]
for p in combinees:
    for cat, valeur in p:
        reducteurs[zlib.crc32(cat.encode()) % 3][cat].append(valeur)
for i, r in enumerate(reducteurs):
    print("machine reduce", i, ":", sorted(r))

```

```

machine reduce 0 : []
machine reduce 1 : ['B']

```

```
machine reduce 2 : ['A', 'C', 'D']
```

Étape 4 — Reduce, puis vérification.

```
moyennes = {}
for r in reduceurs:
    for cat, valeurs in r.items():
        moyennes[cat] = sum(s for s, _ in valeurs) / sum(n for _, n in valeurs)
mr = pd.Series(moyennes).sort_index().round(3)
ref = avis.groupby("categorie")["note"].mean().sort_index().round(3)
print(mr.to_string())
print("identique à pandas :", bool(np.allclose(mr, ref)))
```

```
A    3.828
B    3.841
C    3.871
D    3.838
identique à pandas : True
```

À retenir. Le résultat est **exactement** celui de pandas, et le combineur a réduit le nombre de paires qui voyagent. Il fonctionne parce que la somme et le comptage sont **associatifs**. Pour une **moyenne**, on ne peut pas combiner des moyennes (la moyenne des moyennes est fausse si les morceaux n'ont pas la même taille) : on combine des **couples (somme, effectif)**, puis on divise à la fin.

Application 3.2 — La loi d'Amdahl sur votre machine (section 3.1.5)

Contexte. On veut savoir **quelle part de son calcul est parallélisable**, et prédire le gain de machines supplémentaires. **Objectif.** Mesurer des durées avec 1, 2 et 4 processus, en déduire p , puis extrapoler.

Étape 1 — Mesurer sur votre ordinateur. Ce bloc n'est pas exécuté ici : les durées dépendent de la machine. Lancez-le chez vous (dans un fichier Python, car multiprocessing demande que les fonctions soient définies dans un fichier).

```
import time
from multiprocessing import Pool

def travail(n):
    return sum(i * i % 7 for i in range(n)) # un calcul qui occupe le processeur

if __name__ == "__main__":
    morceaux = [3_000_000] * 8
    for k in (1, 2, 4):
        t0 = time.perf_counter()
        with Pool(k) as pool:
            pool.map(travail, morceaux)
        print(k, "processus :", round(time.perf_counter() - t0, 2), "s")
```

Étape 2 — Estimer p à partir de durées. Supposons que vous ayez mesuré 120 s avec 1 processus, 70 s avec 2 et 45 s avec 4. De $S(n) = T(1)/T(n) = 1/[(1-p) + p/n]$ on tire $p = \frac{1 - 1/S(n)}{1 - 1/n}$.

```
T = {1: 120.0, 2: 70.0, 4: 45.0}
for n in (2, 4):
    S = T[1] / T[n]
    p = (1 - 1 / S) / (1 - 1 / n)
    print(f"n = {n} : accélération {S:.2f} ; fraction parallélisable p = {p:.3f}")
```

```
n = 2 : accélération 1.71 ; fraction parallélisable p = 0.833
n = 4 : accélération 2.67 ; fraction parallélisable p = 0.833
```

Étape 3 — Prédire. Avec le p estimé (il est le même pour $n = 2$ et $n = 4$, ce qui conforte le modèle), que donneraient 8, 16 et 64 machines ? Et quel est le plafond ?

```
p = 5 / 6
for n in (8, 16, 64):
    print(f"n = {n:2d} : accélération prédite {1 / ((1 - p) + p / n):.2f}")
print("plafond :", round(1 / (1 - p), 2))
```

```
n = 8 : accélération prédite 3.69
n = 16 : accélération prédite 4.57
n = 64 : accélération prédite 5.57
plafond : 6.0
```

À retenir. Un p cohérent pour plusieurs valeurs de n valide le modèle ; s'il diminue quand n augmente, c'est que la **communication** pèse (la loi d'Amdahl est alors optimiste). Ici, passer de 16 à 64 machines ne gagne presque rien : le plafond est de 6.

Application 3.3 — Lire des plans d'exécution et choisir sa jointure (sections 3.2.4 et 3.2.7)

Contexte. Un calcul est lent ; avant de toucher au code, on lit son plan. **Objectif.** Compter les mélanges (Exchange) de quatre requêtes et en tirer une règle.

Étape 1 — Un compteur de mélanges.

```
def melanges(df):
    t = plan(df)
    return t.count("Exchange hashpartitioning") + t.count("Exchange rangepartitioning") +
        t.count("Exchange SinglePartitioning")

q1 = ventes.filter(F.col("montant") > 10).select("canal", "montant")
q2 = ventes.groupBy("canal").agg(F.sum("montant"))
q3 = ventes.groupBy("canal").agg(F.sum("montant").alias("ca")).orderBy("ca")
q4 = ventes.groupBy("magasin",
    "canal").agg(F.sum("montant").alias("s")).groupBy("canal").agg(F.sum("s"))
for nom, q in [("filtre + projection", q1), ("groupBy", q2), ("groupBy + orderBy", q3), ("deux
    groupBy", q4)]:
    print(f"{nom:22s} : {melanges(q)} mélange(s)")
```

```
filtre + projection      : 0 mélange(s)
groupBy                 : 1 mélange(s)
groupBy + orderBy       : 2 mélange(s)
deux groupBy            : 2 mélange(s)
```

Étape 2 — Les jointures. On joint les transactions à un catalogue de 500 produits, avec et sans diffusion.

```
catalogue = spark.createDataFrame(pd.DataFrame({"id_produit": np.arange(1, 501, dtype=np.int32),
    "categorie": np.array(["Cuisine", "Maison", "Jardin",
        "Loisirs"])[np.arange(500) % 4])})
avec = ventes.join(F.broadcast(catalogue), "id_produit").groupBy("categorie").agg(F.sum("montant"))
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "-1")
sans = ventes.join(catalogue, "id_produit").groupBy("categorie").agg(F.sum("montant"))
print("avec diffusion :", melanges(avec), "mélange(s) ;", "BroadcastHashJoin in plan(avec)")
print("sans diffusion :", melanges(sans), "mélange(s) ;", "SortMergeJoin in plan(sans)")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "10485760")
```

```
avec diffusion : 1 mélange(s) ; True
sans diffusion : 3 mélange(s) ; True
```

Étape 3 — Même résultat ? Les deux stratégies doivent donner les mêmes totaux.

```
a = avec.orderBy("categorie").toPandas()
b = sans.orderBy("categorie").toPandas()
print("mêmes totaux par catégorie :", bool(np.allclose(a.iloc[:, 1], b.iloc[:, 1])))
```

```
mêmes totaux par catégorie : True
```

À retenir. Un filtre ou une projection **ne mélange pas** ; un `groupBy` ou un `orderBy` mélange une fois chacun ; une jointure sans diffusion en ajoute deux. La diffusion change le **coût** d'une jointure, jamais son **résultat**.

Application 3.4 — Asymétrie des clés et salage (section 3.2.9)

Contexte. Un revendeur (client numéro 1) concentre 30 % des transactions. **Objectif.** Mesurer le déséquilibre, le corriger par le salage, et vérifier que l'agrégation finale est inchangée.

Étape 1 — Créer l'asymétrie et la mesurer.

```
asym = ventes.withColumn("cle", F.when(F.rand(7) < 0.3, F.lit(1)).otherwise(F.col("id_client")))

def par_partition(df):
    t = df.withColumn("p", F.spark_partition_id()).groupBy("p").count().orderBy("p").toPandas()
    return t["count"].to_numpy()

avant = par_partition(asym.repartition(4, "cle"))
print("lignes par partition :", avant.tolist(), "; plus grosse / moyenne :", round(avant.max() /
↳ avant.mean(), 2))
```

```
lignes par partition : [174612, 475813, 174050, 175525] ; plus grosse / moyenne : 1.9
```

Étape 2 — Saler. On ajoute un grain de sel entre 0 et 31 et on répartit par la paire (clé, sel).

```
sel = F.floor(F.rand(8) * 32).cast("int")
sale = asym.withColumn("sel", sel)
apres = par_partition(sale.repartition(4, "cle", "sel"))
print("lignes par partition :", apres.tolist(), "; plus grosse / moyenne :", round(apres.max() /
↳ apres.mean(), 2))
```

```
lignes par partition : [287706, 259473, 221620, 231201] ; plus grosse / moyenne : 1.15
```

Étape 3 — Agréger en deux temps. On somme d'abord par (clé, sel), puis on réagrège par clé. Le résultat doit être identique à l'agrégation directe.

```
direct = asym.groupBy("cle").agg(F.round(F.sum("montant"), 2).alias("ca"))
en_deux_temps = (sale.groupBy("cle", "sel").agg(F.sum("montant").alias("s"))
    .groupBy("cle").agg(F.round(F.sum("s"), 2).alias("ca")))
d = direct.orderBy("cle").toPandas()
e = en_deux_temps.orderBy("cle").toPandas()
print("clés :", len(d), "; mêmes totaux :", bool(np.allclose(d["ca"], e["ca"])))
print("part de la clé 1 dans le chiffre d'affaires :", round(float(d.loc[d["cle"] == 1, "ca"].iloc[0]
↳ / d["ca"].sum()), 3))
```

```
clés : 194037 ; mêmes totaux : True
part de la clé 1 dans le chiffre d'affaires : 0.301
```

À retenir. Le salage répartit la charge ; l'agrégation en deux temps rend le **même résultat** parce

que la somme est associative. Il n'a de sens que pour des opérations qu'on sait **recombinaison** ; il est inutile, voire nuisible, si aucune clé n'est dominante.

Application 3.5 — Un journal de messages et un consommateur idempotent (section 3.3.3)

Contexte. Les ventes arrivent dans un journal de type Kafka. **Objectif.** Se répartir les partitions entre deux consommateurs, subir un plantage, et obtenir tout de même des totaux justes.

Étape 1 — Le journal et les messages. Chaque vente porte un identifiant unique, et la clé est le magasin.

```
class Journal:
    def __init__(self, partitions):
        self.partitions = [[] for _ in range(partitions)]
    def publier(self, cle, valeur):
        p = zlib.crc32(str(cle).encode()) % len(self.partitions)
        self.partitions[p].append((cle, valeur))
        return p, len(self.partitions[p]) - 1
    def lire(self, p, decalage, maximum=10_000):
        return self.partitions[p][decalage:decalage + maximum]

journal = Journal(4)
echantillon = ventes.select("id_transaction", "magasin",
    ↪ "montant").orderBy("id_transaction").limit(2000).toPandas()
for ident, magasin, montant in echantillon.itertuples(index=False):
    journal.publier(magasin, (int(ident), float(montant)))
print("messages par partition :", [len(p) for p in journal.partitions])
```

```
messages par partition : [404, 796, 421, 379]
```

Étape 2 — Deux consommateurs, un groupe. Chacun lit les partitions dont le numéro a la parité de son rang. Le consommateur 0 **plante** après avoir traité 30 messages de sa première partition, **avant** de noter son décalage ; au redémarrage, il relit depuis le dernier décalage noté (0).

```
def consommer(partitions, plante_apres=None):
    """Retourne les messages traités (avec doublons éventuels)."""
    traites = []
    for p in partitions:
        lus = [(cle, ident, montant) for cle, (ident, montant) in journal.lire(p, 0)]
        if plante_apres and p == partitions[0]:
            traites += lus[:plante_apres] # traités, mais le décalage n'a pas été noté avant
    ↪ le plantage...
        traites += lus # ... donc au redémarrage tout est relu depuis le
    ↪ décalage noté (0)
    return traites

c0 = consommer([0, 2], plante_apres=30)
c1 = consommer([1, 3])
tous = c0 + c1
print("messages traités :", len(tous), "; messages publiés :", len(echantillon))
```

```
messages traités : 2030 ; messages publiés : 2000
```

Étape 3 — Naïf, puis idempotent.

```
attendu = echantillon.groupby("magasin")["montant"].sum().round(2)
naif = pd.DataFrame(tous, columns=["magasin", "id",
    ↪ "montant"]).groupby("magasin")["montant"].sum().round(2)
idem = pd.DataFrame(tous, columns=["magasin", "id",
    ↪ "montant"]).drop_duplicates("id").groupby("magasin")["montant"].sum().round(2)
print("naïf juste :", bool(np.allclose(naif, attendu)), "; idempotent juste :",
    ↪ bool(np.allclose(idem, attendu)))
```

```
naïf juste : False ; idempotent juste : True
```

À retenir. Un plantage entre le traitement et la note du décalage produit des **doublons** (garantie « au moins une fois »). La parade est un identifiant unique et un traitement **idempotent**. Ici, `drop_duplicates("id")` joue ce rôle ; en production, on retient les identifiants déjà vus (dans une base, avec une durée de rétention) ou on écrit avec une clé qui écrase au lieu d'additionner.

Application 3.6 — Fenêtres et filigrane (sections 3.3.4 et 3.3.5)

Contexte. Des évènements arrivent dans le désordre. **Objectif.** Compter par fenêtres fixes, glissantes et de session, puis mesurer **ce que coûte un filigrane trop serré**.

Étape 1 — Les évènements. Quatre cents évènements simulés avec une graine fixe : heure de l'évènement en secondes, montant, et **retard d'arrivée** (la plupart arrivent à l'heure, quelques-uns avec un retard allant jusqu'à deux minutes).

```
rng = np.random.default_rng(3)
n = 400
heure = np.sort(rng.uniform(0, 1800, n))                # 30 minutes d'activité
retard = np.where(rng.random(n) < 0.1, rng.uniform(30, 120, n), rng.uniform(0, 5, n))
montant = rng.gamma(2.0, 20.0, n).round(2)
ev = pd.DataFrame({"heure": heure, "montant": montant, "arrivee": heure +
    ↳ retard}).sort_values("arrivee").reset_index(drop=True)
print("évènements :", len(ev), "; arrivés après un évènement plus récent :",
    ↳ int((ev["heure"].cummax() > ev["heure"]).sum()))
```

```
évènements : 400 ; arrivés après un évènement plus récent : 92
```

Étape 2 — Fenêtres fixes et glissantes (calcul par lots, exact).

```
fixes = ev.groupby((ev["heure"] // 300 * 300).astype(int))["montant"].agg(["count", "sum"]).round(1)
print(fixes.to_string())
glissantes = {d: ev[(ev["heure"] >= d) & (ev["heure"] < d + 300)]["montant"].sum() for d in range(0,
    ↳ 1800 - 299, 150)}
print("glissantes (300 s toutes les 150 s) :", {d: round(v) for d, v in glissantes.items()})
```

	count	sum
heure		
0	58	2410.4
300	68	2803.3
600	65	2580.9
900	75	2875.7
1200	73	3316.6
1500	61	2259.0

```
glissantes (300 s toutes les 150 s) : {0: 2410, 150: 3257, 300: 2803, 450: 2645, 600: 2581, 750: 2399,
    ↳ 900: 2876, 1050: 3571, 1200: 3317, 1350: 2322, 1500: 2259}
```

Étape 3 — Le filigrane. On traite les évènements dans l'ordre d'arrivée ; une fenêtre fixe de 300 s est fermée quand le filigrane (plus grand temps d'évènement vu, moins D) dépasse sa fin.

```
def avec_filigrane(ev, D, largeur=300):
    plus_recent, ecartes, total = 0.0, 0.0, 0.0
    for h, m in zip(ev["heure"], ev["montant"]):
        if (h // largeur) * largeur + largeur <= plus_recent - D:
            ecartes += m                                # fenêtre déjà fermée : évènement perdu
        else:
            total += m
            plus_recent = max(plus_recent, h)
    return total, ecartes
```

```
for D in (0, 10, 30, 60, 120):
    garde, perdu = avec_filigrane(ev, D)
    print(f"D = {D:3d} s : montant conservé {garde:8.1f} ; perdu {perdu:6.1f} ({perdu /
    ↪ ev['montant'].sum():.1%})")
```

```
D = 0 s : montant conservé 15827.5 ; perdu 418.4 (2.6%)
D = 10 s : montant conservé 16097.6 ; perdu 148.4 (0.9%)
D = 30 s : montant conservé 16153.4 ; perdu 92.6 (0.6%)
D = 60 s : montant conservé 16190.4 ; perdu 55.5 (0.3%)
D = 120 s : montant conservé 16246.0 ; perdu 0.0 (0.0%)
```

Étape 4 — Le coût du filigrane en latence. Une fenêtre de 300 s ne peut être publiée que lorsque le filigrane dépasse sa fin, c'est-à-dire ***D* secondes après sa fin au plus tôt**.

```
for D in (0, 30, 120):
    print(f"D = {D:3d} s : le résultat de la fenêtre [0, 300) n'est définitif qu'à l'instant {300 + D}
    ↪ s (temps d'évènement)")
```

```
D = 0 s : le résultat de la fenêtre [0, 300) n'est définitif qu'à l'instant 300 s (temps d'évènement)
D = 30 s : le résultat de la fenêtre [0, 300) n'est définitif qu'à l'instant 330 s (temps d'évènement)
D = 120 s : le résultat de la fenêtre [0, 300) n'est définitif qu'à l'instant 420 s (temps d'évènement)
```

À retenir. Plus le filigrane est large, moins on perd, mais plus on attend : la perte tombe à zéro quand *D* dépasse le retard maximal (ici 120 s). Le bon *D* se choisit en regardant la **distribution des retards** observés, pas à l'instinct.

Exercices

Exercice 3.1 ★ — Quelle mémoire pour quelle table ? (section 3.1.1)

Une ligne de transaction occupe 65 octets en mémoire. (a) Quelle mémoire occuperaient 400 millions de lignes ? Tiennent-elles dans un ordinateur de 16 Go ? (b) On n'a besoin que de la colonne montant (8 octets par valeur) : qu'en est-il ?

Exercice 3.2 ★ — Partitionner par hachage (section 3.1.3)

Sept clients ont pour numéros 12, 25, 33, 40, 58, 61, 77. On les répartit sur quatre machines (numérotées 0 à 3) par la règle « numéro modulo 4 ». (a) Quelle machine reçoit chaque client ? (b) Quelle est la machine la plus chargée, et de combien dépasse-t-elle la moyenne ?

Exercice 3.3 ★★ — Un combineur à la main (section 3.1.4)

Trois machines lisent les phrases « a b a », « b c » et « a a c » et comptent les mots. (a) Écrivez les paires émises par la phase map. (b) Combien de paires traversent le réseau sans combineur ? avec combineur ? (c) Donnez le résultat final.

Exercice 3.4 ★ — Appliquer la loi d'Amdahl (section 3.1.5)

Un traitement a 80 % de travail parallélisable. Calculez l'accélération avec 4 et 16 machines, puis le plafond.

Exercice 3.5 ★★ — Retrouver la fraction parallélisable (section 3.1.5)

Avec 8 machines, un traitement est 4 fois plus rapide. (a) Quelle fraction p est parallélisable ? (b) Quel est le plafond ? (c) Combien de machines faut-il pour une accélération de 5 ?

Exercice 3.6 ★★ — Combien de copies ? (section 3.1.7)

Chaque machine tombe en panne, indépendamment, avec la probabilité $q = 0,02$ sur une période. (a) Probabilité de perdre un bloc copié sur 3 machines ? (b) Combien de copies pour descendre sous 10^{-9} ? (c) Sur 1 000 machines, combien tombent en panne en moyenne ?

Exercice 3.7 ★ — Compter les étapes d'un plan (section 3.2.4)

Voici un plan simplifié (de bas en haut, comme dans le livre) :

```
Sort ca DESC
+- Exchange rangepartitioning(ca)
  +- HashAggregate (final) : somme par magasin
    +- Exchange hashpartitioning(magasin)
      +- HashAggregate (partial)
        +- Filter (montant > 0)
          +- Scan parquet [magasin, montant]
```

(a) Combien de mélanges ? (b) En combien d'étapes (*stages*) Spark découpe-t-il ce calcul ? (c) Quelles opérations sont étroites ?

Exercice 3.8 ★★ — Étroit ou large ? (section 3.2.4)

Classez en transformations étroites ou larges : `select`, `filter`, `withColumn`, `groupBy().agg()`, jointure par diffusion (côté gros tableau), jointure par tri et mélange, `distinct`, `orderBy`, `union`, `coalesce`, `repartition`, fonction fenêtre avec `partitionBy`.

Exercice 3.9 ★★ — Combien de données voyagent ? (section 3.2.7)

Un tableau de 2 millions de lignes de 65 octets est joint à un catalogue de 500 lignes de 20 octets, sur une grappe de 10 exécutants. Estimez la quantité de données transmises (a) par la jointure par diffusion, (b) par la jointure par tri et mélange (on suppose que les deux tableaux sont intégralement mélangés). Quel est le rapport ?

Exercice 3.10 ★★ — Dimensionner les partitions (section 3.2.6)

On traite 50 Go sur 5 exécutants de 4 cœurs. (a) Combien de partitions de 128 Mo ? (b) Combien par cœur ? (c) Combien de « vagues » de tâches faut-il pour tout traiter ?

Exercice 3.11 ★★ — L'asymétrie en chiffres (section 3.2.9)

Un million de lignes sont réparties en 4 partitions par hachage de la clé ; une clé unique représente 30 % des lignes, les autres sont uniformes. (a) Combien de lignes dans la plus grosse partition ? Quel rapport à la moyenne ? (b) Même question avec 100 partitions. (c) Que conclure sur l'effet d'ajouter des machines ?

Exercice 3.12 ★★★ — Une médiane ne se combine pas (sections 3.1.4 et 3.2)

- (a) Deux partitions contiennent $\{1, 2, 3, 100\}$ et $\{4, 5, 6\}$. Comparez la médiane exacte à la **moyenne des médianes** des partitions. (b) Sur les montants du million de transactions, comparez la médiane exacte à la médiane approchée de Spark (`percentile_approx`).

Exercice 3.13 ★★ — Petits fichiers (section 3.2.10)

Un `DataFrame` de 40 partitions est écrit avec `partitionBy("canal")` (3 valeurs). (a) Combien de fichiers au plus ? (b) Même question après `coalesce(4)`. (c) Si l'ouverture d'un fichier coûte 50 ms de frais fixes, quel est le coût fixe total dans chaque cas ?

Exercice 3.14 ★ — Partitions et décalages d'un journal (section 3.3.3)

Six messages de clés 7, 8, 9, 10, 13, 16 sont publiés dans cet ordre dans un sujet à trois partitions, avec la règle « clé modulo 3 ». (a) Pour chaque message, donnez (partition, décalage). (b) Un consommateur lit la partition 1 à partir du décalage 2 : que voit-il ? (c) Quel est le défaut de ce partitionnement ?

Exercice 3.15 ★★ — Garanties de livraison (section 3.3.3)

Un consommateur doit traiter dix messages. Il plante **une fois**, pendant le traitement du sixième, puis redémarre. Combien de messages sont traités au total (comptés avec leurs répétitions), et combien manquent, selon que le décalage est noté (a) **avant** le traitement ou (b) **après** ? Quelle garantie chaque ordre réalise-t-il ?

Exercice 3.16 ★★ — Fenêtres fixes et glissantes (section 3.3.4)

Des événements surviennent aux secondes 5, 12, 25, 31, 44 et 58. (a) Comptez-les par fenêtres fixes de 20 s. (b) Comptez-les par fenêtres glissantes de 20 s avancées de 10 s (à partir de la fenêtre $[-10, 10)$). (c) Vérifiez que le nombre total d'appartenances est le nombre d'événements multiplié par 2.

Exercice 3.17 ★★ — Le filigrane à la main (section 3.3.5)

Des événements arrivent dans l'ordre suivant, repérés par leur temps d'évènement (en secondes) : 10, 50, 90, 30, 130, 70, 20. Les fenêtres fixes durent 60 s ; le retard toléré est $D = 40$ s. (a) Quels événements sont écartés ? (b) Quelle valeur minimale de D aurait conservé tous les événements ?

Exercice 3.18 ★★★ — Fenêtres de session (section 3.3.4)

Les événements d'un internaute surviennent aux secondes 4, 10, 15, 40, 44 et 90. Une session se ferme après un silence d'au moins 20 s. (a) Combien de sessions, et lesquelles ? (b) Quelle est la durée de chacune (du premier au dernier événement) ? (c) Écrivez une fonction qui le calcule pour une liste d'heures quelconque.

Corrigés

Corrigé 3.1

- (a) $400 \times 10^6 \times 65 = 26 \times 10^9$ octets, soit **26 Go : non**, cela ne tient pas dans 16 Go. (b) Avec la seule colonne montant : $400 \times 10^6 \times 8 = 3,2$ Go, qui tiennent sans difficulté. **Leçon** : lire seulement les colonnes utiles (stockage en colonnes, section 3.1.9) peut suffire à éviter de distribuer.

```
print("toutes les colonnes :", 400e6 * 65 / 1e9, "Go ; colonne montant seule :", 400e6 * 8 / 1e9, "Go")
```

```
toutes les colonnes : 26.0 Go ; colonne montant seule : 3.2 Go
```

Corrigé 3.2

- (a) $12 \bmod 4 = 0$, $25 \bmod 4 = 1$, $33 \bmod 4 = 1$, $40 \bmod 4 = 0$, $58 \bmod 4 = 2$, $61 \bmod 4 = 1$, $77 \bmod 4 = 1$. (b) La machine 1 reçoit les clients 25, 33, 61 et 77, soit **4 sur 7**. La moyenne est $7/4 = 1,75$; le rapport est $4/1,75 \approx 2,29$: la machine 1 a **2,3 fois la charge moyenne**. Avec si peu de clés, le hasard suffit à déséquilibrer.

```
clients = [12, 25, 33, 40, 58, 61, 77]
charge = collections.Counter(c % 4 for c in clients)
print(dict(sorted(charge.items())), "; plus grosse / moyenne :", round(max(charge.values()) /
↳ (len(charge) / 4), 2))
```

```
{0: 2, 1: 4, 2: 1} ; plus grosse / moyenne : 2.29
```

Corrigé 3.3

- (a) Map : machine 1 : (a,1) (b,1) (a,1) ; machine 2 : (b,1) (c,1) ; machine 3 : (a,1) (a,1) (c,1).
 (b) **Sans combineur**, les 8 paires traversent le réseau. **Avec combineur** : machine 1 envoie (a,2) (b,1) ; machine 2 : (b,1) (c,1) ; machine 3 : (a,2) (c,1) : **6 paires**, soit 25 % de moins.
 (c) Reduce : $a = 2 + 2 = 4$, $b = 1 + 1 = 2$, $c = 1 + 1 = 2$; total 8 mots, comme les 8 mots d'entrée.

```
entree = ["a b a", "b c", "a a c"]
partiel = [collections.Counter(p.split()) for p in entree]
print("paires sans combineur :", sum(len(p.split()) for p in entree), "; avec combineur :", sum(len(c)
↳ for c in partiel))
print("résultat :", dict(sum(partiel, collections.Counter())))
```

```
paires sans combineur : 8 ; avec combineur : 6
résultat : {'a': 4, 'b': 2, 'c': 2}
```

Corrigé 3.4

$S(4) = \frac{1}{0,2 + 0,8/4} = \frac{1}{0,4} = 2,5$; $S(16) = \frac{1}{0,2 + 0,05} = 4$; plafond = $\frac{1}{0,2} = 5$. Quadrupler les machines (de 4 à 16) ne fait gagner que 60 %.

```
amdahl = lambda p, n: 1 / ((1 - p) + p / n)
print(round(amdahl(0.8, 4), 2), round(amdahl(0.8, 16), 2), round(1 / (1 - 0.8), 2))
```

```
2.5 4.0 5.0
```

Corrigé 3.5

- (a) $\frac{1}{4} = (1-p) + \frac{p}{8} = 1 - \frac{7p}{8}$, donc $p = \frac{8}{7} \times \frac{3}{4} = \frac{6}{7} \approx 0,857$. (b) Plafond = $\frac{1}{1-p} = 7$. (c) Pour $S = 5$: $\frac{1}{5} = \frac{1}{7} + \frac{6/7}{n}$, donc $\frac{6/7}{n} = \frac{1}{5} - \frac{1}{7} = \frac{2}{35}$ et $n = \frac{6}{7} \times \frac{35}{2} = 15$. **Quinze machines** pour passer de 4 à 5 : le rendement est faible.

```
p = 6 / 7
print(round(p, 3), round(1 / (1 - p), 2), round(amdahl(p, 8), 2), round(amdahl(p, 15), 2))
```

```
0.857 7.0 4.0 5.0
```

Corrigé 3.6

- (a) $q^3 = 0,02^3 = 8 \times 10^{-6}$. (b) On veut $0,02^k \leq 10^{-9}$, soit $k \geq \frac{9}{-\log_{10} 0,02} = \frac{9}{1,699} \approx 5,3$: **6 copies**. (c) $1000 \times 0,02 = 20$ machines en panne en moyenne : à cette échelle, **les pannes sont la norme**. Le calcul suppose des pannes **indépendantes** : une coupure d'alimentation d'une baie de serveurs touche plusieurs copies à la fois, ce pourquoi on place les copies dans des endroits différents.

```
print(f"{0.02 ** 3:.1e}", int(np.ceil(9 / -np.log10(0.02))), 1000 * 0.02)
```

```
8.0e-06 6 20.0
```

Corrigé 3.7

- (a) **Deux** mélanges : Exchange hashpartitioning(magasin) pour l'agrégation, Exchange rangepartitioning(ca) pour le tri. (b) **Trois étapes** : on coupe à chaque Exchange (lecture-filtre-agrégation partielle ; agrégation finale ; tri). (c) Sont étroits : le Scan, le Filter, l'agrégation **partielle** (qui travaille sur place) ; l'agrégation finale et le tri ne démarrent qu'une fois le mélange terminé.

Corrigé 3.8

Étroites : select, filter, withColumn, jointure par diffusion (côté gros tableau : il ne bouge pas), union, coalesce (fusionne des partitions voisines sans mélange). **Larges** : groupBy().agg(), jointure par tri et mélange, distinct, orderBy, repartition, fonction fenêtre avec partitionBy.

Corrigé 3.9

- (a) **Diffusion** : le catalogue pèse $500 \times 20 = 10$ ko ; il est envoyé à chacun des 10 exécutants : $10 \times 10 = 100$ ko. (b) **Tri et mélange** : tout le gros tableau est mélangé, $2 \times 10^6 \times 65 = 130$ Mo, plus 10 ko. (c) Le rapport est $130 \text{ Mo} / 100 \text{ ko} = 1300$: la diffusion transmet **mille trois cents fois moins** de données.

```
print("diffusion :", 500 * 20 * 10, "octets ; mélange :", 2_000_000 * 65 + 500 * 20, "octets ; rapport  
↪ :", round((2_000_000 * 65 + 500 * 20) / (500 * 20 * 10)))
```

```
diffusion : 100000 octets ; mélange : 130010000 octets ; rapport : 1300
```

Corrigé 3.10

- (a) $50\text{ Go} = 50\,000\text{ Mo}$; $50\,000/128 \approx 390,6$, donc **391 partitions**. (b) La grappe a $5 \times 4 = 20$ cœurs : $391/20 \approx 19,6$ partitions par cœur. (c) Chaque cœur traite une tâche à la fois : il faut $\lceil 391/20 \rceil = 20$ vagues. Plus de quatre partitions par cœur n'est pas un défaut : c'est la **taille** des morceaux (assez petits pour tenir en mémoire, assez gros pour que l'organisation reste négligeable) qui décide.

```
n = int(np.ceil(50_000 / 128)); print(n, round(n / 20, 1), int(np.ceil(n / 20)))
```

```
391 19.6 20
```

Corrigé 3.11

- (a) La clé dominante pèse 300 000 lignes ; les 700 000 autres se répartissent uniformément : $700\,000/4 = 175\,000$ par partition. La plus grosse a $300\,000 + 175\,000 = 475\,000$ lignes ; la moyenne est 250 000 ; le rapport est **1,9**. (b) Avec 100 partitions : $300\,000 + 7\,000 = 307\,000$ pour une moyenne de 10 000 : le rapport est **30,7**. (c) **Ajouter des machines aggrave le déséquilibre** : la partition de la clé dominante ne rétrécit pas (elle contient au moins 300 000 lignes), alors que la moyenne diminue. La durée du calcul reste celle de cette partition : c'est le plafond d'Amdahl dans sa version « une tâche surchargée », et la raison d'être du salage.

```
for k in (4, 100):
    biggest = 300_000 + 700_000 / k
    print(k, "partitions :", int(biggest), "lignes ; rapport", round(biggest / (1_000_000 / k), 1))
```

```
4 partitions : 475000 lignes ; rapport 1.9
100 partitions : 307000 lignes ; rapport 30.7
```

Corrigé 3.12

- (a) La médiane exacte de $\{1, 2, 3, 4, 5, 6, 100\}$ est **4**. Les médianes des partitions valent 2,5 (de $\{1, 2, 3, 100\}$) et 5 ; leur moyenne est 3,75 : **fausse**, parce que la médiane n'est **pas associative** : on ne peut pas la reconstituer à partir de médianes partielles. (b) Il faut un autre algorithme : on résume chaque partition par un **croquis** (histogramme, quantiles approchés), puis on combine les résumés. C'est ce que fait `percentile_approx`, avec une erreur contrôlée.

```
a, b = [1, 2, 3, 100], [4, 5, 6]
print("exacte :", np.median(a + b), "; moyenne des médianes :", (np.median(a) + np.median(b)) / 2)
exacte = float(ventes.toPandas()["montant"].median())
approx = ventes.agg(F.percentile_approx("montant", 0.5, 10000)).first()[0]
print("médiane exacte :", round(exacte, 2), "; approchée :", round(approx, 2), "; écart relatif < 1 %",
      "\u2192 :", abs(approx - exacte) / exacte < 0.01)
```

```
exacte : 4.0 ; moyenne des médianes : 3.75
médiane exacte : 29.98 ; approchée : 29.99 ; écart relatif < 1 % : True
```

Corrigé 3.13

- (a) Chacune des 40 partitions peut contenir des lignes des trois canaux : **jusqu'à** $40 \times 3 = 120$ **fichiers**. (b) Après `coalesce(4)` : **jusqu'à** $4 \times 3 = 12$ **fichiers**. (c) $120 \times 50\text{ ms} = 6\text{ s}$ contre $12 \times 50\text{ ms} = 0,6\text{ s}$: **dix fois moins** de frais fixes, pour exactement les mêmes données. À 100 000 fichiers, ces frais deviennent des heures.

Corrigé 3.14

- (a) $7 \bmod 3 = 1$, $8 \bmod 3 = 2$, $9 \bmod 3 = 0$, $10 \bmod 3 = 1$, $13 \bmod 3 = 1$, $16 \bmod 3 = 1$. Dans l'ordre d'arrivée : clé 7 $\rightarrow$ (1, 0) ; clé 8 $\rightarrow$ (2, 0) ; clé 9 $\rightarrow$ (0, 0) ; clé 10 $\rightarrow$ (1, 1) ; clé 13 $\rightarrow$ (1, 2) ; clé 16 $\rightarrow$ (1, 3). (b) À partir du décalage 2 de la partition 1, il lit les clés **13 et 16**. (c) La partition 1 reçoit **4 messages sur 6** : déséquilibrée, comme dans l'exercice 3.2. Un hachage sur **peu de valeurs** de clé équilibre mal ; un bon sujet a **beaucoup plus de clés que de partitions**.

```
cles = [7, 8, 9, 10, 13, 16]
compte = collections.Counter()
for c in cles:
    p = c % 3; print(c, "→", (p, compte[p])); compte[p] += 1
```

```
7 → (1, 0)
8 → (2, 0)
9 → (0, 0)
10 → (1, 1)
13 → (1, 2)
16 → (1, 3)
```

Corrigé 3.15

- (a) **Décalage noté avant** : au redémarrage, le sixième message est considéré comme lu, mais n'a pas été terminé. Il est **perdu** : 9 messages traités, 1 manquant. C'est la garantie **au plus une fois**. (b) **Décalage noté après** : le sixième message, traité mais non acquitté, est **relu** : il a été traité deux fois, soit 11 traitements pour 10 messages, 0 manquant, 1 doublon. C'est la garantie **au moins une fois**. Dans les deux cas, éviter le problème exige un traitement **idem-potent** (identifiants) ou une transaction qui regroupe le traitement et la note du décalage (**exactement une fois**).

Corrigé 3.16

- (a) $[0, 20) : 5, 12 \rightarrow 2$; $[20, 40) : 25, 31 \rightarrow 2$; $[40, 60) : 44, 58 \rightarrow 2$. (b) $[-10, 10) : 5 \rightarrow 1$; $[0, 20) : 5, 12 \rightarrow 2$; $[10, 30) : 12, 25 \rightarrow 2$; $[20, 40) : 25, 31 \rightarrow 2$; $[30, 50) : 31, 44 \rightarrow 2$; $[40, 60) : 44, 58 \rightarrow 2$; $[50, 70) : 58 \rightarrow 1$. (c) $1 + 2 + 2 + 2 + 2 + 2 + 1 = 12 = 6 \times 2$: chaque évènement appartient à **deux** fenêtres glissantes (durée 20 s, pas de 10 s : durée divisée par pas).

```
ev = [5, 12, 25, 31, 44, 58]
fenetres = {d: [t for t in ev if d <= t < d + 20] for d in range(-10, 60, 10)}
print({d: len(v) for d, v in fenetres.items()}, "; appartenances :", sum(len(v) for v in
↪ fenetres.values()))
```

```
{-10: 1, 0: 2, 10: 2, 20: 2, 30: 2, 40: 2, 50: 1} ; appartenances : 12
```

Corrigé 3.17

À chaque arrivée, on compare la **fin de la fenêtre** de l'évènement au filigrane, calculé avec le plus grand temps vu **avant** cet évènement ($D = 40$).

Arrivée	Plus grand temps vu avant	Filigrane	Fenêtre (fin)	Sort
10	0	-40	[0, 60) (60)	conservé
50	10	-30	[0, 60) (60)	conservé
90	50	10	[60, 120) (120)	conservé

Arrivée	Plus grand temps vu avant	Filigrane	Fenêtre (fin)	Sort
30	90	50	[0,60) (60)	conservé : $60 > 50$
130	90	50	[120,180) (180)	conservé
70	130	90	[60,120) (120)	conservé : $120 > 90$
20	130	90	[0,60) (60)	écarté : $60 \leq 90$

- (a) **Seul l'évènement 20 est écarté.** (b) Il aurait fallu que, à l'arrivée de 20, le filigrane soit inférieur à 60 : $130 - D < 60$, soit $D > 70$ (par exemple $D = 71$ s).

```
def ecartes(ordre, D, largeur=60):
    plus, out = 0, []
    for h in ordre:
        if h // largeur * largeur + largeur <= plus - D:
            out.append(h)
        plus = max(plus, h)
    return out

ordre = [10, 50, 90, 30, 130, 70, 20]
print("D = 40 :", ecartes(ordre, 40), "; D = 70 :", ecartes(ordre, 70), "; D = 71 :", ecartes(ordre, 71))
```

```
D = 40 : [20] ; D = 70 : [20] ; D = 71 : []
```

Corrigé 3.18

- (a) Les silences : $10 - 4 = 6$, $15 - 10 = 5$, $40 - 15 = 25$ (≥ 20 : **nouvelle session**), $44 - 40 = 4$, $90 - 44 = 46$ (**nouvelle session**). Trois sessions : {4,10,15}, {40,44} et {90}. (b) Durées (premier au dernier évènement) : $15 - 4 = 11$ s, $44 - 40 = 4$ s, et 0 s pour la session d'un seul évènement. (c) Le code ci-dessous trie les heures puis coupe à chaque silence d'au moins silence secondes.

```
def sessions(heures, silence=20):
    out = []
    for h in sorted(heures):
        if out and h - out[-1][-1] < silence:
            out[-1].append(h)
        else:
            out.append([h])
    return out

s = sessions([4, 10, 15, 40, 44, 90])
print(s, "; durées :", [x[-1] - x[0] for x in s])
```

```
[[4, 10, 15], [40, 44], [90]] ; durées : [11, 4, 0]
```

Nettoyage

```
spark.stop()
shutil.rmtree(TMP, ignore_errors=True)
print("dossier temporaire supprimé :", not os.path.exists(TMP))
```

```
dossier temporaire supprimé : True
```

Chapitre 4 : MLOps — exercices et applications

Ce chapitre du cahier accompagne le chapitre 4 du livre. Les **applications** reprennent, pas à pas et avec du code, ce que le livre n'a fait que résumer (batterie de tests, API de scoring, déploiement canari avec retour arrière, journal et étiquettes tardives, orchestration, suivi d'expériences, porte de qualité, mois de production simulé) et se terminent par une **chaîne complète** de bout en bout ; les **exercices** sont corrigés à la fin. Il se lit **de haut en bas** : les objets créés dans la section « Préparation » servent partout. Tout s'exécute **hors ligne et dans le processus Python** (API interrogée par un client de test, suivi d'expériences sur une base SQLite temporaire) ; les exemples de Docker, Kubernetes, Airflow ou GitHub Actions ne sont pas exécutés (voir le livre). Les données sont celles du livre : `clients_ml.csv`, des clients d'une boutique simulée, avec la résiliation à 90 jours `churn_90j` comme cible.

Préparation

Une seule cellule charge les bibliothèques, les données, le découpage en trois jeux (60 % d'entraînement, 20 % de test, 20 % de « réservoir de production ») et les deux modèles du chapitre. Les fonctions utilitaires (`donnees`, `modele_v1`, `modele_v2`, `psi`, `semaine`, `creer_app`, `MiniOrchestracteur...`) sont dans `build/outils_ch04.py` ; leur code est celui dont le livre décrit le principe. Un dossier temporaire `WORK` reçoit tous les fichiers créés ; il est **supprimé à la fin**. Les colonnes `depense_6m`, `segment_vrai` et `commandes_apres_cible` ne sont **jamais** des variables d'entrée.

```
import atexit, hashlib, json, math, os, shutil, sys, tempfile, time, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.metrics import roc_auc_score
from outils_ch04 import donnees, modele_v1, modele_v2, hash_fichier, psi, semaine, creer_app,
↳ ligne_json, MiniOrchestracteur

WORK = tempfile.mkdtemp(prefix="cah04_", dir=os.environ.get("TMPDIR"))
atexit.register(shutil.rmtree, WORK, ignore_errors=True)
d = donnees()
```

```

v1, v2 = modele_v1(d), modele_v2(d)
Xtr, Xte, Xpool, ytr, yte = d["Xtr"], d["Xte"], d["Xpool"], d["ytr"], d["yte"].to_numpy()
COLONNES = list(d["X"].columns)
s1_te, s2_te = v1.predict_proba(Xte)[: , 1], v2.predict_proba(Xte)[: , 1]
print(len(Xtr), "clients d'entraînement,", len(Xte), "de test,", len(Xpool), "en réservoir ; AUC de
↳ test :",
      round(roc_auc_score(yte, s1_te), 3), "(v1)", round(roc_auc_score(yte, s2_te), 3), "(v2)")

```

7200 clients d'entraînement, 2400 de test, 2400 en réservoir ; AUC de test : 0.867 (v1), 0.893 (v2)

Applications

Application 4.1 — Une batterie de tests pour un lot de données

Sections du livre : 4.1. **Objectif** : écrire les tests qui protègent le modèle, les essayer sur des incidents réalistes, et découvrir qu'aucun test de plages ne suffit.

Étape 1 — l'empreinte d'un entraînement. Avant de tester quoi que ce soit, on enregistre ce qui permettrait de refaire l'entraînement : le hash des données, la configuration, les versions.

```

import platform, sklearn
empreinte = {"donnees": hash_fichier("donnees/clients_ml.csv"), "config": {"graine": 0, "part_test":
↳ 0.4},
            "python": platform.python_version(), "sklearn": sklearn.__version__, "pandas":
↳ pd.__version__}
print(json.dumps(empreinte))

```

```

{"donnees": "fa48edff3715", "config": {"graine": 0, "part_test": 0.4, "python": "3.13.3", "sklearn":
↳ "1.9.1", "pandas": "3.0.6"}

```

Étape 2 — les tests. Un test est une fonction qui rend vrai ou faux. On en écrit cinq : le schéma, les plages, l'absence de colonne interdite, l'indépendance à l'ordre des lignes, et la **stabilité de la distribution** (le PSI de chaque variable numérique, par rapport à l'entraînement, reste sous 0,25).

```

INTERDITES = {"churn_90j", "commandes_apres_cible", "segment_vrai", "depense_6m", "id_client"}

def verifier(lot, modele):
    res = {"schéma": list(lot.columns) == COLONNES, "pas de colonne interdite": not (INTERDITES &
↳ set(lot.columns))}
    if not res["schéma"]:
        return {"**res", "plages": None, "ordre des lignes": None, "distribution": None}    # inutile
        ↳ d'aller plus loin
    res["plages"] = bool(lot["age"].between(18, 100).all() and
↳ lot["part_achats_promo"].dropna().between(0, 1).all()
                        and lot["recence_jours"].between(0, 365).all())
    p = modele.predict_proba(lot)[: , 1]
    m = lot.sample(frac=1, random_state=0)    # mêmes
    ↳ clients, autre ordre
    res["ordre des lignes"] = bool(np.allclose(pd.Series(modele.predict_proba(m)[: , 1],
↳ index=m.index).loc[lot.index], p))
    res["distribution"] = bool(max(psi(Xtr[c], lot[c]) for c in d["num"]) < 0.25)
    return res

```

Étape 3 — quatre lots. Un lot conforme, puis trois incidents : la part de promotions envoyée en pourcentage, la recence envoyée en semaines, une colonne qui disparaît de l'export.

```
lots = {"conforme": Xte,
        "part promo en %": Xte.assign(part_achats_promo=Xte["part_achats_promo"] * 100),
        "récence en semaines": Xte.assign(recence_jours=Xte["recence_jours"] // 7),
        "colonne manquante": Xte.drop(columns=["ville"])}
tab = pd.DataFrame({nom: verifier(lot, v2) for nom, lot in lots.items()}).T
print(tab.map(lambda v: "-" if v is None else ("oui" if v else "NON")).to_string())
```

	schéma	pas de colonne	interdite	plages	ordre des lignes	distribution
conforme	oui		oui	oui	oui	oui
part promo en %	oui		oui	NON	oui	NON
récence en semaines	oui		oui	oui	oui	NON
colonne manquante	NON		oui	—	—	—

Lecture. Le lot conforme passe tous les tests. La **part de promotions en pourcentage** est arrêtée à la fois par le test de plages et par celui de distribution. La **récence en semaines** est le cas instructif : les valeurs (de 0 à 52) restent **dans les plages permises** (de 0 à 365) ; seul le test de distribution la voit. Quant à la colonne manquante, elle est arrêtée dès le schéma, avant tout calcul.

Pour aller plus loin. Ajoutez un test de **non-régression** (l'AUC sur le jeu de test ne doit pas passer sous 0,85) et un test de **comportement** : à profil égal, un client qui a plus de retours produit ne doit pas avoir un risque plus faible avec la régression logistique v1. Quel test échouerait si l'on remplaçait v1 par un modèle entraîné sur les étiquettes mélangées ?

Application 4.2 — Une API de scoring, de la requête à l'erreur

Sections du livre : 4.2. **Objectif :** servir v2 par une API, vérifier la parité avec la prédiction hors ligne, provoquer les erreurs, mesurer l'ordre de grandeur de la latence.

Étape 1 — le service. On sauvegarde le modèle, on construit l'application, on l'interroge avec un client de test (aucun port réseau n'est ouvert).

```
from fastapi.testclient import TestClient
chemin_v2 = os.path.join(WORK, "churn_v2.joblib"); joblib.dump(v2, chemin_v2)
api = TestClient(creer_app(chemin_v2, COLONNES, version="2.0.0"))
client = ligne_json(Xte.iloc[0])
print(api.get("/health").json())
r = api.post("/predict", json=client)
print(r.status_code, r.json(), "| hors ligne :", round(float(v2.predict_proba(Xte.iloc[[0]])[0, 1]), 4))
```

```
{'status': 'ok', 'version': '2.0.0'}
200 {'probabilite': 0.0049, 'version': '2.0.0'} | hors ligne : 0.0049
```

Étape 2 — la parité sur un lot. On envoie 100 clients à /predict_batch et on compare aux probabilités calculées directement.

```
lot = [ligne_json(Xte.iloc[i]) for i in range(100)]
rep = np.array(api.post("/predict_batch", json=lot).json()["probabilites"])
ecart = np.abs(rep - v2.predict_proba(Xte.iloc[:100])[:, 1]).max()
print("écart maximal entre l'API et le modèle :", round(float(ecart), 5))
```

```
écart maximal entre l'API et le modèle : 5e-05
```

La différence n'est due qu'à l'**arrondi à 4 décimales** de la réponse JSON (l'écart ne peut pas dépasser $0,5 \times 10^{-4}$).

Étape 3 — provoquer les erreurs. Quatre requêtes invalides : âge hors plage, champ manquant, type erroné, fraction hors de [0,1]. Pour chacune, on affiche le code et le champ désigné.

```
essais = {"âge de 17 ans": {**client, "age": 17}, "ville manquante": {k: v for k, v in client.items()
↳ if k != "ville"},
        "âge en lettres": {**client, "age": "vingt"}, "part promo à 90": {**client,
↳ "part_achats_promo": 90}}
for nom, corps in essais.items():
    e = api.post("/predict", json=corps)
    err = e.json()["detail"][0]
    print(f"{nom:16s}→ {e.status_code} · champ {err['loc'][-1]:18s} · {err['type']}")
```

```
âge de 17 ans → 422 · champ age · greater_than_equal
ville manquante → 422 · champ ville · missing
âge en lettres → 422 · champ age · int_parsing
part promo à 90 → 422 · champ part_achats_promo · less_than_equal
```

Étape 4 — la latence. On chronomètre 200 appels successifs. Les durées dépendent de la machine : on n'affiche pas leurs valeurs, seulement si l'**objectif de service** (95 % des réponses sous 500 ms, très large pour ce test) est tenu, et le centile de la latence est calculé par `np.percentile`.

```
durees = []
for i in range(200):
    t0 = time.perf_counter(); api.post("/predict", json=ligne_json(Xte.iloc[i]));
    ↳ durees.append(time.perf_counter() - t0)
p50, p95 = np.percentile(durees, [50, 95])
print("p95 sous 500 ms :", bool(p95 < 0.5), "| p95 plus grand que la médiane :", bool(p95 >= p50))
```

```
p95 sous 500 ms : True | p95 plus grand que la médiane : True
```

Questions. (a) Pourquoi la réponse ne contient-elle que la probabilité et la version, pas la liste des variables ? (b) Que se passerait-il si le modèle était rechargé à **chaque requête** ? (c) Un client envoie "appareil": null : la requête est-elle acceptée ? Pourquoi ? (*Réponses : (a) minimiser ce qui sort (sécurité, 4.6) ; (b) le chargement coûte bien plus que la prédiction, d'où le chargement unique au démarrage ; (c) oui, appareil est facultatif dans le contrat, le modèle gère l'absence de valeur.*)

Application 4.3 — Un déploiement canari avec retour arrière automatique

Sections du livre : 4.2. **Objectif :** simuler une semaine de canari où la nouvelle version a un **défaut** dans sa préparation des données, repérer le défaut par un indicateur, et revenir en arrière en changeant l'alias.

Étape 1 — le routage. Chaque client est dirigé vers la version « champion » ou « canari » par le hash de son identifiant ; on vérifie que la part réelle est proche de la part annoncée.

```
def dans_canari(id_client, part_canari):
    return int(hashlib.sha256(str(id_client).encode()).hexdigest(), 16) % 100 < part_canari

def version_servie(id_client, part_canari, alias):
    return alias["canari"] if dans_canari(id_client, part_canari) else alias["champion"]

alias = {"champion": "1.0.0", "canari": "2.0.0"}
services = np.array([version_servie(i, 10, alias) for i in range(len(Xpool))])
print("part effectivement servie par le canari :", round(100 * (services == "2.0.0").mean(), 1), "%
↳ (annoncé : 10 %)")
```

```
part effectivement servie par le canari : 10.4 % (annoncé : 10 %)
```

Étape 2 — une semaine de trafic. Chaque jour, 2 000 clients sont tirés (avec remise) du réservoir. Le champion (v1) répond aux clients dirigés vers lui ; le canari (v2) aux autres. **À partir du jour 4**, un bug de l'équipe fait que le canari reçoit la récence **en semaines** (comme en 4.1). L'indicateur surveillé est l'écart entre le score moyen du canari et celui du champion ; la règle : si l'écart dépasse

0,03 deux jours de suite, on retourne en arrière.

```

rng = np.random.default_rng(42)
lignes, alerte_veille = [], False
for jour in range(1, 8):
    idx = rng.integers(0, len(Xpool), 2000)
    lot = Xpool.iloc[idx]
    est_canari = np.array([dans_canari(i, 10) for i in idx])
    mod_canari = v2 if alias["canari"] == "2.0.0" else v1 # ce que
    ↪ l'alias désigne
    lot_c = lot[est_canari]
    if jour >= 4 and mod_canari is v2:
        lot_c = lot_c.assign(recence_jours=lot_c["recence_jours"] / 7) # le bug
    ↪ de la version 2
    m_champ = v1.predict_proba(lot[~est_canari])[:, 1].mean()
    m_can = mod_canari.predict_proba(lot_c[:, 1].mean()
    ecart = abs(m_can - m_champ)
    alerte = bool(ecart > 0.03)
    if alerte and alerte_veille and mod_canari is v2:
        alias["canari"] = "1.0.0" # retour
    ↪ arrière : on déplace l'alias
    lignes.append((jour, round(100 * est_canari.mean(), 1), round(m_champ, 3), round(m_can, 3),
    ↪ round(ecart, 3), alerte, alias["canari"]))
    alerte_veille = alerte
print(pd.DataFrame(lignes, columns=["jour", "% canari", "score champion", "score canari", "écart",
    ↪ "alerte", "alias canari ensuite"]).to_string(index=False))

```

jour	% canari	score champion	score canari	écart	alerte	alias canari	ensuite
1	9.8	0.138	0.136	0.002	False		2.0.0
2	9.4	0.141	0.136	0.005	False		2.0.0
3	10.8	0.144	0.124	0.020	False		2.0.0
4	9.9	0.137	0.053	0.083	True		2.0.0
5	10.1	0.132	0.063	0.069	True		1.0.0
6	9.6	0.129	0.148	0.019	False		1.0.0
7	10.8	0.130	0.149	0.020	False		1.0.0

Lecture. Les trois premiers jours, l'écart est de l'ordre du bruit d'échantillonnage (le canari ne voit qu'environ 200 clients par jour) : pas d'alerte. Le jour 4, le bug fait chuter le score moyen du canari ; une alerte seule peut être du bruit, **deux jours de suite** est un signal, et l'alias est ramené sur la version 1 au jour 5. Les jours suivants, tous les clients reçoivent le champion : le service n'a jamais été interrompu et **aucun code n'a été modifié**.

Pour aller plus loin. (1) Remplacez la règle « 0,03 deux jours de suite » par « 0,02 un seul jour » : combien de fausses alertes obtenez-vous avant le jour 4 sur 200 historiques simulés ? (2) Changez la part du canari à 2 % : le bug est-il toujours détecté, et en combien de jours ? Reliez la réponse au calcul du nombre de clients nécessaires donné dans le livre.

Application 4.4 — Le journal des prédictions et les étiquettes tardives

Sections du livre : 4.3. Objectif : journaliser soixante jours de prédictions, mesurer la performance à mesure que les étiquettes arrivent, et comparer à l'estimation sans étiquettes.

Étape 1 — journaliser. Chaque jour, 40 clients du réservoir demandent un score. Le service écrit une ligne JSON par prédiction : identifiant, jour, version, probabilité, quelques entrées de surveillance.

```

def journaliser(fichier, id_pred, jour, client, proba, version):
    ligne = {"id": id_pred, "jour": jour, "version": version, "proba": round(proba, 4),
            "entrees": {k: client[k] for k in ("recence_jours", "satisfaction_moy",
            ↪ "part_achats_promo")}}
    with open(fichier, "a") as f:

```

```

f.write(json.dumps(ligne) + "\n")

fichier_journal = os.path.join(WORK, "journal.jsonl")
sp = v2.predict_proba(Xpool)[: , 1]
for i in range(len(Xpool)):
    journaliser(fichier_journal, i, i // 40, ligne_json(Xpool.iloc[i]), float(sp[i]), "2.0.0")
journal = pd.read_json(fichier_journal, lines=True)
print(len(journal), "lignes sur", journal["jour"].nunique(), "jours ; colonnes :",
      ↪ list(journal.columns))

```

```
2400 lignes sur 60 jours ; colonnes : ['id', 'jour', 'version', 'proba', 'entrees']
```

Étape 2 — les étiquettes arrivent 90 jours plus tard. On définit une fonction qui rapproche le journal des étiquettes disponibles à une date d'observation (seules les prédictions de plus de 90 jours ont leur étiquette) et rend l'AUC avec un intervalle de confiance par rééchantillonnage.

```

etiq = pd.Series(d["ypool"])
rng = np.random.default_rng(1)

def performance_a_la_date(obs, rep=300):
    mur = journal[journal["jour"] + 90 <= obs]
    y_m, p_m = etiq.loc[mur["id"]].to_numpy(), mur["proba"].to_numpy()
    bs = [roc_auc_score(y_m[k], p_m[k]) for k in (rng.integers(0, len(y_m), len(y_m)) for _ in
    ↪ range(rep))]
    return len(mur), roc_auc_score(y_m, p_m), *np.percentile(bs, [2.5, 97.5])

dates = [100, 110, 120, 130, 140, 150]
res = pd.DataFrame([(o, *performance_a_la_date(o)) for o in dates], columns=["jour d'observation",
    ↪ "étiquetées", "AUC", "bas", "haut"])
res["largeur"] = res["haut"] - res["bas"]
print(res.round(3).to_string(index=False))

```

jour d'observation	étiquetées	AUC	bas	haut	largeur
100	440	0.884	0.834	0.925	0.092
110	840	0.871	0.837	0.904	0.067
120	1240	0.874	0.849	0.897	0.048
130	1640	0.874	0.849	0.898	0.049
140	2040	0.881	0.858	0.899	0.041
150	2400	0.888	0.871	0.907	0.036

Lecture. L'intervalle se resserre à mesure que des prédictions mûrissent : de 0,092 de large au jour 100 à 0,036 au jour 150 ; on dispose d'un intervalle plus étroit que 0,06 à partir du jour 120. **C'est le retard de la métrique réelle** : un modèle déployé le jour 0 n'est jugé de façon fiable qu'après plus de trois mois.

Étape 3 — l'estimation sans étiquettes, semaine par semaine. On compare la précision attendue des clients signalés (score au moins égal à 0,30, somme des probabilités) à la précision réellement observée **une fois toutes les étiquettes arrivées**.

```

journal["resilie"] = etiq.loc[journal["id"]].to_numpy()
journal["semaine"] = journal["jour"] // 7
sig = journal[journal["proba"] >= 0.30]
par_semaine = sig.groupby("semaine").agg(signales=("proba", "size"), attendue=("proba", "mean"),
    ↪ reelle=("resilie", "mean")).round(3)
print(par_semaine.to_string())

```

	signales	attendue	reelle
semaine			
0	34	0.606	0.618
1	40	0.601	0.500
2	35	0.631	0.657
3	42	0.609	0.786

4	34	0.623	0.500
5	41	0.581	0.561
6	44	0.637	0.477
7	42	0.626	0.643
8	29	0.601	0.621

Question. Les colonnes attendue et réelle varient d'une semaine à l'autre ; l'écart est-il plus grand que ce que le hasard explique avec une quarantaine de clients signalés ? Calculez l'erreur-type $\sqrt{p(1-p)/n}$ pour une semaine et comparez. (Avec $p \approx 0,6$ et $n \approx 40$, l'erreur-type vaut environ 0,08 : des écarts de 5 à 15 points entre semaines ne prouvent rien.)

Application 4.5 — Orchestrer un pipeline d'entraînement

Sections du livre : 4.4, 4.1. **Objectif :** déclarer le graphe « charger → valider → entraîner → évaluer → publier » dans le mini-orchestrateur, observer reprises, tâches ignorées et rattrapage, et vérifier l'idempotence de la publication.

Étape 1 — les tâches. Chaque exécution (« jour logique ») entraîne une régression logistique sur un instantané des données d'entraînement. Deux incidents sont programmés : le jour 1, le chargement échoue deux fois (panne transitoire) ; le jour 2, l'instantané est **tronqué** (150 clients au lieu de 3 000) et le modèle appris est mauvais. La publication écrit dans un registre JSON par **mise à jour selon la clé** (idempotente).

```
ORCH = os.path.join(WORK, "orch"); os.makedirs(ORCH, exist_ok=True)
registre_json = os.path.join(ORCH, "registre.json")
TRONQUE, PANNES, tentatives, appris = {2}, {1: 2}, {}, {}

def instantane(jour):
    n = 150 if jour in TRONQUE else 3000
    return Xtr.sample(n, random_state=jour)

orch = MiniOrchestrateur(reprises=2)

@orch.tache("charger")
def charger(jour):
    tentatives[jour] = tentatives.get(jour, 0) + 1
    if tentatives[jour] <= PANNES.get(jour, 0):
        raise ConnectionError("entrepôt injoignable")
    instantane(jour).to_csv(os.path.join(ORCH, f"snap_{jour}.csv"))

@orch.tache("valider", depend_de=["charger"])
def valider(jour):
    assert len(pd.read_csv(os.path.join(ORCH, f"snap_{jour}.csv"))) >= 1000, "instantané trop petit"

@orch.tache("entraîner", depend_de=["valider"])
def entrainer(jour):
    snap = pd.read_csv(os.path.join(ORCH, f"snap_{jour}.csv"), index_col=0)
    appris[jour] = modele_vl(dict(d, Xtr=snap, ytr=ytr.loc[snap.index]))

@orch.tache("evaluer", depend_de=["entraîner"])
def evaluer(jour):
    auc = roc_auc_score(yte, appris[jour].predict_proba(Xte)[: , 1])
    assert auc >= 0.85, f"AUC {auc:.3f} sous le seuil"

@orch.tache("publier", depend_de=["evaluer"])
def publier(jour):
    reg = json.load(open(registre_json)) if os.path.exists(registre_json) else {}
    reg[str(jour)] = {"auc": round(roc_auc_score(yte, appris[jour].predict_proba(Xte)[: , 1]), 4)}
    ↪ # clé = jour : écrase, n'ajoute pas
    json.dump(reg, open(registre_json, "w"))
```

Étape 2 — quatre jours d'exécution.

```
etats = {j: orch.lancer(j) for j in range(4)}
print(pd.DataFrame(etats).T.to_string())
journal_orch = pd.DataFrame(orch.journal, columns=["jour", "tâche", "essai", "état"])
print(journal_orch[(journal_orch["jour"] == 1) & (journal_orch["tâche"] ==
↳ "charger")].to_string(index=False))
```

```
charger valider entraîner évaluer publier
0      ok      ok      ok      ok      ok
1      ok      ok      ok      ok      ok
2      ok  échec  ignorée  ignorée  ignorée
3      ok      ok      ok      ok      ok
jour  tâche  essai  état
1 charger      1 échec (ConnectionError)
1 charger      2 échec (ConnectionError)
1 charger      3      ok
```

Lecture. Le jour 1 a réussi malgré la panne (trois essais pour charger) ; le jour 2 est arrêté par la validation (**aucune reprise ne peut corriger** un instantané trop petit) et les trois tâches suivantes sont ignorées : un modèle entraîné sur 150 clients n'a pas été publié.

Étape 3 — corriger et rattraper. La source est réparée. On relance **seulement** le jour 2, puis, pour vérifier l'idempotence, on rejoue **tous** les jours et on regarde le registre.

```
TRONQUE.clear()
print("jour 2 après correction :", orch.lancer(2))
avant = json.load(open(registre_json))
for j in range(4):
    orch.lancer(j)
apres = json.load(open(registre_json))
print("clés du registre :", sorted(apres), "| identique après rejou complet :", avant == apres)
```

```
jour 2 après correction : {'charger': 'ok', 'valider': 'ok', 'entraîner': 'ok', 'évaluer': 'ok',
↳ 'publier': 'ok'}
clés du registre : ['0', '1', '2', '3'] | identique après rejou complet : True
```

Pour aller plus loin. Remplacez la publication par une version qui **ajoute** une ligne à un fichier à chaque appel : rejouez les quatre jours et comptez les lignes. Quelle règle de conception du livre (4.4) cela illustre-t-il ?

Application 4.6 — Suivre une recherche d'hyperparamètres avec MLflow

Sections du livre : 4.5. **Objectif :** enregistrer six essais, choisir sur la **validation croisée**, enregistrer deux versions dans le registre, déplacer un alias et remonter jusqu'au hash des données.

Étape 1 — le magasin de suivi (SQLite temporaire) et la fonction qui enregistre un essai. Le modèle est enregistré au format skops, qui exige de **déclarer** les types que l'on accepte de recharger.

```
import logging, mlflow, mlflow.sklearn
from mlflow.tracking import MlflowClient
from sklearn.base import clone
from sklearn.model_selection import cross_val_score
logging.getLogger("mlflow").setLevel(logging.ERROR)
APPROUVES = ["numpy.dtype", "functools.partial", "sklearn.utils.validation.check_array",
↳ "sklearn.ensemble._hist_gradient_boosting.predictor.TreePredictor"]
mlflow.set_tracking_uri(f"sqlite:///WORK/mlflow.db")
mlflow.create_experiment("recherche", artifact_location=f"file://{WORK}/artefacts");
↳ mlflow.set_experiment("recherche")

def enregistrer(nom, modele, params):
    with mlflow.start_run(run_name=nom):
```

```

mlflow.log_params(**params, "graine": 0, "hash_donnees":
↳ hash_fichier("donnees/clients_ml.csv"))
mlflow.log_metric("auc_cv", cross_val_score(modele, Xtr, ytr, cv=3, scoring="roc_auc").mean())
mlflow.log_metric("auc_test", roc_auc_score(yte, modele.fit(Xtr, ytr).predict_proba(Xte)[: ,
↳ 1]))
mlflow.sklearn.log_model(modele, name="modele", skops_trusted_types=APPROUVES)

```

Étape 2 — six essais tirés au hasard dans une grille (pas d'apprentissage × nombre maximal de feuilles).

```

grille = [(lr, nf) for lr in (0.03, 0.05, 0.1, 0.2) for nf in (7, 15, 31)]
choix = np.random.default_rng(0).choice(len(grille), 6, replace=False)
for k in choix:
    lr, nf = grille[k]
    enregistrer(f"hgb lr={lr} feuilles={nf}", clone(v2).set_params(clf__learning_rate=lr,
↳ clf__max_leaf_nodes=nf),
                {"clf__learning_rate": lr, "clf__max_leaf_nodes": nf})
essais = mlflow.search_runs(experiment_names="recherche", order_by=["metrics.auc_cv DESC"])
print(essais[["tags.mlflow.runName", "metrics.auc_cv",
↳ "metrics.auc_test"]].round(4).rename(columns=lambda c: c.split(".")[1]).to_string(index=False))

```

	runName	auc_cv	auc_test
	hgb lr=0.05 feuilles=7	0.8953	0.9044
	hgb lr=0.03 feuilles=7	0.8932	0.9005
	hgb lr=0.05 feuilles=15	0.8930	0.9023
	hgb lr=0.05 feuilles=31	0.8907	0.8977
	hgb lr=0.03 feuilles=31	0.8901	0.8952
	hgb lr=0.1 feuilles=15	0.8879	0.9000

Étape 3 — le registre. On enregistre le meilleur et le deuxième essai comme versions 1 et 2 du modèle `resiliation_hgb`, on donne l'alias `champion` à la version 1 et `challenger` à la version 2, on promeut le challenger puis on revient en arrière.

```

reg = MlflowClient()
v_a = mlflow.register_model(f"runs://{essais.loc[0, 'run_id']}/modele", "resiliation_hgb").version
v_b = mlflow.register_model(f"runs://{essais.loc[1, 'run_id']}/modele", "resiliation_hgb").version
reg.set_registered_model_alias("resiliation_hgb", "champion", v_a);
↳ reg.set_registered_model_alias("resiliation_hgb", "challenger", v_b)
aliases = lambda: {a: reg.get_model_version_by_alias("resiliation_hgb", a).version for a in
↳ ("champion", "challenger")}
print("départ      :", aliases())
reg.set_registered_model_alias("resiliation_hgb", "champion", v_b); print("promotion :", aliases())
reg.set_registered_model_alias("resiliation_hgb", "champion", v_a); print("retour      :", aliases())

```

```

départ      : {'champion': 1, 'challenger': 2}
promotion    : {'champion': 2, 'challenger': 2}
retour       : {'champion': 1, 'challenger': 2}

```

Étape 4 — remonter la chaîne. À partir de l'alias `champion`, on retrouve l'essai, ses paramètres et le hash des données ; on le compare au hash du fichier actuel, et l'on vérifie que le modèle rechargé par l'alias donne les mêmes probabilités que celui de l'essai.

```

vc = reg.get_model_version_by_alias("resiliation_hgb", "champion")
run = reg.get_run(vc.run_id)
print("paramètres :", {k: v for k, v in run.data.params.items() if k.startswith("clf")}, "| données
↳ identiques :",
      run.data.params["hash_donnees"] == hash_fichier("donnees/clients_ml.csv"))
charge = mlflow.sklearn.load_model("models:/resiliation_hgb@champion")
print("écart maximal avec le modèle de l'essai :", float(np.abs(charge.predict_proba(Xte)[: , 1] -
↳ mlflow.sklearn.load_model(f"runs://{vc.run_id}/modele").predict_proba(Xte)[: , 1]).max()))

```

```
paramètres : {'clf__learning_rate': '0.05', 'clf__max_leaf_nodes': '7'} | données identiques : True
écart maximal avec le modèle de l'essai : 0.0
```

Questions. (a) Le meilleur essai en validation croisée est-il aussi le meilleur sur le jeu de test ? Si ce n'est pas le cas, lequel devrait-on croire, et pourquoi ? (b) Quel élément manque encore à cette chaîne pour que l'essai soit **entièrement** reproductible (4.1) ? (*Réponses : (a) la validation croisée, car le test sert à juger une fois ; (b) la version du code (numéro de commit) et celle de l'environnement, qu'il faudrait ajouter comme étiquettes de l'essai.*)

Application 4.7 — Une porte de qualité, une « intégration continue » et un service protégé

Sections du livre : 4.6. **Objectif :** écrire la porte qui décide si un candidat est publié, la faire tourner comme le ferait un outil de CI (un programme dont le **code de sortie** dit si tout va bien), charger la configuration depuis l'environnement et protéger une API par une clé et une limite de débit.

Étape 1 — la porte. Un candidat est publié s'il atteint un plancher **et** ne régresse pas par rapport au champion.

```
def porte_qualite(candidat, champion, lot, y, plancher=0.85, tolerance=0.005):
    auc_c, auc_p = (roc_auc_score(y, m.predict_proba(lot)[: , 1]) for m in (candidat, champion))
    verdicts = {f"AUC ≥ {plancher}": auc_c >= plancher, f"régression ≤ {tolerance}": auc_c >= auc_p -
    tolerance}
    return all(verdicts.values()), round(auc_c, 3), verdicts

faible = modele_v1(dict(d, Xtr=Xtr.iloc[:300], ytr=ytr.iloc[:300])) # entraîné
    sur 300 clients seulement
for nom, cand in [("boosting (v2)", v2), ("logistique sur 300 clients", faible)]:
    ok, auc, verdicts = porte_qualite(cand, v1, Xte, yte)
    print(f"{nom:28s} AUC {auc} → {'publié' if ok else 'REFUSÉ'} {verdicts}")
```

boosting (v2)	AUC 0.893 → publié	{'AUC ≥ 0.85': True, 'régression ≤ 0.005': True}
logistique sur 300 clients	AUC 0.818 → REFUSÉ	{'AUC ≥ 0.85': False, 'régression ≤ 0.005': False}

Étape 2 — comme un outil de CI. Un outil d'intégration continue ne lit pas un affichage : il lance un programme et regarde son **code de sortie** (0 : tout va bien ; autre chose : échec, la chaîne s'arrête). On écrit le contrôle dans un petit script, que l'on exécute dans un **processus séparé** pour chacun des deux candidats.

```
import subprocess
script = os.path.join(WORK, "controle.py")
open(script, "w").write('''import sys, joblib
sys.path.insert(0, "build")
from outils_ch04 import donnees
from sklearn.metrics import roc_auc_score
d = donnees()
auc = roc_auc_score(d["yte"], joblib.load(sys.argv[1]).predict_proba(d["Xte"])[ : , 1])
print(f"AUC {auc:.3f}")
sys.exit(0 if auc >= 0.85 else 1)
''')
for nom, cand in [("boosting", v2), ("faible", faible)]:
    chemin = os.path.join(WORK, f"cand_{nom}.joblib"); joblib.dump(cand, chemin)
    r = subprocess.run([sys.executable, script, chemin], capture_output=True, text=True)
    print(f"{nom:9s} → code de sortie {r.returncode} ({r.stdout.strip()})")
```

boosting	→ code de sortie 0 (AUC 0.893)
faible	→ code de sortie 1 (AUC 0.818)

Étape 3 — la configuration par l'environnement. Les valeurs qui changent entre test et produc-

tion sont lues dans l'environnement ; l'absence d'une valeur obligatoire est une **erreur immédiate**.

```
def charger_config(env):
    return {"alias": env.get("MODELE_ALIAS", "champion"), "seuil": float(env.get("SEUILALERTE",
    ↪ "0.30")),
            "registre": env["URL_REGISTRE"]}

for env in ({"URL_REGISTRE": "sqlite:///exemple", "SEUILALERTE": "0.25"}, {"MODELE_ALIAS":
    ↪ "challenger"}):
    try:
        print(charger_config(env))
    except KeyError as e:
        print("configuration incomplète, variable absente :", e)
```

```
{'alias': 'champion', 'seuil': 0.25, 'registre': 'sqlite:///exemple'}
configuration incomplète, variable absente : 'URL_REGISTRE'
```

Étape 4 — une API protégée. Chaque appel doit présenter une clé ; chaque clé est limitée à trois appels.

```
from fastapi import FastAPI, Header, HTTPException
protege, appels = FastAPI(), {}
CLES = {"cle-de-test"} # exemple seulement : jamais
    ↪ de secret dans le code

@protege.get("/score")
def score(x_api_key: str = Header(default="")):
    if x_api_key not in CLES:
        raise HTTPException(401, "clé absente ou invalide")
    appels[x_api_key] = appels.get(x_api_key, 0) + 1
    if appels[x_api_key] > 3:
        raise HTTPException(429, "trop de requêtes")
    return {"ok": True}

cp = TestClient(protege)
codes = [cp.get("/score").status_code, cp.get("/score", headers={"x-api-key":
    ↪ "mauvaise"}).status_code]
codes += [cp.get("/score", headers={"x-api-key": "cle-de-test"}).status_code for _ in range(4)]
print(codes)
```

```
[401, 401, 200, 200, 200, 429]
```

Lecture. Le candidat entraîné sur 300 clients est refusé à la fois par le plancher et par la non-régression ; le script de CI sort avec le code 1 et la chaîne s'arrête. Le 401 est rendu **avant** tout calcul, le 429 à **partir du quatrième** appel valide.

Application 4.8 — Un mois de production : quelles variables ont dérivé, et est-ce grave ?

Sections du livre : 4.7. Objectif : repérer quelles variables dérivent (PSI), **pondérer** par leur importance, confronter l'alerte sur les entrées à l'alerte sur la performance, et mesurer la sensibilité du PSI au découpage.

Étape 1 — le PSI de chaque variable, semaine 4, dérive des variables.

```
Xw, yw = semaine(d, v2, 4, "covariable")
psis = pd.Series({c: psi(Xtr[c], Xw[c]) for c in d["num"]}).sort_values(ascending=False)
print(psis.round(3).head(6).to_string())
```

```
part_achats_promo      0.795
satisfaction_moy       0.713
nb_promos_recues_12m   0.460
```

```
panier_moyen      0.258
nb_tickets_support_12m  0.169
taux_ouverture_email  0.161
```

Étape 2 — pondérer par l'importance. Une variable qui dérive mais pèse peu dans le modèle compte moins. On mesure l'importance par **permutation** sur le jeu de test (la baisse d'AUC quand on mélange la variable, volume III).

```
from sklearn.inspection import permutation_importance
imp = pd.Series(permutation_importance(v2, Xte, yte, scoring="roc_auc", n_repeats=3,
    ↪ random_state=0).importances_mean, index=COLONNES)
tab = pd.DataFrame({"PSI": psis, "importance": imp[psis.index]}).head(8)
print(tab.round(3).to_string())
```

	PSI	importance
part_achats_promo	0.795	0.017
satisfaction_moy	0.713	0.039
nb_promos_recues_12m	0.460	0.002
panier_moyen	0.258	0.001
nb_tickets_support_12m	0.169	0.000
taux_ouverture_email	0.161	-0.000
montant_12m	0.101	0.055
age	0.097	0.060

Lecture. Les variables qui dérivent le plus (la part d'achats en promotion, la satisfaction) ne sont **pas** les plus importantes pour le modèle (ce sont ici l'âge et le montant dépensé, dont le PSI reste proche de 0,1). Un PSI très élevé sur une variable peu importante est un signal à surveiller, pas à paniquer : pondérer par l'importance évite des alertes inutiles.

Étape 3 — alerte sur les entrées contre alerte sur la performance. Pour chaque scénario et chaque semaine, on déclenche une alerte « entrées » si le PSI de la part d'achats en promotion dépasse 0,25, et une alerte « performance » si l'AUC (étiquettes arrivées) a perdu plus de 0,05 par rapport à la semaine 1.

```
lignes = []
for scenario in ("covariable", "concept"):
    auc1 = None
    for k in range(1, 5):
        Xw, yw = semaine(d, v2, k, scenario)
        auc = roc_auc_score(yw, v2.predict_proba(Xw)[ :, 1]); auc1 = auc1 or auc
        lignes.append((scenario, k, round(psi(Xtr["part_achats_promo"], Xw["part_achats_promo"]), 3),
    ↪ round(auc, 3),
        psi(Xtr["part_achats_promo"], Xw["part_achats_promo"]) > 0.25, auc < auc1 -
    ↪ 0.05))
res = pd.DataFrame(lignes, columns=["scénario", "semaine", "PSI promo", "AUC", "alerte entrées",
    ↪ "alerte performance"])
print(res.to_string(index=False))
```

scénario	semaine	PSI promo	AUC	alerte entrées	alerte performance
covariable	1	0.015	0.876	False	False
covariable	2	0.106	0.859	False	False
covariable	3	0.345	0.869	True	False
covariable	4	0.795	0.857	True	False
concept	1	0.015	0.876	False	False
concept	2	0.023	0.847	False	False
concept	3	0.008	0.808	False	True
concept	4	0.002	0.776	False	True

Lecture. Dans la dérive des variables, l'alerte sur les entrées se déclenche et l'alerte sur la performance **non** : le modèle reste bon. Dans la dérive du concept, c'est l'inverse : **aucune alerte sur les entrées**, mais la performance s'effondre. Aucune des deux alertes ne suffit seule ; il faut surveiller

les deux.

Étape 4 — le PSI dépend du découpage. Même variable, même semaine, trois nombres d'intervalles.

```
Xw, _ = semaine(d, v2, 4, "covariable")
print({k: round(psi(Xtr["part_achats_promo"], Xw["part_achats_promo"], k=k), 3) for k in (5, 10, 20)})
```

```
{5: 0.753, 10: 0.795, 20: 0.815}
```

Question. Les seuils de 0,1 et 0,25 sont-ils des lois ? Que changer si l'on passe de 10 à 20 intervalles ? (Non : des repères conventionnels, calibrés pour environ 10 intervalles et des échantillons d'au moins quelques centaines ; un découpage plus fin augmente le PSI à effectifs égaux, et il faut recalibrer les seuils sur des périodes calmes.)

Application 4.9 — Une chaîne complète, de l'entraînement à la supervision

Sections du livre : 4.1 à 4.7. **Objectif :** assembler en un seul parcours les briques du chapitre : un candidat est entraîné, testé, soumis à la porte de qualité, enregistré, servi par une API, journalisé en production, puis jugé quand les étiquettes arrivent. Chaque étape **s'arrête** si son contrôle échoue.

```
rapport = {}
# 1. candidat et tests de données
candidat = clone(v2).set_params(clf__learning_rate=0.05)
res_tests = verifier(Xte, v2)
rapport["tests de données"] = all(v for v in res_tests.values() if v is not None)
# 2. porte de qualité (face au champion v1)
candidat.fit(Xtr, ytr)
ok_porte, auc_cand, _ = porte_qualite(candidat, v1, Xte, yte)
rapport["porte de qualité"] = ok_porte
assert all(rapport.values()), rapport # sinon la chaîne
↳ s'arrête ici

# 3. enregistrement dans le registre (suivi + version + alias)
mlflow.create_experiment("chaîne", artifact_location=f"file://{WORK}/artefacts_chaine");
↳ mlflow.set_experiment("chaîne")
enregistrer("candidat lr=0.05", clone(v2).set_params(clf__learning_rate=0.05), {"clf__learning_rate":
↳ 0.05})
run_id = mlflow.search_runs(experiment_names=["chaîne"]).loc[0, "run_id"]
version = mlflow.register_model(f"runs://{run_id}/modele", "resiliation_chaine").version
reg.set_registered_model_alias("resiliation_chaine", "champion", version)
rapport["registre : alias champion"] = reg.get_model_version_by_alias("resiliation_chaine",
↳ "champion").version == version
# 4. service : le modèle est chargé depuis l'alias, puis servi par l'API
en_service = mlflow.sklearn.load_model("models:/resiliation_chaine@champion")
chemin_service = os.path.join(WORK, "service.joblib"); joblib.dump(en_service, chemin_service)
api2 = TestClient(creer_app(chemin_service, COLONNES, version=f"registre-v{version}"))
rapport["service : /health"] = api2.get("/health").status_code == 200

# 5. production : 300 requêtes journalisées (jours 0 à 5, 50 clients par jour)
journal2 = os.path.join(WORK, "journal2.jsonl")
for i in range(300):
    rep = api2.post("/predict", json=ligne_json(Xpool.iloc[i])).json()
    journaliser(journal2, i, i // 50, ligne_json(Xpool.iloc[i]), rep["probabilite"], rep["version"])
j2 = pd.read_json(journal2, lines=True)
# 6. les étiquettes arrivent : performance réelle et dérive des entrées
y_prod = d["ypool"][j2["id"].to_numpy()]
auc_prod = roc_auc_score(y_prod, j2["proba"])
psi_max = max(psi(Xtr[c], Xpool.iloc[:300][c]) for c in d["num"])
rapport["performance réelle ≥ 0,80"] = bool(auc_prod >= 0.80)
rapport["dérive des entrées (PSI max < 0,25)"] = bool(psi_max < 0.25)
```

```
print(pd.Series(rapport).to_string())
print(f"AUC test du candidat : {auc_cand} | AUC sur 300 prédictions étiquetées : {auc_prod:.3f} | PSI
↪ maximal : {psi_max:.3f}")
```

```
tests de données                True
porte de qualité                True
registre : alias champion      True
service : /health              True
performance réelle ≥ 0,80      True
dérive des entrées (PSI max < 0,25) True
AUC test du candidat : 0.898 | AUC sur 300 prédictions étiquetées : 0.901 | PSI maximal : 0.079
```

Lecture. Le parcours passe tous les contrôles. Remarquez deux choses. D'abord, l'AUC sur **300 prédictions seulement** est entourée d'une grande incertitude (application 4.4) : elle sert à vérifier qu'il n'y a pas d'effondrement, pas à classer deux versions. Ensuite, chaque ligne du rapport correspond à une section du chapitre : tests et porte (4.1, 4.6), registre (4.5), service (4.2), journal et étiquettes (4.3), dérive (4.7).

Pour aller plus loin. Cassez volontairement une étape (envoyez la récence en semaines à l'étape 5 ; entraînez le candidat sur 300 clients ; changez 0.80 en 0.95) et notez **quelle ligne du rapport échoue en premier**, et si c'est bien celle que vous attendiez.

Exercices

Exercice 4.1 ★ — Que manque-t-il pour reproduire ? (section 4.1)

Pour chacune des trois situations, dites lequel des cinq éléments de la liste de reproductibilité (graines, versions, empreinte des données, configuration, code) fait défaut. (a) En ré-entraînant le modèle de la semaine dernière sur les mêmes données avec le même code, une forêt aléatoire rend des probabilités légèrement différentes. (b) Après une mise à jour de l'environnement, l'AUC baisse de 0,01 alors que rien n'a changé dans le code ni les données. (c) Un fichier de données a été corrigé à la main sur un poste ; personne ne sait si tel modèle a été appris avant ou après la correction.

Exercice 4.2 ★★ — Une échelle de satisfaction qui change (section 4.1)

La satisfaction moyenne est notée de 1 à 5 à l'entraînement. Un nouveau canal d'export l'envoie **en pourcentage** (note $\times 20$). Mesurez, pour la régression logistique v1 et pour le boosting v2, l'AUC et la probabilité moyenne prédite sur le jeu de test. Lequel des deux modèles est le plus sensible à cette erreur, et pourquoi ?

Exercice 4.3 ★★ — Exporter la régression logistique en ONNX (section 4.2)

Construisez à la main le graphe ONNX de la partie linéaire de v1 (produit matriciel avec biais, puis sigmoïde), exécutez-le avec `onnxruntime` sur les variables **préparées**, et mesurez l'écart avec `predict_proba`. Que se passe-t-il si l'on envoie au graphe les variables **non préparées** (sans imputation ni mise à l'échelle) ?

Exercice 4.4 ★★ — Latence, débit, disponibilité (sections 4.2 et 4.3)

- (a) Le site envoie 300 requêtes par seconde, chacune dure 80 ms, et un pod en traite 5 à la fois. Combien de pods faut-il, avec une marge de 25 % ? (b) Un objectif de disponibilité de 99,9 % sur 30 jours : combien de minutes d'indisponibilité le budget d'erreur autorise-t-il ? (c) Une page fait trois appels successifs au service ; chaque appel a 95 % de chances d'être sous le centile p95 (par définition). Quelle est la probabilité que les trois le soient, en supposant les appels indépendants ? Qu'en conclut-on sur l'engagement de service ?

Exercice 4.5 ★★★ — Choisir la règle d'alerte au moindre coût (section 4.3)

On surveille le score moyen quotidien d'un service (200 clients par jour). Les 30 premiers jours servent de référence ; une dérive modérée commence au jour 61 (celle du livre). Comparez cinq règles : $|z| > 2$, $|z| > 2,5$, $|z| > 3$, et les deux premières exigées **deux jours de suite**. Chaque fausse alerte (sur les 30 jours calmes qui suivent la référence) coûte 1 unité, chaque jour de retard de détection coûte 3 unités. Quelle règle minimise le coût moyen sur 150 historiques simulés ?

Exercice 4.6 ★★ — Rendre une publication idempotente (section 4.4)

Une tâche « publier » ajoute à la fin d'un fichier unique les scores du jour. Après une reprise, les scores du jour sont en double. Écrivez deux versions de la tâche : l'ancienne et une version **idempotente** (une partition par jour que l'on écrase), puis prouvez la différence en exécutant chaque version deux fois pour le même jour.

Exercice 4.7 ★★ — Un seul octet change tout... ou presque (section 4.5)

Créez une copie du fichier de données où l'âge de cinq clients est augmenté de 1. Comparez les hash des deux fichiers, puis entraînez v1 sur chacune des deux versions : de combien l'AUC de test et les probabilités prédites changent-elles ? Que conclure sur l'utilité du hash, plutôt que de la comparaison des performances, pour la traçabilité ?

Exercice 4.8 ★★ — Tester les cas limites d'une API (section 4.6)

Écrivez des tests de contrat pour `/predict_batch` : (a) une liste vide, (b) un seul client, (c) une liste de 10 clients dont le **quatrième** est invalide (âge de 17 ans), (d) un client avec un champ supplémentaire inconnu. Quels codes de statut attendez-vous ? Pour (c), où le message d'erreur désigne-t-il l'élément fautif ?

Exercice 4.9 ★★ — Limiter le débit par fenêtre de temps (section 4.6)

Écrivez un limiteur de débit à **fenêtre fixe** : au plus 3 appels par clé et par fenêtre de 60 secondes. Pour pouvoir le tester sans attendre, la fonction reçoit l'instant courant en argument. Vérifiez : trois appels acceptés, le quatrième refusé, puis de nouveau accepté dans la fenêtre suivante.

Exercice 4.10 ★★ — Un PSI à la main (section 4.7)

La référence a pour parts $p = (0,4; 0,3; 0,2; 0,1)$ dans quatre intervalles, la période récente $q = (0,25; 0,25; 0,25; 0,25)$. Calculez le PSI, la contribution de chaque intervalle, et vérifiez que le PSI est la somme de $KL(q||p)$ et de $KL(p||q)$. Quel intervalle contribue le plus ?

Exercice 4.11 ★★★ — Boucle de rétroaction et groupe témoin (section 4.7)

Sur le jeu de test, simulez une campagne de rétention : les clients dont v2 donne un score d'au moins 0,30 sont retenus (donc étiquetés « n'a pas résilié ») avec une probabilité de 75 %, **sauf** un groupe témoin de 10 % tiré au hasard qui ne reçoit pas la campagne. Comparez l'AUC de v2 mesurée (a) sur les étiquettes d'origine (sans campagne), (b) sur les étiquettes de tous les clients après la campagne, (c) sur le seul groupe témoin. Quelle mesure est la moins biaisée, et pourquoi ?

Exercice 4.12 ★★★ — Écrire une politique de ré-entraînement (section 4.7)

Proposez une règle de ré-entraînement fondée sur la performance mesurée (AUC perdue par rapport à la semaine 1) et appliquez-la aux deux scénarios du livre (dérive des variables, dérive du concept) : en quelle semaine se déclenche-t-elle ? Complétez par les quatre éléments qu'une politique **écrite** doit contenir.

Corrigés

Corrigé 4.1

- (a) **Les graines** : sans graine fixée, l'aléa de la forêt (échantillons bootstrap, variables tirées) change à chaque entraînement. (b) **Les versions** des bibliothèques : une mise à jour a changé un calcul (algorithme d'optimisation, valeur par défaut) sans que le code bouge. (c) **L'empreinte des données** : sans hash enregistré avec le modèle, on ne peut pas dire quelle version du fichier l'a produit.

Corrigé 4.2

```
Xs = Xte.assign(satisfaction_moy=Xte["satisfaction_moy"] * 20)
lignes = []
for nom, m in (("v1 (logistique)", v1), ("v2 (boosting)", v2)):
    p0, p1 = m.predict_proba(Xte[:, 1], m.predict_proba(Xs[:, 1])
    lignes.append((nom, roc_auc_score(yte, p0), roc_auc_score(yte, p1), p0.mean(), p1.mean()))
print(pd.DataFrame(lignes, columns=["modèle", "AUC correct", "AUC avec x20", "proba moyenne
↪ correcte", "proba moyenne avec x20"]).round(3).to_string(index=False))
```

modèle	AUC correct	AUC avec x20	proba moyenne correcte	proba moyenne avec x20
v1 (logistique)	0.867	0.699	0.131	0.013
v2 (boosting)	0.893	0.858	0.119	0.097

La régression logistique est **linéaire** dans la variable mise à l'échelle : une satisfaction multipliée par 20 devient une valeur standardisée de l'ordre de 20 fois plus grande, qui **extrapole** linéairement et écrase le reste du modèle (AUC 0,699 au lieu de 0,867, probabilité moyenne de 1,3 %). Les arbres du boosting, eux, **saturent** : toute valeur au-dessus du maximum vu à l'entraînement tombe dans la dernière coupure, la prédiction cesse de changer (AUC 0,858 au lieu de 0,893). Un modèle à arbres est donc plus **robuste** à ce type d'erreur, mais pas immunisé : il traite désormais tous les clients comme parfaitement satisfaits.

Corrigé 4.3

```
import onnx, onnxruntime as ort
from onnx import helper, TensorProto, numpy_helper
Z = v1.named_steps["pre"].transform(Xte); Z = Z.toarray() if hasattr(Z, "toarray") else Z
clf = v1.named_steps["clf"]; W, b = clf.coef_.T.astype(np.float32), clf.intercept_.astype(np.float32)
graphe = helper.make_graph([helper.make_node("Gemm", ["Z", "W", "b"], ["lin"]),
    ↪ helper.make_node("Sigmoid", ["lin"], ["p"])], "resiliation",
    [helper.make_tensor_value_info("Z", TensorProto.FLOAT, [None, W.shape[0]]),
     [helper.make_tensor_value_info("p", TensorProto.FLOAT, [None, 1])],
     [numpy_helper.from_array(W, "W"), numpy_helper.from_array(b, "b")])
modele_onnx = helper.make_model(graphe, opset_imports=[helper.make_opsetid("", 13)]);
    ↪ modele_onnx.ir_version = 8
onnx.checker.check_model(modele_onnx)
session = ort.InferenceSession(modele_onnx.SerializeToString(), providers=["CPUExecutionProvider"])
p_onnx = session.run(None, {"Z": Z.astype(np.float32)})[0][:, 0]
ecart = float(np.abs(p_onnx - v1.predict_proba(Xte[:, 1]).max())
brut = Xte[df["num"]].fillna(0).to_numpy(np.float32) # numériques
    ↪ bruts, sans imputation ni échelle
brut = np.column_stack([brut, np.zeros((len(brut), Z.shape[1] - brut.shape[1]), np.float32)])
p_brut = session.run(None, {"Z": brut})[0][:, 0]
print("écart maximal avec predict_proba :", f"{ecart:.1e}", "| AUC du graphe sur entrées brutes :",
    ↪ round(roc_auc_score(yte, p_brut), 3))
```

écart maximal avec predict_proba : 1.9e-07 | AUC du graphe sur entrées brutes : 0.713

Sur les variables **préparées**, le graphe rend les mêmes probabilités que scikit-learn à 1.9e-07 près (simple précision). Sur les variables **non préparées**, la sortie n'a plus de sens (AUC 0,713, probabilité moyenne 47,9 %) : le graphe ne contient que la partie linéaire, **la préparation fait partie du modèle** et doit être exportée ou refaite à l'identique en service (4.1).

Corrigé 4.4

```
L = 300 * 0.080
pods = math.ceil(1.25 * L / 5)
budget = (1 - 0.999) * 30 * 24 * 60
p_trois = 0.95 ** 3
print(f"(a) requêtes simultanées L = {L:.0f} ; pods = ceil(1,25 × {L:.0f} / 5) = {pods}")
print(f"(b) budget d'erreur = {budget:.1f} minutes")
print(f"(c) probabilité que les trois appels soient sous le p95 = {p_trois:.3f}")
```

(a) requêtes simultanées $L = 24$; pods = $\text{ceil}(1,25 \times 24 / 5) = 6$
 (b) budget d'erreur = 43.2 minutes
 (c) probabilité que les trois appels soient sous le p95 = 0.857

- (a) La loi de Little donne $L = \lambda W = 300 \times 0,08 = 24$ requêtes simultanées ; à 5 par pod avec 25 % de marge, $\lceil 1,25 \times 24 / 5 \rceil = 6$ pods. (b) $0,001 \times 30 \times 24 \times 60 = 43,2$ minutes par mois : passer de 99,5 % à 99,9 % divise le budget par cinq. (c) $0,95^3 \approx 0,857$: même si chaque appel respecte son objectif 95 fois sur 100, **la page entière ne le respecte que 86 fois sur 100**. Un engagement de service doit donc être fixé sur le **parcours complet** que vit l'utilisateur, pas seulement sur chaque appel.

Corrigé 4.5

```
s2all = v2.predict_proba(Xpool)[: , 1]
def zs(s):
    s = s.fillna(s.median()); return ((s - s.mean()) / s.std()).to_numpy()
w_derive = np.exp(0.15 * (zs(Xpool["part_achats_promo"]) - zs(Xpool["satisfaction_moy"]))); w_derive
    ↪ /= w_derive.sum()
```

```

def historique(graine):
    rng = np.random.default_rng(graine)
    m = np.array([s2all[rng.choice(len(Xpool), 200, p=None if t < 60 else w_derive)].mean() for t in
    ↪ range(90)])
    return (m - m[:30].mean()) / m[:30].std(ddof=1)

REGLES = {"2σ": (2, 1), "2,5σ": (2.5, 1), "3σ": (3, 1), "2σ deux jours": (2, 2), "2,5σ deux jours":
    ↪ (2.5, 2)}
def declenche(z, seuil, consec):
    a = np.abs(z) > seuil
    return a if consec == 1 else a & np.r_[False, a[:-1]]

resultats = {r: [] for r in REGLES}
for h in range(150):
    z = historique(9000 + h)
    for r, (seuil, consec) in REGLES.items():
        a = declenche(z, seuil, consec)
        pos = np.flatnonzero(a[60:])
        resultats[r].append((a[30:60].sum(), pos[0] + 1 if len(pos) else 30))    # (fausses alertes,
    ↪ retard ; 30 si non détectée)
moy = pd.DataFrame({r: np.mean(v, axis=0) for r, v in resultats.items()}, index=["fausses alertes",
    ↪ "retard (jours)"]).T
for nom, (c_fausse, c_retard) in {"coût A (1 par fausse alerte, 3 par jour de retard)": (1, 3), "coût
    ↪ B (30 par fausse alerte, 3 par jour de retard)": (30, 3)}.items():
    moy[nom.split(' ')[0]] = c_fausse * moy["fausses alertes"] + c_retard * moy["retard (jours)"]
print(moy.round(1).to_string())

```

	fausses alertes	retard (jours)	coût A	coût B
2σ	1.7	5.9	19.4	68.9
2,5σ	0.5	11.1	33.8	48.9
3σ	0.2	17.7	53.4	58.6
2σ deux jours	0.1	19.9	59.8	62.3
2,5σ deux jours	0.0	26.5	79.5	79.9

Avec les coûts de l'énoncé (coût A), la règle qui minimise le coût moyen est « 2σ » (coût moyen 19,4). **Le résultat dépend des coûts** : si une fausse alerte coûte 30 unités (une nuit d'astreinte) au lieu de 1 (coût B), la règle optimale n'est plus la même : « 2,5σ » (coût moyen 48,9). Il n'existe pas de « bon seuil » en soi, seulement un seuil adapté au coût relatif des deux erreurs : modifiez les coûts dans le code pour le vérifier.

Corrigé 4.6

```

dossier = os.path.join(WORK, "ex46"); os.makedirs(os.path.join(dossier, "partitions"), exist_ok=True)
fichier_unique = os.path.join(dossier, "scores.csv")
scores_jour = pd.DataFrame({"id": range(40), "proba": v2.predict_proba(Xpool.iloc[:40])[:, 1]})

def publier_ajout(jour, df):                                # non idempotente : ajoute
    ↪ à la fin
    df.assign(jour=jour).to_csv(fichier_unique, mode="a", header=not os.path.exists(fichier_unique),
    ↪ index=False)

def publier_partition(jour, df):                            # idempotente : écrase la
    ↪ partition du jour
    df.assign(jour=jour).to_csv(os.path.join(dossier, "partitions", f"jour={jour}.csv"), index=False)

for _ in range(2):
    publier_ajout(0, scores_jour); publier_partition(0, scores_jour)
n_ajout = len(pd.read_csv(fichier_unique))
n_part = sum(len(pd.read_csv(os.path.join(dossier, "partitions", f))) for f in
    ↪ os.listdir(os.path.join(dossier, "partitions")))

```

```
print("lignes après deux exécutions du jour 0 : par ajout", n_ajout, "| par partition", n_part)
```

```
lignes après deux exécutions du jour 0 : par ajout 80 | par partition 40
```

La version par ajout contient **80 lignes** pour 40 clients : chaque reprise duplique les données. La version par partition en contient **40**, quel que soit le nombre d'exécutions. Les trois façons d'obtenir l'idempotence (livre, 4.4) sont : écraser la sortie du jour, faire un *upsert* selon une clé, ou écrire dans un fichier temporaire puis le renommer d'un coup.

Corrigé 4.7

```
copie = os.path.join(WORK, "clients_modifie.csv")
tab = pd.read_csv("donnees/clients_ml.csv"); tab.loc[:, "age"] += 1; tab.to_csv(copie, index=False)
h0, h1 = hash_fichier("donnees/clients_ml.csv", 16), hash_fichier(copie, 16)
d1 = donnees(copie); v1b = modele_v1(d1)
p0 = v1.predict_proba(Xte)[: , 1]
p1 = v1b.predict_proba(d1["Xte"][: , 1])
print("hash original :", h0, "| hash modifié :", h1)
print("écart d'AUC de test :", round(roc_auc_score(yte, p0) - roc_auc_score(d1["yte"], p1), 5), "|
↳ écart maximal de probabilité :", round(float(np.abs(p0 - p1).max()), 5))
```

```
hash original : fa48edff3715fdc7 | hash modifié : 041b3c3d9ffbd37a
écart d'AUC de test : -4e-05 | écart maximal de probabilité : 0.00642
```

Les deux hash n'ont **rien en commun**, alors que l'AUC de test et les probabilités sont quasiment identiques : la comparaison des performances ne permet **pas** de savoir quelle version des données a servi, tandis que le hash le dit sans ambiguïté. C'est pour cela que l'on enregistre le hash des données avec chaque modèle (4.5) : la traçabilité est une propriété d'**identité**, pas de performance.

Corrigé 4.8

```
api8 = TestClient(creer_app(chemin_v2, COLONNES, version="2.0.0"))
clients10 = [ligne_json(Xte.iloc[i]) for i in range(10)]
cas = {"(a) liste vide": [], "(b) un client": clients10[:1],
      "(c) quatrième invalide": [{**c, "age": 17} if i == 3 else c for i, c in enumerate(clients10)],
      "(d) champ inconnu": [{**clients10[0], "champ_inconnu": 1}]}
for nom, corps in cas.items():
    r = api8.post("/predict_batch", json=corps)
    detail = r.json().get("detail")
    print(f"{nom:24s}→ {r.status_code}", "| loc :", detail[0]["loc"] if detail else "", "|",
          ↳ len(r.json().get("probabilites", [])), "probabilités")
```

```
(a) liste vide          → 200 | loc : | 0 probabilités
(b) un client           → 200 | loc : | 1 probabilités
(c) quatrième invalide → 422 | loc : ['body', 3, 'age'] | 0 probabilités
(d) champ inconnu       → 200 | loc : | 1 probabilités
```

Attendus : (a) 200 avec une liste vide (un lot vide est légitime) ; (b) 200 avec une probabilité ; (c) **422**, et le chemin d'erreur ['body', 3, 'age'] désigne l'élément de **position 3** (le quatrième) : le message dit exactement où est la faute ; (d) 200, car pydantic **ignore** par défaut les champs supplémentaires. Ce dernier comportement est un choix de contrat : si l'on préfère un refus, il faut le demander explicitement (*extra="forbid"*). Un cas limite non testé (ici, la liste vide) est souvent celui qui fait tomber le service en production.

Corrigé 4.9

```
class LimiteurFenetre:
    def __init__(self, max_appels=3, fenetre=60):
        self.max, self.fenetre, self.etat = max_appels, fenetre, {}

    def autoriser(self, cle, maintenant):
        debut, n = self.etat.get(cle, (maintenant, 0))
        if maintenant - debut >= self.fenetre:                # nouvelle fenêtre
            debut, n = maintenant, 0
        if n >= self.max:
            self.etat[cle] = (debut, n); return False
        self.etat[cle] = (debut, n + 1); return True

lim = LimiteurFenetre()
suite = [lim.autoriser("k", t) for t in (0, 10, 20, 30, 59)] + [lim.autoriser("k", t) for t in (61,
↪ 62)]
print(suite)
assert suite == [True, True, True, False, False, True, True]

[True, True, True, False, False, True, True]
```

Trois appels passent, le quatrième et le cinquième (dans la même fenêtre de 60 s) sont refusés (le code HTTP serait 429), puis les appels repassent dès la fenêtre suivante. Cette version a un défaut connu : une rafale à cheval sur deux fenêtres peut faire passer **deux fois** le quota en peu de temps. La **fenêtre glissante** ou le **seau à jetons** corrigent cela, au prix d'un peu plus d'état à conserver.

Corrigé 4.10

```
p = np.array([0.4, 0.3, 0.2, 0.1]); q = np.array([0.25, 0.25, 0.25, 0.25])
termes = (q - p) * np.log(q / p)
kl_qp, kl_pq = (q * np.log(q / p)).sum(), (p * np.log(p / q)).sum()
print("contributions :", termes.round(4), "| PSI =", round(termes.sum(), 4))
print("KL(q||p) + KL(p||q) =", round(kl_qp + kl_pq, 4), "| intervalle le plus contributif :",
↪ int(termes.argmax()) + 1)
assert abs(kl_qp + kl_pq - termes.sum()) < 1e-12
```

```
contributions : [0.0705 0.0091 0.0112 0.1374] | PSI = 0.2282
KL(q||p) + KL(p||q) = 0.2282 | intervalle le plus contributif : 4
```

Les contributions sont toutes positives (les deux facteurs $(q_i - p_i)$ et $\ln(q_i/p_i)$ ont le même signe), le PSI est leur somme, et il coïncide avec $KL(q||p) + KL(p||q)$. L'intervalle 4 contribue le plus : sa part passe de 10 % à 25 %, soit un rapport de 2,5, et le logarithme du rapport est grand même si l'écart absolu (15 points) est le même que pour l'intervalle 1 (qui passe de 40 % à 25 %).

Corrigé 4.11

```
rng = np.random.default_rng(11)
s = v2.predict_proba(Xte)[: , 1]
signale = s >= 0.30
temoin = rng.random(len(yte)) < 0.10
retenu = signale & ~temoin & (rng.random(len(yte)) < 0.75)
y_apres = yte.copy(); y_apres[retenu] = 0
auc_origine = roc_auc_score(yte, s); auc_tous = roc_auc_score(y_apres, s); auc_temoin =
↪ roc_auc_score(y_apres[temoin], s[temoin])
print(pd.Series({"(a) étiquettes d'origine (sans campagne)": auc_origine, "(b) tous, après campagne":
↪ auc_tous, "(c) groupe témoin seul": auc_temoin}).round(3).to_string())
```

(a) étiquettes d'origine (sans campagne)	0.893
(b) tous, après campagne	0.797
(c) groupe témoin seul	0.853

La mesure (b) tombe à 0,797 alors que le modèle n'a pas changé : la campagne a **modifié les étiquettes** des clients que le modèle signale, et ces « succès » de la rétention sont comptés comme des erreurs du modèle. La mesure (c), faite sur le seul groupe témoin (240 clients qui n'ont pas reçu la campagne), retrouve une valeur proche de (a) (0,853 contre 0,893), au bruit d'échantillonnage près (le groupe ne compte que 240 clients, donc une trentaine de résiliations, et l'AUC y est peu précise) : c'est la **seule** qui estime la performance du modèle comme prédicteur du risque en l'absence d'intervention. D'où la règle : réserver un petit groupe témoin aléatoire (4.7), dont les étiquettes servent à la mesure et au ré-entraînement.

Corrigé 4.12

```
lignes = []
for scenario in ("covariable", "concept"):
    aucs = []
    for k in range(1, 5):
        Xw, yw = semaine(d, v2, k, scenario)
        aucs.append(roc_auc_score(yw, v2.predict_proba(Xw)[: , 1]))
    declenchees = [k + 1 for k, a in enumerate(aucs) if a < aucs[0] - 0.05]
    lignes.append((scenario, [round(a, 3) for a in aucs], declenchees[0] if declenchees else "jamais"))
print(pd.DataFrame(lignes, columns=["scénario", "AUC par semaine", "1re semaine de
↳ déclenchement"]).to_string(index=False))
```

scénario	AUC par semaine	1re semaine de déclenchement
covariable	[0.876, 0.859, 0.869, 0.857]	jamais
concept	[0.876, 0.847, 0.808, 0.776]	3

La règle « ré-entraîner dès que l'AUC mesurée a perdu plus de 0,05 par rapport à la semaine 1 » ne se déclenche **jamais** pour la dérive des variables (le modèle reste bon : inutile de payer un ré-entraînement) et se déclenche à la semaine 3 pour la dérive du concept. Attention : cette semaine est celle où **les étiquettes sont arrivées** ; dans la réalité, avec un délai de 90 jours, la décision n'est prise que bien après le début de la dérive, d'où l'intérêt de compléter par des alertes sur les entrées et le score. Une politique **écrite** doit contenir au moins : (1) **les déclencheurs** et leurs seuils (performance, PSI sur variables importantes) ; (2) **le diagnostic préalable** (bug en amont ou changement réel ?) et qui le fait ; (3) **les données** utilisées (fenêtre récente, groupe témoin pour éviter la boucle de rétroaction) ; (4) **les portes** que le nouveau modèle doit franchir (porte de qualité, déploiement progressif, retour arrière), et qui décide.

Chapitre 5 : Ingénierie des données

— exercices et applications

Ce chapitre du cahier accompagne le chapitre 5 du livre. Les **applications** refont pas à pas ce que le livre a résumé : un ETL complet, sa version en SQL, le chargement incrémental, les règles de qualité, le contrat de données, la réconciliation des produits et des clients, le lignage et la pseudonymisation, puis la collecte sur un site local. Les **exercices** (à la main d'abord, puis avec du code) sont corrigés à la fin. Données : les quatre exports bruts de `donnees/sources/` (commandes, CRM, catalogue, fournisseur) et leurs fichiers de vérité (`verite_*.csv`). Prérequis : les sections 5.1 à 5.5 du livre, selon l'exercice.

Préparation

Une seule cellule charge les données brutes **comme du texte** (on ne laisse pas la bibliothèque deviner) et les fonctions du chapitre, rangées dans `build/outils_ch05.py` : lecture, typage des commandes, motifs de rejet, normalisation des libellés, serveur local de démonstration, journal de lignage. Les applications s'en servent.

```
import os, sys, re, hashlib, hmac, time, tempfile, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np
import pandas as pd
import duckdb
from rapidfuzz import fuzz
from rapidfuzz.distance import Levenshtein, JaroWinkler
from outils_ch05 import *

brut = lire("commandes_export")
crm = lire("clients_crm"); crm["id_crm"] = crm["id_crm"].astype(int)
catalogue = lire("produits_catalogue"); catalogue["prix_catalogue"] =
↳ catalogue["prix_catalogue"].astype(float)
fournisseur = lire("produits_fournisseur"); fournisseur["prix_achat"] =
↳ fournisseur["prix_achat"].astype(float)
clients_connus, produits_connus = set(crm["id_crm"]), set(catalogue["id_produit"])
TMP = tempfile.mkdtemp(prefix="cah05_", dir=os.environ.get("TMPDIR"))      # fichiers
↳ intermédiaires, supprimés à la fin
print({"commandes": len(brut), "crm": len(crm), "catalogue": len(catalogue), "fournisseur":
↳ len(fournisseur)})
```

```
{'commandes': 19700, 'crm': 5000, 'catalogue': 48, 'fournisseur': 40}
```

Applications

Application 5.1 — Un ETL pas à pas (sections 5.1.2 à 5.1.5)

Objectif. Transformer les 19 700 lignes de commandes brutes en une table propre, en rendant compte de **chaque** ligne écartée.

Étape 1 : extraire sans deviner, puis regarder ce qu'on a lu. On garde tout en texte et l'on ajoute des métadonnées de chargement. Un premier profil montre les formats rencontrés.

```
lu = brut.copy()
lu["_charge_le"] = "2025-12-31"
lu["_fichier"] = "commandes_export.csv"
format_date = lu["date"].map(lambda s: "AAAA-MM-JJ" if re.fullmatch(r"\d{4}-\d{2}-\d{2}", s)
                             else "JJ/MM/AAAA" if "/" in s else "MM-JJ-AAAA")
print(format_date.value_counts().to_string())
print("prix avec virgule décimale :", int(lu["prix_unitaire"].str.contains(",").sum()))
```

```
date
AAAA-MM-JJ    11786
JJ/MM/AAAA    5897
MM-JJ-AAAA    2017
prix avec virgule décimale : 5824
```

Trois formats de date coexistent (59,8 % au format ISO, 29,9 % JJ/MM/AAAA, 10,2 % MM-JJ-AAAA) et près d'un prix sur trois (5 824 lignes) utilise la virgule décimale : une lecture « intelligente » aurait deviné autre chose pour chacun.

Étape 2 : typer explicitement. Le format MM-JJ-AAAA est ambigu (« 12-03-2025 » : 3 décembre ou 12 mars ?) : nous avons constaté, avec l'équipe de la caisse, qu'un tiret signale le format américain. Cette décision est **écrite** dans la fonction `parser_date`, pas dans la tête de quelqu'un.

```
typ = typer_commandes(lu)
print(typ.dtypes.to_string())
print("dates illisibles :", int(typ["date"].isna().sum()), "| quantités manquantes :",
      int(typ["quantite"].isna().sum()))
```

```
id_commande    int64
date           datetime64[us]
id_client      int64
id_produit     str
quantite       float64
prix_unitaire  float64
_charge_le     str
_fichier       str
dates illisibles : 0 | quantités manquantes : 286
```

Étape 3 : dédoublonner, puis rejeter avec un motif. On compte à chaque étape pour pouvoir écrire l'équation de conservation.

```
sans_doublon_exact = typ.drop_duplicates()
unique = sans_doublon_exact.drop_duplicates("id_commande")
motif = motif_rejet(unique, clients_connus, produits_connus)
propres_a = unique[motif.isna()]
rejets_a = unique[motif.notna()].assign(motif=motif[motif.notna()])
print("lignes lues", len(typ), "| doublons", len(typ) - len(unique), "| rejets", len(rejets_a), "|
      propres", len(propres_a))
print("conservation :", len(typ) - (len(typ) - len(unique)) - len(rejets_a) == len(propres_a))
print(rejets_a["motif"].value_counts().to_string())
```

```
lignes lues 19700 | doublons 1700 | rejets 782 | propres 17218
conservation : True
motif
client inconnu          334
quantité manquante      245
quantité négative ou nulle 203
```

L'équation est vérifiée : $19\,700 - 1\,700 - 782 = 17\,218$. Chacune des 782 lignes écartées a un **motif** : rien n'a disparu en silence.

Étape 4 : tester. Un test sur des cas écrits à la main, et deux garde-fous sur la table produite. Le dernier compare avec la fonction complète du chapitre.

```
assert parser_date("03/12/2025") == pd.Timestamp("2025-12-03")
assert parser_date("12-03-2025") == pd.Timestamp("2025-12-03")
assert pd.isna(parser_date("3 décembre"))
assert propres_a["id_commande"].is_unique and (propres_a["quantite"] >= 1).all()
propres_b, rejets_b, stats = etl_commandes(brut, clients_connus, produits_connus)
assert propres_a.drop(columns=["_charge_le", "_fichier"]).reset_index(drop=True)
↳ ).equals(propres_b.drop(columns="montant").reset_index(drop=True))
print("tous les tests passent ;", stats)
```

```
tous les tests passent ; {'lignes_brutes': 19700, 'apres_doublons_exacts': 18000, 'apres_cle_unique':
↳ 18000, 'rejets': 782, 'propres': 17218}
```

À retenir. Les 1 700 doublons exacts écartent **aussi** toutes les clés répétées : après typage, deux lignes de même `id_commande` sont identiques ligne à ligne (aucune ligne n'est en conflit avec elle-même). Si ce n'était pas le cas, il faudrait **choisir** laquelle garder, et le dire.

Application 5.2 — La même transformation en SQL avec DuckDB (section 5.1.6)

Objectif. Écrire les règles du nettoyage en SQL, les exécuter là où se trouvent les données, et **vérifier** que le résultat est identique à celui de pandas.

Étape 1 : charger le brut dans une table de staging. DuckDB lit directement un DataFrame ; on déclare les trois tables dont les règles ont besoin.

```
con = duckdb.connect()
con.register("staging_commandes", brut)
con.register("crm", crm[["id_crm"]])
con.register("catalogue", catalogue[["id_produit"]])
print(con.execute("SELECT count(*) AS lignes, count(DISTINCT id_commande) AS commandes FROM
↳ staging_commandes").df().to_string(index=False))
```

```
lignes  commandes
19700    18000
```

Étape 2 : la requête de nettoyage. Elle type, dédoubleonne (`DISTINCT ON`), puis applique les règles de gestion. On y retrouve, ligne à ligne, les règles de la fonction Python.

```
con.execute("""
CREATE TABLE propres_sql AS
WITH typees AS (
    SELECT CAST(id_commande AS INT) AS id_commande,
           CASE WHEN date LIKE '____-__-__' THEN strptime(date, '%Y-%m-%d')
                WHEN date LIKE '__/__/____' THEN strptime(date, '%d/%m/%Y')
                ELSE strptime(date, '%m-%d-%Y') END AS date,
           CAST(id_client AS INT) AS id_client, id_produit,
           CAST(quantite AS DOUBLE) AS quantite,
```

```

        CAST(replace(prix_unitaire, ',', '.') AS DOUBLE) AS prix_unitaire
    FROM staging_commandes),
    uniques AS (SELECT DISTINCT ON (id_commande) * FROM typees)
    SELECT *, round(quantite * prix_unitaire, 2) AS montant FROM uniques
    WHERE quantite >= 1 AND prix_unitaire > 0
        AND id_client IN (SELECT id_crm FROM crm) AND id_produit IN (SELECT id_produit FROM catalogue)""")
print(con.execute("SELECT count(*) AS lignes, round(sum(montant), 2) AS ca FROM
↳ propres_sql").df().to_string(index=False))

```

```

lignes      ca
17218  893243.24

```

Étape 3 : comparer les deux chemins. Porter une transformation d'un outil à un autre se teste : on exige l'égalité, ligne à ligne, pas seulement sur le total.

```

a = con.execute("SELECT * FROM propres_sql ORDER BY id_commande").df()
b = propres_b.sort_values("id_commande").reset_index(drop=True)
print("mêmes identifiants :", (a["id_commande"].to_numpy() == b["id_commande"].to_numpy()).all(),
      "| mêmes montants :", np.allclose(a["montant"].to_numpy(), b["montant"].to_numpy()))

```

```

mêmes identifiants : True | mêmes montants : True

```

Étape 4 : un mart en SQL. Le chiffre d'affaires mensuel par catégorie, que le tableau de bord de la gérante consomme.

```

con.register("cat_complet", catalogue[["id_produit", "categorie"]])
mart = con.execute("""SELECT strftime(date_trunc('month', p.date), '%Y-%m') AS mois, c.categorie,
↳ round(sum(p.montant), 2) AS ca
                        FROM propres_sql p JOIN cat_complet c USING (id_produit)
                        GROUP BY 1, 2 ORDER BY 1, 2""").df()
print(mart.head(4).to_string(index=False)); print(len(mart), "lignes (12 mois × 4 catégories) ;
↳ total", round(mart["ca"].sum(), 2))

```

```

    mois categorie      ca
2025-01         A  18296.46
2025-01         B  18099.45
2025-01         C  20840.94
2025-01         D  18256.75
48 lignes (12 mois × 4 catégories) ; total 893243.24

```

Application 5.3 — Un chargement incrémental idempotent (section 5.1.7)

Objectif. Charger les commandes propres par lots, rejouer un lot après un « incident », et comparer l'ajout naïf avec la fusion par clé.

Étape 1 : trois lots par période.

```

propres_b["mois"] = propres_b["date"].dt.month
lots = {1: propres_b[propres_b["mois"] <= 4], 2: propres_b[propres_b["mois"].between(5, 8)], 3:
↳ propres_b[propres_b["mois"] >= 9]}
cols = ["id_commande", "date", "id_client", "id_produit", "quantite", "prix_unitaire", "montant"]
print({k: len(v) for k, v in lots.items()}, "total", sum(len(v) for v in lots.values()))

```

```

{1: 5611, 2: 5893, 3: 5714} total 17218

```

Étape 2 : la fusion par clé (upsert). La clé primaire déclare l'unicité ; ON CONFLICT ... DO UPDATE met à jour au lieu de dupliquer.

```

con.execute("CREATE TABLE cible (id_commande INT PRIMARY KEY, date TIMESTAMP, id_client INT,
↳ id_produit VARCHAR, quantite DOUBLE, prix_unitaire DOUBLE, montant DOUBLE)")
def charger(lot):

```

```

con.register("lot", lot[cols])
con.execute("""INSERT INTO cible SELECT * FROM lot ON CONFLICT (id_commande) DO UPDATE
    SET quantite = excluded.quantite, prix_unitaire = excluded.prix_unitaire, montant =
↪ excluded.montant""")
return con.execute("SELECT count(*) FROM cible").fetchone()[0]
print("après chaque lot :", [charger(lots[k]) for k in (1, 2, 3)])
print("après rejou du lot 2, puis du lot 3 :", charger(lots[2]), charger(lots[3]))

```

```

après chaque lot : [5611, 11504, 17218]
après rejou du lot 2, puis du lot 3 : 17218 17218

```

Étape 3 : le même scénario avec l'ajout naïf. Sans clé, INSERT ajoute tout ce qu'on lui donne.

```

con.execute("CREATE TABLE naive AS SELECT * FROM (SELECT * FROM propres_sql LIMIT 0)")
for k in (1, 2, 2, 3):                                # le lot 2 est rejoué
    con.register("lot", lots[k][cols]); con.execute("INSERT INTO naive (id_commande, date, id_client,
↪ id_produit, quantite, prix_unitaire, montant) SELECT * FROM lot")
print("ajout naïf avec rejou :", con.execute("SELECT count(*) FROM naive").fetchone()[0], "lignes ;
↪ doublons de clé :",
    con.execute("SELECT count(*) - count(DISTINCT id_commande) FROM naive").fetchone()[0])

```

```

ajout naïf avec rejou : 23111 lignes ; doublons de clé : 5893

```

Étape 4 : une correction à la source. Le lot 2 revient avec 50 prix corrigés (+3 %) ; la fusion doit mettre à jour ces lignes sans en ajouter.

```

corrige = lots[2].copy()
idx = corrige.sample(50, random_state=1).index
corrige.loc[idx, "prix_unitaire"] = (corrige.loc[idx, "prix_unitaire"] * 1.03).round(2)
corrige["montant"] = (corrige["quantite"] * corrige["prix_unitaire"]).round(2)
avant = con.execute("SELECT sum(montant) FROM cible").fetchone()[0]
n = charger(corrige)
apres = con.execute("SELECT sum(montant) FROM cible").fetchone()[0]
attendu = corrige.loc[idx, "montant"].sum() - lots[2].loc[idx, "montant"].sum()
print("lignes :", n, "| variation du CA :", round(apres - avant, 2), "| attendue :", round(attendu, 2))

```

```

lignes : 17218 | variation du CA : 76.53 | attendue : 76.53

```

La table compte toujours **17 218** lignes, et son chiffre d'affaires varie de **76,53 €**, exactement la somme des écarts des 50 lignes corrigées. Le rejou du lot 2 ne change rien (17 218 lignes) ; avec l'ajout naïf, le même scénario laisse 5 893 clés en double.

Application 5.4 — Règles de qualité, score et seuils (sections 5.2.1 à 5.2.5)

Objectif. Écrire les règles comme des fonctions, calculer le taux de violation de chacune, un score par dimension, et décider selon une grille de seuils.

Étape 1 : des règles qui renvoient « vrai si la ligne est fautive ».

```

def vide(d, col):                return d[col].isna()
def hors_domaine(d, col, lo, hi): return d[col].notna() & ~d[col].between(lo, hi)
def repete(d, col):              return d.duplicated(col)
def orpheline(d, col, ref):      return ~d[col].isin(ref)

regles = [("complétude", "date renseignée", vide(typ, "date")), ("complétude", "quantité renseignée",
↪ vide(typ, "quantite")),
    ("complétude", "prix renseigné", vide(typ, "prix_unitaire")),
    ("validité", "quantité ≥ 1", hors_domaine(typ, "quantite", 1, 1e6)), ("validité", "prix dans
↪ ]0 ; 1 000]", hors_domaine(typ, "prix_unitaire", 0.01, 1000)),
    ("validité", "date dans 2025", hors_domaine(typ, "date", pd.Timestamp("2025-01-01"),
↪ pd.Timestamp("2025-12-31"))),

```

```

        ("unicité", "id_commande unique", repete(typ, "id_commande")),
        ("cohérence", "client connu", orpheline(typ, "id_client", clients_connus)), ("cohérence",
↪ "produit connu", orpheline(typ, "id_produit", produits_connus))]
bilan = pd.DataFrame([(d, r, int(m.sum()), round(100 * m.mean(), 2)) for d, r, m in regles],
↪ columns=["dimension", "règle", "violations", "taux (%)"])
print(bilan.to_string(index=False))

```

dimension	règle	violations	taux (%)
complétude	date renseignée	0	0.00
complétude	quantité renseignée	286	1.45
complétude	prix renseigné	0	0.00
validité	quantité ≥ 1	226	1.15
validité	prix dans]0 ; 1 000]	0	0.00
validité	date dans 2025	0	0.00
unicité	id_commande unique	1700	8.63
cohérence	client connu	359	1.82
cohérence	produit connu	0	0.00

Étape 2 : un score global et un score par dimension. Une ligne est conforme si elle ne viole **aucune** règle ; on cumule les violations avec un « ou » logique (ne pas additionner les taux : une ligne peut violer plusieurs règles).

```

viol = np.column_stack([m.to_numpy() for _, _, m in regles]).any(axis=1)
par_dim = {d: round(float(100 * (1 - np.column_stack([m.to_numpy() for dd, _, m in regles if dd ==
↪ d]).any(axis=1).mean()))), 1) for d in dict.fromkeys(d for d, _, _ in regles)}
score = round(float(100 * (1 - viol.mean()))), 1)
print("score global :", score, "|", par_dim)
print("somme des taux :", round(bilan["taux (%)"].sum(), 2), "% > taux de lignes touchées :",
↪ round(100 * viol.mean(), 2), "%")

```

```

score global : 87.4 | {'complétude': 98.5, 'validité': 98.9, 'unicité': 91.4, 'cohérence': 98.2}
somme des taux : 13.05 % > taux de lignes touchées : 12.6 %

```

Étape 3 : la décision.

```

def decision(s):
    return "publier" if s >= 95 else "publier et alerter le propriétaire de la source" if s >= 85 else
↪ "arrêter le chargement"
print(score, "->", decision(score))
mois = typ["date"].dt.month
print("taux de lignes fautives par mois (%) :", (pd.Series(viol,
↪ index=typ.index).groupby(mois).mean() * 100).round(1).tolist())

```

```

87.4 -> publier et alerter le propriétaire de la source
taux de lignes fautives par mois (%) : [13.0, 13.6, 12.5, 11.7, 13.0, 11.9, 13.0, 13.6, 11.7, 13.4,
↪ 11.8, 12.1]

```

Étape 4 : le profilage qui change la règle. Quels identifiants se cachent derrière les « clients inconnus » ?

```

inconnus = typ.loc[orpheline(typ, "id_client", clients_connus), "id_client"]
print(inconnus.value_counts().to_string())
sans_anonymes = orpheline(typ, "id_client", clients_connus | {999999})
print("violations de « client connu » en acceptant l'identifiant 999999 :", int(sans_anonymes.sum()))

```

```

id_client
999999    359
violations de « client connu » en acceptant l'identifiant 999999 : 0

```

Les 359 « clients inconnus » sont **un seul identifiant**, 999999, la valeur par défaut des ventes sans compte. Une fois ce cas reconnu comme légitime, la règle n'a plus aucune violation : la règle de

qualité encodait une **décision métier** qu'il fallait expliciter.

Application 5.5 — Un contrat de données exécutable (section 5.2.7)

Objectif. Écrire le contrat d'un export, le faire respecter à l'arrivée de chaque lot, et distinguer ce qui **bloque** de ce qui **alerte**.

Étape 1 : le contrat et son contrôleur. Le contrat dit quelles colonnes, lesquelles ne peuvent jamais être vides, et le taux de vides toléré ailleurs. Le contrôleur renvoie la liste des manquements, séparés en *erreurs* (on arrête) et *avertissements* (on prévient).

```
contrat = {"colonnes": ["id_commande", "date", "id_client", "id_produit", "quantite",
↳ "prix_unitaire"],
          "non_nulles": ["id_commande", "date", "id_client", "id_produit"], "taux_vide_max":
↳ {"quantite": 0.02, "prix_unitaire": 0.0}}

def verifier(lot, contrat):
    erreurs, avertissements = [], []
    erreurs += [f"colonne manquante : {c}" for c in contrat["colonnes"] if c not in lot.columns]
    avertissements += [f"colonne inattendue : {c}" for c in lot.columns if c not in
↳ contrat["colonnes"]]
    erreurs += [f"vide interdit : {c}" for c in contrat["non_nulles"] if c in lot.columns and
↳ lot[c].isna().any()]
    erreurs += [f"trop de vides dans {c} : {lot[c].isna().mean():.1%}" for c, m in
↳ contrat["taux_vide_max"].items() if c in lot.columns and lot[c].isna().mean() > m]
    return erreurs, avertissements
```

Étape 2 : quatre lots d'essai. Le lot du jour, un lot où une colonne est renommée, un lot où une colonne est ajoutée, un lot où un champ obligatoire est vide en partie.

```
essais = {"lot du jour": brut,
          "colonne renommée": brut.rename(columns={"quantite": "qte"}),
          "colonne ajoutée": brut.assign(canal="Site"),
          "identifiant client vide": brut.assign(id_client=brut["id_client"].where(brut.index % 500 !=
↳ 0))}

for nom, lot in essais.items():
    e, a = verifier(lot, contrat)
    print(f"{nom:24s} erreurs={e} | avertissements={a}")
```

lot du jour	erreurs=[] avertissements=[]
colonne renommée	erreurs=['colonne manquante : quantite'] avertissements=['colonne
↳ inattendue : qte']	
colonne ajoutée	erreurs=[] avertissements=['colonne inattendue : canal']
identifiant client vide	erreurs=['vide interdit : id_client'] avertissements=[]

Étape 3 : le brancher sur le chargement. Seules les **erreurs** arrêtent ; un avertissement est consigné et le chargement continue.

```
def charger_si_conforme(lot):
    e, a = verifier(lot, contrat)
    if e:
        return f"REFUSÉ ({len(e)} erreur(s))"
    return f"chargé ({len(a)} avertissement(s))"
print({nom: charger_si_conforme(lot) for nom, lot in essais.items()})
```

```
{'lot du jour': 'chargé (0 avertissement(s))', 'colonne renommée': 'REFUSÉ (1 erreur(s))', 'colonne
↳ ajoutée': 'chargé (1 avertissement(s))', 'identifiant client vide': 'REFUSÉ (1 erreur(s))'}
```

Une colonne **ajoutée** n'empêche pas le chargement (elle est ignorée et consignée) ; une colonne **manquante** ou un champ obligatoire **vide** l'arrête. Le contrat distingue ce qui casse le pipeline de ce qui prévient seulement.

Application 5.6 — Rapprocher les produits (sections 5.3.2 à 5.3.4)

Objectif. Retrouver, pour chacune des 40 désignations du fournisseur, le bon produit du catalogue, et mesurer ce que vaut **chaque niveau** de préparation du texte.

Étape 1 : voir le problème. Aucune clé commune, mais des libellés qui se ressemblent.

```
verite_p = lire("verite_produits")
bonne = dict(zip(verite_p["ref_fournisseur"], verite_p["id_produit"]))
print(fournisseur.merge(verite_p, on="ref_fournisseur").merge(catalogue[["id_produit", "libelle"]],
↳ on="id_produit")
      [["designation", "libelle"]].head(6).to_string(index=False))
```

	designation	libelle
Plat	coton (lot)	Plat en coton
	Verre plat	Plat en verre
Plat	céramique (lot)	Plat en céramique
	Céramique bol	Bol en céramique
	BOL coton	Bol en coton
Bol	laine (lot)	Bol en laine

Étape 2 : une fonction de recherche, paramétrée par la préparation du texte.

```
def meilleur(designation, preparer, mesure=fuzz.ratio):
    scores = [mesure(preparer(designation), preparer(l)) for l in catalogue["libelle"]]
    k = int(np.argmax(scores))
    return catalogue["id_produit"][k], scores[k]

def exactitude(preparer, mesure=fuzz.ratio):
    return sum(meilleur(d, preparer, mesure)[0] == bonne[r] for r, d in
↳ zip(fournisseur["ref_fournisseur"], fournisseur["designation"]))
niveaux = {"textes bruts": str, "minuscules": lambda s: str(s).lower(), "normalisation partielle":
↳ lambda s: normaliser(s, expand=False, trier=False),
      "normalisation complète": normaliser}
print(pd.Series({n: exactitude(p) for n, p in niveaux.items()}, name="bonnes réponses sur
↳ 40").to_string())
```

textes bruts	29
minuscules	36
normalisation partielle	37
normalisation complète	40

Étape 3 : comparer les mesures sur textes bruts. Une mesure plus « intelligente » remplace-t-elle la normalisation ?

```
mesures = {"ratio (Levenshtein normalisé)": fuzz.ratio, "token_sort_ratio": fuzz.token_sort_ratio,
↳ "token_set_ratio": fuzz.token_set_ratio, "partial_ratio": fuzz.partial_ratio}
print(pd.Series({n: exactitude(str, m) for n, m in mesures.items()}, name="bonnes réponses sur 40
↳ (textes bruts)").to_string())
```

ratio (Levenshtein normalisé)	29
token_sort_ratio	29
token_set_ratio	23
partial_ratio	16

Aucune mesure ne passe de 29 à 40 sur textes bruts : `token_sort_ratio` ne dépasse pas `ratio`, parce que la **casse** différencie encore « BOL coton » de « Bol en coton ». La préparation du texte compte plus que la mesure : le gain vient de normaliser, pas d'une formule plus savante.

Étape 4 : l'indice du prix. Le prix d'achat vaut-il toujours entre 45 % et 65 % du prix catalogue ? Si oui, on peut écarter les candidats incompatibles.

```
suivi = fournisseur.assign(id_produit=fournisseur["ref_fournisseur"].map(bonne)).merge(catalogue[[_
↪ "id_produit", "prix_catalogue"]], on="id_produit")
rapport = suivi["prix_achat"] / suivi["prix_catalogue"]
print("rapport prix d'achat / prix catalogue : min", round(rapport.min(), 3), "| max",
↪ round(rapport.max(), 3))
```

```
rapport prix d'achat / prix catalogue : min 0.45 | max 0.647
```

Étape 5 : la jointure exacte sur le texte normalisé. Si la normalisation suffit, aucune mesure floue n'est nécessaire.

```
cle_cat = catalogue.assign(cle=catalogue["libelle"].map(normaliser))
cle_four = fournisseur.assign(cle=fournisseur["designation"].map(normaliser))
jointure = cle_four.merge(cle_cat[["cle", "id_produit"]], on="cle", how="left")
print("désignations sans correspondance :", int(jointure["id_produit"].isna().sum()), "| clés du
↪ catalogue en double :", int(cle_cat["cle"].duplicated().sum()),
    "| bonnes réponses :", int((jointure["id_produit"] ==
↪ jointure["ref_fournisseur"].map(bonne)).sum()))
```

```
désignations sans correspondance : 0 | clés du catalogue en double : 0 | bonnes réponses : 40
```

Une simple **jointure exacte** sur le texte normalisé retrouve les 40 produits, sans doublon de clé côté catalogue : ici, la similarité floue est inutile. Le prix d'achat (45 % à 65 % du prix catalogue) reste un bon **garde-fou** quand deux libellés sont ambigus.

Application 5.7 — Rapprocher les clients (sections 5.3.5 à 5.3.7)

Objectif. Retrouver les doublons du CRM par blocage, similarité de noms et recoupement, mesurer précision et rappel, router les cas douteux, puis fusionner.

Étape 1 : préparer la table de travail. `crm_avec_fautes` ajoute la vérité (`id_vrai`) et des fautes de frappe dans le nom de 35 % des doublons récents.

```
cl, n_fautes = crm_avec_fautes(crm, lire("verite_clients"))
for c in ["prenom", "nom", "ville"]:
    cl[c + "_n"] = cl[c].map(lambda s: normaliser(s, expand=False, trier=False))
vraies = int(cl.groupby("id_vrai").size().pipe(lambda g: g * (g - 1) // 2).sum())
print(len(cl), "fiches |", cl["id_vrai"].nunique(), "personnes |", vraies, "paires de doublons |",
↪ n_fautes, "fautes ajoutées")
```

```
5000 fiches | 4200 personnes | 800 paires de doublons | 288 fautes ajoutées
```

Étape 2 : le coût du blocage. Combien de paires faudrait-il comparer avec chaque clé ?

```
def paires_bloc(cols):
    g = cl.groupby(cols).size()
    return int((g * (g - 1) // 2).sum())
for nom, cols in {"même ville": ["ville_n"], "prénom + ville": ["prenom_n", "ville_n"], "prénom +
↪ ville + date": ["prenom_n", "ville_n", "date_inscription"]}.items():
    print(f"{nom:24s}{paires_bloc(cols):>10,}".replace(", ", " "))
print("toutes les paires :", f"{len(cl) * (len(cl) - 1) // 2:,}".replace(", ", " "))
```

```
même ville          1 041 310
prénom + ville      53 153
prénom + ville + date 828
toutes les paires : 12 497 500
```

Étape 3 : comparer les noms dans les blocs, et évaluer. Le blocage fin ne garde que 828 paires ; on score chacune avec Jaro-Winkler sur le nom, puis on mesure à plusieurs seuils.

```

paires = paires_candidates(cl, ["prenom_n", "ville_n", "date_inscription"])
scores = np.array([JaroWinkler.similarity(cl["nom_n"][i], cl["nom_n"][j]) for i, j in paires])
juste = np.array([cl["id_vrai"][i] == cl["id_vrai"][j] for i, j in paires])
lignes = [(th, int((scores >= th).sum()), round(100 * juste[scores >= th].mean(), 1), round(100 *
↳ juste[scores >= th].sum() / vraies, 1)) for th in (0.7, 0.8, 0.9, 0.95, 1.0)]
print(pd.DataFrame(lignes, columns=["seuil", "paires fusionnées", "précision (%)", "rappel
↳ (%)"]).to_string(index=False))

```

seuil	paires fusionnées	précision (%)	rappel (%)
0.70	803	99.6	100.0
0.80	803	99.6	100.0
0.90	803	99.6	100.0
0.95	720	99.7	89.8
1.00	514	99.6	64.0

Étape 4 : trois zones. Fusion automatique au-dessus de 0,95, revue entre 0,80 et 0,95, rejet en dessous.

```

zones = pd.cut(scores, [-1, 0.80, 0.95, 2], right=False, labels=["séparées", "revue", "fusion
↳ automatique"])
print(pd.DataFrame({"zone": zones, "vrai": juste}).groupby("zone",
↳ observed=True)["vrai"].agg(paires="size", vrais_doublons="sum").to_string())

```

	paires	vrais_doublons
zone		
séparées	25	0
revue	83	82
fusion automatique	720	718

Au seuil 0,95, la fusion automatique ne se trompe que deux fois sur 720 ; la revue concerne 83 paires. Les 25 paires les moins ressemblantes ne sont **aucune** un doublon. Les trois fusions à tort au seuil 0,80 sont des homonymes inscrits le même jour.

Étape 5 : la fiche d'or. On relie les paires fusionnées (composantes connexes d'un graphe), puis on garde la plus ancienne fiche de chaque groupe.

```

from scipy.sparse import coo_matrix
from scipy.sparse.csgraph import connected_components
fusion = [(i, j) for (i, j), s in zip(paires, scores) if s >= 0.80]
m = coo_matrix((1 * len(fusion), ([i for i, _ in fusion], [j for _, j in fusion])), shape=(len(cl),
↳ len(cl)))
cl["groupe"] = connected_components(m, directed=False)[1]
fiche_or = cl.sort_values("id_crm").groupby("groupe").first()
print("fiches :", len(cl), "-> fiches d'or :", len(fiche_or), "| personnes réelles :",
↳ cl["id_vrai"].nunique())

```

```
fiches : 5000 -> fiches d'or : 4198 | personnes réelles : 4200
```

Cinq mille fiches deviennent 4 198 personnes pour 4 200 en réalité : les deux fiches d'écart viennent des homonymes fusionnés à tort.

Application 5.8 — Lignage et pseudonymisation (sections 5.4.2 à 5.4.5)

Objectif. Faire enregistrer ses étapes au pipeline, interroger le graphe qui en résulte, puis pseudonymiser une colonne et mesurer ce qu'un attaquant en retrouve.

Étape 1 : un journal de lignage.

```

journal = Journal()
journal.enregistrer("chargement", ["commandes_export"], ["stg_commandes"], len(brut), len(brut))
journal.enregistrer("typage et dédoublonnage", ["stg_commandes"], ["commandes_typees"], len(brut),
↳ len(unique))

```

```
journal.enregistrer("validation", ["commandes_typees", "clients_crm", "produits_catalogue"],
↳ ["commandes_propres", "commandes_rejets"], len(unique), len(propres_a))
journal.enregistrer("mart par catégorie", ["commandes_propres", "produits_catalogue"],
↳ ["mart_ca_categorie"], len(propres_a), 4)
print(journal.table().assign(entrees=lambda d: d["entrees"].map(", ".join), sorties=lambda d:
↳ d["sorties"].map(", ".join)).to_string(index=False))
```

	etape	entrees
↳ sorties	lignes_in lignes_out	
	chargement	commandes_export
↳ stg_commandes	19700 19700	
typage et dédoublement		stg_commandes
↳ commandes_typees	19700 18000	
	validation commandes_typees, clients_crm, produits_catalogue	commandes_propres,
↳ commandes_rejets	18000 17218	
	mart par catégorie commandes_propres, produits_catalogue	
↳ mart_ca_categorie	17218 4	

Étape 2 : remonter et descendre le graphe.

```
def voisins(table, etapes, sens):
    de, vers = ("sorties", "entrees") if sens == "amont" else ("entrees", "sorties")
    trouves = set()
    for e in etapes:
        if table in e[de]:
            for t in e[vers]:
                trouves |= {t} | voisins(t, etapes, sens)
    return trouves
print("amont de mart_ca_categorie :", sorted(voisins("mart_ca_categorie", journal.etapes, "amont")))
print("aval de produits_catalogue :", sorted(voisins("produits_catalogue", journal.etapes, "aval")))
```

```
amont de mart_ca_categorie : ['clients_crm', 'commandes_export', 'commandes_propres',
↳ 'commandes_typees', 'produits_catalogue', 'stg_commandes']
aval de produits_catalogue : ['commandes_propres', 'commandes_rejets', 'mart_ca_categorie']
```

Étape 3 : l'empreinte nue contre la clé secrète. L'attaquant connaît le format des adresses (prénom.nomN@exemple.test) et dispose des listes de prénoms et de noms : 1,6 million de candidats.

```
def empreinte(v): return hashlib.sha256(v.lower().encode()).hexdigest()
def pseudonyme(v, cle): return hmac.new(cle, v.lower().encode(), hashlib.sha256).hexdigest()
emails = crm["email"].str.lower().unique()
prenoms, noms = sorted(set(crm["prenom"].str.lower())), sorted(set(crm["nom"].str.lower()))
dico = {empreinte(f"{p}.{n}{i}@exemple.test") for p in prenoms for n in noms for i in range(1, 5001)}
nu = [empreinte(e) for e in emails]
protege = [pseudonyme(e, b"cle-secrete-de-la-boutique") for e in emails]
print("candidats :", len(dico), "| adresses :", len(emails))
print("retrouvées avec l'empreinte nue :", sum(x in dico for x in nu), "| avec la clé secrète :",
↳ sum(x in dico for x in protege))
```

```
candidats : 1600000 | adresses : 4200
retrouvées avec l'empreinte nue : 4200 | avec la clé secrète : 0
```

Étape 4 : les quasi-identifiants. Même sans e-mail, combien de personnes sont **uniques** selon les attributs conservés ?

```
personnes = cl.drop_duplicates("id_vrai").assign(annee=lambda d: d["date_inscription"].str[:4])
def uniques(cols):
    return round(float(100 * (personnes.groupby(cols)["id_vrai"].transform("size") == 1).mean()), 1)
print({"ville + prénom": uniques(["ville", "prenom"]), "+ année d'inscription": uniques(["ville",
↳ "prenom", "annee"]),
      "+ date d'inscription": uniques(["ville", "prenom", "date_inscription"])})
```

```
{'ville + prénom': 0.0, "+ année d'inscription": 1.5, "+ date d'inscription": 99.1}
```

Avec la ville et le prénom, personne n'est unique ; avec l'année d'inscription, 1,5 % le sont ; avec la **date précise**, 99,1 %. Une pseudonymisation de l'e-mail seule ne protège donc pas : la **combinaison** d'attributs réidentifie.

Application 5.9 — Collecter sur le site local (sections 5.5.3 à 5.5.6)

Objectif. Lire `robots.txt`, extraire le catalogue d'une page HTML, parcourir une API paginée qui refuse une requête sur sept, et se défendre contre un changement de mise en page. Tout se passe sur votre machine.

Étape 1 : démarrer le site de démonstration. `serveur_local` lance le serveur dans un fil d'exécution et le ferme à la sortie du bloc `with`.

```
import requests
from bs4 import BeautifulSoup
from urllib import robotparser
avis_demo = pd.read_csv("donnees/avis_clients.csv").head(400).fillna("")
with serveur_local(catalogue, avis_demo) as base:
    robots = robotparser.RobotFileParser(base + "/robots.txt"); robots.read()
    print("catalogue autorisé :", robots.can_fetch("...", base + "/catalogue"), "| /prive/ autorisé :",
        ↳ robots.can_fetch("...", base + "/prive/clients"),
        " | délai demandé :", robots.crawl_delay("..."), "s")
    print("accès forcé à /prive/clients : code", requests.get(base + "/prive/clients",
        ↳ timeout=5).status_code)
```

```
catalogue autorisé : True | /prive/ autorisé : False | délai demandé : 1 s
accès forcé à /prive/clients : code 403
```

Étape 2 : scraper la page du catalogue.

```
def extraire(html):
    soupe = BeautifulSoup(html, "html.parser")
    return pd.DataFrame([{"id_produit": d["data-id"], "libelle": d.select_one(".nom").text, "prix":
        ↳ float(d.select_one(".prix").text.replace("€", ""))}
        for d in soupe.select("div.produit")])
with serveur_local(catalogue, avis_demo) as base:
    page = extraire(requests.get(base + "/catalogue", timeout=5).text)
    print(page.shape, "| identique au catalogue :", bool((page["id_produit"] ==
        ↳ catalogue["id_produit"]).all() and np.allclose(page["prix"], catalogue["prix_catalogue"])))
```

```
(48, 3) | identique au catalogue : True
```

Étape 3 : l'API paginée, d'abord sans précaution. Le serveur refuse une requête sur sept (code 429). Combien d'avis le collecteur naïf perd-il, et le sait-il ?

```
with serveur_local(catalogue, avis_demo) as base:
    reponses = [requests.get(f"{base}/api/avis?page={p}&taille=25", timeout=5) for p in range(1, 17)]
    recus = sum(len(r.json()["resultats"]) for r in reponses if r.status_code == 200)
    print("codes :", [r.status_code for r in reponses])
    print("avis reçus :", recus, "sur", len(avis_demo))
```

```
codes : [200, 200, 200, 200, 200, 200, 429, 200, 200, 200, 200, 200, 200, 429, 200, 200]
avis reçus : 350 sur 400
```

Étape 4 : le collecteur robuste. Il réessaie sur 429, avec une attente croissante, et s'arrête après un nombre d'essais borné.

```
def lire_page(session, url, essais=5):
    for k in range(essais):
        r = session.get(url, timeout=5)
        if r.status_code == 429:
            time.sleep(float(r.headers.get("Retry-After", 0)) + 0.01 * 2 ** k); continue
        r.raise_for_status(); return r.json()
    raise RuntimeError(f"abandon après {essais} essais : {url}")

with serveur_local(catalogue, avis_demo) as base:
    s, collectes, page_no = requests.Session(), [], 1
    while page_no:
        rep = lire_page(s, f"{base}/api/avis?page={page_no}&taille=25")
        collectes += rep["resultats"]; page_no = rep["suivante"]
c = pd.DataFrame(collectes)
print(len(c), "avis | identifiants uniques :", c["id_avis"].nunique(), "| identiques à la source :",
      ↪ c["texte"].tolist() == avis_demo["texte"].tolist())
```

400 avis | identifiants uniques : 400 | identiques à la source : True

Étape 5 : la mise en page change. Le sélecteur `div.produit` ne trouve plus rien dans `/catalogue-v2` ; un contrôle de volume le détecte.

```
with serveur_local(catalogue, avis_demo) as base:
    v2 = requests.get(base + "/catalogue-v2", timeout=5).text
    trouves = len(extraire(v2))
    print("produits trouvés dans la nouvelle page :", trouves, "| article[id] en trouve :",
          ↪ len(BeautifulSoup(v2, "html.parser").select("article[id]")))
    print("contrôle de volume :", "OK" if trouves >= 40 else "ALERTE : résultat vide ou tronqué,
          ↪ chargement arrêté")
```

produits trouvés dans la nouvelle page : 0 | article[id] en trouve : 48
 contrôle de volume : ALERTE : résultat vide ou tronqué, chargement arrêté

Exercices

Les exercices sont rangés par difficulté (★ calcul direct, ★★ raisonnement, ★★★ synthèse ou expérience). Essayez-les à la **main d'abord** ; les corrigés suivent.

Exercice 5.1 ★ — L'équation de conservation (section 5.1.4)

Un pipeline lit **12 400** lignes. Il écarte **1 100** doublons, rejette **4 %** des lignes restantes, et charge le reste.

- Combien de lignes sont rejetées ? Combien sont chargées ?
- Écrivez l'équation de conservation et vérifiez-la.
- Le lendemain, il lit **12 600** lignes, écarte **1 150** doublons, rejette **600** lignes et en charge **10 850**. Le seuil d'alerte est un taux de rejet de **5 % des lignes restantes**. Que doit faire le pipeline ?

Exercice 5.2 ★★ — Idempotent ou pas ? (section 5.1.7)

Pour chaque opération, dites si elle est **idempotente** (la relancer donne le même résultat qu'une seule exécution) et, sinon, comment la corriger.

- INSERT INTO ventes SELECT * FROM lot dans une table sans clé.
- INSERT ... ON CONFLICT (id) DO UPDATE SET montant = excluded.montant.
- DELETE FROM ventes WHERE lot = 7 suivi de l'insertion du lot 7.

- (d) `UPDATE stock SET quantite = quantite - 3.`
- (e) Écrire le fichier `ventes_2025-12-31.parquet`, en écrasant le précédent s'il existe.
- (f) Ajouter une ligne « chargement terminé » à la fin d'un fichier de journal.

Exercice 5.3 ★★★ — Le filigrane qui perd des lignes (section 5.1.7)

Chaque commande des trois premiers mois de 2025 arrive à l'entrepôt avec un **retard** : 90 % arrivent le jour même, 10 % entre 1 et 30 jours plus tard (graine 0). Chaque nuit, un pipeline charge les nouvelles lignes, du 1^{er} janvier au 30 avril. Comparez deux méthodes : **A**, charger les lignes dont la date de commande dépasse la plus grande date déjà chargée (le filigrane) ; **B**, charger les lignes arrivées depuis la veille. Combien de lignes chaque méthode perd-elle ?

Exercice 5.4 ★ — Quelle dimension ? (section 5.2.2)

Classez chaque défaut dans une dimension de la qualité (complétude, validité, unicité, cohérence, exactitude, fraîcheur).

- (a) Le champ « téléphone » est vide pour 18 % des clientes.
- (b) Une commande porte la date du 31 février.
- (c) La même cliente apparaît sous deux numéros.
- (d) Le tableau de ventes affiche, le lundi, les chiffres de jeudi dernier.
- (e) Une commande référence un produit qui n'existe pas au catalogue.
- (f) Un produit est vendu 12 € alors que son prix d'achat est de 15 €.

Exercice 5.5 ★★ — Un score sans se tromper de formule (section 5.2.4)

Sur 10 000 lignes, on mesure : 300 valeurs vides (complétude), 150 valeurs hors domaine (validité), 500 doublons (unicité) et 50 références orphelines (cohérence). Cent lignes cumulent un doublon et une valeur vide.

- (a) Combien de lignes sont fautives ? Quel est le score global ?
- (b) Que décide la grille du livre (≥ 95 % publier ; 85 à 95 % publier et alerter ; < 85 % arrêter) ?
- (c) Si l'on ignorait le chevauchement, entre quelles bornes le score pourrait-il se trouver ?

Exercice 5.6 ★★ — Une règle statistique et son seuil (section 5.2.6)

On veut signaler en avertissement les prix « improbables pour leur produit » : ceux qui dépassent **k fois la médiane** des prix de ce produit. (a) Combien de lignes sont signalées pour $k = 3, 4, 5$ et 8 ? (b) Comment choisir k ? (c) Les lignes signalées se répartissent-elles uniformément entre les catégories de produits ?

Exercice 5.7 ★ — Levenshtein à la main (section 5.3.3)

- (a) Quelle est la distance de Levenshtein entre « kitten » et « sitting » ? Donnez une suite d'opérations.
- (b) Quelle est la similarité normalisée $1 - d / \max(|a|, |b|)$ entre « sophie » et « sofie » ?
- (c) « Jean Dupont » et « Dupont Jean » désignent la même personne, mais leur distance de Levenshtein est élevée. Quel traitement du livre règle le problème ?

Exercice 5.8 ★★ — Précision, rappel, coûts (section 5.3.6)

Une base contient **100** vraies paires de doublons. Selon le seuil de similarité :

Seuil	Paires fusionnées	Dont vraies
0,95	60	59
0,80	130	95
0,60	400	100

- Calculez précision, rappel et F1 pour chaque seuil.
- Une fusion à tort coûte **10** (deux personnes fondues en une, difficile à défaire) et un doublon laissé coûte **1**. Quel seuil minimise le coût total ?
- Et si les deux erreurs coûtent 1 ?

Exercice 5.9 ★★★ — Choisir une clé de blocage (section 5.3.5)

Une clé de blocage doit être **économique** (peu de paires) et **sûre** (elle ne sépare pas de vrais doublons). Sur la table c1 de l'application 5.7 (avec ses fautes de frappe), évaluez cinq clés : la ville ; le prénom et la ville ; le prénom et la date d'inscription ; les trois premières lettres du nom et la ville ; la date d'inscription seule. Pour chacune, donnez le nombre de paires à comparer et la part des 800 vraies paires qu'elle conserve (la **complétude des paires**). Ajoutez ensuite des fautes de frappe dans le **prénom** (une lettre manquante au début, pour 35 % des doublons récents) : que devient la complétude de la clé « prénom + date d'inscription » ? Proposez un **blocage multiple** (union de deux clés) et évaluez-le.

Exercice 5.10 ★ — Le k-anonymat à la main (section 5.4.5)

Huit personnes, avec leur ville et leur année de naissance : 1 (A, 1990), 2 (A, 1990), 3 (A, 1991), 4 (B, 1990), 5 (B, 1991), 6 (B, 1991), 7 (B, 1992), 8 (C, 1990). Les villes A et C sont dans la région Nord, B dans la région Sud.

- Quel est le k de la table si l'on ne conserve que la ville ?
- Avec la ville et l'année de naissance : quel est le k , et quelle part des personnes est unique ?
- Proposez une généralisation qui donne $k \geq 4$.

Exercice 5.11 ★★ — Un sel public protège-t-il ? (section 5.4.5)

On remplace l'e-mail par `sha256(sel + e-mail)`, avec un sel **identique pour toutes les lignes et publié** dans la documentation du jeu de données. Un attaquant dispose du même dictionnaire de 1,6 million de candidats qu'à l'application 5.8. (a) Combien d'adresses retrouve-t-il ? (b) Qu'est-ce qui distingue ce sel d'une clé secrète ? (c) Que faudrait-il changer pour que l'attaque échoue ?

Exercice 5.12 ★★ — Un extracteur qui survit au changement de page (sections 5.5.4 à 5.5.6)

- À la main : un collecteur essaie une requête au plus 5 fois, avec une attente de 1, 2, 4, puis 8 secondes entre deux essais. Quelle attente cumulée dans le pire cas ?
- Le serveur refuse une requête sur sept. Pour collecter 16 pages, combien de requêtes sont envoyées au total ?

- (c) Écrivez une fonction `extraire_tout(html)` qui renvoie la même table pour les deux mises en page (`/catalogue` et `/catalogue-v2`), et vérifiez qu'elle retrouve les 48 produits avec leurs prix.

Corrigés

Corrigé 5.1

- (a) Lignes restantes : $12\,400 - 1\,100 = 11\,300$. Rejets : $0,04 \times 11\,300 = 452$. Lignes chargées : $11\,300 - 452 = 10\,848$.
- (b) Entrées – doublons – rejets = sorties : $12\,400 - 1\,100 - 452 = 10\,848$. ✓
- (c) Lignes restantes : $12\,600 - 1\,150 = 11\,450$. Taux de rejet : $600/11\,450 \approx 5,24\%$, au-dessus du seuil de 5 %. Le pipeline **arrête le chargement** et prévient un humain : la veille, le taux était de 4 %, et ce saut est le signe d'un incident à la source. (L'équation reste juste : $12\,600 - 1\,150 - 600 = 10\,850$; une équation de conservation vérifiée n'est **pas** une preuve de qualité, seulement de non-perte.)

Corrigé 5.2

- (a) **Non idempotente** : chaque relance ajoute les lignes une nouvelle fois. Correction : déclarer une clé et fusionner (*upsert*).
- (b) **Idempotente** : une seconde exécution écrit les mêmes valeurs sur les mêmes clés.
- (c) **Idempotente** : on remplace le lot entier par lui-même. À condition que la suppression et l'insertion se fassent dans **une seule transaction** : sinon une panne entre les deux perd le lot.
- (d) **Non idempotente** : chaque relance retire 3 unités. Correction : écrire une **valeur absolue** (`SET quantite = <valeur calculée à partir du lot>`), pas un incrément.
- (e) **Idempotente** : le nom dépend de la date métier, et l'écrasement donne le même fichier.
- (f) **Non idempotente** : la ligne est ajoutée à chaque exécution. Pour un journal c'est voulu (on garde l'historique), mais il faut alors **ne pas le lire comme un état**.

Corrigé 5.3

On simule les retards, puis on joue 120 nuits avec chaque méthode.

```
typ_q1 = propres_b[propres_b["mois"] <= 3].copy()
rng = np.random.default_rng(0)
retard = np.where(rng.random(len(typ_q1)) < 0.10, rng.integers(1, 31, len(typ_q1)), 0)
typ_q1["arrivee"] = typ_q1["date"] + pd.to_timedelta(retard, unit="D")

filigrane, charge_a, charge_b = pd.Timestamp("2024-12-31"), set(), set()
for nuit in pd.date_range("2025-01-01", "2025-04-30"):
    dispo = typ_q1[typ_q1["arrivee"] <= nuit]                                # ce qui est arrivé
    ↪ à l'entrepôt
    neuf_a = dispo[dispo["date"] > filigrane]                                # A : filtre sur la
    ↪ date métier
    neuf_b = dispo[dispo["arrivee"] > nuit - pd.Timedelta(days=1)]          # B : filtre sur la
    ↪ date d'arrivée
    charge_a |= set(neuf_a["id_commande"]); charge_b |= set(neuf_b["id_commande"])
    filigrane = max(filigrane, neuf_a["date"].max()) if len(neuf_a) else filigrane
print("lignes à charger :", len(typ_q1), "| perdues par A :", len(typ_q1) - len(charge_a), "| perdues
    ↪ par B :", len(typ_q1) - len(charge_b))
```

lignes à charger : 4210 | perdues par A : 438 | perdues par B : 0

La méthode A **perd 438 lignes sur 4 210** (10,4 %, soit à peu près la part des lignes en retard) : toutes celles qui arrivent après que le filigrane a dépassé leur date, et plus le retard est long, plus le risque est grand. La méthode B ne perd rien, parce que la date d'arrivée **croît toujours** : une ligne ne peut pas arriver « avant » la veille. Le prix à payer est de stocker cette date technique, mais on la reçoit gratuitement avec la métadonnée de chargement (`_charge_le`).

Corrigé 5.4

- (a) **Complétude** : une valeur attendue est absente.
- (b) **Validité** : la date n'appartient pas au domaine des dates possibles.
- (c) **Unicité** : une même réalité est représentée deux fois (une cliente, deux numéros).
- (d) **Fraîcheur** : la donnée est trop ancienne par rapport à l'usage.
- (e) **Cohérence** : une référence ne trouve pas sa cible entre deux sources.
- (f) **Exactitude** (ou cohérence métier) : la valeur contredit un recoupement de confiance (le prix d'achat dépasse le prix de vente). On ne peut la déceler qu'en la comparant à une **référence**.

Corrigé 5.5

- (a) Lignes fautives distinctes : $300 + 150 + 500 + 50 - 100 = 900$ (on retire les 100 lignes comptées deux fois). Score : $1 - 900/10\,000 = 91\%$.
- (b) 91 % est entre 85 % et 95 % : on **publie et l'on alerte** le propriétaire de la source.
- (c) Sans connaître le chevauchement, le nombre de lignes fautives est au moins $\max(300, 150, 500, 50) = 500$ (si toutes les autres violations recoupent les doublons) et au plus $300 + 150 + 500 + 50 = 1\,000$ (aucun recoupement). Le score est donc **entre 90 % et 95 %** : la décision reste « publier et alerter », sauf si le résultat tombe pile à 95 %. D'où l'intérêt de calculer le « ou » logique ligne à ligne, au lieu d'additionner les taux.

Corrigé 5.6

```
mediane = typ.groupby("id_produit")["prix_unitaire"].transform("median")
for k in (3, 4, 5, 8):
    print(f"k = {k} : {int((typ['prix_unitaire'] > k * mediane).sum())} lignes signalées")
par_cat = typ[typ["prix_unitaire"] > 5 * mediane].merge(catalogue[["id_produit", "categorie"]],
    ↪ on="id_produit")["categorie"].value_counts()
print("répartition par catégorie pour k = 5 :", par_cat.to_dict())
```

```
k = 3 : 660 lignes signalées
k = 4 : 186 lignes signalées
k = 5 : 57 lignes signalées
k = 8 : 7 lignes signalées
répartition par catégorie pour k = 5 : {'D': 16, 'A': 15, 'B': 14, 'C': 12}
```

- (a) Le nombre de lignes signalées s'effondre quand k augmente : 660 pour k = 3, 186 pour k = 4, 57 pour k = 5, 7 pour k = 8.
- (b) Le bon k est celui qui signale un **nombre de lignes qu'un humain peut relire** tout en laissant passer les variations normales : avec k = 3, 660 lignes (3,4 %) noient la relecture ; avec k = 8, sept lignes laissent passer des prix douteux. **k = 5** (57 lignes, 0,3 %) est un bon compromis ; un échantillon relu à la main (20 lignes signalées) dit si l'on cible des erreurs ou de simples promotions.

- (c) Les lignes signalées pour $k = 5$ se répartissent presque également entre les catégories (D : 16, A : 15, B : 14, C : 12) : la règle ne vise pas une catégorie en particulier. Si elles s'étaient concentrées sur l'une d'elles, on aurait suspecté une dispersion naturelle de ses prix, et affiné la règle **par catégorie**.

Corrigé 5.7

- (a) La distance vaut 3 : *kitten* → *sitten* (remplacer k par s) → *sittin* (remplacer e par i) → *sitting* (insérer g).
- (b) *sophie* → *sofie* demande 2 opérations (remplacer p par f, supprimer h) : $d = 2$, longueur maximale 6, similarité $1 - 2/6 \approx 0,667$.
- (c) Les **comparaisons par jetons** (*token sort*), ou la normalisation qui **trie les mots**, rendent l'ordre indifférent.

```
print(Levenshtein.distance("kitten", "sitting"), Levenshtein.distance("sophie", "sofie"),
      round(Levenshtein.normalized_similarity("sophie", "sofie"), 3), "|", round(fuzz.ratio("Jean
↪ Dupont", "Dupont Jean")),
      round(fuzz.token_sort_ratio("Jean Dupont", "Dupont Jean")))
```

3 2 0.667 | 55 100

Les deux premiers nombres retrouvent les distances (3 et 2), puis la similarité 0,667. À droite de la barre, le rapport de similarité (en pourcentage) vaut **55** pour « Jean Dupont » contre « Dupont Jean » et **100** une fois les mots triés.

Corrigé 5.8

- (a)

Seuil	Précision	Rappel	F1
0,95	59/60 = 0,983	59/100 = 0,59	0,738
0,80	95/130 = 0,731	95/100 = 0,95	0,826
0,60	100/400 = 0,25	100/100 = 1	0,4

- (b) Coût = $10 \times$ fusions à tort + $1 \times$ doublons laissés. Seuil 0,95 : $10 \times 1 + 41 = 51$. Seuil 0,80 : $10 \times 35 + 5 = 355$. Seuil 0,60 : $10 \times 300 + 0 = 3\,000$. **Le seuil 0,95** gagne nettement : quand une erreur de fusion coûte cher, on exige une grande précision et l'on envoie le reste à la **file de revue**.
- (c) Avec des coûts égaux : $0,95 \rightarrow 1 + 41 = 42$; $0,80 \rightarrow 35 + 5 = 40$; $0,60 \rightarrow 300$. **Le seuil 0,80** devient le meilleur, de peu. Le seuil se choisit toujours par les coûts, pas par la seule précision ou le seul F1.

Corrigé 5.9

```
vraies_paires = {frozenset(p) for p in paires_candidates(cl, ["id_vrai"])}
cl["nom3"] = cl["nom_n"].str[:3]
def evaluer(d, cols):
    P = {frozenset(p) for p in paires_candidates(d, cols)}
    return len(P), round(100 * len(P & vraies_paires) / len(vraies_paires), 1)
cles = {"ville": ["ville_n"], "prénom + ville": ["prenom_n", "ville_n"], "prénom + date":
↪ ["prenom_n", "date_inscription"],
        "3 lettres du nom + ville": ["nom3", "ville_n"], "date seule": ["date_inscription"]}
print(pd.DataFrame([(n, *evaluer(cl, c)) for n, c in cles.items()], columns=["clé", "paires",
↪ "complétude des paires (%)"]).to_string(index=False))
```

	clé	paires	complétude des paires (%)
	ville	1041310	100.0
	prénom + ville	53153	100.0
	prénom + date	1227	100.0
3 lettres du nom + ville		68548	79.4
	date seule	9595	100.0

La **complétude des paires** est le rappel du blocage : une paire qui n'est pas dans le même bloc ne sera **jamais comparée**, quel que soit le soin du score. Quatre clés conservent les 800 vraies paires, mais à des coûts très différents : « prénom + date » en demande 1 227, « date seule » près de 9 600, « ville » plus d'un million. La clé « trois lettres du nom + ville » est **peu sûre** : elle perd 20,6 % des vraies paires, car la faute de frappe sur le nom touche souvent l'une des trois premières lettres.

Ajoutons maintenant des fautes dans le prénom des doublons récents, et comparons deux clés seules avec leur union.

```
rng = np.random.default_rng(7)
cl2 = cl.copy()
touches = cl2.index[(cl2["id_crm"] > 4200) & (rng.random(len(cl2)) < 0.35)]
cl2.loc[touches, "prenom_n"] = cl2.loc[touches, "prenom_n"].str[1:] # première
↳ lettre manquante
seul_a, seul_b = ["prenom_n", "date_inscription"], ["nom3", "date_inscription"]
union = {frozenset(p) for c in (seul_a, seul_b) for p in paires_candidates(cl2, c)}
print("prénom + date :", evaluer(cl2, seul_a), "| 3 lettres du nom + date :", evaluer(cl2, seul_b))
print("union des deux clés :", len(union), "paires,", round(100 * len(union & vraies_paires) /
↳ len(vraies_paires), 1), "% des vraies paires")
```

```
prénom + date : (882, 61.5) | 3 lettres du nom + date : (1248, 79.4)
union des deux clés : 1706 paires, 92.6 % des vraies paires
```

« Prénom + date », sûre jusque-là, ne conserve plus que **61,5 %** des vraies paires ; « 3 lettres du nom + date » en conserve **79,4 %**. Chaque clé échoue sur des fautes **différentes** (le prénom pour l'une, le nom pour l'autre), si bien que leur **union** monte à **92,6 %** avec 1 706 paires à comparer. Elle ne retrouve pas les doublons dont le prénom **et** le nom portent une faute : un blocage multiple réduit le risque, il ne l'annule pas.

Corrigé 5.10

- Groupes : $A = \{1, 2, 3\}$ (3 personnes), $B = \{4, 5, 6, 7\}$ (4), $C = \{8\}$ (1). Le plus petit groupe compte 1 personne : $k = 1$ (la personne 8 est unique).
- Groupes (ville, année) : $(A, 1990) = 2$, $(A, 1991) = 1$, $(B, 1990) = 1$, $(B, 1991) = 2$, $(B, 1992) = 1$, $(C, 1990) = 1$. $k = 1$, et les personnes 3, 4, 7 et 8 sont uniques : **4 sur 8, soit 50 %**.
- On généralise : la ville devient la **région**, l'année la **décennie** (toutes les années 1990 se confondent). Groupes : Nord = $\{1, 2, 3, 8\}$ (4 personnes), Sud = $\{4, 5, 6, 7\}$ (4). Donc $k = 4$. Le prix est une perte de précision : on ne sait plus ni la ville ni l'année exacte.

Corrigé 5.11

```
sel_public = "sel-publie-dans-la-documentation:"
sale = {hashlib.sha256((sel_public + e).encode()).hexdigest() for e in emails}
dico_sale = {hashlib.sha256((sel_public + f"{p}.{n}{i}@exemple.test").encode()).hexdigest() for p in
↪ prenomns for n in noms for i in range(1, 5001)}
print("adresses retrouvées avec le sel public :", len(sale & dico_sale), "sur", len(sale))
```

```
adresses retrouvées avec le sel public : 4200 sur 4200
```

- (a) L'attaquant **retrouve toutes les adresses** : il lui suffit de recalculer son dictionnaire avec le sel connu. Un sel public a un seul mérite, empêcher les tables précalculées **génériques** (celles que l'on trouve en ligne) ; il n'arrête pas une attaque ciblée.
- (b) Une **clé secrète** n'est connue que de l'organisation : sans elle, l'attaquant ne peut rien précalculer, et chaque essai demanderait un accès au système. C'est le **secret** qui protège, pas la fonction.
- (c) Utiliser un **HMAC avec une clé secrète** (comme à l'application 5.8), conservée **hors du jeu de données** et renouvelée si elle fuit. Et, pour les données à partager, se demander d'abord si l'identifiant est **nécessaire** (minimisation).

Corrigé 5.12

- (a) Les attentes se produisent **entre** deux essais : $1 + 2 + 4 + 8 = 15$ **secondes** dans le pire cas (5 essais, 4 pauses).
- (b) Une requête sur sept est refusée : les requêtes numéro 7 et 14 le sont, et la requête 21 n'est pas atteinte. Pour obtenir 16 pages, il faut $16 + 2 = 18$ **requêtes**.
- (c)

```
def extraire_tout(html):
    soupe = BeautifulSoup(html, "html.parser")
    if soupe.select("div.produit"):
        lignes = [(d["data-id"], float(d.select_one(".prix").text.replace("€", ""))) for d in
↪ soupe.select("div.produit")]
    else:
        lignes = [(a["id"], float(a.select_one("b").text)) for a in soupe.select("article[id]")]
    return pd.DataFrame(lignes, columns=["id_produit", "prix"])

with serveur_local(catalogue, avis_demo) as base:
    v1, v2 = (extraire_tout(requests.get(base + p, timeout=5).text) for p in ("/catalogue",
↪ "/catalogue-v2"))
    for nom, t in (("v1", v1), ("v2", v2)):
        print(nom, len(t), "produits | prix identiques au catalogue :", bool((t["id_produit"] ==
↪ catalogue["id_produit"]).all() and np.allclose(t["prix"], catalogue["prix_catalogue"])))
```

```
v1 48 produits | prix identiques au catalogue : True
v2 48 produits | prix identiques au catalogue : True
```

Deux mises en page, une seule fonction : les **identifiants** (data-id, id) sont les mêmes dans les deux, et c'est eux qu'on exploite, plutôt que des classes de style, plus volatiles. Et le contrôle de volume (au moins 40 produits) reste indispensable : le jour où une **troisième** mise en page arrive, la fonction renvoie un tableau vide, et c'est le contrôle qui donne l'alerte.

```
import shutil
shutil.rmtree(TMP, ignore_errors=True)
```

Chapitre 6 : Plateformes cloud — exercices et applications

Ce chapitre du cahier accompagne le chapitre 6 du livre (cloud : économie, services, coûts, sécurité, choix). Il est, comme le chapitre, **facultatif**. Les **applications** reprennent en code les petits modèles du livre, que vous pouvez refaire avec **vos** prix ; les **exercices** sont corrigés à la fin.

Tous les prix, latences et niveaux de disponibilité de ce cahier sont inventés, à titre d'illustration, et aucun compte chez un fournisseur n'a été utilisé. Ils servent à s'entraîner à poser un calcul, pas à budgéter un projet réel : les tarifs sont à vérifier auprès du fournisseur.

Préparation

Tous les calculs de ce cahier tiennent dans quelques paramètres, rassemblés ici pour que vous les changiez en un endroit. Les montants sont en euros.

```
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

pd.set_option("display.width", 200)

P = dict(
    serveur_achat=6000.0, annees=4, expl_an=1200.0,      # serveur acheté, amortissement, exploitation
    ↪ annuelle
    vm_od=0.40, vm_res=0.25, vm_spot=0.12,             # €/h : à la demande, réservée (facturée tout
    ↪ le mois), spot
    unite_site=0.030, unite_od=0.050,                  # € par unité de calcul et par heure : sur
    ↪ site, cloud
    heures_an=8760, heures_mois=730,
)
print("paramètres définis")
```

paramètres définis

Applications

Application 6.1 — Capex ou opex ? (section 6.1.1)

Question. À partir de quel taux d'utilisation un serveur acheté coûte-t-il moins cher, à l'heure utile, qu'une machine louée ?

Le serveur coûte son prix d'achat réparti sur la durée d'amortissement, plus l'exploitation annuelle ; ce coût est **fixe**, utilisé ou non. Divisé par les heures **utilisées**, il devient plus cher quand l'utilisation baisse.

```
def cout_heure_site(taux):
    """Coût du serveur acheté par heure utile, selon le taux d'utilisation (0 < taux <= 1)."""
    par_an = P["serveur_achat"] / P["annees"] + P["expl_an"]
    return par_an / (P["heures_an"] * taux)

taux = np.array([0.2, 0.4, 0.6, 0.771, 0.9, 1.0])
tab = pd.DataFrame({"taux d'utilisation": taux, "serveur (€/h utile)": cout_heure_site(taux), "cloud
↳ (€/h)": P["vm_od"]})
tab["moins cher"] = np.where(tab["serveur (€/h utile)"] < P["vm_od"], "serveur", "cloud")
print(tab.round(3).to_string(index=False))
print("seuil :", round(cout_heure_site(1.0) / P["vm_od"], 3))
```

taux d'utilisation	serveur (€/h utile)	cloud (€/h)	moins cher
0.200	1.541	0.4	cloud
0.400	0.771	0.4	cloud
0.600	0.514	0.4	cloud
0.771	0.400	0.4	serveur
0.900	0.342	0.4	serveur
1.000	0.308	0.4	serveur

seuil : 0.771

À faire. (1) Refaites le calcul avec un serveur à 9 000 € amorti sur 5 ans. (2) Faites varier le prix du cloud entre 0,25 et 0,60 € l'heure et tracez le seuil en fonction de ce prix. (3) Que devient le seuil si l'exploitation annuelle double ? Quelle hypothèse **omise** par ce modèle pourrait renverser la conclusion ?

Application 6.2 — Dimensionner, et ce que l'élasticité change (section 6.1.2)

Question. Sur une demande qui varie selon l'heure, la semaine et la saison, combien coûtent trois façons de s'équiper ?

On simule une année horaire : un cycle journalier, un creux le week-end, une pointe de fin d'année et un bruit multiplicatif.

```
rng = np.random.default_rng(6201)
h = np.arange(P["heures_an"])
jour = (1 + 0.4 * np.sin(2 * np.pi * (h % 24 - 14) / 24)) * np.where((h // 24) % 7 >= 5, 0.8, 1.0)
saison = np.where(h >= P["heures_an"] * 11 / 12, 2.0, 1.0) # décembre
dem = 40 * jour * saison * rng.lognormal(0, 0.10, h.size) # unités de calcul demandées

cap_pic, cap_p95 = dem.max(), np.percentile(dem, 95)
couts = {
    "sur site, capacité du pic": cap_pic * P["unite_site"] * P["heures_an"],
    "sur site, capacité du 95e centile": cap_p95 * P["unite_site"] * P["heures_an"],
    "cloud élastique": dem.sum() * P["unite_od"],
}
print(f"demande : moyenne {dem.mean():.1f}, 95e centile {cap_p95:.1f}, pic {cap_pic:.1f}")
for k, v in couts.items():
    print(f"{k:36s} {v:9,.0f} €")
```

```
print(f"heures sous-servies avec le 95e centile : {np.mean(dem > cap_p95):.1%}")
```

```
demande : moyenne 41.1, 95e centile 68.0, pic 139.4
sur site, capacité du pic          36,642 €
sur site, capacité du 95e centile  17,881 €
cloud élastique                    17,991 €
heures sous-servies avec le 95e centile : 5.0%
```

À faire. (1) Calculez le rapport pic / moyenne ; à partir de quel rapport le cloud élastique devient-il moins cher que le site dimensionné au pic ? (2) Supprimez la pointe de décembre : que devient l'écart ? (3) Que coûte en **chiffre d'affaires perdu** une heure sous-servie, et à partir de quel prix cela change-t-il le choix du 95e centile ?

Application 6.3 — Disponibilité d'une architecture (section 6.1.4)

Question. Quelle indisponibilité annuelle attendre d'un service dont les composants sont en série, avec ou sans redondance ?

Trois règles : composants **en série** (tous nécessaires) : on **multiplie** les disponibilités ; composants **en parallèle** (un seul suffit) : $1 - (1 - a)^n$; avec une bascule **imparfaite** qui réussit avec la probabilité f : $a^2 + 2a(1 - a)f$ pour deux copies.

```
MIN_AN = 365 * 24 * 60
```

```
def serie(*a):
    return float(np.prod(a))
```

```
def paire(a, f=1.0):
    return a * a + 2 * a * (1 - a) * f
```

```
def minutes(dispo):
    return (1 - dispo) * MIN_AN
```

```
archi = {
    "A : un seul de chaque": serie(0.999, 0.9995, 0.995),
    "B : base doublée, bascule parfaite": serie(0.999, 0.9995, paire(0.995)),
    "C : base doublée, bascule à 90 %": serie(0.999, 0.9995, paire(0.995, 0.9)),
    "D : tout doublé, bascule parfaite": serie(paire(0.999), paire(0.9995), paire(0.995)),
}
print(pd.DataFrame({"architecture": list(archi), "disponibilité": [f"{v:.4%}" for v in
↪ archi.values()], "minutes d'arrêt par an": [round(minutes(v)) for v in
↪ archi.values()]}).to_string(index=False))
```

	architecture	disponibilité	minutes d'arrêt par an
A	A : un seul de chaque	99.3508%	3412
B	B : base doublée, bascule parfaite	99.8476%	801
C	C : base doublée, bascule à 90 %	99.7482%	1323
D	D : tout doublé, bascule parfaite	99.9974%	14

À faire. (1) Quelle architecture respecte un objectif de 99,9 % (environ 526 minutes d'arrêt par an) ? (2) Existe-t-il une probabilité de bascule f qui permette à l'architecture C d'atteindre 99,9 % ? (cherchez par balayage, puis expliquez le plafond) (3) Pourquoi la section 6.1.4 insiste-t-elle sur « le maillon le plus faible » ?

Application 6.4 — Simulateur d'autoscaling (section 6.2.1)

Question. Combien faut-il payer, et combien de requêtes attendent, selon le **délai de démarrage** d'une instance ?

Une journée de 1 440 minutes avec une pointe vers 19 h. À chaque minute, la politique commande assez d'instances pour absorber 1,2 fois les arrivées de la minute précédente ; elles ne sont disponibles qu'après le délai de démarrage. Chaque instance traite 25 requêtes par minute ; ce qui n'est pas traité attend.

```
rng = np.random.default_rng(6204)
t = np.arange(1440)
lam = 40 + 700 * np.exp(-((t - 1140) / 25) ** 2)          # requêtes attendues par minute
arr = rng.poisson(lam)

def simule(delai, marge=1.2, cap=25, plancher=2):
    cible = np.maximum(plancher, np.ceil(marge * np.r_[arr[0], arr[:-1]] / cap)).astype(int)
    actives = np.r_[np.full(delai, plancher), cible[:len(t) - delai]] # même délai à la montée et à
    ↪ la descente
    file, attentes = 0.0, []
    for k in range(len(t)):
        file = max(0.0, file + arr[k] - cap * actives[k])
        attentes.append(file)
    return actives, np.array(attentes)

lignes = []
for delai in [0, 1, 5, 10]:
    act, att = simule(delai)
    lignes.append({"délai (min)": delai, "instances-heures": act.sum() / 60, "coût (€)": act.sum() /
    ↪ 60 * P["vm_od"],
                  "minutes avec file": int((att > 0).sum()), "file maximale (requêtes)":
                  ↪ int(att.max())})
print(pd.DataFrame(lignes).round(1).to_string(index=False))
```

délai (min)	instances-heures	coût (€)	minutes avec file	file maximale (requêtes)
0	82.8	33.1	42	15
1	82.8	33.1	38	15
5	82.7	33.1	89	1236
10	82.7	33.1	90	4558

À faire. (1) Quel est le coût d'une politique **statique** au pic (nombre d'instances nécessaire à la minute la plus chargée) ? (2) Faites varier la marge (1,0 ; 1,2 ; 1,5) : comment arbitrer coût et file d'attente ? (3) Remplacez le délai de montée par un délai de **descente** plus long (cooldown) : quel effet sur le coût ?

Application 6.5 — Quel palier de stockage ? (section 6.2.2)

Question. Selon la part des données relues chaque mois, quel palier (chaud, froid, archive) est le moins cher, et où sont les seuils ?

```
Go = 10_000
tarifs = {"chaud": (0.023, 0.0), "froid": (0.012, 0.010), "archive": (0.002, 0.030)} # (€/Go-mois
    ↪ stocké, €/Go récupéré)

def cout(palier, part):
    stock, recup = tarifs[palier]
    return Go * stock + Go * part * recup

parts = np.linspace(0, 1, 101)
meilleur = [min(tarifs, key=lambda k: cout(k, p)) for p in parts]
changements = [(round(float(parts[i]), 2), meilleur[i]) for i in range(len(parts)) if i == 0 or
    ↪ meilleur[i] != meilleur[i - 1]]
```

```
print("palier le moins cher selon la part relue :", changements)
for p in [0.01, 0.10, 0.50, 1.0]:
    print(p, {k: round(cout(k, p)) for k in tarifs})
```

```
palier le moins cher selon la part relue : [(0.0, 'archive'), (0.5, 'froid')]
0.01 {'chaud': 230, 'froid': 121, 'archive': 23}
0.1 {'chaud': 230, 'froid': 130, 'archive': 50}
0.5 {'chaud': 230, 'froid': 170, 'archive': 170}
1.0 {'chaud': 230, 'froid': 220, 'archive': 320}
```

À faire. (1) Retrouvez à la main les deux seuils (archive contre froid, froid contre chaud). (2) Ajoutez une **durée minimale de conservation** de 90 jours pour l'archive, facturée comme 3 mois de stockage si l'on supprime avant : pour des données gardées 30 jours, que devient le classement ? (3) Proposez une règle de transition automatique (« après N jours sans lecture... ») pour un jeu de journaux dont la lecture décroît avec l'âge.

Application 6.6 — Auditer une politique d'accès (section 6.3.4)

Question. Peut-on noter automatiquement la **gravité** d'une politique d'accès, dans le format générique du livre ?

On donne des points par défaut : principal ouvert à tous (3), action à joker (2), ressource « toutes » (2), aucune condition (1), droit d'écriture ou d'effacement accordé à un groupe qui n'est pas administrateur (1).

```
ECRITURE = ("Ecrire", "Effacer", "")

def gravite(politique):
    points = []
    for r in politique["Statement"]:
        actions = [r["Action"]] if isinstance(r["Action"], str) else r["Action"]
        if r.get("Principal") == "*":
            points.append(("principal ouvert à tous", 3))
        if any(a.endswith("*") for a in actions):
            points.append(("action à joker", 2))
        if r["Resource"] == "*":
            points.append(("ressource : toutes", 2))
        if not r.get("Condition"):
            points.append(("aucune condition", 1))
        if r.get("Principal") != {"Groupe": "admin"} and any(a.endswith(ECRITURE) for a in actions):
            points.append(("écriture accordée hors administration", 1))
    return points

politiques = {
    "P1 lecture publique": {"Statement": [{"Principal": "*", "Action": "stockage:Lire", "Resource":
        ↪ "seau/public/*"}]},
    "P2 tout pour les développeurs": {"Statement": [{"Principal": {"Groupe": "dev"}, "Action":
        ↪ "stockage:*", "Resource": "*"}]},
    "P3 écriture ciblée, chiffrée": {"Statement": [{"Principal": {"Groupe": "dev"}, "Action":
        ↪ ["stockage:Ecrire", "Resource": "seau/dev/*", "Condition": {"ConnexionChiffree": True}]}]},
    "P4 lecture analystes": {"Statement": [{"Principal": {"Groupe": "analystes"}, "Action":
        ↪ ["stockage:Lire", "Resource": "seau/ventes/*", "Condition": {"ConnexionChiffree": True}]}]},
}

for nom, p in politiques.items():
    pts = gravite(p)
    print(f"{nom:32s} score {sum(v for _, v in pts)} {[d for d, _ in pts]}")
```

```
P1 lecture publique          score 4  ['principal ouvert à tous', 'aucune condition']
P2 tout pour les développeurs score 6  ['action à joker', 'ressource : toutes', 'aucune condition',
    ↪ 'écriture accordée hors administration']
P3 écriture ciblée, chiffrée  score 1  ['écriture accordée hors administration']
```

P4 lecture analystes score 0 []

À faire. (1) Classez les quatre politiques de la plus à la moins dangereuse, et vérifiez que la sortie est cohérente avec votre jugement. (2) La politique P1 est-elle forcément un défaut ? Dans quel cas est-elle légitime ? (3) Corrigez P2 pour que son score tombe à 0 ou 1 tout en laissant aux développeurs le moyen de travailler.

Application 6.7 — Comparateur de modes d'achat (section 6.3.1)

Question. Pour une charge faite d'un socle permanent et d'une pointe, quelle stratégie est la moins chère, et comment cela dépend-il de la **durée de la pointe** ?

```
def strategies(base, extra, h_extra, vm_res=None):
    res = P["vm_res"] if vm_res is None else vm_res
    socle = base * res * P["heures_mois"]
    pointe = extra * h_extra
    return {
        "tout à la demande": (base * P["heures_mois"] + pointe) * P["vm_od"],
        "tout réservé au pic": (base + extra) * res * P["heures_mois"],
        "socle réservé + pointe à la demande": socle + pointe * P["vm_od"],
        "socle réservé + pointe en spot": socle + pointe * P["vm_spot"] * 1.15,
    }

lignes = []
for he in [0, 50, 146, 300, 500, 730]:
    c = strategies(10, 20, he)
    lignes.append({"heures de pointe": he, **{k: round(v) for k, v in c.items()}, "moins chère":
        ↪ min(c, key=c.get)})
print(pd.DataFrame(lignes).to_string(index=False))
```

heures de pointe	tout à la demande	tout réservé au pic	socle réservé + pointe à la demande	socle réservé + pointe en spot	moins chère
↪ 0	2920	5475	1825		
↪ 1825	socle réservé + pointe à la demande	5475	2225		
↪ 1963	50	3320	5475		
↪ 2228	146	4088	5475		
↪ 2653	300	5320	5475		
↪ 3205	500	6920	5475		
↪ 3840	730	8760	5475		
	socle réservé + pointe en spot			7665	

À faire. (1) Pour quelle durée de pointe « tout réservé au pic » devient-il plus avantageux que « socle réservé + pointe à la demande » ? Retrouvez la valeur à la main. (2) Faites varier le prix de la réservation (0,20 à 0,35) : comment le classement change-t-il ? (3) Quelle est la limite de la stratégie « pointe en spot » ? (indice : la pointe tombe-t-elle quand le fournisseur manque de capacité ?)

Application 6.8 — Une grille de décision, et sa sensibilité (section 6.3.6)

Question. Comment transformer la liste de contrôle de la section 6.3.6 en une **note** par option, et à quel point le résultat dépend-il des **poinds** que l'on donne aux critères ?

Pour une petite équipe sans compétence d'exploitation, avec une charge irrégulière et peu de contraintes réglementaires, voici des notes de 1 (mauvais) à 5 (excellent) par option et par critère, **invention pédagogique** à discuter.

```

criteres = ["démarrage rapide", "souplesse de charge", "coût stable à forte charge", "maîtrise des
↳ données", "compétences requises", "facilité de sortie"]
notes = pd.DataFrame(
    [[5, 5, 2, 2, 4, 2],      # cloud public
     [1, 1, 4, 5, 1, 4],      # sur site
     [3, 4, 3, 4, 2, 3],      # hybride
     [3, 4, 3, 3, 1, 4]],     # multicloud
    index=["cloud public", "sur site", "hybride", "multicloud"], columns=criteres)

poids0 = np.array([3, 3, 1, 1, 3, 1]) / 12
print("note pondérée :", (notes.values @ poids0).round(2), notes.index.tolist())

rng = np.random.default_rng(6208)
W = rng.dirichlet(np.ones(len(criteres)), size=10_000)          # 10 000 jeux de poids au hasard
gagnant = notes.index[(W @ notes.values.T).argmax(axis=1)]
print(pd.Series(gagnant).value_counts(normalize=True).round(3).to_string())

```

```

note pondérée : [4.    1.83 3.08 2.83] ['cloud public', 'sur site', 'hybride', 'multicloud']
cloud public    0.578
sur site        0.183
hybride         0.137
multicloud      0.101

```

À faire. (1) Quelle option gagne avec les poids de départ ? (2) Avec des poids tirés au hasard, dans quelle proportion des cas gagne-t-elle ? Que conclure sur la robustesse du classement ? (3) Changez les notes pour une entreprise qui a déjà des serveurs, une charge stable et une contrainte de résidence stricte : le classement s'inverse-t-il ?

Exercices

Exercice 6.1 ★ — Le seuil à la main (section 6.1.1)

Un serveur coûte 6 000 € et s'amortit sur 4 ans ; l'exploitation coûte 1 000 € par an. Une machine équivalente se loue 0,40 € l'heure. Il y a 8 760 heures dans une année. (a) Quel est le coût annuel fixe du serveur ? (b) Quel est son coût à l'heure s'il est utilisé en permanence ? (c) À partir de quel taux d'utilisation est-il moins cher que la location ?

Exercice 6.2 ★ — Pic et moyenne (section 6.1.2)

Une demande moyenne de 40 unités de calcul atteint un pic de 120. Sur site, on achète la capacité du pic à 0,030 € l'unité-heure ; dans le cloud élastique, on paie la demande réelle à 0,050 €. (a) Calculez le coût annuel des deux options. (b) À partir de quel rapport pic / moyenne le cloud devient-il moins cher ? (c) Le rapport de cet exemple est-il au-dessus ou en dessous ?

Exercice 6.3 ★★ — Latence minimale (section 6.1.4)

La lumière se propage dans la fibre à environ 200 000 km par seconde. (a) Quel est le temps d'aller-retour minimal entre deux sites distants de 6 000 km ? (b) En pratique, on observe environ 1,5 fois ce minimum plus 1 ms : quelle latence attendre ? (c) Une page déclenche 8 appels **successifs** vers cette région éloignée : quel délai cumulé, et que recommander ?

Exercice 6.4 ★★ — Les maillons d'une chaîne (section 6.1.4)

Un service dépend de trois composants en série, disponibles à 99,9 %, 99,95 % et 99,5 %. (a) Quelle est la disponibilité de l'ensemble et le nombre de minutes d'arrêt par an (une année = 525 600 minutes) ? (b) On double le composant à 99,5 % avec une bascule parfaite : nouvelle disponibilité ? (c) Même question si la bascule ne réussit que 9 fois sur 10 (disponibilité d'une paire : $a^2 + 2a(1 - a)f$).

Exercice 6.5 ★★ — Qui est responsable ? (section 6.1.5)

Pour chaque incident, dites qui est responsable dans un service **IaaS** (machine virtuelle louée), puis dans un service **SaaS** (application clé en main) : (a) un disque physique tombe en panne dans le centre de données ; (b) une faille est corrigée trop tard dans le système d'exploitation ; (c) un employé partage un accès avec trop de droits ; (d) une base de données est exportée par erreur vers un stockage public ; (e) une mise à jour de l'application casse une fonction.

Exercice 6.6 ★ — Quel palier ? (section 6.2.2)

Vous stockez 5 To (5 000 Go). Palier chaud : 0,023 € par Go et par mois. Palier archive : 0,002 € par Go et par mois, plus 0,030 € par Go récupéré. Chaque mois, 4 % des données sont relues. (a) Coût mensuel dans chaque palier. (b) Part relue en dessous de laquelle l'archive est moins chère que le chaud.

Exercice 6.7 ★★ — L'autoscaling à la main (section 6.2.1)

Un service reçoit 600 requêtes par minute à la pointe, deux heures par jour, et 100 le reste du temps. Une instance traite 25 requêtes par minute et coûte 0,40 € l'heure. (a) Combien d'instances faut-il à la pointe et en dehors ? (b) Coût d'une journée avec capacité fixe au pic, puis avec un autoscaling parfait. (c) Quelle économie, en pourcentage ? (d) Si les instances mettent 5 minutes à démarrer et que la pointe monte en 3 minutes, que se passe-t-il ?

Exercice 6.8 ★★ — Quelle base de données ? (section 6.2.3)

Pour chaque besoin, choisissez parmi base relationnelle gérée, base NoSQL, entrepôt de données, et justifiez en une phrase : (a) les commandes d'une boutique en ligne avec transactions ; (b) un tableau de bord qui agrège des milliards de lignes de ventes ; (c) le profil de session de millions d'utilisateurs, lu et écrit par clé ; (d) un catalogue de produits dont les attributs varient fortement d'une catégorie à l'autre.

Exercice 6.9 ★★ — Combiner les modes d'achat (section 6.3.1)

Une charge est de 6 machines en permanence, plus 12 machines supplémentaires pendant 100 heures par mois. Prix : demande 0,40 €/h ; réservé 0,25 €/h facturé 730 h par mois ; spot 0,12 €/h avec 15 % de travail refait. Calculez le coût mensuel de quatre stratégies : tout à la demande, tout réservé au pic, socle réservé + pointe à la demande, socle réservé + pointe en spot. Laquelle choisir, et pour quelle condition sur la pointe ?

Exercice 6.10 ★★ — Fonction ou machine ? (section 6.3.1)

Une fonction coûte 2,40 € par million de requêtes ; une machine équivalente 0,12 € l'heure, soit 730 heures par mois. (a) À partir de combien de millions de requêtes par mois la machine est-elle moins chère ? (b) Pour 10 millions de requêtes par mois, quelle option choisir ? (c) Citez deux raisons non financières de préférer malgré tout l'autre option.

Exercice 6.11 ★★ — La facture de sortie (section 6.3.3)

Un service expédie 8 To par mois ; la sortie coûte 0,09 € le gigaoctet (1 To = 1 000 Go) ; le calcul coûte 120 € par mois. (a) Facture mensuelle de sortie et totale. (b) Avec une compression par 4 et un cache qui évite 60 % de la sortie restante, quelle nouvelle facture ? (c) Quitter le fournisseur avec 80 To stockés : coût de sortie, et durée d'un transfert sur une liaison à 1 Gbit/s utilisée à 70 % ?

Exercice 6.12 ★★★ — Le dossier de décision (section 6.3.6)

Une association de 12 personnes veut publier chaque mois un tableau de bord sur ses dons (quelques Go de données, dont des données personnelles de donateurs), avec une pointe de visites lors des campagnes. Elle n'a aucun administrateur système. Rédigez, en une page, un dossier de décision : (a) le choix recommandé parmi cloud public, sur site, hybride, multcloud ; (b) trois critères de la liste (section 6.3.6) qui pèsent le plus ; (c) trois mesures de sécurité et de conformité à prendre dès le départ ; (d) un plan de sortie en trois actions ; (e) la **limite** de votre recommandation.

Corrigés

Corrigé 6.1

- (a) Amortissement : $6\,000/4 = 1\,500$ € par an, plus 1 000 € d'exploitation : **2 500 € par an**. (b) En usage permanent : $2\,500/8\,760 \approx 0,285$ € l'heure. (c) Le serveur est moins cher dès que $0,285/\tau < 0,40$, c'est-à-dire pour $\tau > 0,285/0,40 \approx 71$ %.

```
par_an = 6000 / 4 + 1000
cout_h = par_an / 8760
print(par_an, round(cout_h, 3), round(cout_h / 0.40, 3))
```

```
2500.0 0.285 0.713
```

Corrigé 6.2

- (a) Sur site : $120 \times 0,030 \times 8\,760 = 31\,536$ € ; cloud : $40 \times 0,050 \times 8\,760 = 17\,520$ €. (b) Sur site, on paie $\text{pic} \times 0,030$; dans le cloud, $\text{moyenne} \times 0,050$. Le cloud gagne si $\text{pic}/\text{moyenne} > 0,050/0,030 \approx 1,67$. (c) Ici le rapport vaut $120/40 = 3$: **au-dessus**, donc le cloud gagne, de 44 % (17 520 contre 31 536).

```
print(round(120 * 0.030 * 8760), round(40 * 0.050 * 8760), round(0.050 / 0.030, 3), round(1 - 17520 /
↪ 31536, 3))
```

```
31536 17520 1.667 0.444
```

Corrigé 6.3

- (a) $2 \times 6\,000 / 200\,000 = 0,06$ s, soit **60 ms**, un minimum physique que rien ne peut réduire. (b) $1,5 \times 60 + 1 = \mathbf{91\ ms}$. (c) Huit appels successifs : $8 \times 91 = \mathbf{728\ ms}$ avant même de calculer quoi que ce soit. Recommandation : rapprocher le service des utilisateurs (région plus proche, cache), ou **regrouper** les appels en un seul pour ne payer la latence qu'une fois.

```
aller_retour = 2 * 6000 / 200_000 * 1000
print(aller_retour, 1.5 * aller_retour + 1, 8 * (1.5 * aller_retour + 1))
```

```
60.0 91.0 728.0
```

Corrigé 6.4

- (a) En série, on multiplie : $0,999 \times 0,9995 \times 0,995 \approx 0,9935$, soit environ **3 412 minutes** d'arrêt par an (plus de deux jours). (b) La paire parfaite vaut $1 - 0,005^2 = 0,999975$; l'ensemble passe à environ **99,85 %**, soit environ 801 minutes. (c) La paire avec bascule à 90 % vaut $0,995^2 + 2 \times 0,995 \times 0,005 \times 0,9 \approx 0,99898$; l'ensemble tombe à environ **99,75 %**, soit environ 1 323 minutes : la redondance mal basculée perd une bonne part de son bénéfice, et le maillon le plus faible domine toujours.

```
a = [0.999, 0.9995, 0.995]
serie = np.prod(a)
paire = lambda a, f: a * a + 2 * a * (1 - a) * f
for nom, dispo in [("série", serie), ("paire parfaite", 0.999 * 0.9995 * paire(0.995, 1.0)), ("paire à
↳ 90 %", 0.999 * 0.9995 * paire(0.995, 0.9))]:
    print(f"{nom:15s} {dispo:.5f} {(1 - dispo) * 525_600:7.0f} min")
```

```
série          0.99351    3412 min
paire parfaite 0.99848      801 min
paire à 90 %   0.99748    1323 min
```

Corrigé 6.5

Incident	IaaS	SaaS
(a) disque physique en panne	fournisseur	fournisseur
(b) faille du système d'exploitation corrigée trop tard	vous (vous administrez le système)	fournisseur
(c) accès partagé avec trop de droits	vous	vous (les identités restent toujours à votre charge)
(d) base exportée vers un stockage public	vous	vous (les données restent toujours à votre charge)
(e) mise à jour de l'application qui casse une fonction	vous (votre application)	fournisseur (mais vous subissez la panne)

Les deux lignes qui **ne changent jamais** sont (c) et (d) : les identités et les données. Le niveau de service déplace la frontière pour le reste.

Corrigé 6.6

- (a) Chaud : $5\,000 \times 0,023 = 115 \text{ €}$. Archive : $5\,000 \times 0,002 + 5\,000 \times 0,04 \times 0,030 = 10 + 6 = 16 \text{ €}$. (b) L'archive est moins chère tant que $5\,000 \times 0,002 + 5\,000x \times 0,030 < 5\,000 \times 0,023$, soit $x < (0,023 - 0,002)/0,030 = 0,70$: jusqu'à 70 % de données relues chaque mois.

```
Go = 5000
print(Go * 0.023, Go * 0.002 + Go * 0.04 * 0.030, round((0.023 - 0.002) / 0.030, 2))
```

```
115.0 16.0 0.7
```

Corrigé 6.7

- (a) Pointe : $600/25 = 24$ instances ; hors pointe : $100/25 = 4$ instances. (b) Capacité fixe : $24 \times 24 = 576$ instances-heures, soit **230,40 €**. Autoscaling parfait : $22 \times 4 + 2 \times 24 = 136$ instances-heures, soit **54,40 €**. (c) Économie : $1 - 136/576 \approx 76 \%$. (d) Les instances arrivent 2 minutes **après** le sommet de la montée : pendant ce délai, la capacité manque et la file d'attente grossit, ce que mesure l'application 6.4. Remèdes : démarrer plus tôt (marge, planification avant l'heure connue de la pointe), garder un plancher d'instances, ou choisir une unité de calcul à démarrage plus rapide (conteneurs, fonctions).

```
ih_fixe, ih_auto = 24 * 24, 22 * 4 + 2 * 24
print(ih_fixe, ih_auto, round(ih_fixe * 0.4, 2), round(ih_auto * 0.4, 2), round(1 - ih_auto / ih_fixe, 3))
```

```
576 136 230.4 54.4 0.764
```

Corrigé 6.8

- (a) **Relationnelle gérée** : transactions et intégrité (commandes). (b) **Entrepôt de données** : requêtes analytiques sur de très gros volumes, facturé à la lecture. (c) **NoSQL clé-valeur** : accès par clé, débit élevé, extensibilité horizontale. (d) **NoSQL documents** : schéma souple adapté à des attributs variables ; une base relationnelle avec une colonne JSON est une alternative défendable si le volume reste modeste.

Corrigé 6.9

Socle réservé : $6 \times 0,25 \times 730 = 1\,095 \text{ €}$; pointe : $12 \times 100 = 1\,200$ instances-heures.

Stratégie	Calcul	Coût mensuel
tout à la demande	$(4\,380 + 1\,200) \times 0,40$	2 232 €
tout réservé au pic	$18 \times 0,25 \times 730$	3 285 €
socle réservé + pointe à la demande	$1\,095 + 1\,200 \times 0,40$	1 575 €
socle réservé + pointe en spot	$1\,095 + 1\,200 \times 0,12 \times 1,15$	1 261 €

La moins chère est **socle réservé + pointe en spot**, à condition que le travail de la pointe supporte les interruptions ; sinon, socle réservé + pointe à la demande. « Tout réservé au pic » ne devient avantageux que si la pointe dure plus de $0,25 \times 730/0,40 \approx 456$ heures par mois, c'est-à-dire plus de 62 % du temps.

```
base, extra, he = 6, 12, 100
socle = base * 0.25 * 730
print(round((base * 730 + extra * he) * 0.4), round((base + extra) * 0.25 * 730), round(socle + extra
↵ * he * 0.4), round(socle + extra * he * 0.12 * 1.15))
```

```
2232 3285 1575 1261
```

Corrigé 6.10

- (a) La machine coûte $0,12 \times 730 = 87,60$ € par mois ; elle est moins chère quand $2,40 \times n > 87,60$, soit $n > \mathbf{36,5 \text{ millions de requêtes par mois}}$. (b) Pour 10 millions : la fonction coûte 24 €, contre 87,60 € : **la fonction**. (c) Deux raisons de préférer quand même la machine : la **latence** (pas de démarrage à froid) et la **portabilité** (un conteneur sur machine se déplace plus facilement qu'une fonction écrite pour une API propriétaire) ; à l'inverse, pour la fonction : aucune administration, et un coût nul à l'arrêt.

```
vm = 0.12 * 730
print(round(vm, 2), round(vm / 2.40, 1), 2.40 * 10)
```

```
87.6 36.5 24.0
```

Corrigé 6.11

- (a) $8\,000 \times 0,09 = \mathbf{720 \text{ €}}$ de sortie, soit **840 €** au total (la sortie pèse six fois le calcul). (b) Compression par 4 : 180 € ; un cache évitant 60 % de ce qui reste : $180 \times 0,4 = 72$ € ; plus 120 € de calcul : **192 €**, une facture 4,4 fois plus basse. (c) Sortir 80 To coûte $80\,000 \times 0,09 = \mathbf{7\,200 \text{ €}}$. À 1 Gbit/s utilisé à 70 %, le débit est de 0,7 Gbit/s : $80 \times 10^{12} \times 8 / (0,7 \times 10^9) \approx 914\,000 \text{ s}$, soit **environ 10,6 jours** de transfert continu.

```
sortie = 8000 * 0.09
print(sortie, sortie + 120, sortie / 4 * 0.4 + 120, 80_000 * 0.09, round(80e12 * 8 / 0.7e9 / 86400, 1))
```

```
720.0 840.0 192.0 7200.0 10.6
```

Corrigé 6.12

Il n'y a pas une seule bonne réponse ; voici une réponse défendable, à discuter.

- (a) **Cloud public** : l'association n'a aucun administrateur ni budget d'investissement, la charge est très variable (pointes lors des campagnes), le volume est de quelques Go. Un **service géré** (application d'analyse ou site statique, stockage objet, base gérée) limite l'exploitation. (b) Critères qui pèsent le plus : **compétences** (aucune en exploitation), **profil de charge** (pointe), **contraintes réglementaires** (données personnelles de donateurs). (c) Mesures : **moindre privilège** avec authentification forte pour les rares comptes ; **chiffrement** et **région** choisie dans une zone juridique adaptée, avec un contrat de sous-traitance clair ; **ne publier que des données agrégées** sur le tableau de bord, les données personnelles restant hors de l'espace public ; budgets et alertes de coût. (d) Plan de sortie : **formats ouverts** (CSV, Parquet), **sauvegarde** mensuelle chez un second hébergeur, et **description** de l'installation par un court document ou fichier de configuration. (e) Limite : les prix et les durées de conservation n'ont pas été vérifiés ; si les volumes ou les contraintes juridiques changent (données de santé, par exemple), la recommandation est à **refaire**, avec un juriste.

Chapitre 7 : ⊕ Applications de démonstration — exercices et applications

Ce chapitre du cahier accompagne le chapitre 7 du livre (applications de démonstration avec Streamlit et Shiny). Il est, comme le chapitre, **facultatif**. Les **applications** construisent pas à pas l'application de résiliation du livre, l'accompagnent de tests, de mesures et d'un journal ; les **exercices** se travaillent d'abord à la main, et leurs **corrigés** viennent à la fin. Chaque exercice indique la section du livre qu'il met en pratique.

Tout se passe sans navigateur. Les applications Streamlit sont écrites dans un dossier temporaire et exécutées avec l'outil de test AppTest ; le Shiny pour R du livre n'est pas repris ici. Les données (donnees/clients_ml.csv, donnees/ventes_quotidiennes.csv) sont **simulées**.

Préparation

Une seule cellule fixe l'environnement : bibliothèques, dossier temporaire où seront écrites les applications, chemins des données, et une petite fonction écrire qui enregistre un fichier d'application (les morceaux de code passés à écrire sont mis bout à bout).

```
import os, sys, json, tempfile, textwrap, warnings, logging
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
logging.getLogger("streamlit").setLevel(logging.CRITICAL)
from streamlit.testing.v1 import AppTest

TMP = tempfile.mkdtemp(prefix="cah7_", dir=os.environ.get("TMPDIR"))
sys.path.insert(0, TMP)
os.environ["APP_DONNEES"] = os.path.abspath("donnees/clients_ml.csv")
os.environ["APP_VENTES"] = os.path.abspath("donnees/ventes_quotidiennes.csv")

def bloc(texte):
    return textwrap.dedent(texte).lstrip("\n")

def ecrire(nom, *morceaux):
    with open(os.path.join(TMP, nom), "w", encoding="utf-8") as f:
        f.write("".join(bloc(m) for m in morceaux))
    return os.path.join(TMP, nom)
print("prêt")
```

prêt

Le **module du modèle** contient l'entraînement et la construction d'un profil de client (les médianes du jeu, modifiées par les valeurs saisies). Les applications l'importeront : le code du modèle reste **en dehors** de l'interface.

```

ecrire("modele_churn.py", '''
    import numpy as np
    import pandas as pd
    import lightgbm as lgb

    VARIABLES = ["recence_jours", "nb_commandes_12m", "satisfaction_moy", "nb_tickets_support_12m",
                 "programme_fidelite", "part_achats_promo", "taux_ouverture_email", "anciennete_mois"]

    def entrainer(chemin):
        """70 % des clients pour apprendre, 30 % pour juger l'incertitude."""
        d = pd.read_csv(chemin)
        X, y = d[VARIABLES], d["churn_90j"]
        n = int(0.7 * len(d))
        m = lgb.LGBMClassifier(n_estimators=120, learning_rate=0.05, num_leaves=15,
                               min_child_samples=40,
                               random_state=0, verbose=-1, n_jobs=1).fit(X.iloc[:n], y.iloc[:n])
        return {"modele": m, "scores_val": m.predict_proba(X.iloc[n:][:, 1]),
                "y_val": y.iloc[n:].to_numpy(), "mediane": X.iloc[:n].median()}

    def profil(mediane, **valeurs):
        x = mediane.to_dict()
        x.update(valeurs)
        return pd.DataFrame([x])[VARIABLES]
''')
```

Un petit module de compteurs sert aux mesures, puis on entraîne le modèle une première fois pour vérifier qu'il se comporte comme dans le livre.

```

ecrire("comptes.py", "entrainements = 0\nnetapes = {'charger': 0, 'filtrer': 0, 'agreguer': 0,
↳ 'lisser': 0}\n")
import modele_churn as mc, comptes
from sklearn.metrics import roc_auc_score
M = mc.entrainer(os.environ["APP_DONNEES"])
print("clients de validation :", len(M["y_val"]), "| AUC :", round(roc_auc_score(M["y_val"],
↳ M["scores_val"]), 3))
```

```
clients de validation : 3600 | AUC : 0.849
```

Les colonnes qui fuient l'avenir (commandes_apres_cible) ou révèlent la vérité programmée (segment_vrai) ne figurent pas dans les huit variables du modèle, comme au volume III.

Applications

Application 7.1 — Construire l'application de résiliation pas à pas (sections 7.1 et 7.2)

Objectif. Partir d'une application d'un seul curseur et arriver à l'application du livre, en testant chaque étape sans navigateur.

Étape 1 : un curseur et un résultat. C'est l'application minimale : elle entraîne le modèle, lit la récence, affiche la probabilité.

```

ecrire("app_v1.py", '''
    import os, sys
    import streamlit as st
    sys.path.insert(0, os.path.dirname(__file__))
    import modele_churn as mc

    st.title("Risque de résiliation à 90 jours")
    M = mc.entrainer(os.environ["APP_DONNEES"])
    rec = st.slider("Jours depuis la dernière commande", 0, 365, 60, key="recence")
    x = mc.profil(M["mediane"], recence_jours=rec)
    p = float(M["modele"].predict_proba(x)[:, 1][0])
    st.metric("Probabilité de résiliation", f"{100 * p:.1f} %")
''')
at = AppTest.from_file(os.path.join(TMP, "app_v1.py"), default_timeout=120).run()
print("au démarrage :", at.metric[0].value)
for r in (10, 200, 365):
    at.slider(key="recence").set_value(r).run()
    print(f"recence {r:3d} jours :", at.metric[0].value)

```

```

au démarrage : 3.1 %
recence 10 jours : 3.4 %
recence 200 jours : 6.5 %
recence 365 jours : 11.5 %

```

La probabilité **croît avec la recence** (de 3,4 % à 10 jours à 11,5 % à 365 jours), ce qui est le sens attendu.

Étape 2 : toutes les entrées, le manque, la suggestion. Le code est écrit en trois morceaux (entrées, calcul, affichage) que l'on pourra recombinaison ensuite. Le premier contient aussi le **cache** du modèle : sans lui, chaque clic entraînerait un nouveau modèle.

```

PARTIE_A = '''
    import os, sys
    import numpy as np
    import pandas as pd
    import streamlit as st
    sys.path.insert(0, os.path.dirname(__file__))
    import comptes
    import modele_churn as mc

    @st.cache_resource
    def charger(chemin):
        comptes.entrainements += 1
        return mc.entrainer(chemin)

    st.title("Risque de résiliation à 90 jours")
    M = charger(os.environ["APP_DONNEES"])
    with st.sidebar:
        rec = st.slider("Jours depuis la dernière commande", 0, 365, 60, key="recence")
        nb = st.number_input("Commandes sur 12 mois", 0, 60, 3, key="commandes")
        sat_nr = st.checkbox("Satisfaction non renseignée", key="sat_nr")
        sat = st.slider("Satisfaction moyenne", 1.0, 5.0, 4.0, 0.1, key="satisfaction",
        ↪ disabled=sat_nr)
        tick = st.number_input("Tickets au support", 0, 20, 0, key="tickets")
        fid = st.checkbox("Programme de fidélité", key="fidelite")
'''

```

```

PARTIE_B = '''
    x = mc.profil(M["mediane"], recence_jours=rec, nb_commandes_12m=nb,
                  satisfaction_moy=np.nan if sat_nr else sat,
                  nb_tickets_support_12m=tick, programme_fidelite=int(fid))
    p = float(M["modele"].predict_proba(x)[:, 1][0])
    st.metric("Probabilité de résiliation", f"{100 * p:.1f} %")
'''

```

```

    st.info("Suggestion : relancer." if p >= 1 / 6 else "Suggestion : pas de relance prioritaire.")
...
ecrire("app_v2.py", PARTIE_A, PARTIE_B)
at = AppTest.from_file(os.path.join(TMP, "app_v2.py"), default_timeout=120).run()
print("démarrage      :", at.metric[0].value, "|", at.info[0].value)
at.slider(key="recence").set_value(300).run()
at.slider(key="satisfaction").set_value(2.0).run()
print("300 j, sat. 2,0 :", at.metric[0].value, "|", at.info[0].value)
at.checkbox(key="sat_nr").check().run()
print("sat. manquante :", at.metric[0].value, "| curseur désactivé :",
↪ at.slider(key="satisfaction").disabled)

démarrage      : 2.8 % | Suggestion : pas de relance prioritaire.
300 j, sat. 2,0 : 57.4 % | Suggestion : relancer.
sat. manquante : 10.1 % | curseur désactivé : True

```

La case « Satisfaction non renseignée » désactive le curseur et envoie une **valeur manquante** au modèle, qui l'a rencontrée à l'entraînement : la probabilité (10,1 %) n'est pas celle d'une satisfaction de 2,0 (57,4 %).

Étape 3 : le cache, mesuré. On compte les entraînements pour le démarrage et trois déplacements de curseur, avec le cache puis sans lui.

```

import streamlit as st

def entraînements(source):
    st.cache_resource.clear()                                # le cache est partagé par les fonctions de même
    ↪ code : on repart de zéro
    comptes.entraînements = 0
    chemin = écrire("app_mesure.py", source)
    a = AppTest.from_file(chemin, default_timeout=120).run()
    for v in (300, 150, 60):
        a.slider(key="recence").set_value(v).run()
    return comptes.entraînements

avec = bloc(PARTIE_A) + bloc(PARTIE_B)
sans = avec.replace("@st.cache_resource\n", "")
print("avec cache :", entraînements(avec), "entraînement(s) |", "sans cache :", entraînements(sans),
↪ "entraînement(s)")

avec cache : 1 entraînement(s) | sans cache : 4 entraînement(s)

```

Avec le cache, le modèle est entraîné **une fois** pour les quatre exécutions du script ; sans lui, **quatre fois**.

À faire ensuite. Changez `n_estimators` dans le module (par exemple 600) et refaites la mesure de durée : le coût de l'oubli du cache croît avec la taille du modèle.

Application 7.2 — L'incertitude et l'explication (sections 7.1.4 et 7.1.5)

Objectif. Ajouter à l'application deux éléments qui la rendent honnête : le taux observé chez des clients comparables, avec son intervalle, et les contributions des variables au score.

Étape 1 : deux fonctions de plus dans le module. `voisins` renvoie le taux de résiliation observé chez les `k` clients de validation dont le score est le plus proche, avec un intervalle de Wilson à 95 % ; `contributions` renvoie la part de chaque variable dans le **log-odds** de la prédiction.

```

ecrire("modele_churn.py", open(os.path.join(TMP, "modele_churn.py"), encoding="utf-8").read(), '''
def voisins(scores_val, y_val, p, k=200):
    idx = np.argsort(np.abs(scores_val - p))[:k]
    n, f, z = len(idx), float(y_val[idx].mean()), 1.96

```

```

    centre = (f + z * z / (2 * n)) / (1 + z * z / n)
    demi = z * np.sqrt(f * (1 - f) / n + z * z / (4 * n * n)) / (1 + z * z / n)
    return n, f, centre - demi, centre + demi

def contributions(modele, x):
    """Dernière valeur : terme constant. Somme = log-odds de la prédiction."""
    return modele.predict(x, pred_contrib=True)[0]
'''
import importlib; importlib.reload(mc)
x = mc.profil(M["mediane"], recence_jours=300, satisfaction_moy=2.0)
p = float(M["modele"].predict_proba(x)[:, 1][0])
n, f, bas, haut = mc.voisins(M["scores_val"], M["y_val"], p)
print(f"profil 300 j / sat. 2,0 : p = {100*p:.1f} % ; parmi {n} voisins, {100*f:.1f} % ont résilié
↳ [{100*bas:.1f} ; {100*haut:.1f}]")

```

```
profil 300 j / sat. 2,0 : p = 57.4 % ; parmi 200 voisins, 55.0 % ont résilié [48.1 ; 61.7]
```

Étape 2 : l'identité à vérifier. La somme des contributions (terme constant compris) doit redonner le log-odds du score, donc la probabilité par la fonction logistique. On le vérifie sur cinq profils tirés au hasard dans le jeu.

```

d = pd.read_csv(os.environ["APP_DONNEES"])[mc.VARIABLES].sample(5, random_state=0)
c = mc.contributions(M["modele"], d)
p_modele = M["modele"].predict_proba(d)[:, 1]
p_somme = 1 / (1 + np.exp(-M["modele"].predict(d, pred_contrib=True).sum(axis=1)))
print("écart maximal entre les deux probabilités :", float(np.abs(p_modele - p_somme).max()))

```

```
écart maximal entre les deux probabilités : 2.498001805406602e-16
```

Étape 3 : l'application complète. Le troisième morceau ajoute l'incertitude, la mention de l'hypothèse de coût et le graphique des contributions.

```

PARTIE_C = '''
    n, taux, bas, haut = mc.voisins(M["scores_val"], M["y_val"], p)
    st.caption(f"Parmi les {n} clients au score le plus proche, {100 * taux:.1f} % ont résilié "
              f"(intervalle à 95 % : {100 * bas:.1f} à {100 * haut:.1f} %).")
    st.caption("Seuil de relance 1/6 : un départ manqué coûte 5 fois une relance inutile (hypothèse).")
    c = mc.contributions(M["modele"], x)[-1]
    st.bar_chart(pd.Series(c, index=mc.VARIABLES), horizontal=True)
'''
ecrire("app_v3.py", PARTIE_A, PARTIE_B, PARTIE_C)
at = AppTest.from_file(os.path.join(TMP, "app_v3.py"), default_timeout=120).run()
print("exceptions :", len(at.exception), "| légendes :", len(at.caption), "| graphiques :",
↳ len(at.get("vega_lite_chart")))
print(at.caption[0].value)

```

```
exceptions : 0 | légendes : 2 | graphiques : 1
Parmi les 200 clients au score le plus proche, 5.0 % ont résilié (intervalle à 95 % : 2.7 à 9.0 %).
```

L'identité est vérifiée à la précision des nombres à virgule flottante (l'écart est de l'ordre de 10^{-16}) : l'explication affichée **reproduit exactement** le score.

À faire ensuite. Comparez l'intervalle du profil de départ (2,7 à 9,0 %, soit 6,3 points de large) à celui du profil de 300 jours et de satisfaction 2,0 (48,1 à 61,7 %, soit 13,6 points). Le nombre de voisins est le même (200) ; l'incertitude est pourtant **plus grande près de 50 %**, parce que la variance d'une proportion est maximale à 50 %.

Application 7.3 — Écrire la suite de tests (section 7.2.5)

Objectif. Protéger l'application contre les régressions, et vérifier qu'un test **détecte** vraiment une erreur.

Étape 1 : un garde-fou à tester. On insère, entre les entrées et le calcul, un contrôle de cohérence : une satisfaction inférieure à 2 avec plus de 20 commandes est une combinaison **absente des données**.

```

GARDE = '''
    if not sat_nr and sat < 2 and nb > 20:
        st.warning("Combinaison absente des données d'entraînement : prédiction non fiable.")
        st.stop()
'''

chemin_v4 = ecrire("app_v4.py", PARTIE_A, GARDE, PARTIE_B, PARTIE_C)
d = pd.read_csv(os.environ["APP_DONNEES"])
print("clients qui ont cette combinaison dans le jeu :", int(((d.satisfaction_moy < 2) &
↪ (d.nb_commandes_12m > 20)).sum()))

```

```
clients qui ont cette combinaison dans le jeu : 0
```

Étape 2 : les tests. Chaque test ouvre l'application, manipule les widgets comme un utilisateur et vérifie ce qui s'affiche. Ils sont écrits comme de simples fonctions ; un test **échoue** quand une assertion est fausse.

```

def ouvrir(chemin):
    return AppTest.from_file(chemin, default_timeout=120).run()

def pct(a):
    return float(a.metric[0].value.replace(" %", ""))

def test_demarrage(chemin):
    a = ouvrir(chemin)
    assert not a.exception and len(a.metric) == 1

def test_recence_fait_monter_le_risque(chemin):
    a = ouvrir(chemin)
    bas = pct(a.slider(key="recence").set_value(10).run())
    haut = pct(a.slider(key="recence").set_value(300).run())
    assert haut > bas, f"{haut} <= {bas}"

```

Deux tests de plus : l'un vérifie un **effet attendu** (la fidélité fait baisser le risque), l'autre le **garde-fou**.

```

def test_fidelite_fait_baisser_le_risque(chemin):
    a = ouvrir(chemin)
    a.slider(key="recence").set_value(300).run()
    a.slider(key="satisfaction").set_value(2.0).run()
    sans = pct(a)
    avec = pct(a.checkbox(key="fidelite").check().run())
    assert avec < sans, f"{avec} >= {sans}"

def test_garde_fou(chemin):
    a = ouvrir(chemin)
    a.slider(key="satisfaction").set_value(1.5)
    a.number_input(key="commandes").set_value(30)
    a.run()
    assert len(a.warning) == 1 and len(a.metric) == 0

```

```

TESTS = [test_demarrage, test_recence_fait_monter_le_risque, test_fidelite_fait_baisser_le_risque,
↪ test_garde_fou]

```

```
def lancer(chemin):
```

```

for t in TESTS:
    try:
        t(chemin)
        print("OK      ", t.__name__)
    except AssertionError as e:
        print("ÉCHEC   ", t.__name__, "-", e)

```

```
lancer(chemin_v4)
```

```

OK      test_demarrage
OK      test_recence_fait_monter_le_risque
OK      test_fidelite_fait_baisser_le_risque
OK      test_garde_fou

```

Étape 3 : un test qui ne détecte rien ne sert à rien. On casse volontairement l'application (le programme de fidélité n'est plus transmis au modèle) et l'on relance la suite : un test doit passer au rouge.

```

cassee = bloc(PARTIE_A) + bloc(GARDE) + bloc(PARTIE_B).replace("programme_fidelite=int(fid)",
↳ "programme_fidelite=0") + bloc(PARTIE_C)
lancer(ecrire("app_cassee.py", cassee))

```

```

OK      test_demarrage
OK      test_recence_fait_monter_le_risque
ÉCHEC   test_fidelite_fait_baisser_le_risque - 57.4 >= 57.4
OK      test_garde_fou

```

Seul le test qui regarde la fidélité détecte cette panne ; sans lui, l'application cassée aurait paru normale (elle démarre, elle affiche un chiffre). Un test vaut ce qu'il **vérifie**, pas le fait de s'exécuter.

Application 7.4 — Mesurer l'effet du cache sur un pipeline (section 7.2.4)

Objectif. Compter les étapes réellement exécutées par une application de ventes à quatre étapes, avec et sans cache, puis découvrir un piège du cache.

Étape 1 : l'application et son compteur. Chaque étape incrémente un compteur ; la variable d'environnement AVEC_CACHE choisit de décorer les fonctions ou non.

```

VENTES_A = '''
import os, sys
import pandas as pd
import streamlit as st
sys.path.insert(0, os.path.dirname(__file__))
import comptes
cache = st.cache_data if os.environ.get("AVEC_CACHE") == "1" else (lambda f: f)

@cache
def charger(chemin):
    comptes.etapes["charger"] += 1
    return pd.read_csv(chemin, parse_dates=["date"])

@cache
def filtrer(d, annee):
    comptes.etapes["filtrer"] += 1
    return d[d["date"].dt.year == annee]

@cache
def agreger(d):
    comptes.etapes["agreger"] += 1
    return d.set_index("date")["ventes"].resample("W").sum()
...

```

Puis l'écran : un sélecteur d'année, un curseur de lissage, et la chaîne d'étapes.

```
VENTES_B = '''
annee = st.selectbox("Année", [2023, 2024, 2025], index=2, key="annee")
fenetre = st.slider("Lissage (semaines)", 1, 12, 4, key="fenetre")
s = agreger(filtrer(charger(os.environ["APP_VENTES"]), annee))
comptes.etapes["lisser"] += 1
st.line_chart(s.rolling(fenetre, min_periods=1).mean())
'''

ecrire("app_ventes.py", VENTES_A, VENTES_B)
```

Étape 2 : la séquence d'interactions. Démarrage, trois déplacements du curseur, un changement d'année. Le tableau donne, pour chaque interaction, les étapes exécutées.

```
ETAPES = ["charger", "filtrer", "agreger", "lisser"]

def compter(avec_cache):
    os.environ["AVEC_CACHE"] = "1" if avec_cache else "0"
    for k in ETAPES: comptes.etapes[k] = 0
    a = AppTest.from_file(os.path.join(TMP, "app_ventes.py"), default_timeout=60).run()
    lignes, prec = [dict(comptes.etapes)], dict(comptes.etapes)
    for cible, v in [("fenetre", 2), ("fenetre", 6), ("fenetre", 9), ("annee", 2024)]:
        if cible == "fenetre":
            a.slider(key="fenetre").set_value(v).run()
        else:
            a.selectbox(key="annee").select(v).run()
    lignes.append({k: comptes.etapes[k] - prec[k] for k in ETAPES}); prec = dict(comptes.etapes)
    return pd.DataFrame(lignes, index=["démarrage", "lissage 2", "lissage 6", "lissage 9", "année
↵ 2024"])

for nom, flag in [("sans cache", False), ("avec cache", True)]:
    t = compter(flag)
    print(nom, "- total :", int(t.to_numpy().sum()))
    print(t.to_string())
```

```
sans cache - total : 20
      charger  filtrer  agreger  lisser
démarrage      1       1       1       1
lissage 2       1       1       1       1
lissage 6       1       1       1       1
lissage 9       1       1       1       1
année 2024      1       1       1       1
avec cache - total : 10
      charger  filtrer  agreger  lisser
démarrage      1       1       1       1
lissage 2       0       0       0       1
lissage 6       0       0       0       1
lissage 9       0       0       0       1
année 2024      0       1       1       1
```

Étape 3 : le piège de la clé de cache. `st.cache_data` reconnaît un calcul déjà fait **d'après les arguments**, pas d'après le contenu du fichier. Si le fichier change mais pas son chemin, le cache renvoie l'ancienne version. Une version qui passe en plus la **date de modification** du fichier évite le piège.

```
ecrire("app_perime.py", '''
import os
import pandas as pd
import streamlit as st

@st.cache_data
def lire(chemin):
    return pd.read_csv(chemin)
```

```

@st.cache_data
def lire_versionne(chemin, version):
    return pd.read_csv(chemin)

chemin = os.environ["CSV_TEST"]
st.write(f"sans version : {len(lire(chemin))} lignes")
st.write(f"avec version : {len(lire_versionne(chemin, os.path.getmtime(chemin)))} lignes")
'''
os.environ["CSV_TEST"] = os.path.join(TMP, "petit.csv")
pd.DataFrame({"x": [1, 2, 3]}).to_csv(os.environ["CSV_TEST"], index=False)
a = AppTest.from_file(os.path.join(TMP, "app_perime.py"), default_timeout=60).run()
print("avant la mise à jour :", [m.value for m in a.markdown])
pd.DataFrame({"x": range(10)}).to_csv(os.environ["CSV_TEST"], index=False)
os.utime(os.environ["CSV_TEST"], (2_000_000_000, 2_000_000_000))    # une date de modification
↪ différente, sans attendre
print("après la mise à jour :", [m.value for m in a.run().markdown])

```

```

avant la mise à jour : ['sans version : 3 lignes', 'avec version : 3 lignes']
après la mise à jour : ['sans version : 3 lignes', 'avec version : 10 lignes']

```

Application 7.5 — Étendre le mini système réactif (section 7.3.2)

Objectif. Reprendre la miniature du livre, vérifier qu'elle ne calcule jamais deux fois le même noeud dans un graphe en **losange**, et qu'un noeud que personne ne demande n'est **jamais** calculé.

Étape 1 : la miniature. C'est le code du livre, en deux blocs.

```

class Noeud:
    """Une entrée (f=None) ou un calcul qui dépend d'autres noeuds."""
    actif = None
    def __init__(self, f=None, valeur=None):
        self.f, self.valeur, self.valide = f, valeur, f is None
        self.lecteurs = set()
    def __call__(self):
        if Noeud.actif is not None:
            self.lecteurs.add(Noeud.actif)
        if not self.valide:
            avant, Noeud.actif = Noeud.actif, self
            self.valeur, self.valide = self.f(), True
            Noeud.actif = avant
        return self.valeur
    def invalider(self):
        self.valide = self.f is None
        lecteurs, self.lecteurs = self.lecteurs, set()
        for n in lecteurs:
            if n.valide:
                n.invalider()

```

```

SORTIES = []
def sortie(f):
    n = Noeud(f); SORTIES.append(n); return n
def entree(n, valeur):
    if valeur != n.valeur:
        n.valeur = valeur
        n.invalider()
def rafraichir():
    for s in SORTIES:
        if not s.valide:
            s()

```

Étape 2 : un losange. L'entrée a alimente deux calculs b et c, qui alimentent tous deux d, affiché. Un cinquième noeud inutile dépend de a mais n'est lu par personne.

```

appels = {k: 0 for k in "bcdi"}
a = Noeud(valeur=1)
def calc_b(): appels["b"] += 1; return a() + 1
def calc_c(): appels["c"] += 1; return a() * 2
b, c = Noeud(calc_b), Noeud(calc_c)
def calc_d(): appels["d"] += 1; return b() + c()
def calc_i(): appels["i"] += 1; return a() * 100
d, inutile = sortie(calc_d), Noeud(calc_i)

rafraichir()
print("démarrage      : d =", d.valeur, "| appels :", appels)
entree(a, 5); rafraichir()
print("a passe à 5    : d =", d.valeur, "| appels :", appels)
entree(a, 5); rafraichir()
print("a reste à 5    : d =", d.valeur, "| appels :", appels)

```

```

démarrage      : d = 4 | appels : {'b': 1, 'c': 1, 'd': 1, 'i': 0}
a passe à 5    : d = 16 | appels : {'b': 2, 'c': 2, 'd': 2, 'i': 0}
a reste à 5    : d = 16 | appels : {'b': 2, 'c': 2, 'd': 2, 'i': 0}

```

Chaque calcul du losange s'exécute **une fois** par changement, et d n'est pas calculé deux fois bien qu'il ait deux parents ; inutile, que personne ne lit, n'est **jamais** calculé (compteur i à zéro) ; enfin, remettre a à la même valeur **ne déclenche rien**.

À faire ensuite. Ajoutez une seconde sortie e qui lit b : un changement de a recalcule-t-il b une ou deux fois ? (Réponse attendue : une, car b est mémorisé.)

Application 7.6 — Préparer la mise en service (sections 7.4.1 à 7.4.4 et 7.4.7)

Objectif. Mettre en œuvre les quatre pratiques de base : une configuration lue de l'extérieur et **validée**, un journal sans valeurs en clair, des versions figées, un inventaire des licences.

Étape 1 : une configuration qui échoue clairement. La fonction lit des variables d'environnement, applique des valeurs par défaut et refuse proprement une valeur invalide ; on la teste dans quatre situations.

```

def lire_config(env):
    cfg = {"donnees": env.get("APP_DONNEES"), "seuil": env.get("APP_SEUIL", "0.1667")}
    if not cfg["donnees"]:
        raise ValueError("APP_DONNEES manquante : indiquez le fichier de données.")
    try:
        cfg["seuil"] = float(cfg["seuil"])
    except ValueError:
        raise ValueError(f"APP_SEUIL invalide : {cfg['seuil']}!r n'est pas un nombre.") from None
    if not 0 < cfg["seuil"] < 1:
        raise ValueError("APP_SEUIL doit être strictement entre 0 et 1.")
    return cfg

for nom, env in [("valide", {"APP_DONNEES": "d.csv"}), ("fichier manquant", {}), ("seuil illisible",
↪ {"APP_DONNEES": "d.csv", "APP_SEUIL": "abc"}), ("seuil hors bornes", {"APP_DONNEES": "d.csv",
↪ "APP_SEUIL": "1.5"})]:
    try:
        print(f"{nom:17s}: {lire_config(env)}")
    except ValueError as e:
        print(f"{nom:17s}: erreur - {e}")

```

```

valide          : {'donnees': 'd.csv', 'seuil': 0.1667}
fichier manquant : erreur - APP_DONNEES manquante : indiquez le fichier de données.
seuil illisible  : erreur - APP_SEUIL invalide : 'abc' n'est pas un nombre.
seuil hors bornes: erreur - APP_SEUIL doit être strictement entre 0 et 1.

```

Étape 2 : un journal sans valeurs en clair, puis son exploitation. L'identifiant de session est haché, les valeurs sont réduites à des tranches ; on lit ensuite le journal pour compter les événements.

```
import hashlib, time
from collections import Counter

def journaliser(chemin, evenement, session, **champs):
    ligne = {"ts": time.time(), "evenement": evenement,
            "session": hashlib.sha256(session.encode()).hexdigest()[:8], **champs}
    with open(chemin, "a", encoding="utf-8") as f:
        f.write(json.dumps(ligne, ensure_ascii=False) + "\n")

def tranche(v, bornes):
    return next((f"<{b}" for b in bornes if v < b), f">={bornes[-1]}")

chemin = os.path.join(TMP, "journal.jsonl")
rng = np.random.default_rng(0)
for i in range(40):
    r, p = int(rng.integers(0, 366)), float(rng.beta(1, 6))
    journaliser(chemin, "simulation", f"session-{i % 7}", recence=tranche(r, [90, 180]),
    ↪ risque=tranche(p, [0.1, 0.3]))
journaliser(chemin, "erreur", "session-3", type="saisie hors bornes")
lignes = [json.loads(l) for l in open(chemin, encoding="utf-8")]
print("événements :", dict(Counter(l["evenement"] for l in lignes)))
print("sessions distinctes :", len({l["session"] for l in lignes}))
print("risques par tranche :", dict(sorted(Counter(l.get("risque") for l in lignes if l["evenement"]
    ↪ == "simulation").items())))
```

```
événements : {'simulation': 40, 'erreur': 1}
sessions distinctes : 7
risques par tranche : {'<0.1': 19, '<0.3': 15, '>=0.3': 6}
```

Étape 3 : versions figées et licences. Un seul tableau pour les deux : version installée (à recopier dans requirements.txt) et licence déclarée, avec un indicateur pour les licences dites **à copyleft** (famille GPL), qui méritent une lecture attentive avant de distribuer un logiciel.

```
from importlib import metadata as md

def licence(nom):
    m = md.metadata(nom)
    classes = [c.split("::")[-1].strip() for c in (m.get_all("Classifier") or []) if
    ↪ c.startswith("License ::")]
    champ = (m.get("License") or "").strip().splitlines()
    return m.get("License-Expression") or " ; ".join(classes) or (champ[0][:40] if champ else "non
    ↪ déclarée")

noms = ["streamlit", "lightgbm", "pandas", "numpy", "scikit-learn", "matplotlib"]
inv = pd.DataFrame({"version": [md.version(n) for n in noms], "licence": [licence(n) for n in noms]}),
    ↪ index=noms)
inv["copyleft ?"] = inv["licence"].str.contains("GPL", case=False)
print(inv.to_string())
```

	version	licence	copyleft ?
streamlit	1.65.0	Apache-2.0	False
lightgbm	4.7.0	MIT	False
pandas	3.0.6	BSD License	False
numpy	2.5.3	BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0	False
scikit-learn	1.9.1	BSD-3-Clause	False
matplotlib	3.11.2	Python Software Foundation License	False

Une licence « non déclarée » ne veut **pas** dire « libre de droits » : elle veut dire qu'il faut chercher l'information ailleurs (dépôt du projet, fichier LICENSE).

Application 7.7 — Application ou API ? (section 7.4.9)

Objectif. Séparer le modèle de l'interface : un petit service de prédiction (une fonction qui reçoit une requête et renvoie une réponse, avec validation), puis une application qui ne connaît plus le modèle.

Étape 1 : le service. Il charge le modèle **une fois**, valide la requête, renvoie la probabilité et la **version** du modèle. Les erreurs sont renvoyées sous forme de liste, comme le ferait une API.

```

ecrire("service_scoring.py", '''
    import os
    import numpy as np
    import modele_churn as mc

    VERSION = "churn-2026.10"
    BORNES = {"recence_jours": (0, 365), "nb_commandes_12m": (0, 60), "satisfaction_moy": (1.0, 5.0),
              "nb_tickets_support_12m": (0, 20), "programme_fidelite": (0, 1)}
    _M = mc.entrainer(os.environ["APP_DONNEES"]) # chargé une seule fois, à l'import

    def scorer(requete):
        erreurs = [f"{k} hors de {b}" for k, b in BORNES.items()
                  if k in requete and requete[k] is not None and not b[0] <= requete[k] <= b[1]]
        erreurs += [f"champ inconnu : {k}" for k in requete if k not in BORNES]
        if erreurs:
            return {"version": VERSION, "erreurs": erreurs}
        x = mc.profil(_M["mediane"], **{k: (np.nan if v is None else v) for k, v in requete.items()})
        return {"version": VERSION, "probabilite": float(_M["modele"].predict_proba(x)[: , 1][0]),
                "erreurs": []}
    '''
import service_scoring as svc
for requete in [{"recence_jours": 300, "satisfaction_moy": 2.0}, {"recence_jours": 900}, {"couleur":
    "bleu"}]:
    print(requete, "->", svc.scorer(requete))

```

```

{'recence_jours': 300, 'satisfaction_moy': 2.0} -> {'version': 'churn-2026.10', 'probabilite':
    0.5735396712863174, 'erreurs': []}
{'recence_jours': 900} -> {'version': 'churn-2026.10', 'erreurs': ['recence_jours hors de (0, 365)']}
{'couleur': 'bleu'} -> {'version': 'churn-2026.10', 'erreurs': ['champ inconnu : couleur']}

```

Étape 2 : une application qui n'embarque plus le modèle. Elle envoie les valeurs saisies au service et affiche la réponse. On vérifie qu'elle affiche **le même chiffre** que l'application autonome de l'application 7.1.

```

ecrire("app_client.py", '''
    import streamlit as st
    import service_scoring as svc

    st.title("Risque de résiliation à 90 jours (client du service)")
    rec = st.slider("Jours depuis la dernière commande", 0, 365, 60, key="recence")
    sat = st.slider("Satisfaction moyenne", 1.0, 5.0, 4.0, 0.1, key="satisfaction")
    r = svc.scorer({"recence_jours": rec, "satisfaction_moy": sat})
    if r["erreurs"]:
        st.error("; ".join(r["erreurs"]))
        st.stop()
    st.metric("Probabilité de résiliation", f"{100 * r['probabilite']:.1f} %")
    st.caption(f"Modèle {r['version']}")
    '''
client = AppTest.from_file(os.path.join(TMP, "app_client.py"), default_timeout=120).run()
client.slider(key="recence").set_value(300).run()
client.slider(key="satisfaction").set_value(2.0).run()
autonome = AppTest.from_file(os.path.join(TMP, "app_v2.py"), default_timeout=120).run()
autonome.slider(key="recence").set_value(300).run()
autonome.slider(key="satisfaction").set_value(2.0).run()
print("application cliente :", client.metric[0].value, client.caption[0].value)

```

```
print("application autonome :", autonome.metric[0].value)
```

```
application cliente : 57.4 % Modèle churn-2026.10
application autonome : 57.4 %
```

Les deux affichent la même probabilité (les autres entrées de l'application autonome ont les mêmes valeurs par défaut que les médianes du service pour les variables non saisies). L'interface a perdu toute connaissance du modèle : on peut changer de modèle en changeant seulement la **version** du service.

Étape 3 : la grille de décision. Quatre signes (section 7.4.9) transforment la question « application ou API ? » en une règle simple que l'on applique à des situations.

```
signes = ["un autre programme appelle le modèle", "plusieurs équipes ou interfaces", "mises à jour du
↳ modèle indépendantes", "disponibilité exigée"]
situations = {
    "démonstration pour la gérante": [0, 0, 0, 0],
    "le site doit afficher un score": [1, 0, 0, 0],
    "logiciel client + site + application": [1, 1, 0, 0],
    "outil du service client, ouvert toute la journée": [0, 0, 1, 1],
}
def recommandation(v):
    n = sum(v)
    return "rester sur l'application" if n == 0 else "API à envisager" if n == 1 else "API recommandée"
grille = pd.DataFrame(situations, index=signes).T
grille["recommandation"] = [recommandation(v) for v in situations.values()]
print(grille.to_string())
```

	un autre programme appelle le modèle	plusieurs équipes ou interfaces	mises à jour du modèle indépendantes	disponibilité exigée	recommandation
démonstration pour la gérante	0	0	0	0	rester sur l'application
↳ 0	0	0	0	0	1
le site doit afficher un score	1	0	0	0	API à envisager
↳ 0	1	0	0	0	1
logiciel client + site + application	1	1	0	0	API recommandée
↳ 1	1	1	0	0	0
outil du service client, ouvert toute la journée	0	0	1	1	API recommandée
↳ 0	0	0	1	1	

La grille est volontairement simple : elle ne remplace pas le jugement, mais elle oblige à **nommer** les signes avant de décider.

Exercices

Exercice 7.1 ★ — Démonstration, prototype ou produit ? (section 7.1.1)

Classez chaque situation comme **démonstration**, **prototype** ou **produit**, et dites quelle exigence manque encore, d'après le tableau de la section 7.1.1 : (a) un outil que douze chargés de clientèle ouvrent chaque matin ; (b) un notebook où l'auteur déplace les curseurs devant la gérante ; (c) une page que trois utilisateurs pilotes essaient sans l'auteur.

Exercice 7.2 ★★ — Une combinaison absente (section 7.1.3)

Avec `clients_ml.csv`, comptez les clients qui ont à la fois plus de 10 commandes sur douze mois et plus de 240 jours depuis la dernière commande. Que peut-on en conclure pour un garde-fou de l'application ? Proposez une règle fondée sur les **données** (un percentile), pas sur l'intuition.

Exercice 7.3 ★★ — Lire l'intervalle (section 7.1.4)

Programmez l'intervalle de Wilson à 95 % pour une proportion observée de 20 % et des échantillons de 50, 200, 800 et 3 200 clients. Calculez la largeur de chaque intervalle. Comment varie-t-elle quand on **multiplie par quatre** le nombre de clients ?

Exercice 7.4 ★ — Combien d'exécutions ? (sections 7.2.1 et 7.2.2)

Une application Streamlit contient un curseur et un bouton, et un compteur incrémente à chaque exécution du script. L'utilisateur ouvre la page, déplace trois fois le curseur et clique deux fois sur le bouton. Combien de fois le script s'est-il exécuté ? Vérifiez avec AppTest.

Exercice 7.5 ★★ — `cache_data` ou `cache_resource` ? (section 7.2.4)

Pour chacun des objets suivants, choisissez `cache_data`, `cache_resource` ou **aucun cache**, et justifiez : (a) un DataFrame de ventes lu dans un fichier ; (b) le modèle entraîné ; (c) une connexion à une base de données ; (d) la liste des valeurs saisies par l'utilisateur courant ; (e) le résultat d'une agrégation qui dépend de l'année choisie. Démontrez ensuite par une petite application que modifier sur place un DataFrame renvoyé par `cache_data` ne modifie pas le cache.

Exercice 7.6 ★★ — Écrire un test (section 7.2.5)

Écrivez deux tests pour l'application de l'application 7.2 : (1) cocher « Satisfaction non renseignée » désactive le curseur de satisfaction ; (2) pour 300 jours de récence et une satisfaction de 2,0, la suggestion est « relancer », alors que pour 10 jours et une satisfaction de 4,5 elle ne l'est pas.

Exercice 7.7 ★★★ — Le compteur qui reste à 1 (section 7.2.6)

Le script ci-dessous devrait compter les clics sur un bouton, mais il affiche toujours 1. Expliquez pourquoi, corrigez-le avec `st.session_state`, et montrez par AppTest que trois clics donnent bien 3.

```
import streamlit as st
n = 0
if st.button("Ajouter", key="ajouter"):
    n += 1
st.write("clics :", n)
```

Exercice 7.8 ★ — Streamlit, Shiny ou autre ? (section 7.3.5)

Pour chaque situation, indiquez l'outil le plus adapté et pourquoi : (a) montrer à la gérante, cet après-midi, le modèle de résiliation écrit en Python ; (b) un tableau de bord de quinze filtres interdépendants, maintenu par une équipe qui travaille en R ; (c) un système qui doit être appelé par le site web de l'entreprise.

Exercice 7.9 ★★ — Qui est recalculé ? (sections 7.3.1 et 7.3.2)

Un graphe contient les entrées a et b , les calculs $c = f(a)$ et $d = g(b, c)$, et deux sorties : $s1$ lit d et $s2$ lit c . Quand b change seul, quels noeuds sont recalculés ? Et quand a change ? Prédisez, puis vérifiez avec la miniature de l'application 7.5.

Exercice 7.10 ★★★ — Lire sans dépendre (section 7.3.4)

Ajoutez à la miniature une fonction `isoler(noeud)` qui renvoie la valeur d'un noeud **sans** créer de dépendance (l'équivalent de `isolate()` de Shiny). Montrez qu'une sortie qui lit x directement est recalculée quand x change, et qu'une sortie qui lit x par `isoler(x)` ne l'est pas.

Exercice 7.11 ★ — Que mettre dans le journal ? (section 7.4.4)

Pour chacun de ces champs, dites s'il faut le **garder tel quel**, le **hacher**, le **réduire en tranche** ou le **ne pas journaliser** : (a) l'horodatage ; (b) l'identifiant de session ; (c) la récence saisie ; (d) l'identifiant du client interrogé ; (e) la version du modèle ; (f) le message d'une exception ; (g) la clé d'API de la plate-forme.

Exercice 7.12 ★★ — Choisir la couleur du texte (section 7.4.6)

Pour chaque couleur de la palette {bleu : #2a78d6, orange : #eb6834, aqua : #1baf7a, violet : #4a3aa7, rouge : #e34948}, calculez le rapport de contraste du texte **blanc** (#ffffff) et du texte **noir** (#0b0b0b) sur ce fond, puis choisissez automatiquement la meilleure couleur de texte. Combien de fonds atteignent le seuil de 4,5:1 avec leur meilleur texte ?

Exercice 7.13 ★★ — Repérer les licences à examiner (section 7.4.7)

Écrivez une fonction qui, pour une liste de paquets installés, renvoie ceux dont la licence déclarée n'est **pas** dans une liste de licences permissives (MIT, BSD, Apache, PSF, Zlib...). Appliquez-la à dix paquets de l'environnement. Quelles sont les **limites** de cette détection automatique ?

Exercice 7.14 ★★★ — Application ou API : le dossier (section 7.4.9)

Une équipe de quarante conseillers utilise l'application de résiliation depuis trois mois. Le service des ventes en ligne demande maintenant que le **site** affiche aussi un score. Écrivez, en une page, le dossier de décision : situation, signes présents (grille de l'application 7.7), architecture proposée, ce qui reste dans l'application, ce qui passe dans le service, et trois risques à surveiller.

Corrigés

Corrigé 7.1

- (a) **Produit** : douze utilisateurs, chaque jour. Il faut de la **fiabilité**, de la **sécurité** (accès, secrets), de la **surveillance** et de la **maintenance** : une démonstration n'a rien de tout cela.
- (b) **Démonstration** : l'auteur est présent et sait ce qu'il ne faut pas toucher. L'exigence est la **clarté** et l'**honnêteté** de l'écran (hypotheses, données simulées).
- (c) **Prototype** : l'enjeu est d'être **utilisable sans l'auteur**, donc des messages d'erreur clairs, des valeurs par défaut, des bornes.

Corrigé 7.2

```
d = pd.read_csv("donnees/clients_ml.csv")
combo = (d.nb_commandes_12m > 10) & (d.recence_jours > 240)
print("clients avec > 10 commandes ET > 240 jours depuis la dernière :", int(combo.sum()))
print("clients avec > 10 commandes :", int((d.nb_commandes_12m > 10).sum()), "| avec > 240 jours :",
      ↪ int((d.recence_jours > 240).sum()))
print("99e percentile des commandes :", d.nb_commandes_12m.quantile(0.99))
```

```
clients avec > 10 commandes ET > 240 jours depuis la dernière : 2
clients avec > 10 commandes : 766 | avec > 240 jours : 2182
99e percentile des commandes : 17.0
```

Seuls 2 clients sur 12 000 cumulent les deux, alors que 766 ont plus de 10 commandes et 2 182 plus de 240 jours sans commande : un client très actif a, presque toujours, commandé récemment. Une règle fondée sur les données : **avertir** quand une valeur dépasse le 99e percentile observé, et quand une **combinaison** n'a aucun représentant dans le jeu. Le modèle n'a rien appris sur ces zones ; il y **extrapole**.

Corrigé 7.3

```
def wilson(f, n, z=1.96):
    centre = (f + z * z / (2 * n)) / (1 + z * z / n)
    demi = z * np.sqrt(f * (1 - f) / n + z * z / (4 * n * n)) / (1 + z * z / n)
    return centre - demi, centre + demi

larg = {}
for n in (50, 200, 800, 3200):
    bas, haut = wilson(0.20, n)
    larg[n] = haut - bas
    print(f"n = {n:4d} : [{100 * bas:.1f} ; {100 * haut:.1f}] %, largeur {100 * larg[n]:.1f} points")
print("rapport des largeurs quand n est multiplié par 4 :", [round(float(larg[a] / larg[b]), 2) for a,
      ↪ b in [(50, 200), (200, 800), (800, 3200)]])
```

```
n = 50 : [11.2 ; 33.0] %, largeur 21.8 points
n = 200 : [15.0 ; 26.1] %, largeur 11.0 points
n = 800 : [17.4 ; 22.9] %, largeur 5.5 points
n = 3200 : [18.7 ; 21.4] %, largeur 2.8 points
rapport des largeurs quand n est multiplié par 4 : [1.97, 1.99, 2.0]
```

La largeur est **divisée par environ 2** (de 1,97 à 2,00) quand l'échantillon est multiplié par 4, ce qui est la loi en $1/\sqrt{n}$: obtenir une estimation deux fois plus précise coûte quatre fois plus de données.

Corrigé 7.4

```
ecrire("app_ex74.py", '''
    import os, sys
    import streamlit as st
    sys.path.insert(0, os.path.dirname(__file__))
    import comptes
    comptes.executions = getattr(comptes, "executions", 0) + 1
    v = st.slider("Valeur", 0, 10, 5, key="v")
    if st.button("Valider", key="b"):
        st.write("validé", v)
''')
comptes.executions = 0
a = AppTest.from_file(os.path.join(TMP, "app_ex74.py"), default_timeout=60).run()
for v in (2, 7, 9):
```

```

a.slider(key="v").set_value(v).run()
for _ in range(2):
    a.button(key="b").click().run()
print("exécutions :", comptes.executions)

```

```
exécutions : 6
```

Une exécution au démarrage, plus une par interaction (trois mouvements du curseur, deux clics) : le script se rejoue à **chaque** interaction, que l'utilisateur ait changé une valeur ou cliqué.

Corrigé 7.5

Objet	Choix	Justification
(a) DataFrame de ventes	cache_data	une donnée : chaque appelant reçoit sa copie, sans risque de modification croisée
(b) modèle entraîné	cache_resource	une ressource lourde et en lecture seule, partagée par tous
(c) connexion à une base	cache_resource	un objet qu'on ne copie pas ; un seul pour tous les utilisateurs
(d) valeurs saisies par l'utilisateur	aucun cache : st.session_state	propre à une personne ; un cache partagé les ferait fuiter
(e) agrégation selon l'année	cache_data	résultat qui dépend d'arguments (l'année) ; un résultat par année

```

ecrire("app_ex75.py", '''
import streamlit as st
import pandas as pd

@st.cache_data
def table():
    return pd.DataFrame({"x": [1, 2, 3]})

t = table()
t.loc[0, "x"] = 99          # modification sur place de l'objet reçu
st.write("valeur relue dans le cache :", int(table().loc[0, "x"]))
''')
a = AppTest.from_file(os.path.join(TMP, "app_ex75.py"), default_timeout=60).run()
print(a.markdown[0].value)

```

```
valeur relue dans le cache : `1`
```

La valeur relue est restée celle d'origine : avec `cache_data`, l'appelant travaille sur une **copie**. Avec `cache_resource`, la modification serait visible de tous.

Corrigé 7.6

```

def test_case_satisfaction_manquante(chemin):
    a = AppTest.from_file(chemin, default_timeout=120).run()
    assert not a.slider(key="satisfaction").disabled
    a.checkbox(key="sat_nr").check().run()
    assert a.slider(key="satisfaction").disabled

def test_suggestion(chemin):
    a = AppTest.from_file(chemin, default_timeout=120).run()

```

```

a.slider(key="recence").set_value(300).run()
a.slider(key="satisfaction").set_value(2.0).run()
assert a.info[0].value.endswith("relancer.")
a.slider(key="recence").set_value(10).run()
a.slider(key="satisfaction").set_value(4.5).run()
assert "pas de relance" in a.info[0].value

for t in (test_case_satisfaction_manquante, test_suggestion):
    t(chemin_v4)
    print("OK", t.__name__)

```

```

OK test_case_satisfaction_manquante
OK test_suggestion

```

Corrigé 7.7

Le script **se rejoue** à chaque clic : la variable ordinaire `n` est recrée à 0 à chaque exécution, puis incrémentée de 1 quand le bouton vient d'être cliqué. Elle ne peut donc valoir que 0 ou 1. Il faut ranger le compteur dans `st.session_state`, qui survit aux rejeux.

```

bug = '''
import streamlit as st
n = 0
if st.button("Ajouter", key="ajouter"):
    n += 1
st.write("clics :", n)
'''

correct = '''
import streamlit as st
if "n" not in st.session_state:
    st.session_state.n = 0
if st.button("Ajouter", key="ajouter"):
    st.session_state.n += 1
st.write("clics :", st.session_state.n)
'''

for nom, src in [("version fautive", bug), ("version corrigée", correct)]:
    a = AppTest.from_file(ecrire("app_ex77.py", src), default_timeout=60).run()
    for _ in range(3):
        a.button(key="ajouter").click().run()
    print(nom, ":", a.markdown[-1].value)

```

```

version fautive : clics : `1`
version corrigée : clics : `3`

```

Corrigé 7.8

- (a) **Streamlit** : le modèle est en Python, il s'agit d'une démonstration, une page suffit ; c'est la prise en main la plus rapide.
- (b) **Shiny pour R** : une équipe R, et des filtres interdépendants, ce que le graphe réactif gère naturellement (chaque filtre n'invalidé que ce qui en dépend).
- (c) **Ni l'un ni l'autre** : ce besoin est celui d'une **API** de scoring (chapitre 4, section 4.2), appelée par un programme et non par une personne.

Corrigé 7.9

Quand b change seul, seul d dépend de b : d et s1 sont recalculés ; c et s2 ne le sont **pas**. Quand a change, c est périmé, donc aussi d (qui lit c), s1 et s2 : tout est recalculé sauf b (une entrée).

```

cmpt = {k: 0 for k in "cd"} | {"s1": 0, "s2": 0}
ea, eb = Noeud(valeur=1), Noeud(valeur=1)
def f_c(): cmpt["c"] += 1; return ea() + 1
nc = Noeud(f_c)
def f_d(): cmpt["d"] += 1; return eb() + nc()
nd = Noeud(f_d)
SORTIES.clear()
def f_s1(): cmpt["s1"] += 1; return nd()
def f_s2(): cmpt["s2"] += 1; return nc()
sortie(f_s1); sortie(f_s2); rafraichir()
def apres(nom, cible, v):
    avant = dict(cmpt); entree(cible, v); rafraichir()
    print(nom, {k: cmpt[k] - avant[k] for k in cmpt})
apres("b change :", eb, 5)
apres("a change :", ea, 5)

```

```

b change : {'c': 0, 'd': 1, 's1': 1, 's2': 0}
a change : {'c': 1, 'd': 1, 's1': 1, 's2': 1}

```

Corrigé 7.10

Il suffit de lire le noeud **en suspendant** le noeud « actif » : sans lecteur courant, aucune dépendance n'est enregistrée.

```

def isoler(n):
    avant, Noeud.actif = Noeud.actif, None
    try:
        return n()
    finally:
        Noeud.actif = avant

SORTIES.clear()
x = Noeud(valeur=1)
cpt = {"direct": 0, "isole": 0}
def lit_direct(): cpt["direct"] += 1; return x()
def lit_isole(): cpt["isole"] += 1; return isoler(x)
sortie(lit_direct); sortie(lit_isole); rafraichir()
entree(x, 2); rafraichir()
entree(x, 3); rafraichir()
print("exécutions après deux changements de x :", cpt, "(le départ compte pour 1)")

```

```

exécutions après deux changements de x : {'direct': 3, 'isole': 1} (le départ compte pour 1)

```

La sortie qui lit x directement est recalculée à chaque changement ; celle qui passe par isoler ne l'est qu'au départ, ce qui est exactement ce que l'on veut pour une valeur « lue au moment du clic » (et non « suivie en permanence »).

Corrigé 7.11

Champ	Traitement	Pourquoi
(a) horodatage	garder	indispensable, sans donnée personnelle
(b) identifiant de session	hacher	permet de compter les sessions sans identifier

Champ	Traitement	Pourquoi
(c) récence saisie	tranche	la valeur exacte n'apporte rien à la supervision
(d) identifiant du client interrogé	ne pas journaliser (ou hacher, avec un motif écrit)	donnée personnelle ; le journal serait un fichier sensible
(e) version du modèle	garder	indispensable pour interpréter une anomalie
(f) message d'exception	garder, après vérification	utile au diagnostic, mais il peut contenir des chemins ou des valeurs
(g) clé d'API	ne jamais journaliser	un secret écrit dans un journal est un secret perdu

Corrigé 7.12

```
def luminance(h):
    c = [int(h[i:i + 2], 16) / 255 for i in (1, 3, 5)]
    c = [x / 12.92 if x <= 0.03928 else ((x + 0.055) / 1.055) ** 2.4 for x in c]
    return 0.2126 * c[0] + 0.7152 * c[1] + 0.0722 * c[2]

def contraste(a, b):
    haut, bas = sorted([luminance(a), luminance(b)], reverse=True)
    return (haut + 0.05) / (bas + 0.05)

fonds = {"bleu": "#2a78d6", "orange": "#eb6834", "aqua": "#1baf7a", "violet": "#4a3aa7", "rouge": "#e34948"}
lignes = []
for nom, f in fonds.items():
    b, n = contraste("#ffffff", f), contraste("#0b0b0b", f)
    lignes.append((nom, round(b, 2), round(n, 2), "blanc" if b >= n else "noir", max(b, n) >= 4.5))
t = pd.DataFrame(lignes, columns=["fond", "blanc", "noir", "meilleur texte", ">= 4,5"])
print(t.to_string())
print("fonds qui atteignent 4,5 avec leur meilleur texte :", int(t[">= 4,5"].sum()), "sur", len(t))
```

```
      blanc  noir meilleur texte  ≥ 4,5
fond
bleu      4.42  4.46          noir  False
orange    3.20  6.15          noir   True
aqua      2.82  6.99          noir   True
violet    8.56  2.30         blanc   True
rouge     3.95  4.98          noir   True
fonds qui atteignent 4,5 avec leur meilleur texte : 4 sur 5
```

Le **bleu** échoue de peu avec les deux textes (4,42 en blanc, 4,46 en noir) : il faut alors **assombrir ou éclaircir le fond**. Pour l'orange, l'aqua et le rouge, le texte **noir** est le bon choix, ce qui est contraire à l'intuition qui voudrait du blanc sur une couleur vive.

Corrigé 7.13

```
PERMISSIVES = ("MIT", "BSD", "APACHE", "PSF", "PYTHON SOFTWARE FOUNDATION", "ZLIB", "ISC", "0BSD",
    ↪ "CC0")

def a_examiner(paquets):
    """Paquets dont la licence déclarée ne contient aucun mot de la liste permissive."""
    return {nom: licence(nom) for nom in paquets if not any(p in licence(nom).upper() for p in
    ↪ PERMISSIVES)}
```

```
dix = ["streamlit", "lightgbm", "pandas", "numpy", "scikit-learn", "matplotlib", "certifi", "tqdm",
      ↪ "psutil", "protobuf"]
print({nom: licence(nom) for nom in ["certifi", "tqdm"]})
print("à examiner :", a_examiner(dix))
```

```
{'certifi': 'Mozilla Public License 2.0 (MPL 2.0)', 'tqdm': 'MPL-2.0 AND MIT'}
à examiner : {'certifi': 'Mozilla Public License 2.0 (MPL 2.0)'}
```

certifi (MPL-2.0) est bien signalé. Mais tqdm, dont la licence est MPL-2.0 AND MIT, **ne l'est pas** : le mot « MIT » suffit à le faire passer. C'est la première limite : la détection se fonde sur un **texte libre** que chaque projet remplit à sa façon, et une licence **composée** contenant un mot permissif passe à tort. Les autres limites : une licence « non déclarée » est signalée mais pas résolue ; la licence d'un paquet ne dit rien de celle de **ses dépendances**, ni de celle des **données** et du **modèle**. L'outil trie, il ne juge pas.

Corrigé 7.14

Un dossier acceptable contient les éléments suivants.

- **Situation.** Quarante conseillers utilisent l'application depuis trois mois ; le service de vente en ligne veut un score sur le site.
- **Signes présents.** *Un autre programme appelle le modèle* (le site) et *plusieurs interfaces* : deux signes sur quatre, donc « API recommandée » avec la grille de l'application 7.7. La disponibilité deviendra exigée dès que le site dépendra du service.
- **Architecture.** Un **service de prédiction** (une API) charge un modèle **versionné** ; l'application des conseillers et le site en sont deux **clients**.
- **Reste dans l'application.** L'interface, les explications affichées, la suggestion et l'avertissement sur les combinaisons inédites.
- **Passe dans le service.** Le chargement du modèle, la **validation des requêtes**, la version, le **journal** et la supervision.
- **Trois risques.** (1) Un site qui appelle le service avec des valeurs hors bornes ou manquantes : la validation doit répondre par une erreur claire. (2) Une **dérive** des données (chapitre 4, section 4.7) qui dégrade le modèle sans que personne ne le voie. (3) Une **indisponibilité** du service qui, sans repli prévu, rend le site incapable d'afficher un score : prévoir un comportement par défaut.

Projet du volume et auto-évaluation — exercices et applications

Ce chapitre du cahier clôt le volume IV. Il contient **le projet du volume** : prendre le modèle de résiliation du volume III et en faire un **système** (un pipeline reproductible et testé, un suivi d'expériences, un service qui valide ses entrées, une supervision qui détecte la dérive), puis l'**auto-évaluation**. Aucune notion nouvelle : chaque étape renvoie à la section du livre qui l'explique. Tout s'exécute **hors ligne**, dans des dossiers temporaires.

Projet du volume

P.1 Le cahier des charges

La gérante a obtenu, au volume III, un modèle qui prédit les clients qui ne commanderont plus dans les 90 jours. Elle veut maintenant que l'équipe relation client reçoive **chaque matin** ces scores. Elle pose cinq exigences :

1. **Reproductible** : on doit pouvoir réentraîner le modèle et retrouver les mêmes résultats.
2. **Versé et traçable** : à tout moment, savoir quel modèle sert, entraîné avec quelles données et quels réglages.
3. **Servi par une API** qui **refuse** les données invalides au lieu de produire un score absurde.
4. **Supervisé** : être prévenue si les clients d'aujourd'hui ne ressemblent plus à ceux de l'entraînement.
5. **Remplaçable sans risque** : un nouveau modèle ne doit passer en service que s'il est meilleur, et on doit pouvoir revenir en arrière.

La méthode suit huit étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Structurer	Où vivent le code, la configuration, les graines ?	4.1
P.3 Pipeline testé	Les données sont-elles saines ? Le modèle est-il reproductible ?	4.1
P.4 Suivre	Quel modèle, quels réglages, quelles métriques ?	4.5
P.5 Servir	Comment exposer le modèle sans accepter n'importe quoi ?	4.2, 4.6
P.6 Journaliser et superviser	Les clients changent-ils ?	4.3, 4.7

Étape	Question	Chapitre du livre
P.7 Décider du réentraînement	La dérive abîme-t-elle les performances ?	4.7, 4.3
P.8 Automatiser	Comment rejouer tout cela sans intervention ?	4.4, 4.6
P.9 Rapporter	Peut-on mettre en production ?	4.1 à 4.3

Les données. On reprend le fichier `donnees/clients_ml.csv` du volume III (12 000 clients, cible `churn_90j`), **simulé**. Rappel important : la colonne `commandes_apres_cible` est la **fuite d'information** du volume III (elle décrit l'avenir) et `segment_vrai` est une vérité cachée : ni l'une ni l'autre ne doit entrer dans le modèle. Une partie du travail de cette étude consiste à **empêcher mécaniquement** qu'elles y entrent.

P.2 Étape 1 : structurer et configurer

On commence par séparer ce qui **change** (la configuration) de ce qui **ne change pas** (le code). Tout le reste du projet lit cette configuration.

```
import json, os, tempfile, warnings
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
ESPACE = tempfile.mkdtemp(prefix="projet_mlops_")      # tout ce que le projet écrit vit ici, puis
↳ disparaît

CONFIG = {
    "graine": 7,
    "colonnes_exclues": ["id_client", "churn_90j", "depense_6m", "segment_vrai",
↳ "commandes_apres_cible"],
    "cible": "churn_90j",
    "part_test": 0.25,
    "seuil_alerte_psi": 0.2,
    "seuil_decision": 0.20,
}
donnees = pd.read_csv("donnees/clients_ml.csv")
print(donnees.shape, "| taux de résiliation :", round(donnees[CONFIG["cible"]].mean(), 3))
```

```
(12000, 24) | taux de résiliation : 0.14
```

P.3 Étape 2 : un pipeline testé

Vérifier les données avant de modéliser (livre, section 4.1). Ces vérifications sont des fonctions : elles s'exécutent à chaque entraînement et **arrêtent** le pipeline si quelque chose cloche.

```
def verifier_donnees(df, cfg):
    problemes = []
    if df["id_client"].duplicated().any():
        problemes.append("identifiants clients en double")
    if not df[cfg["cible"]].isin([0, 1]).all():
        problemes.append("cible non binaire")
    if not df["age"].between(15, 100).all():
        problemes.append("âge hors de [15, 100]")
    if (df["recence_jours"] < 0).any() or (df["montant_12m"] < 0).any():
        problemes.append("valeurs négatives impossibles")
    entrees = [c for c in df.columns if c not in cfg["colonnes_exclues"]]
```

```

interdites = {"commandes_apres_cible", "segment_vrai"} & set(entrees)
if interdites:
    problemes.append(f"colonnes interdites parmi les entrées : {sorted(interdites)}")
return problemes, entrees

problemes, ENTREES = verifier_donnees(donnees, CONFIG)
print("problèmes détectés :", problemes or "aucun", "|", len(ENTREES), "variables d'entrée")

```

```

problèmes détectés : aucun | 19 variables d'entrée

```

On vérifie aussi que **le garde-fou fonctionne** : un test qui ne détecte jamais rien ne prouve rien (livre, section 4.1).

```

fautif = donnees.copy()
fautif.loc[0, "age"] = 250
tricheur = donnees.assign(commandes_apres_cible=donnees["commandes_apres_cible"])
cfg_laxiste = {**CONFIG, "colonnes_exclues": ["id_client", "churn_90j", "depense_6m"]} # on «
↳ oublie » d'exclure les deux colonnes
print("âge impossible      :", verifier_donnees(fautif, CONFIG)[0])
print("fuite non exclue    :", verifier_donnees(tricheur, cfg_laxiste)[0])

```

```

âge impossible      : ['âge hors de [15, 100]']
fuite non exclue    : ['colonnes interdites parmi les entrées : ['commandes_apres_cible',
↳ 'segment_vrai']]

```

Le **pipeline de modèle** applique toutes les transformations **dans** la validation croisée et dans le service (livre, volume III, section 4.1) : imputation avec indicateurs d'absence, encodage des modalités, puis un boosting.

```

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import roc_auc_score, average_precision_score

X, y = donnees[ENTREES], donnees[CONFIG["cible"]]
CATEGORIELLES = ["ville", "canal_acquisition", "appareil", "categorie_preferee"]
NUMERIQUES = [c for c in ENTREES if c not in CATEGORIELLES]

def fabriquer_pipeline(modele):
    prep = ColumnTransformer([
        ("num", SimpleImputer(strategy="median", add_indicator=True), NUMERIQUES),
        ("cat", OneHotEncoder(handle_unknown="ignore", min_frequency=20), CATEGORIELLES)])
    return Pipeline([("prep", prep), ("modele", modele)])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=CONFIG["part_test"],
↳ random_state=CONFIG["graine"], stratify=y)
pipe = fabriquer_pipeline(HistGradientBoostingClassifier(random_state=CONFIG["graine"])).fit(X_tr,
↳ y_tr)
p_te = pipe.predict_proba(X_te)[:, 1]
print(f"AUC test = {roc_auc_score(y_te, p_te):.4f} | précision moyenne =
↳ {average_precision_score(y_te, p_te):.4f}")

```

```

AUC test = 0.8800 | précision moyenne = 0.6105

```

Deux **tests de modèle** complètent la vérification des données : le pipeline doit être **déterministe** (même graine, mêmes scores) et ne doit pas regresser sous un score plancher.

```

pipe2 = fabriquer_pipeline(HistGradientBoostingClassifier(random_state=CONFIG["graine"])).fit(X_tr,
↳ y_tr)

```

```

tests = {
    "déterminisme (même graine, mêmes scores)": bool(np.allclose(pipe2.predict_proba(X_te)[: , 1],
        ↪ p_te)),
    "AUC au-dessus du plancher de 0,85": roc_auc_score(y_te, p_te) > 0.85,
    "scores dans [0, 1]": bool(((p_te >= 0) & (p_te <= 1)).all()),
    "tolère une valeur manquante":
        ↪ bool(np.isfinite(pipe.predict_proba(X_te.head(1).assign(satisfaction_moy=np.nan))[: ,
        ↪ 1]).all()),
}
for nom, ok in tests.items():
    print("OK " if ok else "ÉCHEC", nom)
assert all(tests.values())

```

```

OK déterminisme (même graine, mêmes scores)
OK AUC au-dessus du plancher de 0,85
OK scores dans [0, 1]
OK tolère une valeur manquante

```

P.4 Étape 3 : suivre les expériences

On ne retient pas un modèle « parce que le notebook l'a dit » : on compare des candidats et on **consigne** ce qu'on a fait (livre, section 4.5). **MLflow** enregistre les paramètres, les métriques et le modèle ; ici avec une base SQLite temporaire.

```

import logging, mlflow, mlflow.sklearn
from mlflow.tracking import MlflowClient
from sklearn.linear_model import LogisticRegression

logging.getLogger("mlflow").setLevel(logging.ERROR)
mlflow.set_tracking_uri(f"sqlite:///{{ESPACE}}/mlflow.db")
mlflow.set_experiment("resiliation")

candidats = {
    "logistique": fabriquer_pipeline(LogisticRegression(max_iter=2000)),
    "boosting": fabriquer_pipeline(HistGradientBoostingClassifier(random_state=CONFIG["graine"])),
}
for nom, modele in candidats.items():
    with mlflow.start_run(run_name=nom):
        modele.fit(X_tr, y_tr)
        p = modele.predict_proba(X_te)[: , 1]
        mlflow.log_param("famille", nom)
        mlflow.log_param("graine", CONFIG["graine"])
        mlflow.log_metric("auc_test", roc_auc_score(y_te, p))
        mlflow.log_metric("precision_moyenne_test", average_precision_score(y_te, p))
        mlflow.sklearn.log_model(modele, name="modele", serialization_format="cloudpickle")
runs = mlflow.search_runs(experiment_names=["resiliation"], order_by=["metrics.auc_test DESC"])
print(runs[["tags.mlflow.runName", "params.famille", "metrics.auc_test",
    ↪ "metrics.precision_moyenne_test"]].round(4).to_string(index=False))

```

tags.mlflow.runName	params.famille	metrics.auc_test	metrics.precision_moyenne_test
boosting	boosting	0.8800	0.6105
logistique	logistique	0.8527	0.5519

Sérialiser n'est pas anodin. Le format cloudpickle choisi ici enregistre l'objet Python tel quel ; **charger un fichier de ce type exécute du code** qu'il contient. On ne charge donc **que des modèles produits par soi-même** ou par une source de confiance. MLflow propose par défaut un format plus sûr (skops), qui refuse ici le modèle parce qu'il contient un type qu'il ne connaît pas : ce refus est le comportement voulu d'un outil prudent, et la raison pour laquelle nous précisons le format (livre, section 4.2).

On **enregistre le meilleur** dans le registre de modèles sous un alias « production ». Le service ira chercher le modèle par cet alias : changer de modèle, c'est déplacer l'alias, et revenir en arrière aussi.

```
client = MlflowClient()
meilleur = runs.iloc[0]["run_id"]
version = mlflow.register_model(f"runs:{meilleur}/modele", "modele_resiliation")
client.set_registered_model_alias("modele_resiliation", "production", version.version)
mv = client.get_model_version_by_alias("modele_resiliation", "production")
print("version en production :", mv.version, "| issue du run « boosting » :", mv.run_id == meilleur)
```

```
version en production : 1 | issue du run « boosting » : True
```

P.5 Étape 4 : servir par une API qui valide ses entrées

Le service (livre, sections 4.2 et 4.6) charge le modèle **par alias**, définit un **schéma** d'entrée (types, bornes) et renvoie, avec le score, la **version du modèle**. Une entrée invalide est refusée avec une erreur 422 : c'est la validation qui protège la gérante d'un score absurde.

```
from typing import Literal, Optional
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

class Client(BaseModel):
    age: int = Field(ge=15, le=100)
    ville: str
    canal_acquisition: Literal["Boutique", "Site", "Réseaux"]
    appareil: Optional[Literal["mobile", "ordinateur", "tablette"]] = None
    anciennete_mois: int = Field(ge=0)
    nb_commandes_12m: int = Field(ge=0)
    panier_moyen: Optional[float] = Field(default=None, ge=0)
    montant_12m: float = Field(ge=0)
    recence_jours: int = Field(ge=0, le=365)
    nb_retours_12m: int = Field(ge=0)
    satisfaction_moy: Optional[float] = Field(default=None, ge=1, le=5)
    nb_tickets_support_12m: int = Field(ge=0)
    programme_fidelite: Literal[0, 1]
    nb_promos_recues_12m: int = Field(ge=0)
    part_achats_promo: float = Field(ge=0, le=1)
    taux_ouverture_email: float = Field(ge=0, le=1)
    delai_livraison_moy: Optional[float] = Field(default=None, ge=0)
    categorie_preferree: Literal["A", "B", "C", "D"]
    revenu_zone: float

modele_prod = mlflow.sklearn.load_model("models:/modele_resiliation@production")
app = FastAPI(title="Score de résiliation")

@app.get("/health")
def sante():
    return {"statut": "ok", "version_modele": int(mv.version)}

@app.post("/predict")
def predire(c: Client):
    ligne = pd.DataFrame([c.model_dump()])[ENTREES]
    return {"score": round(float(modele_prod.predict_proba(ligne)[0, 1]), 6), "version_modele":
        ↪ int(mv.version)}

api = TestClient(app)
print(api.get("/health").json())
```

```
{'statut': 'ok', 'version_modele': 1}
```

On teste le **contrat** de l'API : un client valide, un client invalide, une valeur facultative absente, et surtout la **parité** avec le calcul hors ligne (le service ne doit pas calculer autre chose que le notebook).

```
exemple = X_te.iloc[0].to_dict()
exemple = {k: (None if isinstance(v, float) and np.isnan(v) else (v.item() if hasattr(v, "item") else
↳ v)) for k, v in exemple.items()}
rep = api.post("/predict", json=exemple)
print("client valide      :", rep.status_code, rep.json())
print("parité hors ligne :", abs(rep.json()["score"] - float(pipe.predict_proba(X_te.iloc[[0]])[0,
↳ 1])) < 1e-4)

invalide = {**exemple, "age": 250, "canal_acquisition": "Autre"}
r = api.post("/predict", json=invalide)
print("client invalide   :", r.status_code, "| champs refusés :", sorted(e["loc"][-1] for e in
↳ r.json()["detail"]))

sans_satisfaction = {**exemple, "satisfaction_moy": None}
print("valeur absente    :", api.post("/predict", json=sans_satisfaction).status_code)
```

```
client valide      : 200 {'score': 0.017973, 'version_modele': 1}
parité hors ligne  : True
client invalide    : 422 | champs refusés : ['age', 'canal_acquisition']
valeur absente     : 200
```

P.6 Étape 5 : journaliser et superviser

Un modèle en production se **regarde** : on journalise chaque prédiction, puis on compare ce que le modèle voit aujourd'hui à ce qu'il a vu à l'entraînement (livre, sections 4.3 et 4.7). Simulons **huit semaines** d'exploitation. Les clients de chaque semaine sont tirés dans le jeu de test ; à partir de la semaine 5, une **campagne de promotions** change la clientèle : on sur-échantillonne les clients très sensibles aux promotions et peu satisfaits (c'est une **dérive de covariables** : les clients changent, pas la règle qui relie leurs caractéristiques à la résiliation).

```
rng = np.random.default_rng(CONFIG["graine"])
base = X_te.copy()
base["vrai"] = y_te.to_numpy()

def semaine(i, n=500):
    poids = np.ones(len(base))
    if i >= 5:
        intensite = 1 + 2.5 * (i - 4) / 4
        poids = np.exp(intensite * (2.0 * (base["part_achats_promo"].to_numpy() - 0.3) - 0.6 *
↳ (np.nan_to_num(base["satisfaction_moy"].to_numpy(), nan=3.5) - 3.5)))
        idx = rng.choice(len(base), n, replace=False if i < 5 else True, p=poids / poids.sum())
        return base.iloc[idx].copy().assign(semaine=i)

journal = pd.concat([semaine(i) for i in range(1, 9)], ignore_index=True)
journal["score"] = pipe.predict_proba(journal[ENTREES])[:, 1]
print(journal.groupby("semaine")[["part_achats_promo", "score"]].mean().round(3).T.to_string())
```

semaine	1	2	3	4	5	6	7	8
part_achats_promo	0.274	0.289	0.300	0.274	0.572	0.684	0.748	0.799
score	0.127	0.130	0.134	0.132	0.188	0.233	0.249	0.311

Pour **mesurer** la dérive d'une variable, on utilise l'**indice de stabilité de population** (PSI), écrit à la main : on découpe la variable en classes d'après l'**entraînement**, puis on compare les proportions observées aujourd'hui à celles d'hier.

$$\text{PSI} = \sum_k (a_k - e_k) \ln \frac{a_k}{e_k}$$

où e_k est la part attendue (entraînement) dans la classe k et a_k la part actuelle. Règle usuelle : moins de 0,1 stable, de 0,1 à 0,25 à surveiller, plus de 0,25 dérive marquée.

```
def psi(attendu, actuel, classes=10):
    attendu, actuel = np.asarray(attendu, float), np.asarray(actuel, float)
    bornes = np.unique(np.quantile(attendu[~np.isnan(attendu)], np.linspace(0, 1, classes + 1)))
    bornes[0], bornes[-1] = -np.inf, np.inf
    e = np.histogram(attendu[~np.isnan(attendu)], bornes)[0] / np.sum(~np.isnan(attendu))
    a = np.histogram(actuel[~np.isnan(actuel)], bornes)[0] / np.sum(~np.isnan(actuel))
    e, a = np.clip(e, 1e-4, None), np.clip(a, 1e-4, None)
    return float(np.sum((a - e) * np.log(a / e)))

surveillees = ["part_achats_promo", "satisfaction_moy", "recence_jours", "nb_commandes_12m", "age",
               ↪ "montant_12m"]
table = pd.DataFrame({f"s{i}": {v: psi(X_tr[v], journal.loc[journal.semaine == i, v]) for v in
               ↪ surveillees} for i in range(1, 9)})
scores_tr = pipe.predict_proba(X_tr)[:, 1]
table.loc["score du modèle"] = [psi(scores_tr, journal.loc[journal.semaine == i, "score"]) for i in
               ↪ range(1, 9)]
print(table.round(3).to_string())
```

	s1	s2	s3	s4	s5	s6	s7	s8
part_achats_promo	0.017	0.031	0.026	0.009	0.963	1.864	2.812	4.083
satisfaction_moy	0.011	0.009	0.020	0.009	0.539	1.191	1.724	3.016
recence_jours	0.014	0.007	0.003	0.022	0.064	0.118	0.201	0.462
nb_commandes_12m	0.010	0.011	0.008	0.013	0.091	0.077	0.146	0.229
age	0.025	0.021	0.025	0.009	0.169	0.449	0.515	0.992
montant_12m	0.013	0.031	0.020	0.030	0.170	0.296	0.421	0.601
score du modèle	0.009	0.026	0.030	0.048	0.229	0.357	0.604	0.910

Une **règle d'alerte** transforme ces nombres en action : ici, alerte si le PSI d'au moins **deux** variables, ou celui du **score**, dépasse le seuil de la configuration.

```
def alerte(colonne, seuil=CONFIG["seuil_alerte_psi"]):
    v = table.drop(index="score du modèle")[colonne]
    return bool((v > seuil).sum() >= 2 or table.loc["score du modèle", colonne] > seuil)

etat = pd.DataFrame({"semaine": range(1, 9), "alerte": [alerte(f"s{i}") for i in range(1, 9)]})
print(etat.T.to_string(header=False))
print("première alerte à la semaine :", int(etat.loc[etat["alerte"], "semaine"].min()) if
       ↪ etat["alerte"].any() else "aucune")
```

semaine	1	2	3	4	5	6	7	8
alerte	False	False	False	False	True	True	True	True
première alerte à la semaine :	5							

P.7 Étape 6 : la dérive abîme-t-elle les performances ?

Une alerte de dérive n'est **pas** une alerte de performance (livre, section 4.7). Les étiquettes (a-t-il résilié ?) n'arrivent qu'**au bout de 90 jours** : pendant ce temps, on est aveugle. Une fois les étiquettes connues, on peut mesurer.

```
perf = journal.groupby("semaine").apply(lambda g: pd.Series({"AUC": roc_auc_score(g["vrai"],
               ↪ g["score"]), "taux de résiliation": g["vrai"].mean(), "score moyen": g["score"].mean()}))
print(perf.round(3).T.to_string())
```

semaine	1	2	3	4	5	6	7	8
AUC	0.875	0.908	0.863	0.853	0.779	0.811	0.830	0.823
taux de résiliation	0.138	0.140	0.158	0.154	0.212	0.274	0.254	0.304
score moyen	0.127	0.130	0.134	0.132	0.188	0.233	0.249	0.311

Dans notre simulation, la dérive est une **dérive de covariables** pure : les clients changent, mais la relation entre leurs caractéristiques et la résiliation reste la même. Deux constats, tous deux instructifs :

- **L'AUC baisse** (environ 0,875 en moyenne sur les semaines 1 à 4, environ 0,81 sur les semaines 5 à 8) **alors que la règle n'a pas changé**. La clientèle devenue plus homogène (beaucoup de clients sensibles aux promotions) est simplement **plus difficile à classer** : l'AUC dépend de la population sur laquelle on la mesure, et ne se compare pas d'une population à l'autre.
- **La calibration tient** : le taux de résiliation observé suit le score moyen (par exemple 0,304 observé pour 0,311 prédit en semaine 8). Le modèle ne se « trompe » pas de niveau ; il y a seulement plus de clients à risque.

Une baisse d'AUC n'est donc **pas** à elle seule la preuve que le modèle est cassé. Voyons maintenant la situation où **la règle elle-même change** (dérive conceptuelle) : à partir de la semaine 5, les clients sensibles aux promotions résilient **plus que ne le prévoit le modèle** (simulation : leur probabilité réelle de résiliation est multipliée par deux).

```
rng2 = np.random.default_rng(11)
j2 = journal.copy()
sensibles = (j2["part_achats_promo"] > 0.5) & (j2["programme_fidelite"] == 0)
doubler = (j2["semaine"] >= 5) & sensibles
p_vraie = np.clip(j2["score"].to_numpy() * np.where(doubler, 2.0, 1.0), 0, 0.98)
j2["vrai_concept"] = np.where(j2["semaine"] >= 5, rng2.binomial(1, p_vraie), j2["vrai"])
perf2 = j2.groupby("semaine").apply(lambda g: pd.Series({"AUC": roc_auc_score(g["vrai_concept"],
↪ g["score"]), "écart (observé - prédit)": g["vrai_concept"].mean() - g["score"].mean()}))
print(perf2.round(3).T.to_string())
```

semaine	1	2	3	4	5	6	7	8
AUC	0.875	0.908	0.863	0.853	0.917	0.932	0.934	0.952
écart (observé - prédit)	0.011	0.010	0.024	0.022	0.098	0.129	0.123	0.127

Dans ce scénario, l'AUC **ne signale rien** : elle monte même (de 0,85-0,91 en semaines 1 à 4 à 0,92-0,95 ensuite), parce que le sous-groupe concerné devient plus facile à repérer. Seul l'**écart entre le taux observé et le score moyen** (le défaut de calibration, volume III, section 5.2) révèle le problème : il passe d'environ 0,01-0,02 à environ 0,10-0,13. D'où la règle pratique : surveiller **à la fois** la dérive des entrées (immédiate, sans étiquettes) et la **calibration** dès que les étiquettes arrivent ; l'AUC seule peut rassurer à tort comme alarmer à tort.

Réentraîner. Quand la décision est prise, on réentraîne sur une **fenêtre récente**, on compare le candidat à l'ancien modèle sur des données **récentes**, et on ne promeut le candidat que s'il est meilleur (**portail de promotion**).

```
recent = j2[j2["semaine"].between(5, 6)]
a_reentraîner = recent.drop(columns=["vrai", "score", "semaine", "vrai_concept"])
nouveau = fabriquer_pipeline(HistGradientBoostingClassifier(random_state=CONFIG["graine"],
↪ max_iter=60)).fit(pd.concat([X_tr, a_reentraîner]), pd.concat([y_tr, recent["vrai_concept"]]))
eval_ = j2[j2["semaine"].between(7, 8)]
ancien_auc = roc_auc_score(eval_["vrai_concept"], pipe.predict_proba(eval_[ENTREES])[:, 1])
nouveau_auc = roc_auc_score(eval_["vrai_concept"], nouveau.predict_proba(eval_[ENTREES])[:, 1])
print(f"semaines 7-8 : AUC de l'ancien modèle {ancien_auc:.4f} | du candidat {nouveau_auc:.4f}")
print("promotion du candidat :", "OUI" if nouveau_auc > ancien_auc + 0.005 else "NON (gain
↪ insuffisant)")
```

```
semaines 7-8 : AUC de l'ancien modèle 0.9439 | du candidat 0.9381
promotion du candidat : NON (gain insuffisant)
```

Le portail refuse la promotion : le candidat, entraîné sur peu de données récentes, **ne fait pas mieux** que l'ancien modèle (son AUC est même un peu plus basse). Mais le vrai problème de ce scénario est un **défaut de calibration**, et il existe un remède bien moins coûteux que le réentraînement : **recalibrer** les scores de l'ancien modèle sur les données récentes (méthode de Platt, volume III, section 5.2).

```
from sklearn.linear_model import LogisticRegression as LR

def logit(p):
    p = np.clip(p, 1e-4, 1 - 1e-4)
    return np.log(p / (1 - p)).reshape(-1, 1)

platt = LR(C=1e6).fit(logit(recent["score"].to_numpy()), recent["vrai_concept"])
s78 = eval_["score"].to_numpy()
recal = platt.predict_proba(logit(s78))[:, 1]
print(f"semaines 7-8 : taux observé {eval_['vrai_concept'].mean():.3f} | score moyen avant
↳ {s78.mean():.3f} | après recalibrage {recal.mean():.3f}")
print(f"AUC avant {roc_auc_score(eval_['vrai_concept'], s78):.4f} | après
↳ {roc_auc_score(eval_['vrai_concept'], recal):.4f} (le classement ne change pas : la
↳ transformation est croissante)")
```

```
semaines 7-8 : taux observé 0.405 | score moyen avant 0.280 | après recalibrage 0.404
AUC avant 0.9439 | après 0.9439 (le classement ne change pas : la transformation est croissante)
```

Le recalibrage ramène le score moyen au niveau du taux observé **sans toucher au classement** : c'est la première réponse à tenter quand seule la calibration dérive. Il reste soumis au même portail de promotion et se versionne comme un modèle.

P.8 Étape 7 : automatiser

Tout ce qui précède doit pouvoir être **rejoué sans intervention**. On l'écrit sous forme de commandes et de fichiers de configuration. Ils ne sont **pas exécutés ici** (ni Docker ni service d'intégration continue ne sont disponibles dans notre environnement) ; chaque ligne est expliquée dans le livre, sections 4.4 et 4.6.

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY service/ service/
ENV MODEL_ALIAS=production
EXPOSE 8000
CMD ["uvicorn", "service.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .github/workflows/ci.yml : à chaque modification, tester avant de pouvoir livrer
name: ci
on: [push, pull_request]
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.13" }
      - run: pip install -r requirements.txt
      - run: pytest -q # tests de données, de modèle et de contrat d'API
```

P.9 Étape 8 : le rapport de mise en production

On termine par une **liste de décision** (go / no-go), générée à partir des résultats : une mise en production se justifie par des critères écrits, pas par un sentiment.

```
criteres = {
    "données validées (aucun problème détecté)": not problemes,
    "tests de modèle réussis": all(tests.values()),
    "AUC de test au-dessus de 0,85": roc_auc_score(y_te, p_te) > 0.85,
    "API : entrée invalide refusée (422)": r.status_code == 422,
    "API : parité avec le calcul hors ligne": abs(rep.json()["score"] -
        float(pipe.predict_proba(X_te.iloc[[0]])[0, 1])) < 1e-4,
    "modèle enregistré avec alias « production »": mv.aliases == ["production"] or "production" in
        mv.aliases,
    "supervision : la règle d'alerte détecte la dérive simulée (semaine 5)":
        bool(etat["alerte"].any()) and int(etat.loc[etat["alerte"], "semaine"].min()) == 5,
}
for nom, ok in criteres.items():
    print("OK " if ok else "À REVOIR", nom)
print("\nDécision :", "GO pour la mise en production" if all(criteres.values()) else "NO-GO")
```

```
OK données validées (aucun problème détecté)
OK tests de modèle réussis
OK AUC de test au-dessus de 0,85
OK API : entrée invalide refusée (422)
OK API : parité avec le calcul hors ligne
OK modèle enregistré avec alias « production »
OK supervision : la règle d'alerte détecte la dérive simulée (semaine 5)
```

Décision : GO pour la mise en production

À retenir. Un modèle en production, c'est un **pipeline testé, un modèle versionné, une API qui valide, une supervision qui regarde et un moyen de revenir en arrière**. Le modèle n'est qu'une des briques ; la plupart des pannes viennent d'ailleurs : données qui changent, entrées invalides, nouvelle version non testée, étiquettes qui arrivent trop tard.

P.10 Les limites de l'étude

- **Simulation de production.** Les « huit semaines » sont tirées (avec remise à partir de la semaine 5) dans les 3 000 clients du jeu de test ; la dérive est fabriquée. En vraie production, la dérive est moins nette et ses causes sont à chercher (changement de la collecte, saison, campagne, panne d'un capteur).
- **Pas de service réel.** L'API est testée **en mémoire** (TestClient) : nous n'avons pas mesuré le comportement sous charge, ni la sécurité (authentification, limitation du débit), ni le déploiement conteneurisé.
- **Pas d'orchestration.** Le réentraînement est déclenché à la main dans ce projet ; un orchestrateur (livre, section 4.4) le déclencherait sur critère.
- **Seuil de décision supposé.** Le seuil de 0,20 de la configuration n'a pas été optimisé : il dépend des coûts réels de la relation client (volume III, section 4.3.5).
- **Équité et vie privée non traitées ici** : un score de résiliation utilisé pour cibler des clients pose ces questions (volume III, section 5.4 ; chapitre 5 de ce volume, section 5.4).

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des chapitres facultatifs ($\oplus$ 3.3, 4.4 à 4.7, 6 et 7) comptent si vous les avez lues.

Deep learning (chapitre 1)

1. Combien de paramètres compte un réseau dense $784 \rightarrow 128 \rightarrow 10$ (avec les biais) ?
2. Pourquoi un neurone à sortie sigmoïde, entraîné avec la perte logistique, est-il « une régression logistique » ?
3. Une couche convolutive prend une image 28×28 à un canal et applique 8 filtres 3×3 sans rembourrage. Combien de paramètres, et quelle taille de carte de sortie ?
4. Pourquoi les gradients disparaissent-ils dans un réseau profond à activations sigmoïdes ? Donnez un ordre de grandeur pour 10 couches, et deux remèdes.
5. Pour un neurone sigmoïde et la perte logistique, quel est le gradient de la perte par rapport à la pré-activation z quand la probabilité prédite est 0,8 et que l'étiquette vaut 1 ?
6. Votre réseau a une perte d'entraînement qui baisse et une perte de validation qui remonte. Que diagnostiquez-vous, et quels leviers avez-vous ?
7. Que fait la porte d'oubli d'un LSTM, et quel problème du réseau récurrent simple le LSTM atténue-t-il ?
8. Sur le tableau des clients (résiliation), le réseau obtient une AUC de 0,869 et le boosting 0,896. Que concluez-vous, et que faut-il vérifier avant de généraliser ?

NLP et modèles de langage (chapitre 2)

9. Calculez le poids TF-IDF ($\text{tf} \cdot \ln \frac{N}{\text{df}}$) d'un mot présent deux fois dans un avis et dans un seul des trois avis du corpus.
10. Quel est le cosinus entre les vecteurs de comptage $(1, 0, 1)$ et $(1, 1, 0)$?
11. Un classifieur de sentiments obtient 94 % sur un test tiré du même moule que l'entraînement. Pourquoi ce chiffre peut-il tromper, et comment le vérifier ?
12. Pourquoi divise-t-on les scores d'attention par $\sqrt{d_k}$? Quel est l'écart-type des scores bruts si $d_k = 64$ et si les composantes sont indépendantes de variance 1 ?
13. Environ combien de paramètres un bloc transformer de dimension $d = 256$ compte-t-il (ordre de grandeur $12 d^2$) ?
14. Pourquoi l'attention coûte-t-elle « le carré de la longueur », et que devient le coût quand on double la longueur de la séquence ?
15. Que fait la température du décodage ? Donnez les probabilités de softmax(2, 1, 0) à $T = 1$ et à $T = 0,5$.
16. Un système RAG répond faussement avec assurance. Quels maillons examinez-vous, dans quel ordre ?

Big data et calcul distribué (chapitre 3)

17. Quel est l'accélération maximale d'un programme dont 90 % du temps se parallélise, sur 10 machines puis sur un nombre infini ?
18. Citez trois situations où l'on ne devrait **pas** distribuer le calcul.
19. Pourquoi une moyenne ne se combine-t-elle pas simplement par « moyenne des moyennes » dans un MapReduce, et comment la réécrire ?

20. Qu'est-ce que l'asymétrie des clés dans une jointure ou un regroupement, et comment le salage la corrige-t-il ?
21. Qu'appelle-t-on évaluation paresseuse dans Spark, et que signale un Exchange dans un plan ?
22. Un consommateur de messages offre la garantie « au moins une fois ». Que doit faire le traitement pour que le résultat reste correct ?

MLOps (chapitre 4)

23. Un service reçoit 50 requêtes par seconde et chaque requête dure 0,2 s en moyenne. Combien de requêtes sont en cours en moyenne (loi de Little) ?
24. Pourquoi ne faut-il charger un modèle sérialisé par pickle que depuis une source de confiance ?
25. Qu'est-ce que le décalage entre entraînement et service ? Donnez un exemple qui ne fait planter aucun programme.
26. Quelle différence entre un déploiement fantôme et un déploiement canari ?
27. Calculez le PSI pour une variable dont la répartition attendue est (0,5 ; 0,5) et la répartition observée (0,3 ; 0,7).
28. Distinguez dérive des variables et dérive du concept : quel indicateur détecte chacune ?
29. Pourquoi une estimation de la performance sans étiquettes peut-elle être aveugle ? Citez le cas vu au chapitre.
30. Comment revient-on à la version précédente d'un modèle en moins d'une minute ?

Ingénierie des données (chapitre 5)

31. Un fichier compte 19 700 lignes, dont 1 700 doublons et 782 rejets. Combien de lignes propres attendez-vous, et pourquoi cette « équation » est-elle un test ?
32. Pourquoi le rejeu d'un lot gonfle-t-il la table avec un simple ajout, et pas avec une fusion (*upsert*) ?
33. Combien de paires faudrait-il comparer pour 5 000 clients sans blocage ? À quoi sert le blocage ?
34. Pourquoi une empreinte de l'adresse électronique (SHA-256 sans clé) ne protège-t-elle pas les personnes ? Que faire à la place ?
35. Citez quatre dimensions de la qualité des données et, pour deux d'entre elles, une règle écrite comme une petite fonction.

Plateformes cloud (chapitre 6)

36. Deux composants de disponibilité 99,9 % en série, puis le même composant en double (en parallèle) : quelle disponibilité chaque architecture donne-t-elle ?
37. Une machine réservée coûte 60 % du tarif à la demande, que l'on utilise la machine ou non. À partir de quel taux d'usage la réservation devient-elle rentable ?
38. Pourquoi les frais de sortie peuvent-ils changer le choix d'un fournisseur ?

Applications de démonstration (chapitre 7)

39. Une application Streamlit réentraîne un modèle à chaque clic. Quelle est la cause, et comment la corriger ?
40. Quand faut-il remplacer l'application par une API ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np

print("Q1 paramètres :", 784 * 128 + 128 + 128 * 10 + 10)
print("Q3 filtres : ", 8 * (3 * 3 * 1) + 8, "paramètres ; carte", 28 - 3 + 1, "x", 28 - 3 + 1)
print("Q4 0,25^10 =", f"{0.25**10:.2e}")
print("Q5 p - y =", round(0.8 - 1, 1))
print("Q9 TF-IDF =", round(2 * np.log(3 / 1), 3))
a, b = np.array([1, 0, 1]), np.array([1, 1, 0])
print("Q10 cosinus =", round(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)), 2))
print("Q12 écart-type des scores =", np.sqrt(64))
print("Q13 12 d^2 =", 12 * 256**2)
softmax = lambda z, T: np.round(np.exp(np.array(z) / T) / np.exp(np.array(z) / T).sum(), 3)
print("Q15 T=1 :", softmax([2, 1, 0], 1), " T=0,5 :", softmax([2, 1, 0], 0.5))
print("Q17 Amdahl :", round(1 / (0.1 + 0.9 / 10), 2), "sur 10 ; plafond", 1 / 0.1)
print("Q23 Little : L =", 50 * 0.2)
ps, qs = np.array([0.5, 0.5]), np.array([0.3, 0.7])
print("Q27 PSI =", round(float(((qs - ps) * np.log(qs / ps)).sum()), 4))
print("Q31 lignes propres =", 19700 - 1700 - 782)
print("Q33 paires =", 5000 * 4999 // 2)
print("Q36 série =", round(0.999 * 0.999 * 100, 4), "% ; parallèle =", round((1 - 0.001**2) * 100, 4),
      ↪ "%")
```

```
Q1 paramètres : 101770
Q3 filtres : 80 paramètres ; carte 26 x 26
Q4 0,25^10 = 9.54e-07
Q5 p - y = -0.2
Q9 TF-IDF = 2.197
Q10 cosinus = 0.5
Q12 écart-type des scores = 8.0
Q13 12 d^2 = 786432
Q15 T=1 : [0.665 0.245 0.09 ] T=0,5 : [0.867 0.117 0.016]
Q17 Amdahl : 5.26 sur 10 ; plafond 10.0
Q23 Little : L = 10.0
Q27 PSI = 0.1695
Q31 lignes propres = 17218
Q33 paires = 12497500
Q36 série = 99.8001 % ; parallèle = 99.9999 %
```

1. $784 \times 128 + 128 + 128 \times 10 + 10 = 101\,770$ (code ci-dessus) : une matrice de poids et un biais par couche (1.1.3).

2. Un neurone calcule $\sigma(w \cdot x + b)$, exactement la forme de la régression logistique, et la perte logistique est la même : seul l'**algorithme** change (descente de gradient plutôt qu'une résolution dédiée) (1.1.1, 1.1.4).

3. $8 \times (3 \times 3 \times 1) + 8 = 80$ paramètres, quelle que soit la taille de l'image ; la carte de sortie est de $28 - 3 + 1 = 26$ sur 26 (par filtre). Le **partage** des filtres explique ce faible compte (1.2.4).

4. La dérivée de la sigmoïde vaut au plus 0,25 ; multipliée à travers 10 couches, elle donne au mieux $0,25^{10} \approx 10^{-6}$: les premières couches n'apprennent presque plus. Remèdes : **ReLU** (dérivée 1 pour les valeurs positives), **initialisation de He**, normalisation, connexions résiduelles (1.1.6).

5. Pour la sigmoïde et la perte logistique, $\partial L / \partial z = p - y = 0,8 - 1 = -0,2$: le gradient est l'écart entre la probabilité et l'étiquette, sans facteur parasite (1.1.5).

6. **Surapprentissage**. Leviers : **arrêt précoce**, **décroissance des poids**, **dropout**, réseau plus petit, **augmentation** des données, davantage de données ; on juge sur **plusieurs graines**, pas sur une

seule exécution (1.1.8).

7. La porte d'oubli décide, à chaque pas, quelle fraction de la mémoire de la cellule on **garde**. En laissant passer le signal presque intact, le LSTM atténue la **disparition du gradient dans le temps** du réseau récurrent simple (1.3.2, 1.3.3).

8. Sur ce tableau, le **boosting fait mieux** (0,896 contre 0,869) : le réseau n'est pas le bon outil par défaut sur des données tabulaires de cette taille. Avant de généraliser, il faut **répéter sur plusieurs graines et découpages**, comparer avec un **test apparié** et vérifier que le réseau a été réglé aussi sérieusement que le boosting (1.6.3).

9. $\text{tf} \cdot \ln(N/\text{df}) = 2 \ln 3 \approx 2,197$ (code ci-dessus) : un mot rare dans le corpus pèse plus qu'un mot partout présent, dont $\ln(N/N) = 0$ (2.1.3).

10. Produit scalaire 1 ; normes $\sqrt{2}$ chacune ; cosinus = $1/2 = 0,5$ (2.1.3).

11. Un test **tiré du même moule** (mêmes gabarits, mêmes tournures) mesure surtout la mémorisation du gabarit. Il faut un test **écrit à la main, hors gabarit** : dans le chapitre, la même méthode tombe de 94,2 % à 62,5 % (2.1.4, 2.1.5).

12. Le produit scalaire de deux vecteurs aléatoires de dimension d_k a une variance égale à d_k ; sans division, le softmax **sature** (une probabilité proche de 1) et les gradients s'effondrent. Écart-type pour $d_k = 64$: $\sqrt{64} = 8$ (2.2.3).

13. $12 \times 256^2 = 786\,432$ paramètres environ : $4d^2$ pour l'attention (requêtes, clés, valeurs, sortie) et $8d^2$ pour le réseau de la couche suivante, avec un facteur 4 (2.2.5).

14. Chaque jeton se compare à **tous** les autres : n^2 scores. Doubler la longueur multiplie donc le coût par **quatre** (2.2.6).

15. La température **divise les logits** avant le softmax : basse, elle concentre la masse sur le jeton le plus probable ; haute, elle l'étale. Résultats ci-dessus : à $T = 1$, (0,665; 0,245; 0,090) ; à $T = 0,5$, la tête de distribution monte à 0,867 (2.3.4).

16. Du début à la fin : **la recherche** (les bons passages sont-ils retrouvés ?), puis **le contexte fourni** (est-il complet, non contradictoire ?), puis **la génération** (la réponse reste-t-elle fidèle aux passages ?), avec des citations pour vérifier. Un maillon faux donne une réponse fausse et assurée (2.5.3).

17. $S(10) = 1/(0,1 + 0,09) \approx 5,26$; plafond $1/(1 - p) = 10$ quel que soit le nombre de machines, et le coût du mélange abaisse encore ces valeurs (3.1.5).

18. Quand les données **tiennent en mémoire** d'une machine, quand le **temps de coordination** dépasse le gain, quand une **requête SQL sur un fichier Parquet** avec DuckDB répond en secondes, quand l'équipe n'a pas les moyens d'**exploiter** une grappe (3.1.10, 3.2.11).

19. La moyenne n'est pas **associative** : la moyenne de moyennes de groupes de tailles différentes est fausse. On combine des **sommes et des effectifs** : chaque partition renvoie (somme, effectif), puis on divise à la fin (3.1.4).

20. Quelques clés regroupent une grande part des lignes : une partition travaille bien plus que les autres et ralentit tout. Le **salage** ajoute un suffixe aléatoire à la clé, répartit la clé chaude sur plusieurs partitions, puis recombine (3.2.9).

21. Les **transformations** ne calculent rien ; elles construisent un plan, que seule une **action** déclenche. Un Exchange marque un **mélange** : des données voyagent entre machines, c'est là que se paie la performance (3.2.3, 3.2.4).

22. Être **idempotent** : traiter deux fois le même message ne doit pas changer le résultat (clé de message, fusion plutôt qu'ajout) (3.3.3, 3.3.5).
23. $L = \lambda W = 50 \times 0,2 = 10$ requêtes en cours en moyenne : de quoi dimensionner le nombre de processus (4.2).
24. Charger un fichier pickle **exécute du code** choisi par son auteur : un fichier piégé prend la main sur la machine. On ne charge que des fichiers de source maîtrisée, ou l'on préfère un format qui n'exécute rien (ONNX, formats déclarés) (4.2, 4.5).
25. Les données vues par le service ne sont pas préparées comme à l'entraînement. Exemple du chapitre : une **erreur d'unité** dans une entrée ne plante rien mais fait chuter l'AUC de 0,867 à 0,538 sans aucune erreur visible (4.1).
26. En **fantôme**, la nouvelle version reçoit les mêmes requêtes mais **ses réponses ne sont pas utilisées** : on compare sans risque. En **canari**, une petite fraction du trafic réel est servie par la nouvelle version : on **mesure** avant d'élargir (4.2).
27. $\sum (q - p) \ln(q/p) = 0,1695$ (code ci-dessus), soit entre 0,1 et 0,25 : un changement **modéré** selon l'usage courant (4.7).
28. **Dérive des variables** : la distribution des entrées change ; le **PSI** ou le test de Kolmogorov-Smirnov la repère. **Dérive du concept** : le lien entre entrées et résultat change, les entrées restent identiques (PSI sous 0,02) ; seuls la **performance** ou l'**écart de calibration** la repèrent (4.7, projet P.7).
29. L'estimation repose sur l'hypothèse que **le lien entre variables et cible n'a pas changé** : si le concept dérive, elle reste confiante à tort. Dans le chapitre : 63 % annoncés, 10 % réels (4.3).
30. Le service charge le modèle par un **alias** (« production ») du registre : revenir en arrière, c'est **déplacer l'alias** vers la version précédente, sans redéployer le code (4.2, 4.5).
31. $19\,700 - 1\,700 - 782 = 17\,218$ commandes propres (code ci-dessus). Chaque ligne lue doit se retrouver **quelque part** (propre, doublon, rebut) : si l'équation ne tombe pas juste, le pipeline perd ou invente des lignes en silence (5.1.4).
32. Un ajout simple **recopie** les lignes du lot rejoué ($17\,218 \rightarrow 23\,111$ dans le chapitre). Une **fusion** identifie la ligne par sa clé : la ligne existante est remplacée, rien n'est dupliqué (5.1.7).
33. $5\,000 \times 4\,999/2 = 12\,497\,500$ paires, soit environ 12,5 millions. Le **blocage** ne compare que les enregistrements partageant une clé grossière (code postal, initiale) : 828 paires dans le chapitre, pour un rappel qu'il faut **mesurer** (5.3.5).
34. Une empreinte sans clé se **retrouve par dictionnaire** : on hache les adresses possibles et on compare (4 200 adresses retrouvées sur 4 200 dans le chapitre). Il faut une **clé secrète** (HMAC), conservée à part, et se souvenir que les **quasi-identifiants** réidentifient (5.4.5).
35. Par exemple : **complétude, validité, unicité, cohérence, exactitude, fraîcheur**. Une règle de validité : `def age_valide(x): return 18 <= x <= 110` ; une règle d'unicité : `df["id"].is_unique` (5.2.2, 5.2.3).
36. En série : $0,999^2 = 99,8001\%$, **moins** que chaque maillon ; en parallèle (redondance qui bascule correctement) : $1 - 0,001^2 = 99,9999\%$. Un maillon unique plafonne l'ensemble (6.1.4).
37. On paie 0,6 par heure de l'année, utilisée ou non ; la machine à la demande coûte u par heure d'usage : la réservation est rentable dès $u > 60\%$ d'usage. Avec les prix inventés du chapitre, le seuil est de 62,5 % (6.3.1).
38. On paie la **sortie** des données (et non l'entrée) : quand les volumes sortants sont grands, ces frais peuvent valoir **dix fois** le calcul, et rendre la migration coûteuse (**gravité des données**)

(6.1.6, 6.3.3).

39. Streamlit **rejoue le script** à chaque interaction : sans cache, l'entraînement se refait à chaque clic. On le place dans une fonction décorée par `cache_resource` (ou `cache_data` pour des données) pour ne le payer qu'une fois (7.2.1, 7.2.4).

40. Quand **un autre programme** doit appeler le modèle, quand **plusieurs équipes** le partagent, quand le modèle est **versionné** ou qu'une **exigence de disponibilité** existe : une application est faite pour un humain, une API pour des programmes (7.4.9, 4.6).

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Calculer une rétropropagation, compter des paramètres	1.1
Régler optimiseur et régularisation, juger sur plusieurs graines	1.1.7, 1.1.8
Expliquer convolution, pooling, récurrence, LSTM	1.2, 1.3
Comparer un réseau à une référence sur un tableau	1.6
Représenter un texte et bâtir une référence solide	2.1
Calculer l'attention, décrire un transformer	2.2
Régler un décodage, connaître les limites des modèles de langage	2.3
Construire et évaluer un RAG, un agent	2.5
Décider s'il faut distribuer, borner un gain	3.1
Lire un plan Spark, traiter asymétrie et petits fichiers	3.2
Distinguer lot et flux, écrire des traitements idempotents	3.3
Rendre un modèle reproductible, le tester, le servir	4.1, 4.2
Superviser, régler des alertes, mesurer la dérive	4.3, 4.7
Orchestrer, suivre les essais, automatiser la livraison	4.4, 4.5, 4.6
Concevoir un ETL rejouable et en mesurer la qualité	5.1, 5.2
Réconcilier des enregistrements, protéger les personnes	5.3, 5.4
Raisonner sur coûts et sécurité dans le cloud	6
Concevoir, tester et publier une application de démonstration	7
Déployer et surveiller un modèle de bout en bout	Projet du volume